



PYTHON PROGRAMLAMA



Elektronik Yüksek Mühendisi İlhan ÖZKAN
Ankara, Şubat 2026

ÖNSÖZ

Ülkemizde her alanda olduğu gibi yazılım alanında da bir terminoloji karmaşasının yaşandığı yazılan kitaplardan anlaşılmaktadır. Son dönemde yazılan kitaplar, adeta çeviri yapan programların bir gecede ürettiği çıktılar birleştirilerek hazırlanmaktadır. Bu kitaplar incelendiğinde, bilindiği varsayılan kavramların bile tanımlanırken birbirinin içerisine girdiği görülmektedir. 1989 yılından beri birçok programlama dili kullanarak yazılım geliştiriyor olmama rağmen, bu kitapların içeriğini anlama konusunda oldukça zorlanıyorum.

Günümüz dünyasında yazılım geliştirme, sadece profesyonel programcıların değil; bilim insanlarının, mühendislerin, öğrencilerin ve hatta sanatçıların bile ihtiyaç duyduğu bir beceri haline geldi. Bu ihtiyaçlar doğrultusunda, bilgisayar mimarisini bilmeden öğrenmesi kolay, esnek ve güçlü bir programlama dili olan Python, dünya genelinde en çok tercih edilen dillerden biri haline geldi.

Bu kitap, Python programlama dilini sıfırdan başlayarak adım adım öğrenmek isteyen herkese hitap etmektedir. İster bilgisayar mühendisliği öğrencisi olun ister veri bilimine ilgi duyan bir araştırmacı, isterse sadece hobi olarak programlamaya başlamak isteyenler için bu kitap, temel kavramlardan ileri düzey konulara kadar rehberlik edecektir. Bu dilin eğitimini verecek tüm kişiler için bu kitap serbestçe kullanılabilir.

Kitap boyunca sadece sözdizimini değil, aynı zamanda iyi programlama alışkanlıklarını, algoritmik düşünmeyi ve gerçek dünya problemlerine Python ile nasıl çözüm geliştirileceğini öğrenme fırsatı bulacaksınız. Her bölüm, açıklayıcı örnekler ve uygulamalarla desteklenerek, bilgilerin pekişmesini amaçlamaktadır.

Python'un sadeliği, okunabilirliği ve geniş kütüphane desteği, onu sadece öğrenilmesi kolay değil; aynı zamanda güçlü ve üretken bir dil yapar. Bu kitabın sonunda, kendi projelerinizi geliştirebilecek ve Python'un sağladığı zengin ekosistemden faydalanabilecek düzeye geleceksiniz.

Bu kitabın güncel sürümüne <https://github.com/HoydaBre/python> adresinden erişilebilir. Amacım Türkçe düşünen ve konuşan öğrencilerimize kavramları doğru bir şekilde aktarmaktır. Katkılarınızı bekler bu yolculukta size rehberlik etmekten büyük bir mutluluk duyuyorum. Kodlama serüveninizde başarılar dilerim!

İlhan ÖZKAN
ilhanozkan[at]outlook.com
www.linkedin.com/in/ilhanozkan/

İÇİNDEKİLER

ÖNSÖZ	1
İÇİNDEKİLER.....	2
TEMEL BİLGİSAYAR KAVRAMLARI	8
Mühendis ve Teknisyen	8
Bilgisayara Niçin İhtiyaç Duyulur?	8
Veri ve Bilgi	8
En Basit Bilgisayar	8
İşlemcinin Emri Alma, Çözme ve İcra Döngüsü.....	10
İşlemci Tasarlama Stratejileri.....	11
Program ve Programlama	11
Genel Amaçlı Bilgisayarlar ve İşletim Sistemleri.....	12
TEMEL YAZILIM KAVRAMLARI	14
Tarihçe.....	14
Genel Amaçlı Yüksek Düzey Diller.....	15
Arapsaçı Kod.....	15
Değişken	16
Fonksiyon.....	17
Yapısal Programlama	18
Nesne Yönelimli Programlama İhtiyacı	19
Nesne Yönelimli Programlama.....	20
Python Programlama Dili	20
PYTHON YORUMLAYICISI.....	22
Derleyiciler ne yapar?	22
Yorumlayıcı nedir?	23
Python Yorumlayıcısı.....	23
PYTHON DİLİNE GİRİŞ	25
Python Dili Söz Dizim Kuralları.....	25
Talimatlar ve Anahtar Kelimeler	25
Açıklamalar.....	26
Değişkenler.....	26
Değişken Tanımlama.....	26
Çoklu Değişken Tanımlama	27
Değişken İsimlendirme Kuralları.....	27
Değişkenlerin Bellek Adresleri	28
Mevcut Değişkenleri Görme	28
Değişkenleri Silme	29
Değişken Tiplerini Test Etme	29
Veri Tipleri.....	29
Dahili Veri Tipleri.....	29
Diziler ve Koleksiyonlar	32
Numaralandırma	34
Dahili Sabitler	35
ARDIŞIK İŞLEMLER	36
Ardışık İşlem Nedir?.....	36
Konsoldan Girdi Okuma ve Konsola Veri Yazma	36
Tip Dönüşümleri.....	37
İşleçler.....	38
Aritmetik İşleçler.....	38
Karşılaştırma İşleçleri.....	39
Atama İşleçleri.....	39
Mantıksal İşleçler	40

Bit Düzeyi İşleçler	40
Üyelik İşleçleri.....	41
Kimlik İşleçleri.....	42
İşleç Öncelikleri.....	42
Kayan Noktalı Sayı Hesapları.....	43
Konsola Biçimlendirilmiş Metin Yazma.....	44
Dizgi modül işleci	44
format() Yöntemi.....	44
f-Dizgisi	45
Hangisini Kullanmalıyız?	46
KONTROL İŞLEMLERİ	47
Kontrol İşlemleri.....	47
Satırlar ve Girintileme.....	47
Akış Diyagramları.....	48
Duruma Göre Seçimler.....	49
If Talimatı.....	49
If-Else Talimatı.....	50
Match Talimatı.....	52
İlişkisel Döngüler.....	54
For Talimatı.....	54
Aralık Nesneleri.....	55
For-Else Talimatı.....	56
While Talimatı.....	57
İç İç Döngü Kodu	57
Gözcü Kontrollü Döngüler.....	58
Dallanmalar.....	58
Break Talimatı.....	58
Continue Talimatı	59
Pass Talimatı.....	60
Return talimatı	61
FONKSİYONLAR.....	62
Fonksiyon Nedir?	62
Hazır Fonksiyonlar	62
Kullanıcı Tanımlı Fonksiyonlar.....	65
Fonksiyon Tanımlama.....	65
Fonksiyonlara Parametre Geçirme	66
Fonksiyon Açıklamaları	68
Değişken Kapsamı	69
Özyinelemeli Fonksiyonlar.....	69
Kendi Modülümüzü Oluşturma	71
Paket (Package) yönetimi.....	71
Lamda İfadeleri.....	73
NESNE YÖNELİMLİ PROGRAMLAMA	76
Nesne Yönelimli Programlama Paradigması.....	76
Sınıf ve Nesne	76
Yöntemler.....	80
Erişim Belirleyiciler	82
Soyutlama (Abstraction)	83
Sarmalama (Encapsulation).....	84
Kalıtım (Inheritance).....	84
Aşırı Yükleme	88
Geçersiz Kılma (Overriding).....	89
Çok Biçimlilik (Polymorphism).....	89

Soyut Sınıf (Abstract Class).....	91
Ara Yüzler yani Roller (Interface).....	92
İç Sınıf (Inner Class).....	94
Tekil Nesne Sınıfı (Singleton).....	94
Yansıma (Reflection).....	95
type() fonksiyonu.....	95
isinstance() fonksiyonu.....	95
issubclass() fonksiyonu.....	96
callable() fonksiyonu.....	96
getattr() fonksiyonu.....	96
setattr() fonksiyonu.....	97
hasattr() fonksiyonu.....	97
dir() fonksiyonu.....	97
Dekoratörler.....	98
Fonksiyonların Nesne Olarak Kullanılması.....	98
Dekoratör Söz Dizimi ve @ Operatörü.....	98
Temel Bir Dekoratör Tasarımı.....	98
Parametrelili Dekoratörler ve *args, **kwargs.....	99
@functools.wraps Kullanımı.....	99
Neden Dekoratör Kullanırız?.....	99
VERİ YAPILARI.....	100
Dizi (array).....	100
Liste (list).....	103
Dizgi (str).....	106
Demet (tuple).....	108
Küme (set).....	109
Sözlük (dict).....	112
İSTİSNA YÖNETİMİ.....	117
try-except Bloğu.....	117
Neden try-except Kullanmalıyız?.....	117
İstisnai Duruma Düşme.....	118
Kendi İstisna Sınıflarımızı Oluşturma.....	119
PYTHON STANDART KÜTÜPHANESİ.....	120
Zaman ve Tarih Yönetimi.....	120
Zamanın Temsili ve Temel Sınıflar.....	120
Sistem Zamanına Erişim.....	120
Zaman Aritmetiği.....	120
Biçimlendirme.....	120
Zaman Dilimleri.....	121
Metin İşleme ve Düzenli İfadeler.....	121
Temel Karakterler ve Belirleyiciler.....	121
re Modülü Temel Fonksiyonları.....	121
Gruplandırma ve Veri Ayıklama.....	121
Açgözlü Eşleşme.....	122
Derlenmiş Desenler ve Performans.....	122
Dosya Yönetimi ve Bağlam Yöneticileri.....	122
Dosya Açma ve Erişim Modları.....	122
Bağlam Yöneticileri.....	123
Dosya Okuma Teknikleri.....	123
Yapılandırılmış Veri Saklama.....	123
Dosya ve Dizin Kontrolleri.....	123
Sistem Parametreleri ve Etkileşimi.....	123
Komut Satırı Argümanları.....	124

Bellek Yönetimi.....	124
Standart Giriş, Çıkış ve Hata Akışları.....	124
Dosya İşlemlerinde Özel Hata Türleri.....	124
Dosyaların Ön Tanımlı Olarak Aranacağı Dizinler	124
Programdan Çıkış.....	125
Yorumlayıcı ve Platform Bilgileri.....	125
Matematiksel Hesaplamalar	125
Matematiksel Sabitler	125
Sayı Yuvarlama ve Sınır Değerler	125
Üs, Logaritmik ve Güç Fonksiyonları	125
Trigonometrik İşlemler.....	126
Özel Durumlar	126
Bilimsel Hesaplama	126
NumPy Dizileri.....	127
Özel Dizi Oluşturma Fonksiyonları	127
Dizi Özellikleri.....	128
Dizi İndisleri ve Dilimleme	128
Matematiksel İşlemler.....	130
Yayın	132
Pratik Örnekler	133
Bilimsel Hesaplama Özeti.....	135
Olasılık ve İstatistiksel Analiz Araçları.....	136
Rastlantısallık ve Simülasyon	136
Temel Veri Analizi.....	136
Yüksek Performanslı Döngü Yönetimi	136
Sonsuz ve Koşullu Yineleyiciler	136
Kombinasyonel Araçlar	136
İleri Seviye Veri Yapıları.....	136
JSON Veri Değişim Formatı	137
JSON Yapısı ve Python Nesneleriyle İlişkisi	137
Serileştirme.....	137
Dosya Tabanlı İşlemler	137
JSON ile Veri Bütünlüğü.....	137
Örnek Uygulama	137
TEST VE HATA AYIKLAMA	139
Yazılım Doğrulamada Birim Testi	139
unittest Modülü Temel Yapısı.....	139
İddia Yöntemleri	139
Test Hazırlığı ve Temizliği	139
pytest Modülü.....	140
assert Fonksiyonu.....	141
Birden Fazla Girdiyle Test.....	142
Sahte Nesneler	143
pytest-mock Modülü.....	144
Veri Tabanı Testi.....	144
Test Kapsama Oranı	145
Testleri Gruplandırma.....	145
Paylaşımlı Demirbaşlar.....	146
Zaman Uyumsuz Fonksiyon Testleri.....	146
Test Odaklı Yazılım Geliştirme	147
En İyi Uygulamalar.....	147
Test İçeren Örnek Proje.....	147
ZAMAN UYUMSUZ PROGRAMLAMA.....	149

Zaman Uyumlu ve Uyumsuz Programlama	149
async ve await Anahtar Kelimeleri	150
Birden Fazla Görevi Birlikte Çalıştırma.....	150
Görev Oluşturma.....	150
Asenkron Dosya İşlemleri	151
Asenkron HTTP İstekleri	152
Asenkron Bağlam Yöneticileri	152
Asenkron Yineleyiciler	153
Zaman Aşımı	153
Asenkron Kuyruk	153
Hata Yönetimi.....	154
İlk Tamamlanan.....	154
Özet ve En İyi Uygulamalar.....	155
TÜR İPUÇLARI.....	156
Tür İpuçlarının Temelleri	156
Temel Veri Tipleri.....	156
Gelişmiş Tür İpuçları	156
Python 3.10+ Modern Özellikler	157
Eşleşme Desenleri.....	157
Union İşleci.....	157
Statik Tip Kontrolü.....	157
Generic Tipler ve TypeVar	158
Literal ve Final.....	158
Protocol ve Structural Subtyping.....	158
TypedDict.....	159
Callable.....	159
Özet ve En İyi Uygulamalar.....	159
WEB GELİŞTİRME.....	161
İlk FastAPI Uygulaması	161
Path Parametreleri.....	161
Query Parametreleri.....	162
Request Body	162
HTTP Yöntemleri	163
Response Models.....	163
Hata Yönetimi.....	164
Dependency Injection.....	164
Asenkron Endpoint'ler.....	164
Dosya Yükleme.....	165
Çapraz Kaynak Paylaşımı	165
Arka Planda Görev Çalıştırma.....	165
Veritabanı Entegrasyonu (SQLAlchemy).....	166
Modüler Yapı.....	167
Gerçek Zamanlı İletişim.....	167
API'leri Test Etme	168
En İyi Uygulamalar	168
Web Proje Yapısı	168
EN İYİ UYGULAMALAR VE TEMİZ KOD.....	169
Python Stil Rehberi	169
Girinti.....	169
Satır Uzunluğu.....	169
İsimlendirme Kuralları.....	169
Modülleri İçer Aktarma Düzeni.....	170
Boşluklar	170

Dokümantasyon Metinleri	170
Tür İpuçlarını Kullanın.....	171
SOLID İlkeleri.....	171
S-Single Responsibility (Tek Sorumluluk).....	171
O-Open/Closed (Açık/Kapalı).....	172
L-Liskov Substitution (Liskov İkame).....	173
I-Interface Segregation (Arayüz Ayrımı).....	173
D-Dependency Inversion	174
Tekrarlardan Kaçınma İlkesi.....	174
Aptala Anlatır Gibi Basit İlkesi.....	175
Code Smell'ler ve Çözümleri.....	175
Long Method (Uzun Fonksiyon).....	175
Magic Numbers (Sihirli Sayılar).....	176
Kod Kalitesi Araçları.....	176
İz Kayıtları Oluşturma.....	176
Konfigürasyon Yönetimi.....	177
Bağlam Yöneticisi.....	177
Liste veya Sözlük Kavrama.....	178
En İyi Uygulamalar	178
ŞEKİL LİSTESİ	180
TABLO LİSTESİ.....	181
DİZİN	182

TEMEL BİLGİSAYAR KAVRAMLARI

Mühendis ve Teknisyen

Mühendis (**engineer**), ihtiyaç duyulan ürünü; **ekonomik** (**cost**), **güvenli** (**safe**) ve **kullanışlı** (**functional**) olarak **zamanında** (**on time**) ortaya koyan kişilerdir. Bunu yapmak için; **keşif** (**invent**), **tasarım** (**design**), **analiz** (**analysis**), **uygulama** (**build**) ve **sinama** (**test**) yaparlar¹.

Konumuz yazılım olduğundan, yazılım mühendisleri ise istenilen bilgisayar yazılımını ekonomik, güvenli ve kullanışlı olarak zamanında ortaya koyan kişilerdir. Programcılar ise teknisyenlere karşılık gelir. Yani geliştirme aşamasında bazı yazılım kısımlarının kodlanmasını veya geliştirilen yazılımın çalışır tutulmasını sağlarlar.

Bilgisayara Niçin İhtiyaç Duyulur?

Bir çiftçi niçin traktöre ihtiyaç duyar? Bir berber niçin elektrikli tıraş makinesi kullanır? Benzer şekilde bir mühendis niçin bilgisayara ihtiyaç duyar?

İşte bütün bu soruların cevabı yapılacak işleri daha kısa sürede yapmaktır. Yani bütün araç gereç ve makinalara olan ihtiyacımız işlerimizi daha **hızlı** (**speed**) yapabilmek içindir. Bütün makineler gibi bilgisayar da veriden hızlıca bilgi elde etmemizi ve tekrarlanan hesaplamaları hatasız yapmamızı hızlıca sağlayan makinelerdir.

Veri sorumlusu, veri hazırlama kontrol işletmeni gibi unvanlar bu sebeple ortaya çıkmıştır. Bu unvanlara ihtiyaç duyulmasının sebebi verinin belli bir şekle uygun olarak bilgisayara girilmesi ve işlenen verinin saklanması ve raporlanması bilgisayar dünyasında önemli bir yer tutmasıdır.

Bilgisayarlar ilk zamanlarda daha çok cebirsel ifadeleri hatasız ve hızlı bir şekilde yapmak için kullanılmışlardır. Bunu daha sonradan **ALGOL** (**ALGO**ritmic **L**anguage) adını alacak **IAL** (**I**nternational **A**lgebraic **L**anguage) ya da **FORTTRAN** (**FOR**mula **TRAN**slation) dilinden de anlayabiliriz.

Veri ve Bilgi

Veriler (**data**), genellikle bir ölçüm ya da sayım sonucu elde edilen ve hakkında az şey bilinen varlıklardır. Örneğin ölçülen hava sıcaklığı, derse giren öğrenci sayısı, tartılan sebzenin ağırlığı gibi şeyler.

Bilgi (**information**) ise verinin işlenerek ortaya çıkan anlamlı verilerdir. Örneğin ortalama hava sıcaklığı, ölçülen yerde bir ölçüm yapmaya gerek kalmadan sıcaklığı yaklaşık olarak bilmeye yarayan bir bilgidir. Bir sınıftaki not ortalaması, o sınıftan rastgele seçilen bir öğrencinin notunu yaklaşık belirleyen bir bilgidir. Kısacası bilgi, işlenmiş veridir.

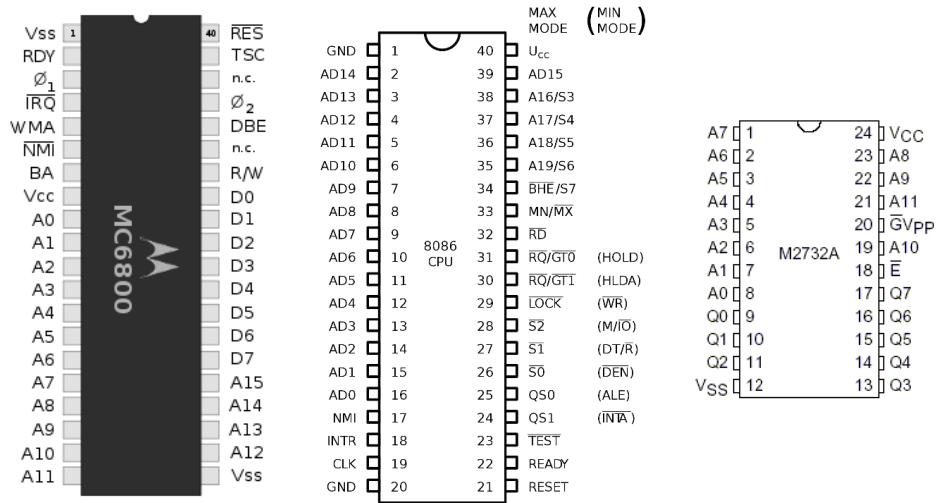
En Basit Bilgisayar

En basit bilgisayar;

- **Merkezi işlem birimi** (**Central Processing Unit-CPU**) kısaca işlemci,
- **Bellek** (**memory**) ve
- Bunları birbirine bağlayan elektrik **yollarından** (**bus**) oluşur.

Bellek ise işlemcinin işleyeceği emirleri barındırır. Belleğin hem **adres** (**address bus**) hem de **veri** (**data bus**) bacakları bulunur. Benzer şekilde işlemcinin de bellek ile iletişimini sağlayan adres ve veri bacakları bulunur. Belleğin bu bacakları elektrik yolları ile işlemcinin adres ve veri bacaklarına bağlıdır. Bunların yanında hem işlemcinin hem de belleğin **okuma** (**read**), **yazma** (**write**), **kesme** (**interrupt**) veya kristal bağlanan bacakları gibi **kontrol** (**control**) bacakları da bulunur.

¹ <https://www.bls.gov/>



Şekil 1. Motorola 6802, Intel 8086 İşlemciler ile 2732 EPROM Belleği

Şekil 1’de görüldüğü üzere 6800 işlemcisinin 15 adet adres bacağı, 8 adet veri bacağı bulunmaktadır. 8086 işlemcisinin ise 20 adet adres bacağı bulunmakta olup bu bacakların 16 adedi aynı zamanda veri bacağı olarak kullanılmaktadır. 2732 belleğin ise 12 adet adres bacağı, Q ile gösterilen 8 adet veri bacağı bulunmaktadır.

İşlemci ile bellek arasında bağlantı sağlayan elektrik yolları (bus) adres veya veri taşır. Bu yolların veri taşıyanlarına veri yolu (data bus), adres taşıyanına ise adres yolu (address bus) adı verilir. Veri yolu; hem işlemci tarafından belleğe veri yazmak veya bellekten işlemciye taşınacak veriler için kullanıldığından çift yönlüdür. Adres yolu ise belleğin hangi bölgesindeki veriye ulaşılacağını belirleyen hatlar olup tek yönlüdür.

Bilgisayarlar elektrikle çalışan aletlerdir. İşlemci ile bellek arasında bir hatta elektrik varsa "1" yoksa "0" olarak anlamlandırılır. Bunlar ikili sayı sisteminin rakamlarıdır (Binary DigiT) ve kısaca bit olarak adlandırılır. Dolayısıyla; 1 elektrik hattının taşıyacağı veri 0 ya da 1 olabilir. 2 elektrik hattının taşıyacağı veri 00, 01, 10 ve 11 olabilir. Yani 2^2 kadar farklı veri taşıyabilir. N adet elektrik hattının taşıyacağı veri 2^N farklı alternatifte bilgi olacaktır.

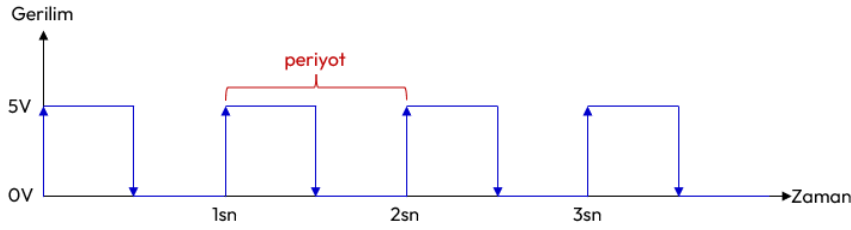
İşlemcinin veri yolunun (data bus) genişliği kelime uzunluğudur ve işlemcinin aynı anda işlem yaptığı bit sayısını da verir. 8 Bitlik veriye bayt (byte) adı verilir ve bellek miktarı genellikle bayt olarak ölçülür. Veri yolu, 8 bit ise 8 bitlik-BYTE işlemci, 16 bit ise 16 bitlik-WORD işlemci, 32 bit ise 32 bitlik-DWORD (Double WORD) işlemci, 64 bit ise 64 bitlik-QWORD (Quad WORD) işlemci olarak adlandırılır. Bu nedenle veri yolu genişliği ile işlemcinin akümülatör ve kaydedicilerin bit sayısı da aynı olur.

Adreslenebilir bellek miktarı ise işlemcinin adres hatlarının sayısı yani adres yolu (address bus) ile belirlenir. Adres yolu ne kadar geniş ise adreslenebilir bellek miktarı da o kadar artar.

İşlemci	Veri Yolu	Adres Yolu	Adreslenebilir Bellek: bayt	Kilo bayt	Mega bayt	Giga bayt
Motorola 6802	8 Bit	16 Bit	$2^{16}=65.536$	64		
Intel 8086	16 Bit	20 Bit	$2^{20}=1.048.576$	1024	1	
Intel 80286	16 Bit	24 Bit	$2^{24}=16.777.216$		16	
Intel Pentium	32 Bit	32 Bit	$2^{32}=4.294.967.296$		4096	4
Intel i7	64 Bit	36 Bit	$2^{36}=68.719.476.736$			64

Tablo 1. Bazı İşlemcilerin Adres ve Veri Yolu Genişlikleri

Bilgisayarlarda kablodaki gerilimin değeri genelde 5V, 3.3V, 2.8V, 2V, 1.5V veya 1V gibi değerler alır. Kullanıcılar için gerilimin ne olduğu değil varlığı önemlidir. Gerilim değerin yüksek olması fazla akım çekme ve ısınma anlamına gelir. Bu nedenle zamanla daha düşük gerilimle çalışan işlemciler tasarlanmıştır.



Şekil 2. Frekansı 1 olan Kare Dalga İşareti

İşlemciler kare dalga olarak oluşturulmuş elektriksel bir işareti saat olarak kullanır. Bu işaret işlemciye $\emptyset$ ya da CLK bacaklarından uygulanır. Saniyede 1 kez bir dalga üreten işaretin frekansı $1/1\text{sn} = 1 \text{ Hertz} = 1 \text{ Hz}$. olarak hesaplanır. Bir mikro saniyede bir kez dalga üreten işaretin frekansı; $1/1\mu\text{s} = 1/10^{-6}\text{s} = 10^6 \text{ Hertz} = 1.000.000 \text{ Hertz} = 1 \text{ MHz}$ olarak hesaplanır.

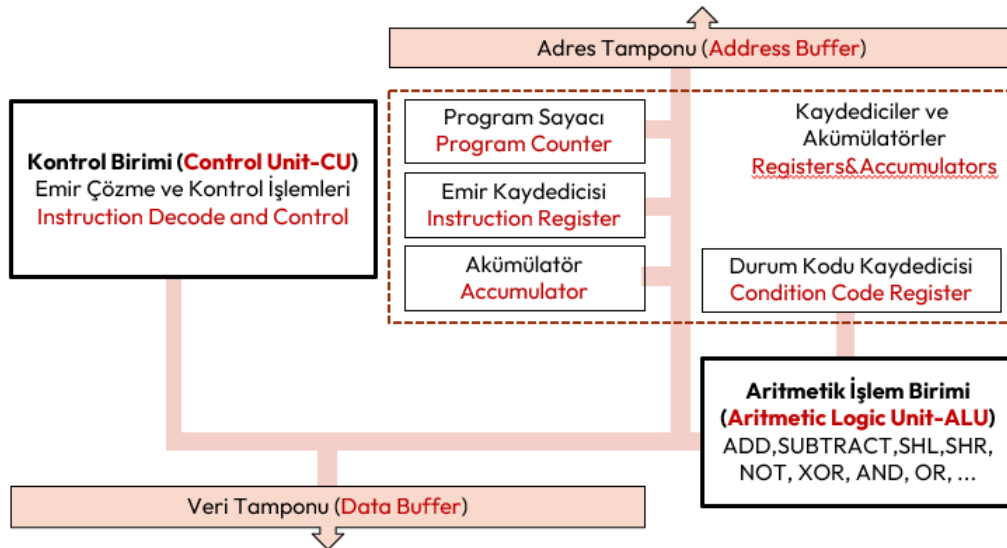
İşlemciye uygulanan saatin frekansı ne kadar yüksek olursa işlemci o kadar hızlanır. Ancak işlemcilerin de tasarımından ötürü karşılayabileceği belli bir frekans değeri bulunmaktadır. Günümüzde 5 GHz frekansını karşılayan işlemciler bulunmaktadır.

İşlemcinin Emri Alma, Çözme ve İcra Döngüsü

Basit bir işlemcide;

- **Kaydediciler (register)**, işlemci içerisinde adres ve veri saklayan mini belleklerdir.
- **Akümülatörler** ise bellekten alınan verileri saklamak için en sık kullanılan kaydedicilerdir.
- **Program sayacı (program counter)**, işlemcinin icra edeceği emri (**instruction**) takip için kullanılır. Getirilecek bir sonraki emrin bellek adresini içerir.
- **Emir kaydedicisi (instruction register)** işlemcinin o an icra ettiği emirdir.
- **Koşul kodu kaydedicisi (condition code register)**, herhangi bir işlemin durumunu gösteren farklı bayraklar (**flag**) içerir.

Bu işlemci tasarımı aynı zamanda **depolanmış program (stored program)** kavramı olarak adlandırılır ve *Von Neumann Mimarisi* olarak da bilinir². *John Von Neumann* tarafından 1946-1949 yılları arasında tasarlanan bu mimari günümüzde modern bilgisayarlarda hala kullanılmaktadır.



Şekil 3. Basit bir İşlemcinin Yapısı

Basit bir bilgisayara elektrik verildiğinde aşağıdaki üç adımlık çevrim elektrik kesilene kadar sürekli tekrarlanır;

² <https://www.youtube.com/watch?v=ByllwN8q2ss>

1. **Emri Alma** (**fetch**): Program sayacının (**program counter**) tuttuğu adresteki emir (**instruction**) bellekten okunur.
2. **Çözme** (**decode**): İşlemci içerisindeki kontrol birimi (**control unit**) tarafından emrin ne anlama geldiği ve buna bağlı nelerin yapılacağını (örneğin ulaşılacak bellek adresi değiştirilir) belirler ve program sayacı bir sonraki emir için bir artırılır.
3. **İcra** (**execute**): Çözülen emir koduna bağlı olarak gerekirse yine işlemci içinde bulunan aritmetik işlem birimini (**Aritmetic Logic Unit-ALU**) çalıştırılır ve emrin işlenmesi sonucunda ortaya çıkan değerler ise işlemci içinde bulunan **akümülatörlere** (**accumulator**) veya **kaydedicilerde** (**register**) saklanır.

Burada çözülecek **emirlerin** (**instruction**) her biri ikili sistemde bir tamsayıya karşılık gelir. Bu sayılar **emir kodu** (**instruction code**) veya **makine kodu** (**opcode**) olarak da bilinirler. İşlemciler tüm sayıları emir kodu olarak anlamaz. Yani sınırlı sayıda sayı emri tanır. Yukarıda verilen Motorola 6802 işlemcisi 72 farklı emri, Intel 8086 işlemcisi ise 116 farklı emri tanır. Çözülebilecek tüm emirlerin oluşturduğu kümeye **emir seti** (**instruction set**) adı verilir.

İşlemci Tasarlama Stratejileri

İşlemciler, aşağıda verilen üç strateji ile tasarlanırlar;

1. **İndirgenmiş Emir Setli İşlemciler** (**Reduced Instruction Set Computing-RISC**): Az sayıda emir alan yüksek hızlı işlemci tasarımı. RISC işlemciler az sayıda emre sahip olduğundan emrin çözülmesi ve icrası da kısa zaman almaktadır. Bu da çalıştırılacak kodu büyütür.
2. **Karmaşık Emir Setli İşlemciler** (**Complex Instruction Set Computing-CISC**): Çok fazla emri anlayan karmaşık nispeten yavaş işlemci tasarımı. CISC işlemciler, çok sayıda emir tanıyabildiğinden emrin çözülmesi ve icra edilmesi nispeten daha uzun sürer. Ancak yüksek düzey dillerin daha kolay makine koduna çevrilmesini kolaylaştırır.
3. **Özel İşlemciler**: Bunlar grafik kartları ve yapay zekâ gibi belli amaçlar için tasarlanmış işlemcilerdir.

Genel olarak işlemcilerde “*fetch-decode-execute*” çevrimi 1 saat periyodunda tamamlanmaz. RISC işlemciler için bu çevrim, birçok emir için 1 saat periyodunda tamamlanır. CISC işlemciler için ise birden fazla saat periyodu gerekir. Çünkü CISC işlemcilerin emir seti daha geniş olduğundan emrin çözülmesi daha fazla vakit alır. Bu nedenle RISC işlemciler CISC işlemcilere göreceli olarak daha hızlıdır.

Günümüzde ticari olarak satılan işlemcilerin çoğu ortak emir seti kullanmaya başlamışlardır;

- Masaüstü bilgisayarların birçoğu Intel ve AMD Marka işlemciler kullanır ve bunlar IA-16, IA-32, x86-16, x86-64, AMD64, ... emir setleri ve eklentilerini desteklemektedir.
- Bilinen birçok telefon işlemcisi Acorn Risc Machine-ARM tabanlı işlemciler kullanır. Bu işlemcilerde ARMv7, ARMv8, ... emir setleri ve eklentileri kullanılmaktadır.

Program ve Programlama

Belli bir işlemi gerçekleştirmek üzere, işlemciye verilen bir dizi **emre** (**instruction**), **bilgisayar programı** (**computer program**) denir³. Bu bir dizi emrin, işlemcinin çalıştıracağı şekilde hazırlaması işlemine **programlama** (**programming**) denir. Programlama **emir kodları** (**instruction code/opcode**) ile yapıldığından bu programın dili, **makine dili** (**machine language**) olarak adlandırılır.

Her işlemci üreticisi, her bir emre farklı makine kodu verdiği için programlama oldukça zorlu ve teknik bilgi gerektirmekteydi. Burada daha önce yazılmış bir programın ne yaptığını anlamak için her defasında bu listelere defalarca bakmak gerekiyordu.

Sembolik İsim (Mnemonic)	16 Tabanında Emir Kodu (Hexadecimal Opcode)	Tanım
ABA	1B	ADD A to B → A

³ "ISO/IEC 2382:2015". ISO. 2020-09-03. [Software includes] all or part of the programs, procedures, rules, and associated documentation of an information processing system.

Sembolik İsim (Mnemonic)	16 Tabanında Emir Kodu (Hexadecimal Opcode)	Tanım
INCA	4C	INCREMENT A
INCB	5C	INCREMENT B
SBA	10	SUBTRACT B from A → A
LDA	96	LOAD M to A, M: Memory
LDB	D6	LOAD M to B, M: Memory

Tablo 2. 6802 Emir Seti Örneği ve Sembolik İsim Listesi

Programın okunurluğunu artırmak için her bir emre **sembolik isimler** (**mnemonic**) verilmeye başlandı. Bu sembolik isimlerle yazılan program **montaj kodu** (**assembly code**) olarak adlandırılır. Montaj kodları bir metin dosyasına yazılarak bir dosyaya saklanır. Bu dosyalar montaj dosyaları olarak adlandırılır. Montaj dosyalarını makine diline çeviren programlar ilk **derleyicilerdir** (**compiler**) olup sembolik isimlerle yazılmış program kodlarını makine koduna çeviren programlardır.

Aşağıda x86 emir seti kullanan Linux işletim sisteminde örnek **montaj** (**assembly**) kodu örneği verilmiştir; Program genel amaçlı CL kaydedicisine koyulan sayıya kadar 2 ve 3'e bölünebilen sayıların toplamını bulup yine genel amaçlı DX kaydedicisine koyar⁴.

```
section .text                ; programı oluşturan emir kodları buradan başlar
global _start

_start:
    mov     dx, 0            ; dx 0 olsun. Toplamı tutacaktır.
    mov     cl, 99           ; 99 kez tekrarlanacak.

sum:
    mov     ax, cx           ; cx, ax e kopyalanır. Bölme işlemi için.
    mov     bl, 3            ; bl 3 olsun.
    div     bl               ; ax/bl.
    cmp     ah, 0            ; Kalan olup olmadığı kontrol ediliyor.
    jne     cont             ; Kalan yoksa, 'cont' etiketine git.
    mov     bl, 2            ; Sonra bl 2 olsun.
    div     bl               ; ax/bl
    cmp     ah, 0            ; Kalan olup olmadığı kontrol ediliyor.
    jne     cont             ; Kalan yoksa, 'cont' etiketine git.
    add     dx, c            ; Değilse, cx deki sayı 2 ve 3'e bölünür. Toplama ekle.

cont:
    loop    sum              ; Bir sonraki sayı için 'sum' etiketine dön.

exit:
    mov     eax, 1           ; Artık programdan çıkıyoruz.
    mov     ebx, 0           ; Return 0.
    int     80h              ; linux çekirdeğini çağır. (soft interrupt 80h!).
```

Genel Amaçlı Bilgisayarlar ve İşletim Sistemleri

Genel amaçlı yani herkesin kendi amacına göre kullanacağı bilgisayarlar; Veri girişini sağlayan klavye, verileri görüntüleneceği monitör, işlenen verilerin elektrik kesildiğinde saklanacağı disk ve benzeri donanımlara sahip olmalıdır.

Genel amaçlı bilgisayarın kullanıcı isteklerine cevap verebilmesi için;

1. Çeşitli programlara sahiptirler. İşte bu programlar bütününe **işletim sistemi** (**operating system**) adı verilir. DOS, Windows, Unix, Linux, MacOS, ChromeOS, OS/2 bunların en çok bilinenleridir.
2. Elektrik verildiğinde klavye, monitör ve disk olup olmadığı kontrol eden **Basic Input Output System-BIOS** programı çalıştırılır. BIOS gerekli kontrollerden sonra bilgisayara işletim sistemini yükler. İşletim sistemi yüklendikten sonra, işletim sistemi kullanıcının vereceği komutları bekler.
3. Kullanıcı işletim sistemine vereceği komutlarla yeni bir program derleyebilir, derlenmiş bir programı çalıştırabilir.

⁴ <https://github.com/armut/x86-Linux-Assembly-Examples>

4. Kullanıcının vereceği komutlar grafik olmayan işletim sistemlerinde (DOS, Unix, Linux gibi) komut yazılarak verilir. Komut yazılan bu ekrana **konsol**, **terminal** ya da **komut satırı** adı verilir. Sunucu işletim sistemlerinde hala tercih edilmektedir.
5. Grafik ara yüzlerine sahip işletim sistemlerinde (Windows ve MacOS İşletim Sistemleri ile KDE, Mate, GNome, Cinnamon, LXQt, XFce ve Deepin gibi grafik ara yüzleri içeren Linux İşletim Sistemleri) fare tıklamasıyla çoğu komut verilebilmektedir.

İşletim sistemlerinin yaygınlaşması ile kullanıcının bilgisayarı donanım bilmeden yapabilmesi sağlanmıştır. Günümüzdeki işletim sistemlerinin temel programlama dili olarak C dili kullanılmaktadır.

TEMEL YAZILIM KAVRAMLARI

Tarihçe

1642 Yılında *Blaise Pascal* tarafından icat edilen Pascalline Hesap Makinesi, eldeli toplama, ödünç almalı çıkarma yapıyordu.

1671 Yılında *Gottfried Wilhelm Leibniz* tarafından icat edilen Leibniz Çarkı, toplama, ödünç almalı çıkarmanın yanında bölme, çarpma ve karekök işlemlerini yapıyordu.

1801 Yılında *Joseph Maria Jacquard* dokuma tezgahlarında desenleri oluşturmak için **delikli kartlar** (**punch card**) kullanmayı icat etmiştir. Sonradan bilgisayara veri girdisi sağlamak ve sonuçların çıktısını göstermek için kullanılacaktır.

1830 Yılında *Charles Babbage* fark makinesi olan analitik makineyi icat etmiştir. Buhar gücü kullanan bu makinenin mantıksal işlem birimi, veri depolama birimi, giriş çıkış üniteleri bulunuyordu. *Augusta Ada Byron*, Babbage ile beraber çalışmıştır. Byron, analitik makinenin bir dizi matematiksel işlemi gerçekleştirebildiğini fark etti ve bu makineyi *Bernoulli* sayılarını üretmek için kullandı. Bu sayıların hesaplanması için yazdığı algoritma tarihteki ilk bilgisayar programı olarak kabul edilir. Bu sebeple *Byron*, ilk programcı olarak kabul görür.

1854 Yılında *George Boole* tarafından ikili sayı sistemini geliştirmiş bugün bilgisayarlarımızın kullandığı bool cebiri üzerine çalışmalar yapmıştır.

1890 Yılında *Herman Hollerith* tarafından delikli kartlarla bilgilerin yüklenebildiği ve bu bilgiler üzerinde toplama işlemlerinin yapılabilirdiği bir elektro mekanik araç geliştirdi. ABD'nin 1890 nüfus sayımında başarılı biçimde kullanıldı. *Hollerith* başarılı olunca bir şirket kurdu ve daha sonra üç firma işle birleşerek 1924 yılında adını IBM olarak değiştirdi.

1941 Yılında *Konrad Zuze Z3* isimli elektrik motorları ile çalıştırılan mekanik bir bilgisayar yaptı. Bu (Z1, Z2, Z3 ve Z4 serisi) program kontrollü ilk bilgisayardır.

1944 Yılında *Howard Aitken*, IBM ile iş birliği yaparak MARK I'i yaptı. Bilgiler, MARK I'e delikli kartlarla veriliyor ve sonuçlar yine delikli kartlarla alınıyordu. Saniyede 5 işlem yapabiliyordu. Boyutları, 18 m uzunluğunda ve 2,5 m yüksekliğinde idi. İnsan müdahalesi olmadan sürekli olarak, hazırlanan programı yürüten ilk bilgisayar idi ve tam olarak elektronik bir bilgisayar değildi.

1943-1945 Yılları arasında *Konrad Zuze*, ilk yüksek düzey programlama dili olan Plankalkül'ü, Z1 bilgisayarı için geliştirmiştir. **Emir kodlarını** (**instruction code**) ve **sembolik isimleri** (**mnemonic**) içermeyen programlama dilleri **yüksek düzey programlama dili** (**high level programming language**) olarak adlandırılır. Yüksek düzey programlama dilleri çoğunlukla İngilizce sözcükleri kullanır.

1946 Yılında *John Von Neumann* önderliğinde Pensilvanya Üniversitesinde ENIAC (Elektronik Sayısal Hesaplayıcı ve Doğrulayıcı) isimli sayısal elektronik bilgisayar tamamlandı. Askeri amaçla üretildi ve top mermilerinin menzillerini hesaplamak için kullanıldı. *Neumann* tarafından geliştirilmiş bu mimari (Stored-program computer and Universal Turing machine § Stored-program computer), günümüzdeki bilgisayarlarda hala kullanılmaktadır. 18,000 adet elektronik tüp kullanılan ENIAC; 150 kW gücünde idi ve 50 ton ağırlığıyla 167 m² yer kaplıyordu. Saniyede 5.000 toplama işlemi yapabiliyordu. Mark-I'den 1000 kat daha hızlıydı.

Bir önceki bölümde anlatıldığı üzere emir kodu veya sembolik isimlerle yazılan programlar **düşük düzey programlama** (**low level programming**) olarak adlandırılır. Bu programlamanın yapılabilmesi için hem işlemcinin hem de hafıza ve işlemciye bağlanan diğer bileşenlerin nasıl çalıştığı ve tasarlandığını çok iyi bilinmesi gerekir. Daha çok elektronik ve bilgisayar mühendisleri ya da sistemi tasarlayan matematikçiler bu programlama türünü kullanır.

Genel Amaçlı Yüksek Düzey Diller

Sembolik isimler veya makine kodu yerine bildiğimiz konuşma diline yakın talimatlarla metin olarak yazılan programlara dilleri, **yüksek düzey programlama dilleri** (**high level programming language**) olarak adlandırılır. Yani **emir kodu** (**instruction code**) ve **sembolik isim** (**mnemonic**) kullanmayan programlama dilleri yüksek düzey programlama dilleridir.

Yüksek düzey dillerde **talimatlar** (**statement**) olarak yazılan programlar yine kendine has derleyiciler tarafından makine diline çevrilir. Her talimat, genellikle birden fazla **emirden** (**instruction**) oluşur.

1954 Yılında **FORTRAN** (**FORmula TRANslation**), *John Backus* liderliğinde IBM 704 için geliştirildi. 32 farklı **talimata** (**statement**) sahiptir. İlk genel amaçlı, yüksek düzey bir programlama dilidir.

Yüksek seviyeli bir dilde yazılmış bir **talimat** (**statement**), bilgisayara belirtilen bir işlemi gerçekleştirmesini söyler. Yüksek seviyeli bir dildeki tek bir talimat, birkaç **emri** (**instruction**) içerebilir. Talimatlar, yapılaması gerekeni kısa ve net olarak belirten **mantıksal satırlardır** (**logical sequence**).

Aşağıda üç kenarı teyp ünitesinden alınan bir üçgenin alanını hesaplayan FORTRAN II programı verilmiştir⁵. Görüldüğü üzere talimatlar kısa öz ve anlaşılırdır.

```
C AREA OF A TRIANGLE - HERON'S FORMULA
C INPUT - CARD READER UNIT 5, INTEGER INPUT
C OUTPUT -
C INTEGER VARIABLES START WITH I,J,K,L,M OR N
  READ(5,501) IA,IB,IC
501 FORMAT(3I5)
  IF (IA) 701, 777, 701
701 IF (IB) 702, 777, 702
702 IF (IC) 703, 777, 703
777 STOP 1
703 S = (IA + IB + IC) / 2.0
  AREA = SQRT( S * (S - IA) * (S - IB) * (S - IC) )
  WRITE(6,801) IA,IB,IC,AREA
801 FORMAT(4H A= ,I5,5H B= ,I5,5H C= ,I5,8H AREA= ,F10.2,
  $13H SQUARE UNITS)
  STOP
END
```

1956 Yılında ticari amaçlarla kullanılabilen ve seri halde üretimi yapılan ilk bilgisayar UNIVAC I oldu. UNIVAC, giriş-çıkış birimleri manyetik bant idi ve bir yazıcıya sahipti. UNIVAC 1951-1959 arasında üretilen bilgisayarlarda vakum tüpleri kullanıldı. Bu tüpler bir ampul büyüklüğünde, çok fazla enerji harcamakta ve çok fazla ısı yaymakta idiler. Veri ve programlar manyetik teyp ve tambur gibi bilgi saklama araçlarıyla saklandı. Veriler ve programlar bilgisayara delgi kartları ile yükleniyordu. 1959 yılı sonrasında üretilen bilgisayarlarda transistörler (yaklaşık 10 bin adet) kullanıldı. Artık transistor içeren ikinci kuşak bilgisayarlar hayatımıza girmiştir.

Arapsaçı Kod

1955-1959 Yılları arasında FLOW-MATIC B0 (Business Language Version 0) Dili, *Grace Hopper* tarafından UNIVAC I için geliştirildi. İngilizceye benzeyen ilk yüksek düzey programlama dilidir. Aşağıda B0 Dilinde örnek bir program verilmiştir⁶.

```
(0) INPUT INVENTORY FILE-A PRICE FILE-B ; OUTPUT PRICED-INV FILE-C UNPRICED-INV
FILE-D ; HSP D .
(1) COMPARE PRODUCT-NO (A) WITH PRODUCT-NO (B) ; IF GREATER GO TO OPERATION 10 ;
IF EQUAL GO TO OPERATION 5 ; OTHERWISE GO TO OPERATION 2 .
(2) TRANSFER A TO D .
```

⁵ https://github.com/scivision/fortran-II-examples/blob/main/herons_formula.f

⁶ <https://gist.github.com/marcg/f9f2da31a973ba319809>


```

(3) WRITE-ITEM D .
(4) JUMP TO OPERATION 8 .
(5) TRANSFER A TO C .
(6) MOVE UNIT-PRICE (B) TO UNIT-PRICE (C) .
(7) WRITE-ITEM C .
(8) READ-ITEM A ; IF END OF DATA GO TO OPERATION 14 .
(9) JUMP TO OPERATION 1 .
(10) READ-ITEM B ; IF END OF DATA GO TO OPERATION 12 .
(11) JUMP TO OPERATION 1 .
(12) SET OPERATION 9 TO GO TO OPERATION 2 .
(13) JUMP TO OPERATION 2 .
(14) TEST PRODUCT-NO (B) AGAINST ZZZZZZZZZZ ; IF EQUAL GO TO OPERATION 16 ;
    OTHERWISE GO TO OPERATION 15 .
(15) REWIND B .
(16) CLOSE-OUT FILES C ; D .
(17) STOP . (END)

```

Görüldüğü üzere bu tarihlere kadar yazılan programlarda bir sürü GO TO ya da JUMP TO **talimatı** (**statement**) bulunmaktadır. Bu durumda programın ne yaptığı konusunda fikir yürütmeye çalıştığımızda okunaklılık oldukça azdır. İşte okunaklılığın az olduğu bu program kodlarına **arapsaçı kod** (**spaghetti code**) adı verilir. Günümüzde ne yaptığı kolay anlaşılamayan kodlar için bu kavram çokça kullanılmaktadır.

1958 Yılında LISP programlama dili, *John McCarthy* önderliğinde *Steve Russell* tarafından geliştirilmiş ikinci yüksek düzey programlama dilidir.

1959 Yılında Common Business-Oriented Language-COBOL programlama dili, *CODASYL* komitesi tarafından İngilizceye benzer ikinci yüksek düzey bir dil olarak geliştirilmiştir.

1958 Yılında daha sonra ALGOritmic Language-ALGOL adını alan International Algebraic Language-IAL Programlama Dili geliştirilmiştir. Bu dil daha sonra ortaya çıkan birçok dile önderlik etmiştir. Bu dil ile birlikte **kod blokları** (**code block**), **iç içe fonksiyon** (**nested function**), değişken **kapsamı** (**scope**) ve **dizi** (**array**) kavramları programcının hayatına girmiştir. ALGOL dili, barındırdığı özellikler sebebiyle, kendisinden sonra ortaya çıkan bütün programlama dilleri için bir esin kaynağı olmuştur.

İşin doğası gereği makine diline çevrilecek tüm metinler gözle okunabilmektedir. Yani bütün kaynak kodlar gözle okunabilir.

Değişken

Değişkenler (**variable**) aslında cebirden bildiğimiz değişkenlerdir. **Cebir** (**algebra**) işlemlerinde doğrudan problemde verilen sayıları değil, onları temsil eden değişkenleri kullanırız.

$$3x^2 + 2x + 10$$

$$c^2 = a^2 + b^2$$

$$c = 2\pi r$$

Yukarıdaki cebirsel ifadelerin ilkinde bir polinom, ikincisinde ise bir dik üçgenin kenar uzunlukları arasındaki ilişki verilmiştir. Bu ifadelerde **x**, **a**, **b** ve **r** bağımsız değişken, **c** ise bağımlı değişkendir. Aslında problemi çözerken bunlar sayılara karşılık gelir. Örneklerdeki 3,2,10,2 ve π ise **değişmez** (**literal**) sayılardır.

Bilgisayarda ise sayıları tutacak değişkenler, bir miktar bellek bölgesini işgal eder ve bu bellek bölgesinde çeşitli biçimlerde tutulurlar. Tamsayılar, **ikili** (**binary**) sayı olarak bellekte tutulur. Bu ikili sayının en anlamlı bitinin işaret biti olarak kullanılıp kullanılmayacağına göre bellekte biçimlendirilir;

Bit Sayısı	Bellek Miktarı	Sayı Sınırları
8 Bit	1 bayt	-127 ile +128 arasında veya 0 ile 255 arasında
16 bit	2 bayt	-32.767 ile +32.768 arasında veya 0 ile 65.535 arasında
32 bit	4 bayt	-2.147.483.648 ile +2.147.483.647 arasında veya

Bit Sayısı	Bellek Miktarı	Sayı Sınırları
		0 ile 4.294.967.295 arasında
64 bit	8 bayt	-9.223.372.036.854.775.808 ile +9.223.372.036.854.775.807 arasında veya 0 ile 18,446,744,073,709,551,615 arasında

Tablo 3. Tamsayı Değişkenler ve Sayı Sınırları

Gerçek sayılar **kayan noktalı sayı** (floating point number) olarak biçimlendirilir; 1914 yılında *Leonardo Torres Quevedo* tarafından Charles Babage'in analitik makinesine dayalı kayan nokta analizi yayınladı.

$$12345 = 1234,5 \times 10^1 = 123,45 \times 10^2 = 12,345 \times 10^3 = 1,2345 \times 10^4 = 0,12345 \times 10^5$$

Yukarıda ondalık tabanda verilen gerçek bir sayıya ilişkin kesir noktasının kaydırılması ile aynı sayı ifade edildiği bir örnek verilmiştir. Kesir noktası sola kaydırıldığında 10 ile çarpılmış, sağa kaydırıldığında ise 10 ile bölünmüş olur. Kayan noktalı sayının üs kısmı dışında kalan kısmı **taban** (base) veya **kesir** (fraction) olarak adlandırılır. Tabanın kesir noktası sağında kalan hanelerine ise **mantis** (mantissa) adı verilir.

1938 yılında *Konrad Zuse*, ilk **ikili** (binary) programlanabilir mekanik bilgisayar olan Z1'i yapmıştı. Bu bilgisayar, 7 bitlik işaretli üs, 17 bitlik anlamlı değer ve bir işaret biti içeren 24 bitlik ikili tabanda kayan noktalı sayı kullanmıştır. Bilgisayarlar da tamsayılar da olduğu gibi kayan noktalı sayılar da **ikili sayı tabanında** tutulur⁷. Örneğin ondalık tabanda 50,25 sayısı ikili tabanda 1.1001001×2^5 olarak ifade edilir.

Bit Sayısı	Bellek Miktarı	Sayı Sınırları
32 bit	4 bayt	$1,2 \times 10^{-38}$ ile $3,4 \times 10^{+38}$ arasında tek hassasiyetli gerçek sayılar; en soldaki 1 bit işaret biti, sonraki 8 bit üs ve geri kalan 23 bit ise kesir olarak kullanılan ikilik tabanda sayılardır.
64 bit	8 bayt	$2,3 \times 10^{-308}$ ile $1,7 \times 10^{+308}$ arasında çift hassasiyetli gerçek sayılar; en soldaki 1 bit işaret biti, sonraki 11 bit üs ve geri kalan 52 bit ise kesir olarak kullanılan ikilik tabanda sayılardır.

Tablo 4. Kayan Noktalı Sayılar ve Sınırları

Program yazılırken tanımlanacak değişkenlere ilişkin bellek ihtiyacı programcı tarafından belirlenir. Programcı değişkene tahsis edilecek bellek miktarına ve tutacağı içeriğe göre **veri tipi** (data type) belirler. Programcı tarafından kullanılacağı amaca göre belirleyeceği veri tipinde istediği kadar değişken tanımlayabilir.

Bellek sızıntıları Python ve tüm programlama dillerinde büyük bir sorundur. Ancak Python dilinde sorun ciddidir çünkü Python dilindeki değişkenler çok fazla yer kaplar. Örneğin C/C++/C# ve Java dillerinde;

- **float** bir değişken bellek 4 bayt yer kaplarken Python dilinde **24 bayt** yer kaplar.
- **int** bir değişken bellek 4 bayt yer kaplarken Python dilinde **28 bayt** yer kaplar.
- **char** bir değişken bellek 1-2 bayt yer kaplarken Python dilinde en az **49 bayt** yer kaplar⁸.

Python dilinde değişkenler çok fazla yer kaplar ama şüphesiz harika bir dildir. Çünkü değişkenleri **nesne** (object) olarak ele alır ve bilgisayarın doğal olarak bildiği makine kodu biçiminde tutmaz.

Fonksiyon

Matematikte **fonksiyon** (function), bir veya daha fazla kümenin elemanını tek bir kümenin elemanına eşleyen **ilişkidir** (relationship).

Girdi	İlişki	Çıktı
1	x2	2

⁷ IEEE Computer Society (2019-07-22). IEEE Standard for Floating-Point Arithmetic. IEEE STD 754-2019. IEEE. pp. 1-84.

⁸ <https://www.askpython.com/python/built-in-methods/variables-memory-size-in-python>

Girdi	İlişki	Çıktı
2	x2	4
3	x2	6
4	x2	8
...		

Tablo 5. İki ile çarpma fonksiyonu.

Girdi olan bağımsız değişkenler ile çıktı olan bağımlı değişken arasındaki bu ilişki, matematiksel olarak aşağıdaki şekilde ifade (expression) edilir.

$$f(x) = 2x$$

Burada f, fonksiyon anlamında x ise girdi anlamında olup parametre olarak adlandırılır. Böylece tabloda verilen fonksiyon tablosuna her girdiyi yazmak zorunda kalmayız. Bazen fonksiyona aşağıdaki örneği verilen şekilde bir ad verilmez;

$$y = 2x$$

Bazı durumlarda ise girdi daha detaylı tanımlanır;

$$x \in R \text{ Olmak üzere } f(x) = 2x + 3$$

Bu fonksiyon, reel sayı kümesindeki her x elemanını, hesaplanan f(x) değerine eşleyen bir ifadedir. Yani x=1.0 argümanını f(1.0)=5.0 reel sayısına eşler. Kısaca fonksiyon 5.0 reel sayısını hesaplayıp döndürür (return).

$$a \in R \text{ ve } b \in Z \text{ Olmak üzere } g(a, b) = 2a + b$$

Bu fonksiyon ise reel sayı kümesindeki her a elemanı ile tamsayı kümesindeki b elemanını, hesaplanan g(a,b) değerine eşleyen bir ifadedir. Yani a=1.0 ve b=1 argümanlarını g(1.0,1) fonksiyonu yine reel sayı kümesinden 3.0 reel sayısına eşler. Kısaca fonksiyon, 3.0 reel sayısını hesaplayıp döndürür (return). Buradaki x, a ve b parametre olarak adlandırılır. f(1.0) ifadesindeki 1.0 ise parametrenin o an andığı değeri belirtir ve argüman (argument) ya da gerçek parametre (actual parameter) olarak adlandırılır. Bu fonksiyonlar;

$$f(3.0) + g(3.0,2) + f(2.0) + g(1.0,3)$$

Şeklinde tanımlandıktan sonra tekrar tekrar çağrılabilirler (call).

Yapısal Programlama

1958 Yılındaki FORTRAN-II diline çıktı üretmeyen fonksiyonlar için SUBROUTINE, değer üreten fonksiyonlar için FUNCTION, CALL ve RETURN özellikleri eklenmiştir.

1958 ile 1969 Yılları arasında ALGOL diline çıktı üretmeyen fonksiyonlar için PROCEDURE ve BEGIN-END arasında kod bloğu özellikleri eklenmiştir.

1966 Yılındaki FORTRAN-IV diline INTEGER, REAL, DOUBLE PRECISION, COMPLEX ve LOGICAL veri tipleri (data type) ile Mantıksal IF, aritmetik (üç yollu) IF ve DO döngüsü talimatları (statement) eklenmiştir.

Bu yıllarda her ne kadar çıktı üretmeyen fonksiyonlar için SUBROUTINE, birden fazla çağrı noktası olabilen COROUTINE ve FUNCTION kullanılıyor olmasına rağmen hala GO TO/JUMP TO ifadeleri kullanılmakta ve kafa karışıklığına devam etmekteydi. 1968 Yılında Edsger W. Dijkstra önerliğinde GO TO/JUMP TO ifadesini zararlı olarak ilan edilmiştir.

1970 Yılında Niklaus Wirth tarafından Pascal dili geliştirildi. Bu dil ilk geliştirilen yapısal programlama dilidir. Yapısal programlamada (structural programming) verileri taşıyan veri yapıları (değişkenler ve diziler) ile bunu işleyen kontrol yapıları (if, for, do, repeat) birbirinden ayrılmıştır. Pascal dili, kendi

kendini çağıran **özyinelemeli** (**recursive**) fonksiyonlar ile değişkenlerin adreslerini tutan **göstericileri** (**pointer**) desteklemektedir⁹.

Yapısal programlamanın genel çerçevesi;

1. Programın başladığı yeri belirten bir **ana fonksiyon** (**main function**) tanımlanır. Kodlaması yapılacak işi birbirini çağıran fonksiyonlar olarak bu ana fonksiyonda yazarız. Yani yapısal programlamada programlama, fonksiyonların birbirini çağırmasıyla yapılır.
2. Her fonksiyonda ilk önce **veri yapıları** (**data structure**) tanımlanır. Veri yapıları;
 - a. En basit veri yapısı **char**, **integer**, **float** ve **double** gibi **ilkel veri tiplerinden** (**primitive data type**) olabilen **değişken** (**variable**) olabileceği gibi,
 - b. İlkel veri tiplerinden meydana gelen **dizi** (**array**),
 - c. İlkel veri tiplerinin birkaçından veya dizilerinden bir araya gelmesiyle oluşan **yapılar** (**structure**),
 - d. Ya da dinamik olarak yani çalıştırma anında birbirine bağlanmış verilerden oluşan **bağlı liste** (**linked list**) olabilir.
3. Her fonksiyonda tanımlanan veri yapılarının ardından bu yapıları işleyecek **kontrol yapıları** (**control structure**) tanımlanır. Kontrol yapıları bir veya daha fazla veya iç içe **if**, **if-else**, **switch**, **while**, **do-while**, **for**, **continue**, **break**, **goto** ve **return** **talimatlardan** (**statement**) oluşur.

Yapısal programlamanın en çok uygulandığı Pascal dilinde ana fonksiyon nokta ile biten BEGIN-END bloğudur. C dilinde ana fonksiyon, **main()** fonksiyonudur.

Nesne Yönelimli Programlama İhtiyacı

Yapısal programlamada **talimatlar** (**statements**) art arda koda yazılarak programlama yapılır ve programların neler yaptığı bu talimatlar izlenerek anlaşılabilir. Talimatların zincirin halkaları gibi birbirinin peşi sıra yazılarak yapılan programlamaya **emreden programlama** (**imperative programming**) paradigması adı verilir. Bu paradigmaya sahip diller, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağını belirtirler. Emreden paradigmanın bir örneği olan yapısal programlamada;

- Kod ne kadar büyürse, modüllere ayırma imkânı olmasına rağmen, fonksiyonlar arasındaki bağımlılık da o kadar artar. Bir fonksiyonun parametrelerindeki değişiklik, onun kullanıldığı tüm yerlerde değişiklik gerektirir. **Değişim yönetimi** (**change management**) çok zordur ve uzun zaman alır.
- Hata durumunda yapılacak işlemlere ilişkin kod ile iş sürecini gerçekleştiren kod iç içedir. Aynı fonksiyonun bazı durumlarda hata, bazı durumlarda ise değer döndürmesi mümkündür. Çoğu durumda **hataların izini sürmek** (**error handling**) işi zorlaşır.
- Yapısal programlamada, **gösterici** (**pointer**) kullanımında erişilmesi istenmeyen bellek bölgelerine erişilmesi halinde istenmeyen program davranışları ortaya çıkar. Kodun **güvenli** (**safe code**) olarak çalıştığının incelenmesi çok zordur.
- Sürekli olarak benzer projelerde **aynı kodları tekrar yazmak** (**duplicate code**) gereklidir.
- Emreden paradigmanın burada belirtilen sebepler başta olmak üzere çeşitli problemleri bulunmaktadır. Bu nedenle 1980'li yıllarda birçok yazılım projesi **başarısız** (**fail**) olmuştur. Bir yazılım; Kullanıcı ihtiyaçlarının karşılanamaması, Öngörülen bütçenin aşılması ve Zamanında teslim edilememesi durumlarında başarısız olur;

Yapısal programlamadaki bu sıkıntıları gören yazılımcıların imdadına Simula ve Smalltalk programlama dillindeki yaklaşım yetişmiştir. Bu diller oldukça yavaş olmasına rağmen getirdiği çözümler oldukça yenilikçiydi. Smalltalk, 1972 yılında Xerox Park şirketinde **Alan Kay** önderliğinde **Dan Ingalls**, **Adele Goldberg**, **Ted Kaehler**, **Diana Merry**, **Scott Wallace** ve **Peter Deutsch** tarafından geliştirilmiş bir dildir. Smalltalk dilinde nesneler (**object**) temel yapıtaşlarıdır. Nesneler;

- **Durum** (**state**) ve **davranışlara** (**behavior**) sahiptir.
- Programlama, nesnelerin birbirlerine **ileti göndermesiyle** (**message-passing**) yapılır.

⁹ Wirth, Niklaus (1976). Algorithms + Data Structures = Programs. Prentice-Hall. p. 126

- Nesnelerin kendi ya da bir başka nesnenin davranış ve durumlarını öğrenebilme yeteneği yani **yansıma** (**reflection**) özelliği vardır.

Emreden paradigma (**imperative programming**) aksine nesnelerin birbirlerine ileti göndermesi bakış açısıyla yapılan programlamaya **nesne yönelimli programlama** (**object oriented programming-OOP**) paradigması adı verilir.

Nesne Yönelimli Programlama

Yapısal programlamada **talimatlar** (**statement**) art arda koda yazılarak programlama yapılır ve programların neler yaptığı bu talimatlar izlenerek anlaşılabilir. Talimatların zincirin halkaları gibi birbirinin peşi sıra yazılarak yapılan programlamaya **emreden programlama paradigması** (**imperative programming**) adı verilir. Bu paradigmaya sahip diller, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağını belirtirler

Emreden programlama (**imperative programming**) bir örneği olan yapısal programlamada **talimatlar** (**statements**) art arda koda yazılarak programlama yapılır. 1980'li yıllara kadar yazılım ihtiyaçları yapısal programlama ile çözülmüştür. Kodlar büyüdükçe sorunlar artmış ve bir önceki başlıkta anlatılan problemler ortaya çıkmıştır. Bu yıllarda Simula ve Smalltalk yaklaşımı yazılımcıların imdadına yetişmiştir.

Bu dillerde **nesneler** (**object**) programın temel yapıtaşlarıdır. Nesneler; durum (**state**) ve davranışlara (**behavior**) sahiptir. Programlama, nesnelerin birbirlerine ileti göndermesiyle (**message-passing**) yapılır. **Emreden paradigmanın** aksine programlama, nesnelerin birbirlerine ileti göndermesi ile yapılır. Bu bakış açısıyla yapılan programlamaya **nesne yönelimli programlama** (**object oriented programming**) paradigması adı verilir.

C++ programlama dili, C diline yapılan nesne yönelimli programlama eklentileri yapılarak geliştirilmiş bir programlama dilidir. Danimarkalı bilgisayar bilim adamı olan Bjarne Stroustrup, sonradan C++ olarak bilinecek olan "C with Classes" üzerinde çalışmaya başladı ve 1985 yılında ilk sürümünü yayınladı. C++, nesne yönelimli programlama kavramlarına sahip C programlama dilinin bir uzantısıdır. C++ gibi **genel amaçlı** (**general purpose**), Java ve C# gibi nesne yönelimli birçok programlama dili daha sonra geliştirilmiş ve günümüzde popüler olmuştur.

Şimdiye kadar anlatılan bu dillerin dolaylı da olsa bilgisayar mimarisi ile ilişkisi vardır. Gelişmiş yazılımlar yapabilmek için bir miktar mimariden anlamak gereklidir. Ayrıca Java, C# ve C++ nesnelerle ilgili birçok kütüphaneye sahiptir. Bu kütüphaneleri kullanabilmek için birçok kavramı bir anda özümsemek gerektiğinden öğrenme eğrisi diktir.

Python Programlama Dili

Python dili ise, Guido van Rossum tarafından geliştirilmiş ve ilk olarak 1991'de yayınlanan, **genel amaçlı** (**general purpose**) programlama için yaygın olarak kullanılan **yüksek düzey** (**high level**) bir programlama dilidir.



Guido van Rossum, dili *Monty Python* komedi grubundan esinlenerek isimlendirmiştir.

Python, **dinamik bir tip sistemine** (**dynamic type system**) ve **otomatik bellek yönetimine** (**automatic garbage collection**) sahiptir ve **nesne yönelimli** (**object oriented**), **emreden** (**imperative**), fonksiyonel programlama ve **yordamlı** (**procedural**) programlama paradigmalarını destekler. Python programlama dili;

- Fonksiyonel ve yapısal programlama paradigmanın yanı sıra nesne yönelimli programlamayı da destekler.
- Bir betik dili olarak kullanılabilir veya büyük uygulamalar oluşturmak için bayt koduna derlenebilir.
- Yüksek seviyede dinamik veri tipleri sağlar ve denetimini destekler.
- Otomatik çöp toplamayı destekler.

- C, C++, COM, ActiveX, CORBA ve Java ile kolayca entegre edilebilir.

Geniş ve kapsamlı bir standart kütüphanesi vardır;

- Veri Biliminde, veri analizi ve görselleştirme için Numpy, Pandas ve Matplotlib gibi Python kütüphaneleri kullanılır.
- Django ve Pyramid gibi Python çerçeveleri, Web Uygulamalarının geliştirilmesini ve dağıtımını kolaylaştırır.
- Uygulama alanları arasında bilgisayarlı görme ve görüntü işleme alanları da vardır.
- Otomasyon, İş Planlama, GUI geliştirme gibi birçok görevde de tercih edilmektedir.

💡 Python şüphesiz harika bir dildir. Çünkü sağladığı işlevsellik miktarı başka hiçbir dilde mevcut değildir.

Bilgisayarın mimarisini bilmeden programlama yapmayı sağlayacak nesneler kullanılarak ve tasarlanarak programlama yapılır. Ancak Python dilindeki bellek tüketimi çok yüksek olduğundan bellek kısıtlamalarının olduğu bir yerde kötü bir seçim haline getirir.

PYTHON YORUMLAYICISI

Derleyiciler ne yapar?

Şimdiye kadar anlatılan yüksek seviyeli dillerde yazılan kodların çalıştırmak için **derleyiciler** (compiler) kullanılır. Derleme işlemi sırasında aşağıdaki adımlar izlenir;

- Koda ilişkin derleyiciye argüman olarak kod dosyası verilir.
- Derlenecek koda hata yoksa işlemci tarafından çalıştırılacak **icra edilebilir** (executable) dosya oluşur. Oluşan bu dosya içinde işlemcinin anladığı **emirler** (instruction) yani makine kodları bulunur. Bu nedenle bu dosyaya **doğal kod** (native code) adı da verilir.
- Kullanıcı tarafından verilen bir komutla icra edilebilir bu dosya işletim sistemi kontrolünde işlemci tarafından çalıştırılır. Çalıştırılan bu program ihtiyacımızı gördüğü sürece yeniden derlemeye gerek yoktur.
- Derleyiciler kaynak kodu makine koduna dönüştürülür. Makine kodu değil de bir başka dile ya da koda dönüştüren programlara **dönüştürücü** (translator) adı verilir.

Aşağıda bir C programının derleme e çalıştırma süreci verilmiştir;

1. Daha sonra metin editörü ile ilk kaynak kodumuzu oluşturmak için komut satırında aşağıdaki komut yazılır;

```
Microsoft Windows [Version 10.0.22621.4317]
(c) Microsoft Corporation. Tüm hakları saklıdır.
```

```
C:\Users\ILHANOZKAN>notepad hello.c
```

2. Eğer böyle bir dosya yoksa editör bu dosyayı oluşturmak için onay ister. Açılan metin editörü penceresine ilk C programı yazılır ve **hello.c** olarak dosyaya kaydedilir. Eğer böyle bir dosya yoksa editör bu dosyayı oluşturmak için onay ister. Açılan metin editörü penceresine C programı yazılır ve **hello.c** olarak dosyaya kaydedilir.

```
#include <stdio.h>
int main() {
    /* İlk C Programı */
    printf("Hello, World! \n"); //Konsola Hello, World! Yazılacak Talimat (Statement)
    return 0; // Programdan çıkış talimatı
}
```

3. Yazılan C programı iki satırlık bir programdır. Yazılım dilinde satır, aslında **mantıksal satır** (logical sequence) anlamındadır ve C dilinde her mantıksal satır bir **talimat** (statement) olup noktalı virgül karakteri ile biter.
4. Aşağıdaki komut ile **hello.c** olarak hazırlanan kaynak kodumuz GCC derleyicisi tarafından icra edilebilir **hello.exe** dosyası olarak derlenir. (GCC derleyicisinin bilgisayarda kurulu olduğu varsayılmıştır.)

```
C:\Users\ILHANOZKAN>gcc hello.c -o hello.exe
```

5. Derlenen programı çalıştırmak için yine aynı komut satırında **hello.exe** yazarak icra edilebilir programımız işletim sistemi tarafında işlemciye hedef gösterilerek çalıştırılır.

```
C:\Users\ILHANOZKAN>hello.exe
Hello, World!
C:\Users\ILHANOZKAN>
```

6. Burada çalıştırılabilir dosya olan **hello.exe** bir istenildiği kadar çalıştırılabildiğine dikkat edilmelidir;

```
C:\Users\ILHANOZKAN>hello.exe
Hello, World!
C:\Users\ILHANOZKAN>hello.exe
Hello, World!
C:\Users\ILHANOZKAN>hello.exe
Hello, World!
C:\Users\ILHANOZKAN>
```

7. Eğer konsola bir başka şey yazdıran bir program isteseydik birinci adımdaki `hello.c` dosyasını yeniden açtıktan sonra kod üzerinde gerekli değişiklikleri kaydedip sonraki adımları tekrar yapmalıyız.

Yorumlayıcı nedir?

Yorumlayıcı (**interpreter**), bir programlama dilinde veya **betik** (**script**) dilde yazılmış talimatları, daha önce bir makine dili programına derlenmiş olmalarına gerek kalmadan doğrudan çalıştıran bir bilgisayar programıdır.

Programlama dilleri makine koduna **derlenebilir** (**compile**). Ancak betik diller için derleme süreci yoktur. Betik diller her zaman bir ev sahibi uygulama tarafından her seferinde çalıştırılır. Örnek olarak;

- JavaScript için ev sahibi uygulama bir tarayıcıdır.
- PHP için ev sahibi uygulama web sunucusudur.
- Windows'taki toplu işlem dosyaları (***.bat**) için ev sahibi uygulama komut satırıdır.
- Linux için kabuk dosyaları (***.sh**) için ev sahibi uygulama **/bin/bash** kabuğu uygulamasıdır.

Kısaca yorumlayıcı, bir programlama dilinde yazılmış kaynak kodu **satır satır** (**logical sequence**) okuyup çalıştıran bir programdır. Yorumlayıcılarda hata oluşursa, hata satırında durur ve sonrasını çalıştırmaz.

Yorumlayıcılar derleyicilere göre genellikle daha yavaştır çünkü her seferinde kodu çözümlemesi gerekir. Bir yorumlayıcı genellikle program çalıştırmak için aşağıdaki stratejilerden birini kullanır:

1. Kaynak kodu **ayrıştırır** (**parse**) ve çalıştırılacak satırları belirleyerek sırayla icra eder.
2. Kaynak kodu daha kolay işleyecek bir biçime veya nesne koduna çevir ve ardından icra eder.
3. Python dilinin de dahil olduğu bu stratejide kaynak kod derleyici tarafından yorumlayıcının sanal makinesiyle eşleşen bir bayt kodu üretir. Yorumlayıcı sanal makinesi bu kodu doğrudan çalıştırır.

Python Yorumlayıcısı

Python programlama yapabilmek için resmî sitesinden yorumlayıcı indirilir ve kurulur¹⁰. Bu kitapta üçüncü sürüm ve sonrası için örnek verilecektir. Kurulum tamamlandıktan sonra komut satırından aşağıdaki komut ile kontrol edilebilir;

```
Microsoft Windows [Version 10.0.22621.4317]
(c) Microsoft Corporation. Tüm hakları saklıdır.

C:\Users\ILHANOZKAN>python --version
Python 3.13.4

C:\Users\ILHANOZKAN>
```

Python, **yorumlayıcı** tabanlı bir dildir. Python dosyaları (***.py**) doğrudan python yorumlayıcısı tarafından satır satır çalıştırılır. Python dosyalarını çalıştırmak için iki tür ev sahibi uygulama vardır.

Birincisi komut satırında çalışan **python** kabuk programıdır. Kabuk çalıştırılınca üçlü büyük karakteri ile beliren bir **komut istemi** (**command prompt**) ile çalıştırılacak python satırı beklenenecektir. Kabuktan çıkmak için **exit()** ya da **quit()** yazılır. Alternatif olarak, **CTRL + D** kabuk programını sonlandırır ve sizi terminalinizin komut satırına geri götürür

```
C:\Users\ILHANOZKAN>python
Python 3.13.4 (tags/v3.13.4:8a526ec, Jun 3 2025, 17:46:04) [MSC v.1943 64 bit (AMD64)] on
win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

¹⁰ <https://www.python.org/downloads/>

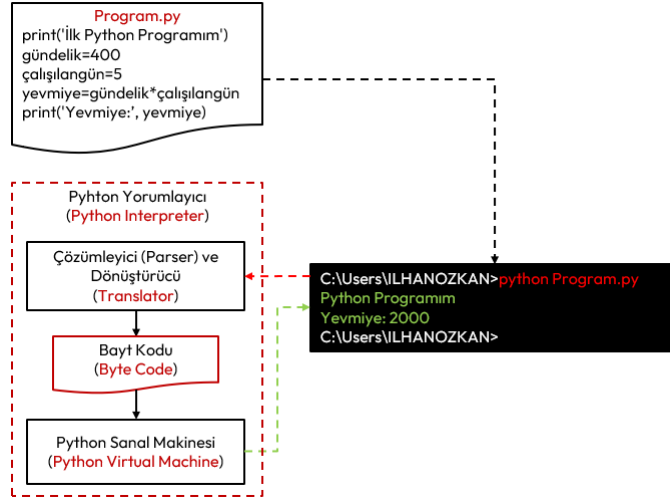
Komut isteminde Python talimatı yazıldıktan sonra çalıştırılması için **yenı satır** (new line) karakteri yani Enter (↵) tuşuna basılır.

```
C:\Users\ILHANOZKAN>python
Python 3.13.4 (tags/v3.13.4:8a526ec, Jun 3 2025, 17:46:04) [MSC v.1943 64 bit (AMD64)] on
win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print('Hello, World')
Hello, World
>>>
```

Bir dosya içine kaydedilmiş python talimatlarını çalıştırmak istiyorsak komut satırında aşağıdaki komut verilebilir.

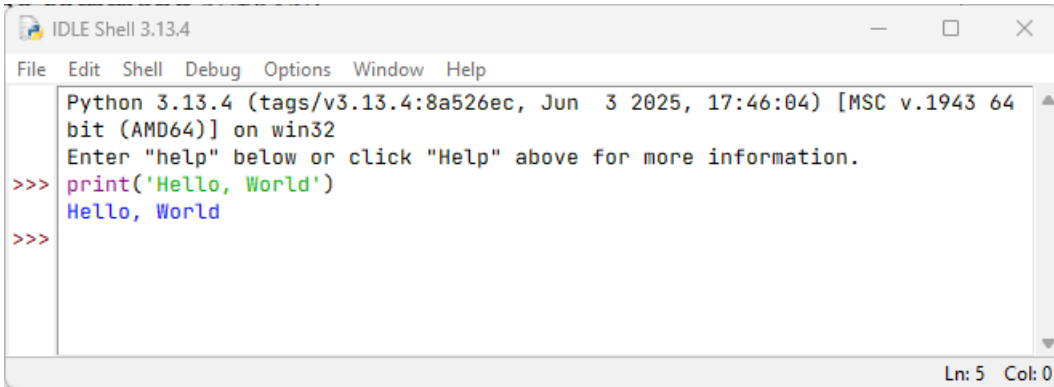
```
C:\Users\ILHANOZKAN>python hello.py
Hello, World
C:\Users\ILHANOZKAN>
```

Python yorumlayıcısı aşağıda gösterildiği şekilde çalışır. Argüman olarak verilen kod dosyası ilk önce çözümlenir ve bir çeşit derleyici olan **dönüştürücü** (translator) tarafından **bayt koda** (byte code) dönüştürülür. Oluşturulan bayt kod, sanal makine tarafından çalıştırılır.



Şekil 4. Python Yorumlayıcısının Çalışması

İkincisi ise aşağıda görseli verilen grafik işletim sistemlerinde çalışan IDLE kabuk programıdır. IDLE, Python ile birlikte gelen basit bir Python kod editörüdür. Burada da komut istemi aynı karakterlerdir.



Şekil 5. IDLE Kabuk Programı

PYTHON DİLİNE GİRİŞ

Python Dili Söz Dizim Kuralları

Python dilinde kaynak kod programcı tarafından yazılırken belli **söz dizim kullarına** (**syntax**) uyar. Bunlar aşağıda başlıklar halinde verilmiştir.

Kaynak kodumuz aşağıdaki karakterlerinden oluşabilir. Python 3.0 ve üzeri sürümlerde Unicode desteği yerleşik olarak bulunmaktadır. Python kaynak kodu için varsayılan kodlama UTF-8'dir. Ancak etik olarak aşağıdaki karakterler kullanılır;

1. Alfabadeki büyük ve küçük harfler (Unicode desteği bulunduğundan yerel karakterler de kullanılabilir):

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z

2. Rakamlar:

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

3. Özel Karakterler:

<	>	.	,	_	(	)	;	\$	:	%	[	]	#	?	'	&	{	}	"	^	!	*	/		-	\	~	+
---	---	---	---	---	---	---	---	----	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--	---	---	---	---

4. Metinde görünmeyen ancak metni biçimlendiren **beyaz boşluk** (**white space**) karakterleri;
 - a. Boşluk (**space**)
 - b. ↵ Yeni satır (**new line**)
 - c. → Sekme (**tab**)

Talimatlar ve Anahtar Kelimeler

Yüksek seviyeli bir dilde yazılmış bir **talimat** (**statement**), işlemciye belirtilen bir eylemi gerçekleştirmesini söyleyen cümledir. Yüksek seviyeli bir dildeki tek bir talimat, birkaç makine dili **emir kodunu** (**instruction code**) temsil edebilir. Talimatlar, yapılaması gerekeni kısa ve net olarak belirten **mantıksal satırlardır** (**logical sequence**).

Anahtar **kelimeler** (**keyword**), programlamada kullanılan ve Python yorumlayıcısı tarafından özel anlama sahip, önceden tanımlanmış, **ayrılmış kelimeler** (**reserved word**) olup programı oluşturan **talimatlar** (**statement**) bu kelimeler kullanılarak yazılırlar.

```
>>> help()
Welcome to Python 3.13's help utility! If this is your first time using
Python, you should definitely check out the tutorial at
https://docs.python.org/3.13/tutorial/.

help> keywords

Here is a list of the Python keywords. Enter any keyword to get more help.

False      class      from      or
None       continue  global    pass
True       def       if       raise
and        del      import    return
as         elif     in       try
assert     else     is       while
async      except   lambda   with
await      finally nonlocal yield
break      for      not
```

Python dilinde;

- **Anahtar kelimeler**, küçük-büyük harf ayrımı yapıldığından her zaman **küçük harflerle yazılırlar**.
- **Talimatlar** (**statement**), **anahtar kelimeler içerecek şekilde yazılır**.

Normal olarak her satırda bir talimat yazılır. Ancak bazı durumlarda bir satırda birden fazla talimat yazılmak istenirse noktalı virgül (;) karakteri ile talimatlar ayrılır.

```
>>> toplam = 0.0
>>> sayaç = 1; adım = 10;
>>>
```

Açıklamalar

Özellikle dosyalarda saklanan **talimatları** (**statement**) yazarken bir açıklama yapmak istenildiğinde karma (#) karakteri kullanılır. Bu karakterden sonra satır sonuna kadar açıklama kabul edilir.

```
>>> indis=0 # bu değişken indis olarak kullanılacak
>>> toplam=0.0 # bu değişken işlemler toplamını tutacak
```

Birden fazla satırda açıklama üç adet tek tırnak veya çift tırnak karakterini art arda yazarak açıklamaya başlayabilir ve yine aynı şekilde açıklamayı bitirebiliriz.

```
>>> '''
...     İlhan Özkan
... Haziran 2025
... Ankara
... '''
'\nİlhan Özkan\nHaziran 2025\nAnkara\n' #Yukarıdakinin aynısıdır.
>>>
```

Değişkenler

Değişken Tanımlama

C/C++/C# ve Java gibi dillerin aksine Python dilinde tüm **değişkenler** (**variable**) aslında var olan bir **nesneye** (**object**) tanımlanan bir referanstır. Atama işlemi ile nesneye dönüştürülen **değişmezler** (**literal**) ile değişken tanımlanabilir. **Kimliklendirmede küçük büyük harf ayrımı yapılır. Ayrıca bir değişmeze atama yapamazsınız!**

```
Değişken-kimliği = değişmez
#aslında
Değişken-adı = herhangi-bir-nesne
```

Değişkenin hangi **veri tipinde** (**data type**) veri tutacağı dinamik olarak çalıştırma anında belirlenir. Burada belirlenen kimlikler aslında var olan nesnelere ilişkin referanslardır. Buradan anlaşılacağı üzere tüm değişmezler de nesne dönüştürülür ve sonrasında değişkene referansı verilir. Aşağıda örnekleri verilmiştir;

```
>>> tamsayı=1 #1 değişmezi bir int nesnesine dönüştürülür ve değişkene referans verilir
>>> print(tamsayı)
1
>>> büyüsayı=987654321987654321 #değişmez nesneye dönüştürülür ve değişkene referans verilir
>>> print(büyüsayı)
987654321987654321
>>> ondalık=3.14 #değişmez float nesnesine dönüştürülür ve değişkene referans verilir
>>> print(ondalık)
3.14
>>> avagadro=6.02214076e23
>>> print(avagadro)
6.02214076e+23
>>> karakter='A' #değişmez bir str nesnesine dönüştürülür ve değişkene referans verilir
>>> print(karakter)
A
>>> metin='İlhan Özkan'
>>> print (metin)
```

```

İlhan Özkan
>>> dogrumudegilmi=True
>>> print(dogrumudegilmi)
True
>>> tipibellidegil=None
>>> print(tipibellidegil)
None
>>> print(type(tamsayı))
<class 'int'>
>>> print(type(büyüksayı))
<class 'int'>
>>> print(type(ondalık))
<class 'float'>
>>> print(type(avagadro))
<class 'float'>
>>> print(type(karakter))
<class 'str'>
>>> print(type(metin))
<class 'str'>
>>> print(type(dogrumudegilmi))
<class 'bool'>
>>> print(type(tipibellidegil))
<class 'NoneType'>
>>>

```

Verilen örnekteki ilk satırda `tamsayı=1` talimatında `1` **değişmezi** (**literal**) için bellekte bir nesne oluşturulur ve tamsayı kimliği ile bu nesneye erişilebilir. Benzer şekilde `3.14`, `6.02214076e23`, `'A'` v e `True` değişmezleri için aynı durum geçerlidir.

Çoklu Değişken Tanımlama

Birden fazla değişkeni aralarında virgül kullanarak tek bir satırda tanımlayabilirsiniz;

```

>>> num1,num2,num3=10,20,30
>>> print(num1,num2,num3)
10 20 30
>>>

```

Birden fazla değişkeni aynı nesne ile tanımlayabilirsiniz;

```

>>> num1=num2=num3=100
>>> print(num1,num2,num3)
100 100 100
>>>

```

Değişken İsimlendirme Kuralları

Değişkenlere isim tanımlarken (kimlik verilirken) aşağıdaki **kurallara** (**naming convention**) uyulur;

1. Kimliklendirmede anahtar kelimeler kullanılamaz. Bu durumda `SyntaxError` hatası alırsınız.
2. Kimlikler küçük harf (a-z), büyük harf (A-Z), rakamlar (0-9) ve alt çizgi (_) içerebilir. UTF karakterleri de desteklendiğinden yerel alfabedeki karakterler de kullanılabilir.
3. Kimlikler bir rakamla başlayamaz. Örneğin, `1no` geçersiz bir değişken kimliğidir.
4. Kimlikler yalnızca rakamlardan oluşamaz. Örneğin, `888` geçersiz bir değişken kimliğidir.
5. Kimlikler altçizgi karakteri ile başlayabilir. Örneğin `_test` geçerli bir değişken kimliğidir.
6. Kimliklendirmede karakter sınırlaması yoktur. İstedığınız uzunlukta kimlik tanımlayabilirsiniz. Ancak, çok uzun bir değişken kimliği programcıyı yorar ve güvenilirliğinizi zedeler.
7. Kimliklendirmede **büyük küçük harf ayrımı yapılır**. Bu nedenle, `abc` ve `ABC` iki farklı değişken kimliğidir.

Kimliklendirmede, kodun okunaklılığı fazla olması için, etik olarak aşağıdaki üç desenden biri kullanılır;

- **Camel case:** İlk harf küçük harftir, ancak sonraki her kelimenin ilk harfi büyük harftir. Örneğin: öğrenciYaşı, birimFiyat
- **Pascal case:** Her kelimenin ilk harfi büyük harftir. ÖğrenciYaşı, BirimFiyat
- **Snake case:** Kelimeleri ayırmak için tek alt çizgi karakteri kullanılır. Örneğin öğrenci_yaşı, birim_fiyat

Değişkenlerin Bellek Adresleri

Bir değişkenin bellek adresi `id()` ile öğrenilebilir. Aşağıdaki örnekten de anlaşılacağı üzere değeri tamsayı bir olan değişmezden bir nesne imal edilip a değişkeni tanımlanmıştır. Aydında b değişkeni de a değişkeninin referans gösterdiği nesneyi gösterecek şekilde tanımlanmıştır. Her iki değişkenin de gösterdiği bellek adresi aynı olduğu gözden kaçırılmamalıdır.

```
>>> a=1
>>> b=a
>>> print(id(a),id(b))
140712917549992 140712917549992
>>>
```

C/C++/C# ve Java gibi dilleri **statik tip sistemine** (**static type system**) sahip olduklarından bu yapılamaz. Yani bir değişken kimliklendirildikten sonra veri tipi değiştirilemez. Çünkü veri tipine bağlı kimliklendirme yapılır. Aksine Python dilinde çalışma anında değişkenin işlediği veri tipi değiştirilebilir. Bu **dinamik veri tip sistemi** (**dynamic type system**) programcıya kolaylık sağlar;

```
>>> numara = 100
>>> print (numara)
100
>>> print(type(numara))
<class 'int'>
>>> numara = '0n'
>>> print (numara)
0n
>>> print(type(numara))
<class 'str'>
>>>
```

Mevcut Değişkenleri Görme

Yorumlayıcıda tanımlanan değişkenler tanımlandığı satır icra edildikten sonra hemen kaybolmazlar. Python **komut isteminde** (**command prompt**) aşağıdaki komut çalıştırılarak halihazırda tanımlı değişkenleri `dir()` komutuyla görebilirsiniz.

```
>>> a=10
>>> b=20
>>> print('toplam',a+b)
toplam 30
>>> print('çarpım',a*b)
çarpım 200
>>> dir()
['__annotations__', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
'__name__', '__package__', '__spec__', 'a', 'b']
>>>
```

Tanımlı değişkenler listelenirken alt çizgi karakterleri arasında yer alan **ön tanımlı** (**default**) değişkenler de listelenir. Bu komutun dışında `locals()` ile yerel değişkenleri, `globals()` ile evrensel değişkenleri listeleyebilir ve değerlerini görebilirsiniz. İleride detayı verilecektir.

```
>>> locals()
{'__name__': '__main__', '__doc__': None, '__package__': '_pyrepl', '__loader__': None,
'__spec__': None, '__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>,
'__file__':
```

```
'C:\\Users\\ILHANOZKAN\\AppData\\Local\\Programs\\Python\\Python313\\Lib\\_pyrepl\\__main__.py', '__cached__':
'C:\\Users\\ILHANOZKAN\\AppData\\Local\\Programs\\Python\\Python313\\Lib\\_pyrepl\\__pycache__\\__main__.cpython-313.pyc', 'a': 10, 'b': 20}
>>> globals()
{'__name__': '__main__', '__doc__': None, '__package__': '_pyrepl', '__loader__': None,
'__spec__': None, '__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>,
'__file__':
'C:\\Users\\ILHANOZKAN\\AppData\\Local\\Programs\\Python\\Python313\\Lib\\_pyrepl\\__main__.py',
'__cached__':
'C:\\Users\\ILHANOZKAN\\AppData\\Local\\Programs\\Python\\Python313\\Lib\\_pyrepl\\__pycache__\\__main__.cpython-313.pyc', 'a': 10, 'b': 20}
>>>
```

Değişkenleri Silme

Halihazırda tanımlı değişkenleri silmek için `del` kullanılır.

```
>>> toplam=12
>>> print(toplam)
12
>>> del toplam
>>> print(toplam)
Traceback (most recent call last):
  File "<pyshell#28>", line 1, in <module>
    print(toplam)
NameError: name 'toplam' is not defined
>>>
```

Değişken Tiplerini Test Etme

Değişkenin veri tipi ön tanımlı `type()` fonksiyonu ile öğrenilir.

```
>>> a = 12
>>> print(type(a))
<class 'int'>
>>> b = '12'
>>> print(type(b))
<class 'str'>
>>> c = (1, 3.14)
>>> print(type(c))
<class 'tuple'>
>>>
```

Koşullu ifadelerde veri tipini `isinstance()` ile test etmek mümkündür. İleride anlatılacaktır.

Veri Tipleri

Dahili Veri Tipleri

bool

Doğru anlamında `True` veya yanlış anlamında `False` değer alabilen veri tipidir. `True` tamsayı olarak 1, `False` ise 0 sayısına karşılık gelir

```
>>> devammi=True
>>> print('Devam mı', devammi)
Devam mı True
>>> print(True+True)
2
>>> print(True*True)
1
>>>
```

int

Tamsayı değer alabilen veri tipidir. Python son sürümündeki bu veri tipi C dilindeki `Long` veri tipine karşılık gelir.

```
>>> a = 1
>>> b = 300
>>> c = -123456789
>>> d = 398765432109876543
>>> print(a,b,c,d)
1 300 -123456789 398765432109876543
>>>
```

Değişmezler yukarıdaki gibi onluk sistemdeki tamsayılar olabileceği gibi **ikili** (**binary**), **sekizli** (**octal**) veya **on altılı** (**hexadecimal**) sayı tabanında olabilir. İkili sayılara ilişkin sabit örneği aşağıda verilmiştir.

```
>>> a=0b0101 # ikili sayı sabitinin başında 0b karakterleri bulunur.
>>> print(a," type a", type(a))
5 type a <class 'int'>
>>> b=int("0b1000",2)
>>> print(b," type b", type(b))
8 type b <class 'int'>
>>> c=bin(b)
>>> print(c,"type c:",type(c))
0b1000 type c: <class 'str'>
>>>
```

Sekizli sayı sisteminde bir sayıya ait değişmezin önünde sıfır ile birlikte küçük ya da büyük o harfi bulunur. Bir sayı ya da metinden sekizlik sistemde bir dizgi elde etmek istersek `oct()` fonksiyonunu kullanırız.

```
>>> a=0o10
>>> print(a,"type a",type(a))
8 type a <class 'int'>
>>> b=0020
>>> print(b,"type b",type(b))
16 type b <class 'int'>
>>> c=int('10',8)
>>> print(c,"type c",type(c))
8 type c <class 'int'>
>>> d=int('0o21',8)
>>> print(d,"type d",type(d))
17 type d <class 'int'>
>>> e=oct(16)
>>> print(e, "type e", type(e))
0o20 type e <class 'str'>
>>>
```

On altılık sistemde bir değişmezin önünde ise sıfır ile birlikte x harfi bulunur. Bir sayı ya da metinden on altılık sistemde bir dizgi elde etmek istersek `hex()` fonksiyonunu kullanırız.

```
>>> a=0x10
>>> print(a,'type a', type(a))
16 type a <class 'int'>
>>> b=0X20
>>> print(b,'type b', type(b))
32 type b <class 'int'>
>>> c=int('10',16)
>>> print(c,"type c",type(c))
16 type c <class 'int'>
>>> d=int('0x21',16)
>>> print(d,"type d",type(d))
```

```
33 type d <class 'int'>
>>> e=hex(10)
>>> print(e, "type e", type(e))
0xa type e <class 'str'>
>>>
```

float

Kayan noktalı sayıları tutan veri tipidir. Python son sürümündeki bu veri tipi C dilindeki `double` veri tipine karşılık gelir.

```
>>> a = 3.14
>>> b = 7.5e0
>>> c = 1e-1
>>> d = 0.5e8
>>> print(a,b,c,d)
3.14 7.5 0.1 50000000.0
>>>
```

Herhangi bir sayı ya da metni kayan noktalı sayıya çevirmek için `float()` fonksiyonu kullanılır. Matematikte sonsuzluk soyut bir kavramdır. Fiziksel olarak, sonsuz büyüklükteki sayılar hiçbir zaman herhangi bir miktardaki bellekte saklanamaz. Ancak bilgisayar donanım yapılandırmalarının çoğu için 10'un 400. kuvveti olan çok büyük bir sayı `Inf` ile gösterilir. `float()` fonksiyonu için argüman olarak "Infinity" kullanırsanız, `Inf` döndürür. Tanımlanmamış veya temsil edilemeyen herhangi bir değeri `Nan` (Not a Number) temsil eder.

```
>>> a=3.14E0
>>> print(a,type(a))
3.14 <class 'float'>
>>> b=float('Infinity')
>>> print(b,type(b))
inf <class 'float'>
>>> c=float('Nan')
>>> print(c,type(c))
nan <class 'float'>
>>>
```

complex

Karmaşık sayıları tutan veri tipidir. Sanal kısım `j` harfi ile biter.

```
>>> a = 3 + 5j
>>> b = -1.5 + 4.2j
>>> print (a,b)
(3+5j) (-1.5+4.2j)
```

str

Unicode karakterleri içeren metinleri veri olarak tutan `dizgilerdir` (`string`). `bytes` ise metin bayt dizisi karakterlerini içeren dizidir.

```
>>> metin='Uşagum Merhaba'
>>> batylar= u'Uşagum'
>>> print(metin)
Uşagum Merhaba
>>> dizgi = 'Hello World'
>>> print(dizgi[0]) #İlk karakteri konsola yazar. H
>>> print(dizgi[0:5]) #İlk 5 Karakteri konsola yazar. Hello
```

Python, dizgi `değişmezlerini` (`literal`) tanımlamak için tek tırnak (`'`), çift tırnak (`"`) ve üçlü tek veya çift tırnak (`"""`) işaretlerini, dizinin başında ve sonunda aynı tür tırnak işareti kullanıldığını sürece kabul eder.

```
>>> harf='a'
>>> kelime="sözcük"
>>> cümle="Türkçe güzel bir dildir."
```



```
>>> print(cümle)
Türkçe güzel bir dildir.
>>> paragraf="""Python dili çok esnek bir dildir.
Şöyleki C ve C++ dillerindeki dizki karakterleri gibi tanımlama yapabilirsiniz."""
>>> print(paragraf)
Python dili çok esnek bir dildir.
Şöyleki C ve C++ dillerindeki dizki karakterleri gibi tanımlama yapabilirsiniz.
```

`str` veri tipi Unicode karakterleri içerebilir, dolayısıyla tek, çift veya üçlü tırnaklı dize sözdizimi kullanılarak oluşturulan herhangi bir `dizgi (string)` Unicode olarak saklanır.

```
>>> string='ÜĞİŞÇÖİÜğışçöı'
>>> tb=string.encode('utf-8')
>>> print(tb)
b'\xc3\x9c\xc4\x9e\xc4\xb0\xc5\x9e\xc3\x87\xc3\x96\xc3\xbc\xc4\x9f\xc5\x9f\xc3\xa7\xc3\xb6\xc4\xb1'
>>> str=tb.decode('utf-8')
>>> print(str)
ÜĞİŞÇÖİÜğışçöı
```

Diziler ve Koleksiyonlar

Dizi ve koleksiyonlar `yinelenebilir` (`iterable`). Yani elemanları üzerinde `for` döngüsü ile gezinilebilir. Burada kısaca değinilen veri yapıları bir bölüm olarak detaylı incelenecektir.

str ve bytes

Bu veri tipleri karakterlerden oluşan `dizilerdir (array)`. *Dizi* başlığında anlatılacaktır.

```
>>> kelime="sözcük"
>>> print(kelime[0])
s
>>> print(kelime[1])
ö
```

tuple

`Demet (tuple)`, herhangi bir veri tipinde n adet değer bir araya gelerek oluşturduğu sıralı koleksiyondur. Elemanlar yuvarlak parantez içerisinde yazılarak ilk değer verilir.

```
>>> a = (1, 2, 3) # tamsayılardan oluşan demet
>>> print(a)
(1, 2, 3)
>>> b = (1, 'İlhan', 3.14, 'Özkan') # karışık veri tiplerinden oluşan demet
>>> print(b)
(1, 'İlhan', 3.14, 'Özkan')
>>> c = (1, 'ilhan', ('Özkan', 3.14)) # içinde bir başka demet olan demet
>>> print(c)
(1, 'ilhan', ('Özkan', 3.14))
>>> print(c[0])
1
>>> print(c[1])
ilhan
>>> print(c[2])
('Özkan', 3.14)
>>> d=(a,b,c) # başka demet elemanlar ile yeni demet tanımlama
>>> print(d)
((1, 2, 3), (1, 'İlhan', 3.14, 'Özkan'), (1, 'ilhan', ('Özkan', 3.14)))
>>> e=(100,) #tek elemanlı bir demet:
... #öncelik için kullanılan parantez ile karışmaması için eleman sonrasına bir virgül konulur
>>> print(e)
(100,)
```

İndisler (index) sıfırdan başlar ve her bir elemana köşeli parantez ([]) içerisinde indis kullanılarak erişilebilir. **Demet** (tuple) koleksiyonunda elemanlar **değiştirilemez** (immutable).

list

Listeler (list) herhangi bir veri tipindeki değerlerin sıralı bir koleksiyonudur. Elemanları **değiştirilebilir** (mutable). Elemanları köşeli parantez içerisinde yazılarak ilk değer verilir.

```
>>> a = [1, 2, 3]
>>> print(a)
[1, 2, 3]
>>> print(a[0]) #birinci eleman
1
>>> print(a[1]) #ikinci eleman
2
>>> print(a[2]) #Üçüncü eleman
3
>>> print(a[3]) #dördüncü eleman!!
Traceback (most recent call last):
  File "<pysHELL#5>", line 1, in <module>
    print(a[3])
IndexError: list index out of range
>>> print(a[-1]) #sondan birinci eleman
3
>>> print(a[-2]) #sondan ikinci eleman
2
>>> print(a[-3]) #sondan üçüncü eleman
1
>>> b = [1, 'ilhan', (3,5.0), ['özkan',12345]]
>>> print(b)
[1, 'ilhan', (3, 5.0), ['özkan', 12345]]
>>> b[0]=-1 # birinci eleman değiştiriliyor
>>> b[1]='özkan' # ikinci eleman değiştiriliyor
>>> b[2]= 3.14 # üçüncü eleman değiştiriliyor
>>> print(b)
[-1, 'özkan', 3.14, ['özkan', 12345]]
>>> c=[] #boş liste
>>> c.append(1) #listenin sonuna 1 sayısı eklendi.
>>> c.insert(0,100) #listenin sıfırıncı indisine 100 sayısı eklendi.
>>> print(c)
[100, 1]
>>> c.index(100) #100 sayısı listede hangi indiste?
0
>>> len(c) #c listesinin kaç elemanı var? Uzunluğu nedir?
2
>>> c.reverse() #c listesinin elemanlarını sondan başa yer değiştir. Tersini Al
>>> c
[100, 1]
>>> c[::-1] #c.reverse() ile aynı işi görür
[1, 100]
>>> c.pop() #varsayılan olarak son elemanı listeden çıkarır. Yada belirtilen indistekini.
1
>>> c
[100]
```

Dilim (slice) kavramı dizi, demet ve listenin elemanlarının bir veya birkaçını ifade etmek için kullanılır. Dilim, [başlangıç:bitiş:adım] şeklinde ifade edilir. Bu dilimin çıktısı, başlangıç indisinden **bitiş-1** indisine olan öğeleri içeren yeni bir listedir. **adım**, başlangıç ve **bitiş-1** arasında ile belirtilen aralıklar ile öğe seçimini ifade eder.

```
>>> liste = [1, 2, 3, 4]
>>> liste[1:]
```

```
[2, 3, 4]
>>> liste[:3]
[1, 2, 3]
>>> liste[::2]
[1, 3]
>>> liste[::-1]
[4, 3, 2, 1]
>>> liste[-1:0:-1]
[4, 3, 2]
>>> liste[4:9]
[]
>>> liste[1:10]
[2, 3, 4]
```

set

Küme (set), herhangi bir veri tipinde ve **tekil** (unique) elemanlardan oluşan sırasız bir koleksiyondur.

Bir nesne, ömrü boyunca asla değişmeyen bir **karma** (hash) değerine sahipse (bir `__hash__()` yöntemine ihtiyaç duyar) ve diğer nesnelerle karşılaştırılabilirse (bir `__eq__()` yöntemine ihtiyaç duyar) **karmalanabilir** (hashable). Eşitliği karşılaştıran karmalanabilir nesnelerin, aynı karma değerine sahip olması gerekir. Karma, bir nesneye eşsiz bir sayı olarak karşılık gelen değerlerdir.

```
>>> a = { 1, 2, 'a' }
>>> b = { 1, 1, 2, 2 } # bir değer bir kümede yalnızca 1 kez yer alabilir.
>>> print(b)
{1, 2}
```

dict

Sözlük (dict), **anahtar-değer çiftlerini** (key-value pair) tekil olarak tutan bir koleksiyondur. Anahtarlar **karmalanabilir** (hashable).

```
>>> a={ 1: 'Elma', 2: 'Armut', 3: 'Muz' }
>>> print(a)
{1: 'Elma', 2: 'Armut', 3: 'Muz'}
>>> b= { 'a': [1,2,3], 'b': (1,1), 'c': 'Özkan' }
>>> print(b)
{'a': [1, 2, 3], 'b': (1, 1), 'c': 'Özkan'}
```

Numaralandırma

Numaralandırma (enumeration), benzersiz, sabit bir değere bağlı bir dizi sembolik adlara sahip üyelerden oluşan değerlerdir. Bir numaralandırma içinde, üyeler kimliğe göre karşılaştırılabilir ve Numaralandırmanın kendisi **yenilenebilir** (iterable).

💡 Şimdiye kadar verilen örnekler Python **yorumlayıcısı** (interpreter) ile gerçekleştirilebilir. Bundan sonra verilecek örnekler bir Python dosyasına yazılmış gibi anlaşılacak **python dosya.py** şeklinde çalıştırılmalıdır.

Python 3.4 ve üzeri sürümlerde numaralandırma (enum) modülü standart kütüphanede bulunur, ayrı kurulum gerekmez.

```
#oynek.py
from enum import Enum
class Renkler(Enum):
    kırmızı = 1
    yeşil = 2
    mavi = 3
    sarı = 4
print(Renkler.kırmızı)
print(Renkler(1))
```

```
print(Renkler['kırmızı'])
print()
for renk in Renkler: #Numaralandırmalar yinelenabilir.
    print(renk)
```

Yukarıdaki örnek dosya `python ornek.py` şeklinde çalıştırılırsa aşağıdaki çıktı elde edilir.

```
Renkler.kırmızı
Renkler.kırmızı
Renkler.kırmızı

Renkler.kırmızı
Renkler.yeşil
Renkler.mavi
Renkler.sarı
```

Dahili Sabitler

Program içerisinde değerleri sabit olan nesneleri kullanırız. Bunlar;

- `True`: Dahili `bool` tipinin doğru değeri olan nesne.
- `False`: Dahili `bool` tipinin yanlış değeri olan nesne.
- `None`: Bir değer olmadığını belirtmek için kullanılan **tekil** (**unique**) bir nesnedir.
- `Ellipsis` veya `...`: İleride anlatılacaktır.
- `NotImplemented`: Bir yöntemin belirli argümanları desteklemediğini belirtmek için kullanılan tekil bir nesnedir.

ARDIŞIK İŞLEMLER

Ardışık İşlem Nedir?

Yapısal programlamada ana fonksiyondaki ilk talimatla program başlar. Python dilinde ise programın icrası kodun başındaki ilk talimattan başlar ve sırasıyla program bitene kadar sırayla devam eder. İşte programın icrasında talimat sırasını değiştirmeyen bu tür işlemlere **ardışık işlem** (**sequential operation**) adı verilir.

Ardışık işlemler aşağıdaki üç tür **talimattan** (**statement**) oluşur.

1. Değişkenlerin kimliklendirildiği **değişken tanımlamaları** (**identifier definition**):

```
tamsayı = 1
metin = "İlhan"
```

2. **İfadelerden** (**expression**) yani sabit, değişken ve **işleç** (**operator**) içeren sözdizimleri:

```
daireninAlanı = PI*yarıCap*yarıCap;
daireninÇevresi = 2.0*PI*yarıCap;
```

3. Klavyeden veri okuma ve konsola veri yazma gibi **giriş çıkış işlemleri** (**input output operation**):

```
print(daireninAlanı);
dosyaAdı = input()
```

Konsoldan Girdi Okuma ve Konsola Veri Yazma

Her bilgisayar uygulamasının çalışırken kullanıcıdan girdi kabul etme olanağı olmalıdır. Bu, uygulamayı etkileşimli hale getirir. Nasıl geliştirildiğine bağlı olarak, bir uygulama kullanıcı girdisini konsola (**sys.stdin**) girilen metin, grafiksel bir düzen veya web tabanlı bir ara yüz biçiminde kabul edebilir. Bunu sağlayan **input()** fonksiyonudur. Bu fonksiyonunda okunan değerın aktarılacağı bir **dizgi** (**string**) değişken bulunmalıdır;

```
adı = input()
soyadı = input("Soyadınızı Giriniz:")
telefon = input("Telefon Numaranızı Giriniz:")
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
İlhan
Soyadınızı Giriniz:Özkan
Telefon Numaranızı Giriniz:3121234567
```

Yukarıdaki örnekte okunan telefon numarası metin bir değişkene atanmıştır. Sayıya çevirmek gerekir.

```
yarıcapstr = input('Yarıçapı Giriniz:')
yarıcap = float(yarıcapstr)

# yarıcap = float(input('Yarıçapı Giriniz:')) # şeklinde yazılabilirdi. Bu daha "Pythonic"

alan = 3.14*yarıcap*yarıcap
print('Dairenin Alanı:',alan)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Yarıçapı Giriniz:2.5
Dairenin Alanı: 19.625
```

Konsola biçimlendirilmiş merin **print()** fonksiyonu ile yazılır. Bu fonksiyon her çağrıldığında konsola yazar ve alt satıra iner.

```
boy=180
kilo=90.5
print(boy)
# Çıktı: 180
```

```
print(boy,kilo)
# Çıktı: 180 90.5
```

Eğer birden fazla değişken var ve aralarındaki **ayraç** (**separator**) değiştirilmek isteniyorsa isimlendirilmiş **sep** parametresi değiştirilerek bu yapılabilir. Bunu isimlendirilmiş **end** parametresini değiştirerek yapabiliriz.

```
boy=180
kilo=90.5
print(boy,kilo,sep=' - ')
# Çıktı: 180 - 90.5
print(boy,'-', end='')
# Çıktı: 180- #bunu yazar ve alt satıra geçmez!
print(kilo)
# Çıktı: 180-90.5 #önce yazılan yerden devam eder!
```

Tip Dönüşümleri

Yukarıda verilen örnekte **dizgi** (**string**) metninden kayan noktalı sayıya bir dönüşüm yapılmıştır. Benzer şekilde birçok dönüşüme ihtiyaç duyarız.

Herhangi bir dil derleyicisi/yorumlayıcısı bir veri tipindeki nesneyi otomatik olarak başka bir veri tipine dönüştürdüğünde buna otomatik veya **örtük dönüşüm** (**implicit casting**) denir. Python **güçlü veri tiplerine** (**strongly typed**) sahip bir dildir. İlgisiz veri tipleri arasında otomatik tip dönüşümüne izin vermez. Örneğin, bir **dizgi** (**string**) herhangi bir sayı tipine dönüştürülemez. Ancak, bir tam sayı (**int**), kayan noktalı bir sayıya (**float**) dönüştürülebilir.

Bir tamsayı, kayan noktalı sayıdan daha az yer kaplar. Bir tamsayı kayan noktalı sayıya örtük olarak dönüştürülebilir. Ancak bir kayan noktalı sayı, kesirli kısmı kaybolacağından, doğrudan tamsayıya dönüştürülemez.

```
tamsayı=10
kayannoktalısayı=15.5
sonuç=tamsayı+kayannoktalısayı
print(sonuç)
# Çıktı: 25.5
print(type(sonuç))
# Çıktı: <class 'float'>
bool=True # True sabitinin değeri 1, False sabitinin değeri 0'dır
sonuç=sonuç+bool
print(sonuç)
# Çıktı: 26.5
```

Otomatik veya örtük dönüşüm yalnızca **int** veri tipinden **float** veri tipine dönüşümle sınırlı olsa da **int()**, **float()** ve **str()** fonksiyonlarını kullanarak **str** veri tipinden **int** veri tipine dönüşüm gibi **açık dönüşümleri** (**explicit casting**) gerçekleştirebilirsiniz.

```
tamsayımetni='321'
tamsayı=int(tamsayımetni)
print(tamsayı)
# Çıktı: 321
kayannoktalısayımetni='7.5'
kayannoktalısayı=float(kayannoktalısayımetni)
print(kayannoktalısayı)
# Çıktı: 7.5
tamsayı=int(kayannoktalısayı)
print(tamsayı)
# Çıktı: 7
str=str(kayannokyalısayı)
print('Str:', str, 'type:', type(str))
# Çıktı: Str: 7.5 type: <class 'str'>
```

`int()` fonksiyonu ayrıca ikili, sekizli ve on altılı dizgilerden tam sayı döndürür. Bunun için fonksiyonun sırasıyla 2, 8 veya 16 olması gereken bir taban parametresine ihtiyacı vardır. Dizgi, verilen tabanda geçerli bir metni içermelidir.

```
ikilitabandan=int("1010",2)
print(ikilitabandan)
# Çıktı: 10
sekizlitabandan=int("12",8)
print(sekizlitabandan)
# Çıktı: 10
onaltiliktabandan=int("A",16)
print(onaltiliktabandan)
# Çıktı: 10
```

İşleçler

İşleçler (**operator**), bir veya daha fazla **işlenen** (**operand**) üzerinde belirli işlemleri gerçekleştirmek için kullanılan özel sembollerdir. Aslında işleçler matematikten bildiğimiz işleçlerdir.

$$y = 3 + 2$$

Yukarıda verilen cebirsel ifadede toplama işleci (+) iki tarafına bulunan argümanları toplar. Bu argümanlara **işlenen** (**operand**) adı verilir. Yüksek düzey dillerde de aritmetik işleçler ve bunun yanında birçok işleç de bulunur. En çok kullanılan işleç, atama (=) işlecidir. Atama işleci, sağdaki işleneni soldakine atar. Bu nedenle kodlama yapılırken `2+2=x` veya `a+b=x` gibi sağa eşitleme yazılamaz!

İşlenenler, değişken, değer veya **ifade** (**expression**) olabilir. İfadelerin değişmez, değişken ve işleç içeren sözdizimleri olduğu daha önce belirtilmişti.

Aritmetik İşleçler

Matematik ifadelerini gerçekleştirmek için kullanılır.

İşleç	Adı
+	Toplama: Sağdaki ve soldaki işlenenin toplamını verir. Eğer sadece işlenenin solunda kullanılıyorsa işleneni pozitifçe çevirir.
-	Çıkarma: Soldaki işlenenden ve sağdaki işlenenden farkını verir. Eğer sadece işlenenin solunda kullanılıyorsa işleneni negatifçe çevirir.
*	Çarpma: Sağdaki ve soldaki işlenenin çarpımını verir.
/	Bölme: Soldaki işlenenin ve sağdaki işlenene bölümünü verir.
%	Kalan (modulus) : Soldaki işlenenin sağdakine bölümünden kalanı verir.
**	Üs (exponent) : Soldaki işlenenin sağdaki kadar kuvvetini verir.
//	Kat bölümü (floor division) : Soldaki işlenenin sağdaki işlenenin kaç katı olduğunu verir.

Tablo 6. Aritmetik İşleçler.

Aşağıda bu işleçler için örnek verilmiştir;

```
a=10
b=20
c = a + b
print ("a: {} b: {} a+b: {}".format(a,b,c))
# Çıktı: a: 10 b: 20 a+b: 30
c = a - b
print ("a: {} b: {} a-b: {}".format(a,b,c))
# Çıktı: a: 10 b: 20 a-b: -10
c = a * b
print ("a: {} b: {} a*b: {}".format(a,b,c))
# Çıktı: a: 10 b: 20 a*b: 200
c = b / a
print ("a: {} b: {} b/a: {}".format(a,b,c))
# Çıktı: a: 10 b: 20 b/a: 2.0
```

```

a=2
b=3
c= a**b
print ("a: {} b: {} a**b: {}".format(a,b,c))
# Çıktı: a: 2 b: 3 a**b: 8
a=30
b=6
c= a//b
print ("a: {} b: {} a//b: {}".format(a,b,c))
# Çıktı: a: 30 b: 6 a//b: 5
a = 1
sonuç = -a
print ("a: {} -a: {}".format(a,sonuç))
# Çıktı: a: 1 -a: -1

```

Karşılaştırma İşleçleri

Karşılaştırma işleçleri (**comparision operator**) ilişkisel işleçler (**relational operator**) olarak da adlandırılır. Duruma göre seçim ve döngülerde kullanılır. Sağ ve solunda olan iki **işlenen** (**operand**) üzerinde karşılaştırma yapar.

İşleç	Açıklama
<	Küçük Mü?
>	Büyük Mü?
<=	Küçük ya da Eşit Mi?
>=	Büyük ya da Eşit Mi?
==	Eşit Mi?
!=	Farklı Mı?

Tablo 7. Karşılaştırma İşleçleri

Aşağıda bu işleçler için örnek verilmiştir;

```

a=5
b=6
c= a > b
print ("a: {} b: {} a>b: {}".format(a,b,c))
# Çıktı: a: 5 b: 6 a>b: False
c= a<b
print ("a: {} b: {} a<b: {}".format(a,b,c))
# Çıktı: a: 5 b: 6 a<b: True
c= a>=b
print ("a: {} b: {} a>=b: {}".format(a,b,c))
# Çıktı: a: 5 b: 6 a>=b: False
c= a<=b
print ("a: {} b: {} a<=b: {}".format(a,b,c))
# Çıktı: a: 5 b: 6 a<=b: True
c= a==b
print ("a: {} b: {} a==b: {}".format(a,b,c))
# Çıktı: a: 5 b: 6 a==b: False
c= a!=b
print ("a: {} b: {} a!=b: {}".format(a,b,c))
# Çıktı: a: 5 b: 6 a!=b: True

```

Atama İşleçleri

Eşittir (=) sembolü **atama işleci** (**assignment operator**) olarak tanımlanır. Sağındaki işlenen değeri solundaki tek bir değişkene atar. Atama işleçleri her zaman sola atama yapar.

İşleç	Adı
=	Sağdaki işleneni sola atama yapar. Soldan sağa atama yapılmaz!
+=	Soldaki işlenene sağdakini ekler ve sonucu sola atama yapar

-=	Soldaki işlenenden sağdakini çıkarır ve sonucu sola atama yapar
*=	Soldaki işlenenle sağdakini çarpar ve sonucu sola atama yapar
/=	Soldaki işleneni sağdakine böler ve sonucu sola atama yapar
%=	Soldaki işleneni sağdakine böler ve kalanı sola atama yapar
**=	Soldaki işlenenin sağdaki kadar kuvvetini alır ve sonucu sola atama yapar
//=	Soldaki işlenenin sağdaki kaç katı olduğu bulunur ve sonucu sola atama yapar

Tablo 8. Atama işlemleri

Aşağıda bu işlemler için örnek verilmiştir;

```
a = 10
print(a) #10
a += 10
print(a) #20
a -= 10
print(a) #10
a *= 10
print(a) #100
a /= 10
print(a) #10.0
a %= 4
print(a) #2.0
a **= 2
print(a) #4.0
a //= 4
print(a) #1.0
```

Mantıksal İşlemler

Mantıksal işlemler (**logical operator**) bileşik Boole ifadeleri oluşturmak için kullanılır. Bu mantıksal işlemlerin her bir işleneni kendi başına bir Boole ifadesidir. Üç mantıksal işlem vardır. Bunlar küçük harfle yazılan **and**, **or** ve **not** işlemleridir.

İşlem	Adı
and	Mantıksal VE: Sağ ve soldaki işlenenler True ise True sonucunu verir. Aksi halde False sonucunu verir.
or	Mantıksal VEYA: Sağ ve soldaki işlenenlerden bir True ise True sonucunu verir. Aksi halde False sonucunu verir.
not	Mantıksal DEĞİL: Sağındaki işlenen True ise False sonucunu verir. Aksi halde False sonucunu verir.

Tablo 9. Mantıksal İşlemler

Aşağıda bu işlemler için örnek verilmiştir;

```
yaş=19
cinsiyet='E'
sonuç=yaş>18 and cinsiyet=='E'
print ("Askerlik Yapabilir: {}".format(sonuç))
# Çıktı: Askerlik Yapabilir: True
sağlıkdurumu='Engelli'
sonuç=yaş>60 or sağlıkdurumu=='Engelli'
print ("Toplu Taşımada Öncelikli: {}".format(sonuç))
# Çıktı: Toplu Taşımada Öncelikli: True
```

Bit Düzeyi İşlemler

Bit düzeyi işlemler (**bitwise operator**) tamsayı veri tipindeki nesneler üzerinde bit seviyesinde işlemler gerçekleştirmek için kullanılır. Tamsayı veri tipindeki nesneyi bir bit dizisi olarak ele alır. Altı farklı bit düzeyi işlem vardır.

İşleç	Adı
&	İki işlenen tamsayının karşılıklı olarak bitlerini VE (AND) işlemine tabi tutar.
	İki işlenen tamsayının karşılıklı olarak bitlerini VEYA (OR) işlemine tabi tutar.
^	İki işlenen tamsayının karşılıklı olarak bitlerini ÖZEL VEYA (XOR) işlemine tabi tutar.
~	Sağdaki işlenen tamsayının her bir bitini DEĞİL (NOT) işlemine tabi tutar.
<<	Soldaki işlenenin bitlerini sağdaki işlenen kadar sola kaydırır. Her kaydırma 2 ile çarpma anlamına gelir.
>>	Soldaki işlenenin bitlerini sağdaki işlenen kadar sağa kaydırır. Her kaydırma 2 ile bölme anlamına gelir.

Tablo 10. Bir Düzeyi İşleçler

Aşağıda bu işleçler için örnek verilmiştir. Bir tamsayıyı ikili olarak konsola yazdırmak için `bin()` fonksiyonu kullanılır;

```
a=10
b=6
sonuç=a&b
print ("a: {} b: {} a&b: {}".format(bin(a),bin(b),bin(sonuç)))
# Çıktı: a: 0b1010 b: 0b110 a&b: 0b10
sonuç=a|b
print ("a: {} b: {} a|b: {}".format(bin(a),bin(b),bin(sonuç)))
# Çıktı: a: 0b1010 b: 0b110 a&b: 0b1110
sonuç=a^b
print ("a: {} b: {} a^b: {}".format(bin(a),bin(b),bin(sonuç)))
# Çıktı: a: 0b1010 b: 0b110 a^b: 0b1100
a=2
b=1
sonuç= a<<b
print ("a: {} b: {} a<<b: {}".format(bin(a),b,bin(sonuç)))
# Çıktı: a: 0b10 b: 1 a<<b: 0b1000
sonuç= a>>b
print ("a: {} b: {} a>>b: {}".format(bin(a),b,bin(sonuç)))
# Çıktı: a: 0b10 b: 1 a>>b: 0b1
b=-1 # -1 tamsayısının bütün bitleri 1 dir
sonuç=~b
print ("b: {} ~b: {}".format(bin(b),bin(sonuç)))
# Çıktı: b: -0b1 ~b: 0b0
```

Üyelik İşleçleri

Üyelik işleçleri (**membership operator**), bir öğenin belirli bir dizi ve koleksiyonda bulunup bulunmadığını veya üyesi olup olmadığını belirlememize yardımcı olur.

İşleç	Adı
in	Üye, kapsayıcı nesnede bulunursa mantıksal sonucunu <code>True</code> verir.
not in	Üye, kapsayıcı nesnede bulundurmuyorsa mantıksal sonucunu <code>True</code> verir.

Tablo 11. Üyelik İşleçleri

Aşağıda bu işleçler için örnek verilmiştir.

```
metin = "Bugün bayram."
bul='.'
print (bul, "in", metin, ":", bul in metin)
# Çıktı: . in Bugün bayram. : True
liste = [10,20,30,40,50]
```

```

a = 20
b = 3
print(a, "in", liste, ":", a in liste)
# Çıktı: 20 in [10, 20, 30, 40, 50] : True
print(b, "in", liste, ":", b in liste)
# Çıktı: 3 in [10, 20, 30, 40, 50] : False
print(a*b, "in", liste, ":", a*b not in liste)
# Çıktı: 60 in [10, 20, 30, 40, 50] : True

```

Kimlik İşleçleri

Kimlik işleçleri (**identity operator**), nesnelerin aynı belleği paylaşıp paylaşmadıklarını ve aynı veri tipinde nesneye başvurup başvurmadıklarını belirlemek için nesneleri karşılaştırır.

İşleç	Adı
is	Sağ ve solundaki nesne aynı nesne ise True verir.
is not	Sağ ve solundaki nesne farklı nesne ise True verir.

Tablo 12. Üyelik İşleçleri

Aşağıda bu işleçler için örnek verilmiştir.

```

liste1=[10,20,30,40,50]
liste2=[10,20,30,40,50]
liste3=liste1
print('liste1 ile liste2 aynı mı? ', liste1 is liste2)
# Çıktı: liste1 ile liste2 aynı mı? False
print('liste1 ile liste3 aynı mı? ', liste1 is liste3)
# Çıktı: liste1 ile liste3 aynı mı? True

```

İşleç Öncelikleri

Cebirsel ifadelerde nasıl ki önce çarpma ve bölme sonra toplama işlemleri yapılıyor. Python programlama dilinde yazılan ifadelerde de **işleç öncelikleri** (**operator precedence**) vardır. İfadelerde bir işlemi öncelikli hale getirmek için parantez () içerisine alınır.

En içteki parantez en öncelikli olarak gerçekleştirilir. Aşağıda verilen tabloda aynı satırda olan işleçler aynı önceliğe sahiptir.

Öncelik Sırası	İşleçler
1	() , [], {}
2	[index]
3	**
4	+x, -x, ~x
5	*, /, //, %
6	+, -
7	<<, >>
8	&
9	^
10	
11	in, not in, is, is not, <, <=, >, >=, !=, ==
12	not x
13	and
14	or

Tablo 13. İşleç Öncelikleri

Başka işlemler de bulunmaktadır bunlar ilerleyen bölümlerde yeri geldikçe anlatılacaktır¹¹.

Kayan Noktalı Sayı Hesapları

Kayan noktalı sayıların IEEE-754 standardında ikili tabanda tutulduğu *Değişken* başlığında anlatılmıştı. Kayan noktalı sayılar olarak değişkenlere verdiğimiz onluk tabanda değerler aslında arka planda ikilik tabanda tutulur ve bu durum yazdığımız kodlarda garip davranışların ortaya çıkmasına sebep olur. Buna örnek olarak; hemen hemen her programcının yaptığı ilk hata, aşağıdaki kodun amaçlandığı gibi çalışacağını varsaymaktır;

```
total = 0;
while total!=2.0:
    total += 0.01
    print(total)
print (total)
```

Acemi programcı bunun 0, 0.01, 0.02, 0.03, gibi artarak, 1.97, 1.98, 1.99 aralığındaki her bir sayıyı toplayacağını ve 2.0 sonucuna ulaşacağını varsayar. Bu kodda doğru yürümeyen iki şey olur: Birincisi yazıldığı haliyle program asla sonuçlanmaz. `total` asla 2'ye eşit olmaz ve döngü asla sonlanmaz. İkincisi ise döngü mantığını `total < 2.0` şartını kontrol edecek şekilde yeniden yazarsak, döngü sonlanır, ancak toplam 2.0'dan farklı bir şey olur. IEEE754 uyumlu işlemlerde, genellikle bunun yerine yaklaşık 2.0000000000000004'e ulaşır. Bunun olmasının nedeni, kayan noktalı sayıların onluk tabanda kendilerine atanan değerlerin, ikilik tabanda yaklaşık değerlerini temsil etmesidir.

```
total=0
while total<2:
    total +=0.1
print (total)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
0.1
0.2
0.30000000000000004
0.4
0.5
0.6
0.7
0.7999999999999999
0.8999999999999999
0.9999999999999999
1.0999999999999999
1.2
1.3
1.4000000000000001
1.5000000000000002
1.6000000000000003
1.7000000000000004
1.8000000000000005
1.9000000000000006
2.0000000000000004
```

Bu duruma aşağıdaki klasik örnek verilir;

```
a=0.1
b=0.2
c=0.3
print("a: {}, b: {}, c:{}, a+b==c {}".format(a,b,c,a+b==c))
#a: 0.1, b: 0.2, c:0.3, a+b==c False
```

¹¹ https://www.tutorialspoint.com/python/python_operator_precedence.htm

Programcı olarak gördüğümüz şey onluk tabanda yazılmış üç sayı olsa da derleyicinin (ve altta yatan donanımın) gördüğü şey ikili sayılardır. 0.1, 0.2 ve 0.3, ona mükemmel bölmeyi gerektirdiğinden (ki bu onluk sayı sisteminde oldukça kolaydır), ancak ikili sayı sisteminde imkansızdır (bu sayılar, onluk sayı sistemindeki $1/3$ sayısı gibi $0.3333333333333333...$ şeklinde kesin olmayan biçimde depolanması gerektiğindendir). Bunun çözümü için aşağıdaki kod örneği verilebilir;

```
from decimal import Decimal
# Kayan nokta sorunu
print(0.1 + 0.2) # 0.30000000000000004
# Çözüm
print(Decimal('0.1') + Decimal('0.2')) # 0.3
```

Konsola Biçimlendirilmiş Metin Yazma

Python dilinde çıktı **biçimlendirme** (**formatting**), verilerin yazdırıldığında veya kaydedildiğinde sunulma biçimini ifade eder. Doğru biçimlendirme, bilgileri daha anlaşılır ve uygulanabilir hale getirir. Python dilinde, eski tip biçimlendirmeden yeni **f-dizgisi** (**f-string**) yaklaşımına kadar birçok yöntem bulunur.

Dizgi modül işleci

Dizgileri biçimlendirmenin en eski yollarından biri dizgi modül (%) işlecidir. Biçim belirteçlerini dizeye yerleştirerek değerleri bir dizgiye yerleştirmenize olanak tanır. Her belirteç bir % ile başlar ve biçimlendirilen değerin türünü temsil eden bir karakterle biter.

```
print("Toplam Adet: %2d, Ortalama : %5.2f" % (27, 07.444))
# Toplam Adet: 27, Ortalama : 7.44

print("Öğrenci Sayısı : %3d, Erkek : %2d, Kadın:%2d" % (98, 25, 73))
# Öğrenci Sayısı : 98, Erkek : 25, Kadın:73

print("%7o" % (64))
# 100 # sekizlik (octal) sayı sisteminde çıktı

print("%10.3E" % (3.141527))
# 3.561E+02 #bilimsel gösterim olarak biçimlendirme
```

format() Yöntemi

Bu yöntem, dizgiye değerleri yerleştirmek için yer tutucu olarak süslü parantezler {} kullanarak dizgi interpolasyonunu daha esnek bir şekilde ele alma olanağı sağlar. Hem konumsal hem de adlandırılmış argümanları desteklediğinden, çeşitli biçimlendirme ihtiyaçları için çok yönlüdür. Dizgi metnindeki her tırnaklı parantez sırasıyla format() fonksiyonundaki parametrelere karşılık gelir.

```
print("Meyveler: {0},{1},{2}".format("Elma", "Armut","Vişne"))
#Meyveler: Elma,Armut,Vişne

print("{0} adet Elma var.".format(25))
#25 adet Elma var.

print("Pi Sayısı:{0}, Öğrenci Sayısı:{1}".format(3.1415,100))
#Pi Sayısı:3.1415, Öğrenci Sayısı:100.
```

Bu yöntem ayrıca genişlikleri, hizalamayı, sayı biçimlendirmesini ve daha fazlasını ayarlama gibi ayrıntılı biçimlendirme seçeneklerini de destekler. Tablo şeklide veriler veya raporlar oluşturmak için son derece yararlı olabilir.

```
sablon = "Toplam: {0}, Ortalama: {1} and Standart Sapma:{sapma}."
print(sablon.format(1000, 100, sapma=1.0))
# Çıktı: Toplam: 1000, Ortalama: 100 and Standart Sapma:1.0.

print("Adet: {0:4d}, Pi:{1:4.2f}".format(12, 3.141527))
```

```
# Çıktı: Adet: 12, Pi:3.14
print("Pi:{1:4.2f}, Adet: {0:4d}".format(12, 3.141527))
# Çıktı: Pi:3.14, Adet: 12

print("Adet: {adet:4d}, Pi:{pi:4.2f}".format(pi=3.141527, adet=12))
# Çıktı: Adet: 12, Pi:3.14

boy=180
kilo=90.5
print('boy:',boy,'kilo:',kilo)
# Çıktı: boy: 180 kilo: 90.5
print('boy:{}, kilo:{}'.format(boy,kilo))
# Çıktı: boy:180, kilo:90.5
print('12345678901234567890')
# Çıktı: 12345678901234567890
print('{0:>10}'.format(boy))
# Çıktı: 180
print('{0:<10}-{0:>10}'.format(boy,kilo))
# Çıktı: 180 180
```

Biçimlendirme metni aynı zamanda kendisinden sonra gelecek parametrelerin de hangi biçimde konsola yazılacağını belirler. Her bir parametrenin konsola nasıl yazılacağını **biçim belirleyicisi** (**format specifier**) belirler. Biçim belirleyicileri iki karakter olup ilk karakteri yüzde (%) karakteridir. Biçim belirleyici karakterler C dilinde olduğu gibi aşağıdakilerden biri olabilir;

Biçim Karakterleri	Biçimlendirme metni çıktısı
%d	Konsola ondalık sayı olarak yazılır
%b	Konsola ikili (binary) olarak yazılır.
%c	Konsola karakter olarak yazılır.
%f	Konsola kayan noktalı sayı olarak yazılır.
%e	Konsola kayan noktalı sayı üs içerecek şekilde yazılır.
%g	Konsola genel olarak sayı yazılır.
%x, %X	Konsola onaltılık (hexadecimal) sayı olarak yazılır.
%o	Konsola sekizlik (octal) sayı olarak yazılır.
%s	Konsola metin olarak yazılır.
%%	Konsola % karakteri yazılır.

Tablo 14. Biçim Belirleyiciler

f-Dizgisi

Bu dizgi biçimlendirmeyi ve enterpolasyonu kolaylaştırır. f-dizgileriyle, değişkenleri ve ifadeleri süslü parantezler {} kullanarak doğrudan dizgilerin içine yerleştirebiliriz. F-dizgisi, bir dizginin başına **f** eklenir ve değişkenleri veya ifadeleri {} içine yerleştirerek kullanılır. **str.format()** gibi çalışır, ancak daha öz ve okunabilir bir şekilde.

```
boy=180
kilo=90.5
print(f"boy:{boy}, kilo: {kilo}") #boy:180, kilo: 90.5
print(f"kilo: {kilo},boy:{boy}") #kilo: 90.5,boy:180
print(f"kilo: {kilo:5.2f},boy:{boy:4d}") #kilo: 90.50,boy: 180
```

⚠ f-Dizgisinde ters bölü karakteri (\) kullanılmaz!

Ekrana tarih yazmak için aşağıdaki kod kullanılabilir. Burada **B** ay ismini, **d** rakam olarak günü ve **Y** rakam olarak yılı temsil eder.

```
import datetime
bugun = datetime.datetime.today()
print(f"{bugun:%B %d, %Y}") # October 17, 2025
```

Hangisini Kullanmalıyız?

Konsola veya bir başka yer için metin biçimlendirirken dizgi modül işleci olan % işleci eski yöntem olup kullanılmamalıdır. `format()` yöntemi karmaşık durumlarda tercih edilmelidir. **f-dizgisi** (**f-string**) yeni nesil metin biçimlendirme olup her zaman tercih edilmelidir.

```
isim = "Ali"  
yas = 25  
print(f"{isim} {yas} yaşında")
```

KONTROL İŞLEMLERİ

Kontrol İşlemleri

Kontrol işlemleri (**control operation**) veya **kontrol akışı** (**control flow**) programın icra sırasını değiştiren kontrol yapıları olup bu **talimatlar** (**statement**) olup üç türü vardır;

1. **Duruma göre seçimler** (**conditional choice**): if, if-else ve match talimatları.
2. **İlişkisel döngü** (**relational loop**): for, for-else ve while talimatları.
3. **Dallanmalar** (**jump**): Fonksiyon çağrılar ve continue, break, pass ve return talimatları.

Kontrol işlemlerinde; kodlana mantıksal satırlar (**logical sequence**) ile fiziksel olarak icra edilen satırlar (**physical sequence**) birbirinden farklıdır.

Satırlar ve Girintileme

Python programlama, sınıf (**class**), fonksiyon (**function**) tanımları veya kontrol işlemlerinde (**control operation**) kullanılacak kod blokları için tırnaklı parantez kullanmaz. C/C++/C# ve Java dillerinde kod bloğu tırnaklı parantezle başlar ve biter. Ancak Python dilinde kod blokları, katı bir şekilde uygulanan **satır girintisi** (**indentation**) ile yapılır.

💡 Standart olarak 4 boşluk karakteri ile girinti verilir!

Girintileme, iki nokta üst üste (:) karakteri kod bloğunu başlatır. Bu karakterden sonra talimatlar girintili yazılır. Girinti bittiğinde kod bloğu bitmiş sayılır.

```
yaş=150
if yaş>120: # If koşulu doğrulanırsa icra edilecek blok buradan başlıyor
    print('Hatalı Yaş Değeri:',yaş) # Bu satır blok içinde
    yaş=120 # Bu satır da girintili yazıldığından bu satırda aynı blok içindedir.
print(yaş)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Hatalı Yaş Değeri: 150
120
```

Yukarıdaki örnekte if koşulundan sonraki iki nokta üst üste karakterinden sonra bir alt satıra geçildiğinde 4 karakter girintileme yapılmıştır. Girintileme bir alt satırda da devam etmiştir. Bu durumda if koşulu gerçekleşirse bu iki satır icra edilecektir. Bir başka örnek aşağıda verilmiştir;

```
if True:
    print ("Doğru") #Doğru ekrana basılır
else:
    print ("Yanlış")
Doğru

if True:
    print ("Cevap") #Cevap
    print ("Doğru") #Doğru
else:
    print ("Cevap")
    print ("Yanlış")
```

Eğer tek talimat yazılacak ise aşağıdaki gibi girintileme yapılmayabilir;

```
yaş=150
if yaş>120: print('Hatalı Yaş Değeri!') #Hatalı Yaş Değeri! Ekrana basılır
else: print('Yaş:',yaş)
```


Ayrıca Python dilinde eğer bir **talimat** (**statement**) birden fazla satırda yazılacak ise ters bölü karakteri ile birden fazla satırda yazılabilir. Bu karakterden sonra talimatın devamına alt satırda girintili bir şekilde devam edilir.

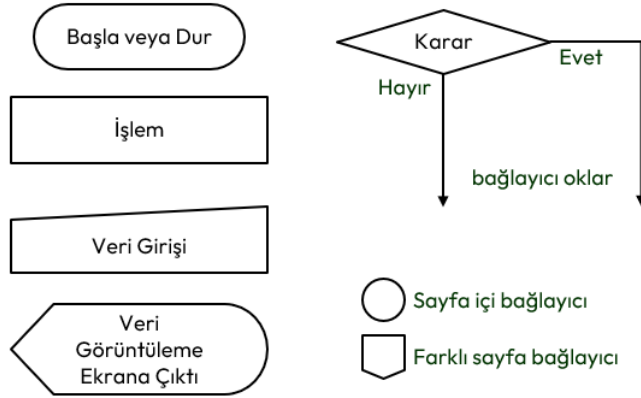
```
birincisatış = 10000
ikincisatış = 30000
üçüncüsatış = 40000
toplam = birincisatış + \
        ikincisatış + \
        üçüncüsatış
print(toplam) #80000
```

[], {} veya () parantezleri içinde yer alan ifadelerin satır devam karakterini kullanması gerekmez.

```
günler=['Pazartesi', 'Salı',
        'Çarşamba', 'Perşembe',
        'Cuma']
```

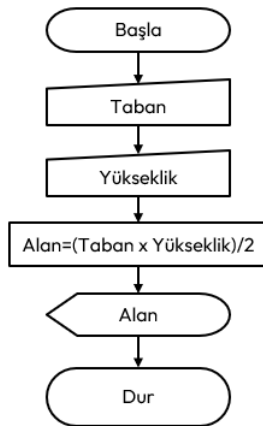
Akış Diyagramları

Kontrol işlemi içeren programın icrasını anlamak için akış diyagramlarını da anlamak gerekir. Aşağıda akış diyagramlarındaki temel şekiller verilmiştir.



Şekil 6. Akış Diyagramları Genel Şekilleri

Akış diyagramları (**flowchart**), adımların grafiksel gösterimleridir. Algoritmaları ve programlama mantığını temsil etmek için bir araç olarak bilgisayar biliminden ortaya çıkmıştır. Ancak diğer tüm işlem türlerinde kullanılmak üzere genişletilmiştir. Günümüzde akış diyagramları, bilgileri göstermede ve akıl yürütmeye yardımcı olmada son derece önemli bir rol oynamaktadır. Karmaşık süreçleri görselleştirmemize veya sorunların ve görevlerin yapısını açık hale getirmemize yardımcı olurlar. Bir akış şeması, uygulanacak bir süreci veya projeyi tanımlamak için de kullanılabilir.



Şekil 7. Örnek Akış Diyagramı

Akış diyagramı çizilirken aşağıdaki kurallara uyulur;

- Akış şemalarında tek bir başlangıç simgesi olmalıdır
- Bitiş simgesi birden çok olabilir.
- Karar simgesinin haricindeki simgelere her zaman tek giriş ve tek çıkış yolu bulunur.
- Bağlaç simgesi sayfanın dolmasından ötürü parçalanmış akış şemasının öğelerini birleştirmede kullanılır.
- Simgeler birbirleri ile tek yönlü okla bağlanırlar.
- Okların yönü algoritmanın mantıksal işlem akışını tanımlar.

Aşağıda taban ve yüksekliği klavyeden girilen bir üçgenin alanını hesaplamak için bir akış diyagramı örneği verilmiştir.

Bu akış diyagramına ilişkin program aşağıda verilmiştir;

```
tabanstr=input('Tabanı Giriniz:')
yükseklikstr=input('Yüksekliği Giriniz:')
alan=float(tabanstr)*float(yükseklikstr)/2
print(alan)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Tabanı Giriniz:3.5
Yüksekliği Giriniz:4
7.0
```

Duruma Göre Seçimler

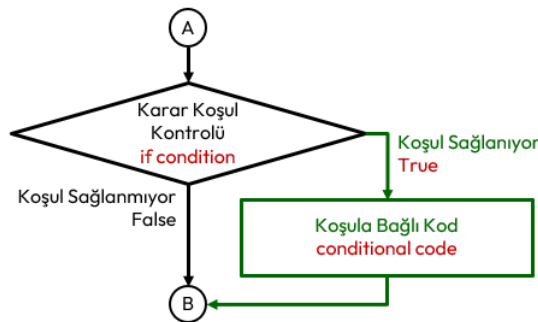
Duruma göre seçimler (**conditional choice**) programın akışını bir koşula göre değiştiren talimatlardır.

If Talimatı

If talimatı, **karar vermeye** (**decision making**) ilişkin bir talimat olup bir koşula bağlı olarak programın icrasını değiştirir. Mutlaka Boolean bir ifade olan **koşul** (**condition**) ifadesi test edilir sonuç **True** ise **koşul kodu** (**conditional code**) icra edilir, değilse icra edilmez.

Sözde kodu aşağıda verilmiştir.

```
if koşul:
    koşul-kodu
```



Şekil 8. If Talimatı İcra Akışı

If talimatı içeren bir kodun icrası;

		yaş	cinsiyet
1	yaş=18	18	?
2	cinsiyet='K'	18	'K'
3	if yaş<30:	18	'K'
4	print("Genç")	18	'K'
5	if cinsiyet=='E':	18	'K'
	print("Erkek")	18	'K'

Tablo 15. If Talimatı İçeren Bir C Programının İcrası

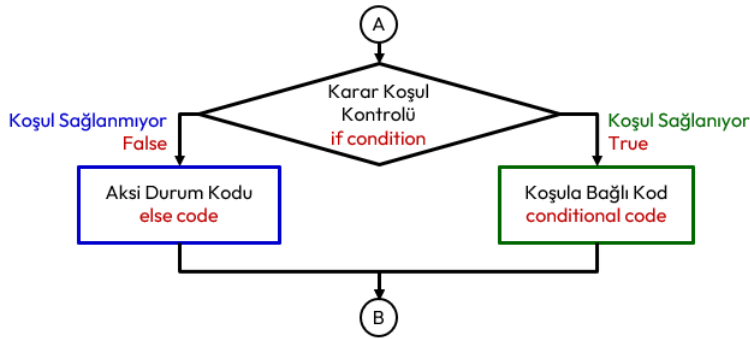
Programın icrasında neler olduğu aşağıda verilmiştir;

1. yaş değişkenine 18 değeri atanır.
2. cinsiyet değişkenine 'K' atanır.
3. if talimatındaki **koşul** (condition) test edilir. `yaş<30` yani `18<30` testinde sonuç **True** olduğundan izleyen **koşul kodu** (conditional code) icra edilir.
4. `print("Genç")` koşul kodu olduğundan icra edilir.
5. if talimatındaki koşul test edilir. `cinsiyet=='E'` yani `'E'=='K'` testinde sonuç **False** olduğundan izleyen koşul kodu icra edilmez.

If-Else Talimatı

If-Else Talimatı, bir başka **karar verme** (decision making) talimatı olup bir koşula bağlı olarak programın icrasını değiştirir. If talimatına benzer şekilde **koşul** (condition) ifadesi True ise **koşul kodu** (conditional code) icra edilir, değilse **aksi durum kodu** (else code) icra edilir.

```
if koşul:
    koşul-kodu
else:
    aksi-durum-kodu
```



Şekil 9. If-Else Talimatı Sözde Kodu ve İcra Akışı

Bu talimata ilişkin örnek aşağıda verilmiştir;

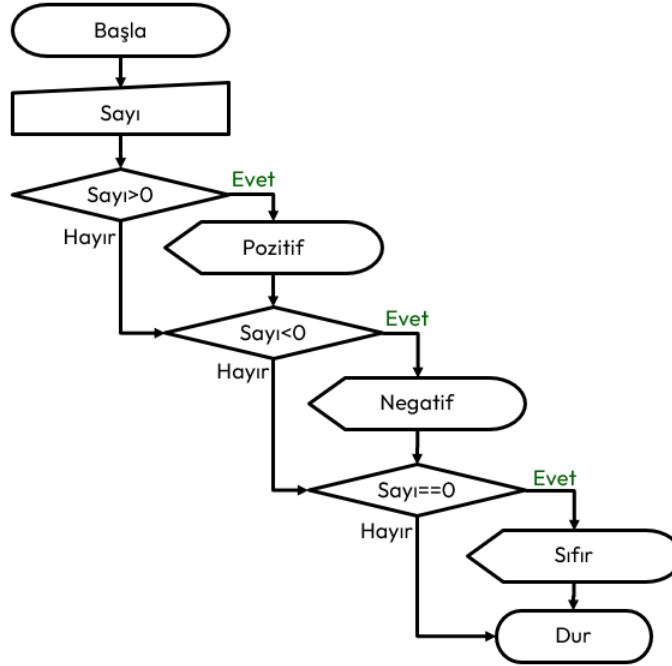
```
yaş=18
cinsiyet='K'
if yaş<30: # yaş test ediliyor
    print("Genç") #burada yaş değerinin 30 dan küçük olduğu biliniyor. Genç Yazılır
else:
    print("Yaşlı") #burada yaş değerinin 30 dan küçük olmadığı biliniyor
if cinsiyet=='E': #cinsiyet test ediliyor
    print("Erkek") #burada cinsiyet değerinin 'E' olduğu biliniyor
else:
    print("Kadın") #burada cinsiyet değerinin 'E' olmadığı biliniyor. Kadın Yazılır
```

If talimatında test edilen değerler **elif** eklenerek çoğaltılabilir. Bu durumda her eklenen **elif** için ayrı koşul kodu yazılır;

```
yaş=18
cinsiyet='K'
if yaş<30:
    print("Genç") # bu blokta yaş değerinin 30 dan küçük olduğu biliniyor
elif yaş <60:
    print("Orta Yaşlı") # bu blokta yaş değerinin 30 dan küçük olmadığı yani
                        # önceki testten geçmediği ve 60 den küçük olduğu
                        # biliniyor
else:
    print("Yaşlı") # bu blokta yaş değerinin önceki iki testten geçmediği
                  # ve 60 den küçük olmadığı biliniyor
if cinsiyet=='E': #cinsiyet test ediliyor
    print("Erkek") #burada cinsiyet değerinin 'E' olduğu biliniyor
else:
```

```
print("Kadın") #burada cinsiyet değerinin 'E' olMAdığı biliniyor
```

Örnek olarak klavyeden girilen bir sayının sıfırdan küçük mü? Büyük mü? ya da sıfıra eşit mi? Olduğunu bulan bir programı yazalım. Programın icra akışı aşağıdaki şekilde olacaktır.



Şekil 10. Programın If Talimatlarıyla Yazılması Halinde İcra Akışı

Bu akışı çalıştıran program kodu aşağıda verilmiştir;

```
sayistr=input("Bir Tamsayı Giriniz:")
sayı=int(sayistr)
if sayı>0:
    print("Pozitif")
if sayı<0:
    print("Negatif")
if sayı==0:
    print("Sıfır")
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Bir Tamsayı Giriniz:12
Pozitif
```

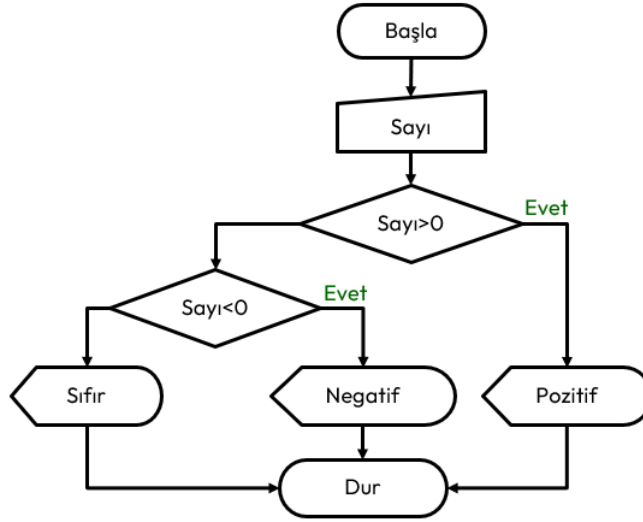
İcra akışı incelendiğinde ilk **if** talimatında yapılan test doğru çıkarsa geri kalan testlerin yapılmasına gerek kalmaz. Benzer şekilde ilk iki test geçilirse üçüncü **if** talimatındaki teste gerek yoktur. Aynı program if-else talimatlarıyla aşağıdaki şekilde yeniden yazabiliriz;

```
sayistr=input("Bir Tamsayı Giriniz:")
sayı=int(sayistr)
if sayı>0:
    print("Pozitif")
elif sayı<0:
    print("Negatif")
else:
    print("Sıfır")
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Bir Tamsayı Giriniz:12
Pozitif
```

Bu durumu aşağıdaki akış diyagramında daha net görebiliriz.



Şekil 11. Programın If-Else Talimatlarıyla Yazılması Halinde İcra Akışı

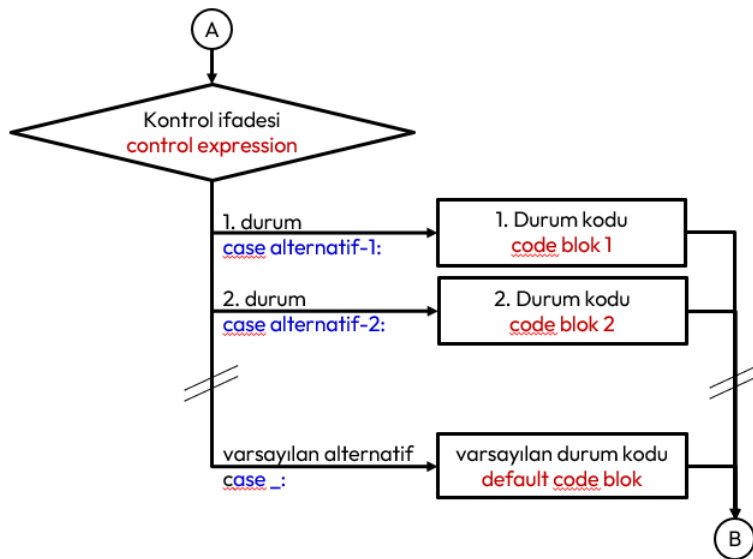
Program incelendiğinde sayının pozitif girilmesi halinde sadece ilk **if** talimatındaki test geçecek ve sonrasındaki aksi durum kodlarına ilişkin bloklar icra edilmeyecektir. Sayının negatif girilmesi halinde **elif** bloğuna girilecek ve sonrasındaki **else** kısmı çalıştırılmayacaktır. Sayı sıfır girilmiş ise **else** kodu çalıştırılacaktır.

Match Talimatı

Seçme (**match**) talimatı, bir kontrol **ifadesi** (**expression**) sonucunda birden fazla alternatif arasında seçim yapılmasını sağlar. Alternatifler, kontrol ifadesini seçen **desenlerdir** (**pattern**). Sözde kodu aşağıda verilmiştir;

```

match kontrolifadesi:
    case alternatif-1:
        alternatif-1-kodu
    case alternatif-2:
        alternatif-2-kodu
    # ...
    case _:
        varsayılan-alternatif-kodu
  
```



Şekil 12. Switch Talimatı Sözde Kodu ve İcra Akışı

⚠ match-case yapısı Python 3.10 ve üzeri sürümlerde kullanılabilir. Eski sürümlerde **if-elif-else** kullanılmalıdır.

Seçim talimatı, bir ifadeyi alır ve değerini bir veya daha fazla durum bloğu olarak verilen ardışık desenlerle karşılaştırır. İfadeye uyan desendeki durum kodu icra edilir. `match` talimatına ilişkin kurallar;

1. **Kontrol ifadesi** (**control expression**), sonucu tamsayı olan ifadedir ve bir kez değerlendirilir.
2. Bloğu olmayan bir `match` talimatı yazılamaz!
3. Tüm alternatifleri içerecek şekilde alternatiflerin tümü yazılmayabilir.
4. Blok sonunda yer alacak şekilde üzerinde yer alan alternatifler dışında kalan tüm alternatifler için `_`: etiketli alternatif yazılabilir.

Örnek program aşağıda verilmiştir;

```
gün=input("Gün Giriniz:")
match gün:
    case "Pazartesi" | "Salı" | "Çarşamba" | "Perşembe": #birden fazla desen
        print("Girdiğiniz Gün:{}".format(gün))
    case "Cuma": print("Mübarek Gün.") #tek desen
    case "Cumartesi": print("Yarım Gün Çalışma.") #tek desen
    case "Pazar": print("Tatil.") #tek desen
    case _: print("Geçerli bir gün adı Girmediniz!") #yukarıdaki desenlere uymayanlar
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Gün Giriniz:Cuma
Mübarek Gün
```

Seçim talimatı ifadeyi herhangi bir sabitle eşleştirmez. Bir listeyi durum değeri olarak kullanılabilir.

```
liste=["İyi Çalışmalar", "İlhan"] #iki elemanlı bir liste
match liste:
    case [selamlama,isim]: # her bir eleman bir değişkenile işlenebilir.
        print(selamlama,"!",isim)

hesaplistesi=[True,2000] # iki elemanlı liste: ilk elemam haftasonu çalışmış mı?
match hesaplistesi:
    case [haftasonumu,yevmiye]:
        if haftasonumu: print("Haftasonu Yevmiyesi:",yevmiye)
        else: print("Hafta İçi Yevmiyesi:", yevmiye*0.8)

hesaplistesi=[False,2000] # iki elemanlı liste: ilk elemam haftasonu çalışmış mı?
match hesaplistesi:
    case [haftasonumu,yevmiye]:
        if haftasonumu: print("Haftasonu yevbiyesi:",yevmiye)
        else: print("Hafta İçi Yevmiyesi:", yevmiye*0.8)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
İyi Çalışmalar ! İlhan
Haftasonu Yevmiyesi: 2000
Hafta İçi Yevmiyesi: 1600.0
```

Ayrıca, listedeki değişken sayıda öge için, yıldız (*) operatörüyle bir diziye ayrıştırılabilirler.

```
liste=["İlhan","Pazartesi","Çarşamba","Perşembe","Cuma","Cumartesi"] # 6 elemanlı liste

match liste:
    case [isim,*günler]: #ikiden fazla elemanlı liste girilirse ilk elemandan sonrası
                        #diziye ayrıştırılıyor.
        print(isim," Şu günler Çalıştı:")
        for gün in günler:
            print(gün,end=",")
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
İlhan Şu günler Çalıştı:
Pazartesi,Çarşamba,Perşembe,Cuma,Cumartesi,
```

Bir başka örnek;

```
liste=["İlhan","Pazartesi"]
match liste:
    case [isim,gün]: # Eğer iki elemanlı bir liste girilirse
        print(isim,",", gün, " Günü çalıştı")
    case [isim,*günler]: # İki'den fazla elemanlı liste girilirse
        print(isim," Şu günler çalıştı:")
        for gün in günler:
            print(gün,",")
```

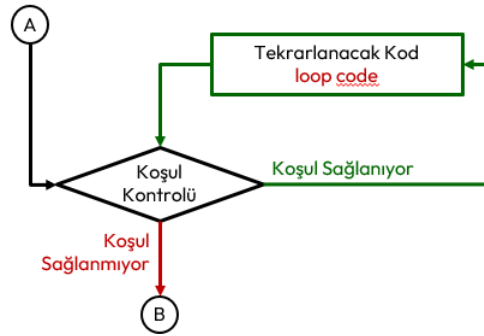
Örneğin program akışı aşağıdaki gibi olacaktır.

İlhan , Pazartesi Günü çalıştı

İlişkisel Döngüler

Döngüler (loop) bir ifadeyi veya ifadeler grubunu birden çok kez yürütmemize olanak tanır. Bunu yapmak için tekrarlanacak kodun başına gitmek için eskiden GOTO talimatı kullanılırdı. Bu talimatın kullanılması kodun okunaklılığını azaltan bir şeydir. GOTO 1968 Yılında *Edsger W. Dijkstra* GOTO/JUMP TO ifadelerini zararlı olarak ilan edilmiştir¹². GOTO kullanmamak için; **for**, **for-next** ve **while** gibi **kontrol talimatları** (control statement) yapısal programlamaya eklenmiştir.

Python dili de yapısal programlamayı destekleyen bir dildir.

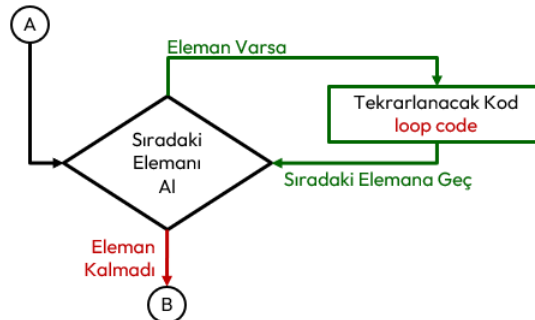


Şekil 13. Bir Döngünün İcra Akışı

For Talimatı

Python dilindeki **for** döngüsü, bir liste, **demet** (tuple) veya bir **dizgi** (string) gibi herhangi bir dizinin elemanları üzerinde döngü yapma yeteneği sağlar. Dizinin her elemanına aynı eylemi gerçekleştirir.

```
for yineleme-elemanı-değişkeni in yilenebilir-yapı:
    döngü-kodu
```



Şekil 14. For Talimatı İcra Akışı

¹² <https://homepages.cwi.nl/~storm/teaching/reader/Dijkstra68.pdf>

Örnek program aşağıda verilmiştir;

```
metin="Merhaba!"
for karakter in metin:
    print(karakter)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
M
e
r
h
a
b
a
!
```

Bir liste üzerinde for döngüsüne örnek;

```
satışlar=[10000,12000,10000,5000] #liste üzerinde for döngüsü
toplam=0
for satış in satışlar:
    toplam +=satış
print("Satış Toplamı",toplam)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Satış Toplamı 37000
```

Tek sayılardan oluşan demet (tuple) üzerine for döngüsü

```
ondanküçükteksayılar=(1,3,5,7,9)
for teksayı in ondanküçükteksayılar:
    print(teksayı,end="-")
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
1-3-5-7-9-
```

Sözlük üzerinde for döngüsü

```
sözlük={10:"On", 20:"Yirmi", 30:"Otuz"}
for eleman in sözlük:
    print(eleman)
for eleman in sözlük:
    print(eleman,"-", sözlük[eleman])
for eleman in sözlük.items():
    print(eleman)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
10
20
30
10 - On
20 - Yirmi
30 - Otuz
(10, 'On')
(20, 'Yirmi')
(30, 'Otuz')
```

Aralık Nesneleri

Dahili range() fonksiyonu, bir tamsayı dizisi oluşturarak bu dizi üzerinde bir yineleyici (iterable) nesnesi döndürür. Bu aralık (range) nesnesi, adım parametresiyle ayrılmış, başlangıçtan bitişe kadar tamsayılar içerir.

Sözde kodu aşağıda verilmiştir.

```
range (başlangıç, bitiş, adım)
```

Burada **başlangıç**, aralığın başlangıç değeri. İsteğe bağlı olup varsayılan değeri 0'dır. **bitiş** değerini almayacak son elemana kadar dizi oluşturulur. **adım** kadar dizi elemanlarının arasındaki değer farkıdır. **adım** için varsayılan değer 1'dir. Aşağıda örnek program verilmiştir;

```
aralık=range(4)
for sayı in aralık:
    print(sayı,end=',') # end-ayraç (separator) anlamındadır
print()
aralık=range(1,5)
for sayı in aralık:
    print(sayı,end='-') # end-ayraç (separator) anlamındadır
print()
aralık=range(10,20,2)
for sayı in aralık:
    print(sayı,end=':') # end-ayraç (separator) anlamındadır
```

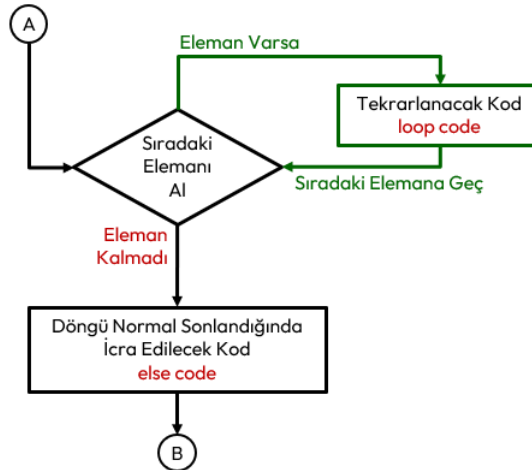
Örneğin program akışı aşağıdaki gibi olacaktır.

```
0,1,2,3,
1-2-3-4-
10:12:14:16:18:
```

For-Else Talimatı

Python, bir **for** döngüsüyle ilişkilendirilecek isteğe bağlı bir **else** bloğunu destekler. Bu **else** bloğu bir **for** döngüsüyle kullanılırsa, yalnızca **for** döngüsü normal şekilde sonlandığında icra edilir. Aşağıda sözde kodu verilmiştir.

```
for değişken in yinelenenebilir-yapı:
    #tekrarlanacak talimatlar
else:
    #döngü normal bir şekilde sonlandığında icra edilecek talimatlar
```



Şekil 15. For-Else Talimatı İcra Akışı

Aşağıda örnek bir program verilmiştir;

```
for sayaç in range(5):
    print('{}. Adım...'.format(sayaç))
else:
    print('For Döngüsü Bitti.!')
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
0. Adım...
```

```

1. Adım...
2. Adım...
3. Adım...
4. Adım...
For Döngüsü Bitti.!

```

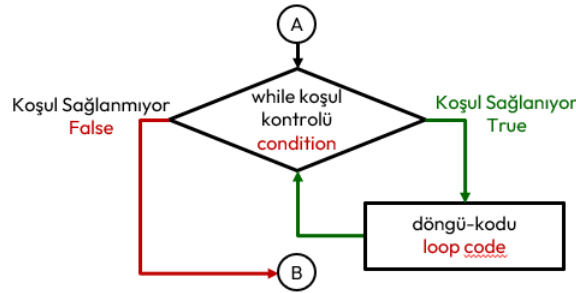
While Talimatı

While döngüsünde, **koşul** (**condition**) kontrolü **True** olduğu sürece **döngü kodu** (**loop code**) icra edilir. Koşul herhangi bir ifade olabilir ve **True** sıfır olmayan herhangi bir değerdir. İfade **False** olur olmaz, program kontrolü döngü dışındaki izleyen talimatla devam eder. Sözde kodu aşağıda verilmiştir;

```

while koşul-ifadesi:
    döngü-kodu

```



Şekil 16. While Talimatı İcra Akışı

Örnek uygulama aşağıda verilmiştir.

```

sayaç=1
while sayaç<=10:
    print(sayaç, ".Adım")
    sayaç+=1

```

Örneğin program akışı aşağıdaki gibi olacaktır.

```

1 .Adım
2 .Adım
3 .Adım
4 .Adım
5 .Adım
6 .Adım
7 .Adım
8 .Adım
9 .Adım
10 .Adım

```

İç İçe Döngü Kodu

Döngüler içinde yer alan **döngü kodu** (**loop code**) bir başka döngüyü içerebilir. **İç içe döngülerde** (**nested loop**), bir döngü diğerinin içinde tanımlanır. Başka bir döngüyü yani iç döngüyü çevreleyen döngüye **dış döngü** (**outer loop**) denir. Örneğin ekrana satır ve sütun içeren bir şekil oluşturmak istediğimizde iç içe döngü kullanırız. Aşağıda 3 satır 10 adet yıldız basan bir örnek verilmiştir. İlk **for** döngüsünün tekrarlanan kodu ikinci **for** döngüsüdür.

```

for satır in range(1,4):
    for sütun in range(1,10):
        print('*',end='')
    print() #alt satıra geçmek için.

```

Örneğin program akışı aşağıdaki gibi olacaktır.

```

*****
*****
*****

```

Aşağıda elemanları, satır ve sütun indislerinin çarpımı olan bir 4x5 boyutlu bir matris ekrana yazdırılmıştır;

```
for satır in range(1,5):
    for sütun in range(1,6):
        print('{0:<4}'.format(satır*sütun), end=' ')
    print()
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
1  2  3  4  5
2  4  6  8  10
3  6  9  12  15
4  8  12  16  20
```

Gözcü Kontrollü Döngüler

Döngüler her zaman bir sayaca bağlı çalıştırılmaz. Bir döngüden çıkış koşulu döngü kodu içerisinde üretilen bir değere bağlı ise bu değere **gözcü değeri** (**sentinel value**) adı verilir. Buradaki gözcü değeri beklenen bir değer dışındaki bir değerdir. Buna örnek olarak klavyeden 0 girilene kadar girilen rakamları toplayan bir program verilmiştir.

```
sayı=-1 # while döngüsüne girmek için -1 olarak ayarlandı. 0 olsaydı girilmezdi!
toplam=0
while sayı!=0: # sentinel value 0
    sayıstr=input('Sayı Giriniz (Bitirmek İçin 0):')
    sayı=int(sayıstr)
    toplam+=sayı
print(toplam)
```

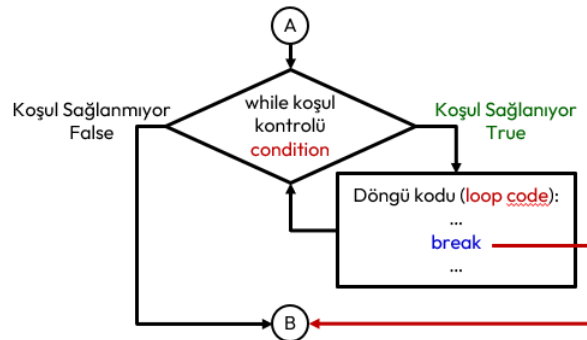
Örneğin program akışı aşağıdaki gibi olacaktır.

```
Sayı Giriniz (Bitirmek İçin 0):10
Sayı Giriniz (Bitirmek İçin 0):30
Sayı Giriniz (Bitirmek İçin 0):40
Sayı Giriniz (Bitirmek İçin 0):50
Sayı Giriniz (Bitirmek İçin 0):0
130
```

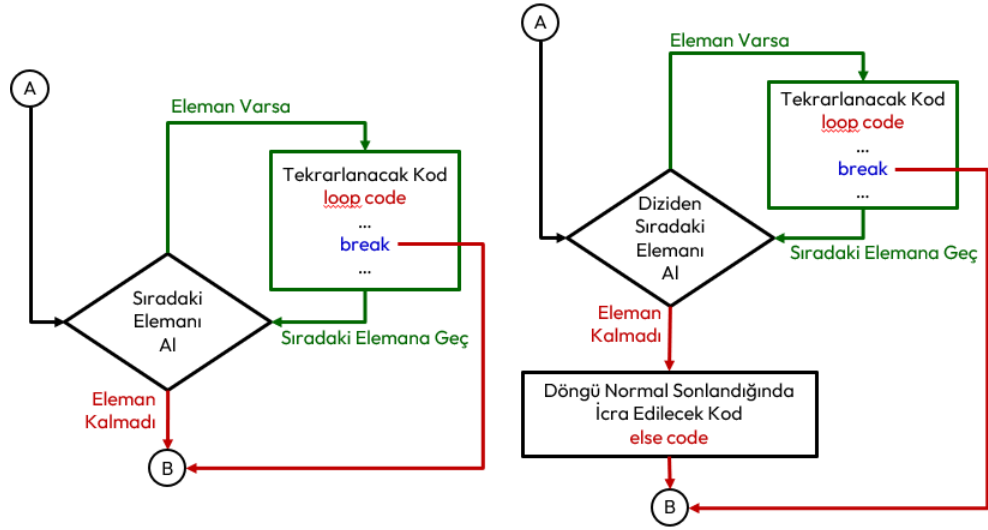
Dallanmalar

Break Talimatı

C dilindeki geleneksel **break** talimatı gibi geçerli döngüyü sonlandırarak döngü sonrasındaki ilk talimatı icra etmek için kullanılır. Bu ifade, bir döngüden ani bir çıkış gerektiren bazı koşullarda kullanılır. Hem **while** hem de **for** döngülerinde kullanılabilir.



Sekil 17. While Döngüsünde Break Talimatı İcra Akışı



Şekil 18. For ve For-Else Döngülerinde Break Talimatı İcra Akışı

Aşağıda bir dizide beşe bölünen ilk elemanı bulan ve döngüyü sonlandıran program örneği verilmiştir.

```
dizi=[1,3,6,7,15,31,27]
for eleman in dizi:
    if eleman%5==0:
        print('Dizide 5 e bölünen ilk eleman:',eleman)
        break
```

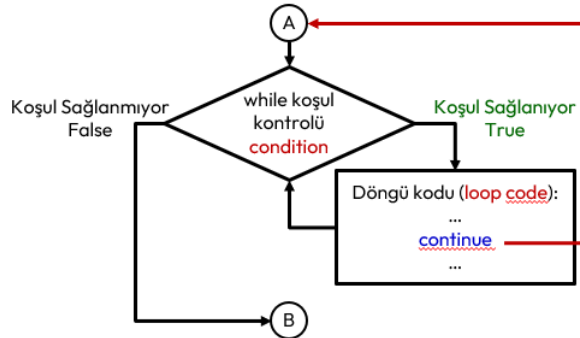
Örneğin program akışı aşağıdaki gibi olacaktır.

Dizide 5 e bölünen ilk eleman: 15

İç içe döngülerde break talimatı yalnızca içinde bulunduğu döngüyü sonlandırır!

Continue Talimatı

Döngülerde tekrarlanan kodun icrasını keserek bir sonraki elemandan döngü kodunun icrasını devam ettirmek için `continue` talimatı kullanılır. Yani geçerli döngünün başına döndürerek bir sonraki yinelemeye başlamak için kullanılır. Hem `while` hem de `for` döngülerinde kullanılabilir.



Şekil 19. While Döngüsünde Continue Talimatı İcra Akışı

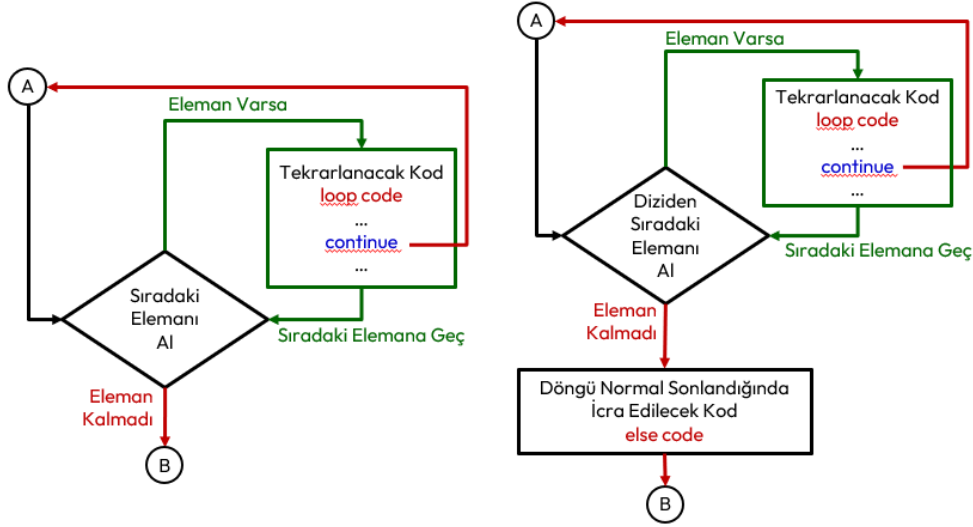
Aşağıda bir sayının asal çarpanlarını bulan programın örneği verilmiştir.

```
sayı=30
print(sayı, ' sayısının asal çarpanları:')
sayaç=2
while sayı>1:
    if sayı%sayaç==0:
        print(sayaç)
        sayı=sayı/sayaç
        continue
    sayaç+=1
```

Örneğin program akışı aşağıdaki gibi olacaktır.

30 sayısının asal çarpanları:

2
3
5



Şekil 20. For ve For-Else Döngülerinde Continue Talimatı İcra Akışı

Bir başka örnek olarak içinde 'h' harfi ayıklanmış bir metin örneği verilebilir;

```
for harf in 'Merhaba':
    if harf=='h':
        continue
    print(harf)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

M
e
r
a
b
a

Pass Talimatı

Bir talimatın gerekli olduğu ancak icra edilmesi istenmediği durumlarda `pass` talimatı kullanılır. Yani `pass` talimatı icra edildiğinde hiçbir şey olmayacağı anlamına gelir. Bu talimat, kod parçasının daha sonra ekleneceği yerlerde de kullanışlıdır.

```
for sayac in range(11):
    if sayac%2==0: #sayac çift ise
        print('{} sayısi çifttir.'.format(sayac))
    else:
        pass # tek olması halinde icra edilecek talimatlar sonra yazılacak.
```

Örneğin program akışı aşağıdaki gibi olacaktır.

0 sayısi çifttir.
2 sayısi çifttir.
4 sayısi çifttir.
6 sayısi çifttir.
8 sayısi çifttir.
10 sayısi çifttir.

Bir sonsuz döngü talimatı aşağıdaki şekilde yazılabilir. Bitirmek için `Ctrl+C` tuşuna basılmalıdır.

```
>>> while True:pass
```

Return talimatı

Bu talimat fonksiyon başlığında incelenecektir.

FONKSİYONLAR

Fonksiyon Nedir?

Bir Python programı, genel olarak, tek başına tüm görevleri yerine getiren tek kod içeren dosyadan veya kod bloğundan oluşmaz. Bu durumda ana problemin büyüklüğüne göre kod bloğumuz uzar, okunabilirliği azalır, müdahale etmek zorlaşır. Bu tip büyük programları kendi içerisinde, her biri özel bir işi çözmek için tasarlanmış parçacıklarından oluşturmak daha mantıklı olacaktır. Yani **böl ve yönet** (**divide and conquer**) tekniği uygulanır.

💡 Büyük bir problemin toptan çözülmesi, problemin parçalar halinde çözülmesinden her zaman daha yorucu ve zordur.

Yazılım geliştirmede problemi büyük ana parçalara ayırırız. Daha sonra bu büyük parçaları daha küçük parçalara ayırarak çözeriz. Buna **yukarıdan aşağıya** (**top down**) yaklaşım adı verilir.

Yapısal Programlama mantığında çözülecek problem genel olarak fonksiyonların birbirini çağırmasıyla yapıldığı anlatılmıştı. Yani kodlaması yapılacak işi birbirini çağıran fonksiyonlar yazarak ana fonksiyondan bu fonksiyonları çağırarak yaparız. Python dili de fonksiyonları destekler. Ayrıca birbiriyle ilişkili fonksiyon, değişken ve nesneler bir modül olarak tasarlanır.

Bir Python modülü, Python tanımları ve ifadeleri içeren bir dosyadır. Bir modül, fonksiyonları, sınıfları ve değişkenleri tanımlayabilir. Bir modül ayrıca çalıştırılabilir kod da içerebilir. Birçok Python modülü vardır.

Python dilinde fonksiyonlar;

- Fonksiyonlar, belirli bir görevi gerçekleştiren bir kod bloğudur.
- Parametrelili alabilir, girdilerle ya da parametresiz bir şeyler yapar ve ardından cevabı verebilir. Kısaca bilgiyi fonksiyona argümanlar ile aktarırız.
- Her fonksiyonun bir kimliği vardır. **Değişken kimliklendirme kuralları fonksiyonlar için de geçerlidir!**
- Bir fonksiyon program içerisinde istenildiği kadar farklı yerlerden ulaşılarak çalıştırılabilir ya da çağrılabilir. Bir başka deyişle tekrar kullanılabilir.
- Bir fonksiyon, çağıran koda bir değer döndürebilir.
- Bir fonksiyon, bir başka fonksiyondan çağrılabilir.

İki tip fonksiyon bulunmaktadır; başlık dosyalarında bize sunulan **hazır fonksiyonlar** ve programcının tanımlayacağı **kullanıcı tanımlı fonksiyonlar** (**user defined function**).

Hazır Fonksiyonlar

Programcıların kullanmaları için, önceden yazılmış `print()` ve `input()` gibi hazır fonksiyonlar Python dilinde **dahili fonksiyon** (**built-in function**) olarak sunulur. Bu fonksiyonlar `builtins` modülünde bulunur. Python yorumlayıcısı, her zaman kullanılabilir olan bu modüle ait bir dizi yerleşik fonksiyona ve veri tiplerine sahiptir. Bunlar `dir(__builtins__)` talimatı ile görüntülenebilir¹³.

```
>>> dir( __builtins__ )
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException', 'BaseExceptionGroup',
'BlockingIOError', 'BrokenPipeError', 'BufferError', 'BytesWarning', 'ChildProcessError',
'ConnectionAbortedError', 'ConnectionError', 'ConnectionRefusedError', 'ConnectionResetError',
'DeprecationWarning', 'EOFError', 'Ellipsis', 'EncodingWarning', 'EnvironmentError',
'Exception', 'ExceptionGroup', 'False', 'FileExistsError', 'FileNotFoundError',
'FloatingPointError', 'FutureWarning', 'GeneratorExit', 'IOError', 'ImportError',
'ImportWarning', 'IndentationError', 'IndexError', 'InterruptedError', 'IsADirectoryError',
'KeyError', 'KeyboardInterrupt', 'LookupError', 'MemoryError', 'ModuleNotFoundError',
```

¹³ <https://docs.python.org/3/library/functions.html>

```
'NameError', 'None', 'NotADirectoryError', 'NotImplemented', 'NotImplementedError', 'OSError',
'OverflowError', 'PendingDeprecationWarning', 'PermissionError', 'ProcessLookupError',
'PythonFinalizationError', 'RecursionError', 'ReferenceError', 'ResourceWarning',
'RuntimeError', 'RuntimeWarning', 'StopAsyncIteration', 'StopIteration', 'SyntaxError',
'SyntaxWarning', 'SystemError', 'SystemExit', 'TabError', 'TimeoutError', 'True', 'TypeError',
'UnboundLocalError', 'UnicodeDecodeError', 'UnicodeEncodeError', 'UnicodeError',
'UnicodeTranslateError', 'UnicodeWarning', 'UserWarning', 'ValueError', 'Warning',
'WindowsError', 'ZeroDivisionError', '_', '_IncompleteInputError', '__build_class__',
'__debug__', '__doc__', '__import__', '__loader__', '__name__', '__package__', '__spec__',
'abs', 'aiter', 'all', 'anext', 'any', 'ascii', 'bin', 'bool', 'breakpoint', 'bytearray',
'bytes', 'callable', 'chr', 'classmethod', 'compile', 'complex', 'copyright', 'credits',
'delattr', 'dict', 'dir', 'divmod', 'enumerate', 'eval', 'exec', 'exit', 'filter', 'float',
'format', 'frozenset', 'getattr', 'globals', 'hasattr', 'hash', 'help', 'hex', 'id', 'input',
'int', 'isinstance', 'issubclass', 'iter', 'len', 'license', 'list', 'locals', 'map', 'max',
'memoryview', 'min', 'next', 'object', 'oct', 'open', 'ord', 'pow', 'print', 'property',
'quit', 'range', 'repr', 'reversed', 'round', 'set', 'setattr', 'slice', 'sorted',
'staticmethod', 'str', 'sum', 'super', 'tuple', 'type', 'vars', 'zip']
```

Bu fonksiyonların bir kısmı anlatılmıştır. Yeri geldikçe diğerleri anlatılacaktır. Bu fonksiyonlara ilişkin yardım dokümanına `help` komutu ile aşağıdaki şekilde ulaşılabilir.

```
>>> help
Welcome to Python 3.13's help utility! If this is your first time using
Python, you should definitely check out the tutorial at
https://docs.python.org/3.13/tutorial/.

Enter the name of any module, keyword, or topic to get help on writing
Python programs and using Python modules. To get a list of available
modules, keywords, symbols, or topics, enter "modules", "keywords",
"symbols", or "topics".

Each module also comes with a one-line summary of what it does; to list
the modules whose name or summary contain a given string such as "spam",
enter "modules spam".

To quit this help utility and return to the interpreter,
enter "q", "quit" or "exit".

help> id
Help on built-in function id in module builtins:

id(obj, /)
    Return the identity of an object.

    This is guaranteed to be unique among simultaneously existing objects.
    (CPython uses the object's memory address.)

help> q

You are now leaving help and returning to the Python interpreter.
If you want to ask for help on a particular object directly from the
interpreter, you can type "help(object)". Executing "help('string')"
has the same effect as typing a particular string at the help> prompt.
>>>
```

Benzer şekilde `math` modülünde ise yaygın matematik fonksiyonları ve sabitleri bulunur. Her Python programı için ön tanımlı olan `builtin` modülü dışındaki modülleri koda dahil etmek için `import` kelimesi kullanılır. Örnek olarak aşağıda `math` modülü verilmiştir.

```
# ÜÇGENİN ALANI - HERON FORMÜLÜ
import math
kenar1str=input('Üçgenin birinci kenarını Giriniz:')
kenar1=float(kenar1str)
```



```
kenar2str=input('Üçgenin ikinci kenarını Giriniz:')
kenar2=float(kenar2str)
kenar3str=input('Üçgenin üçüncü kenarını Giriniz:')
kenar3=float(kenar3str)
if (kenar1+kenar2)<=kenar3 or (kenar1+kenar3)<=kenar2 or (kenar2+kenar3)<=kenar1:
    print('Böyle bir üçgen olamaz!')
else:
    kenarlarToplamınınYarısı=(kenar1+kenar2+kenar3)/2
    alan=math.sqrt( kenarlarToplamınınYarısı*
                    (kenarlarToplamınınYarısı-kenar1)*
                    (kenarlarToplamınınYarısı-kenar2)*
                    (kenarlarToplamınınYarısı-kenar3) )
    print('Üçgenin Alanı:',alan)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Üçgenin birinci kenarını Giriniz:3.0
Üçgenin ikinci kenarını Giriniz:4.0
Üçgenin üçüncü kenarını Giriniz:5.0
Üçgenin Alanı: 6.0
```

Matematik modülü fonksiyon ve sabitleri aşağıda talimatla öğrenilebilir¹⁴. Benzer şekilde **aktarılan** (**import**) edilerek tüm modül fonksiyon, sınıf ve sabitleri listelenebilir.

```
>>> import math
>>> dir (math)
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acosh', 'asin',
'asinh', 'atan', 'atan2', 'atanh', 'cbrt', 'ceil', 'comb', 'copysign', 'cos', 'cosh',
'degrees', 'dist', 'e', 'erf', 'erfc', 'exp', 'exp2', 'expm1', 'fabs', 'factorial', 'floor',
'fma', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf', 'isclose', 'isfinite',
'isinf', 'isnan', 'isqrt', 'lcm', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf',
'nan', 'nextafter', 'perm', 'pi', 'pow', 'prod', 'radians', 'remainder', 'sin', 'sinh',
'sqrt', 'sumprod', 'tan', 'tanh', 'tau', 'trunc', 'ulp']
```

Aynı modüldeki sabitlerin kullanımına örnek olarak yarıçapı girilen bir dairenin çevre ve alan hesabı verilebilir.

```
import math
yarıçapstr=input('Yarıçapı Giriniz:')
yarıçap=float(yarıçapstr)
print('Dairenin Çevresi:',2*math.pi*yarıçap)
print('Dairenin Alanı:',math.pi*yarıçap*yarıçap)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Yarıçapı Giriniz:12.0
Dairenin Çevresi: 75.39822368615503
Dairenin Alanı: 452.3893421169302
```

Yukarıdaki örnekte `math.pi` sabitini kullanmak için tüm `math` modülü **aktarılmıştır** (**import**). Eğer bir modülden sadece belirlenen bir üye aktarılacak ise `from` talimatı kullanılır. Verilen örnek bu kapsamda aşağıdaki şekilde yazılabilir.

```
from math import pi
yarıçapstr=input('Yarıçapı Giriniz:')
yarıçap=float(yarıçapstr)
print('Dairenin Çevresi:',2*pi*yarıçap)
print('Dairenin Alanı:',pi*yarıçap*yarıçap)
```

Dahili modül yanında, Python dili birçok modülü bize standart olarak sunar¹⁵.

¹⁴ <https://docs.python.org/3/library/math.html#math.sqrt>

¹⁵ <https://docs.python.org/3/library/index.html>

Kullanıcı Tanımlı Fonksiyonlar

Python dilinde dahili olan fonksiyonlar ya da `math` modülündeki fonksiyonlar gibi kendi ihtiyaçlarımızı karşılayan yeniden kullanılabilir fonksiyonlar tanımlayabilir ve modüller oluşturabiliriz.

Fonksiyon Tanımlama

Bir Python fonksiyonu aşağıdaki kurallar ile tanımlanır;

1. Fonksiyon blokları `def` anahtar sözcüğüyle başlar, bunu fonksiyon adı ve parantezler `()` izler.
2. Herhangi bir giriş parametresi veya argümanı bu parantezlerin içine yerleştirilmelidir. Parametreleri bu parantezlerin içine de tanımlayabilirsiniz.
3. Her fonksiyon içindeki kod bloğu iki nokta üst üste `:` ile başlar ve girintilidir.
4. Fonksiyon bloğunun ilk satırında fonksiyonun ne yaptığını kısaca anlatan bir dokümantasyon metni **dizgi** olarak (**docstring**) yer alabilir. Bu **docstring**, modülleri, sınıfları, fonksiyonları ve yöntemleri belgelemek için kullanılan çok satırlı bir yorumdur. Bileşenin ilk talimatı olmalıdır.
5. Fonksiyon bloğu içinde `return [expression]` ifadesi bir fonksiyondan çıkar ve isteğe bağlı olarak çağırana bir ifade geri iletebiliriz. Argümanı olmayan bir `return` ifadesi `return None` ile aynıdır.

Fonksiyon tanımına ilişkin sözde kod aşağıda verilmiştir.

```
def fonksiyon-kimliği( parametreler ):
    "Fonksiyon Tanımı (docstring)"
    # fonksiyon gövdesi
    return [expression]
```

Aşağıda fonksiyon tanımlamalarına ve bu fonksiyonların çağırılmasına kısa bir örnek verilmiştir.

```
#Fonksiyon Tanımları
def ikiSayıTopla(x,y): # Parametrelili fonksiyon
    "Bu Fonksiyon Parametreleri Toplar" #Fonksiyon Dokümantasyon Metni
    toplam=x+y
    return toplam
def sayıYaz(sayı): # Geri değer döndürmeyen fonksiyon
    print(sayı)
    return # yazılmayabilir
def sayıOku(): # Parametresiz fonksiyon
    str=input('Bir Sayı Giriniz:')
    return float(str)
#Artık Tanımlanmış Fonksiyonları Çağırabiliriz
sayı1=sayıOku() # sayıOku fonksiyonu çağırılıyor
sayı2=sayıOku() # sayıOku fonksiyonu tekrar çağırılıyor
sonuc=ikiSayıTopla(sayı1,sayı2) # ikiSayıTopla fonksiyonu çağırılıyor
sayıYaz(sonuc) # sayıYaz fonksiyonu çağırılıyor
print(ikiSayıTopla.__doc__) # ikiSayıTopla fonksiyonunun tanımı nedir?
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Bir Sayı Giriniz:10
Bir Sayı Giriniz:20
30.0
Bu Fonksiyon Parametreleri Toplar
```

Dokümantasyon metni (**docstring**) için günümüzde PEP 257 standardı kullanılmaktadır. Aşağıda örneği verilmiştir;

```
def toplam(a, b) :
    """
    İki sayıyı toplar.

    Args:
        a : İlk sayı
        b : İkinci sayı
```

```

Returns:
    İki sayının toplamı

Examples:
    >>> toplam(2, 3)
    5
"""
return a + b

```

Fonksiyonlara Parametre Geçirme

Bir fonksiyonun tanımlanması sırasında parantez içinde bildirilen değişkenlerin listesi **resmi argümanlardır** (**formal argument**) ve bu argümanlar **konumsal argümanlar** (**positional argument**) olarak da bilinir. Bir fonksiyon istenildiği kadar resmi argümanla tanımlanabilir. Fonksiyonlara geçirilen parametreler tanımlı ise çağrılırken mutlaka argüman verilmelidir.

```

def negatif(x):
    "Bu fonksiyon parametresiz çağrılmaz."
    return -x;
sonuc=negatif(-1)
print(sonuc) #1
# sonuc=negatif() hata verir.
a=-1;
sonuc=negatif(a) #bir değişken de argüman olarak verilebilir
print(sonuc) #1

```

Fonksiyonlara parametre geçirirken bazı parametreler **ön tanımlı argümanlara** (**default argument**) sahip olabilir.

```

# Fonksiyon Tanımı
def ikiSayıTopla(x,y=1): # y parametresi için 1 argümanı ön tanımlıdır
    "Bu Fonksiyon Parametreleri Toplar" #Fonksiyon Dokümantasyon Metni
    toplam=x+y
    return toplam
# Artık Fonksiyonu Kullanabiliriz
sonuc=ikiSayıTopla(10) #ikinciargman 1 kabul edildi
print(sonuc) #11

```

Tüm parametreler için de ön tanımlı argüman belirlenebilir.

```

def ikiSayıTopla(x,y=1): # y parametresi için 1 argümanı ön tanımlıdır
    "Bu Fonksiyon Parametreleri Toplar" #Fonksiyon Dokümantasyon Metni
    toplam=x+y
    return toplam
def ikinciSayıyıÇıkar(x=10,y=1):
    "Bu Fonksiyon Birinci Parametreden İkincisini Çıkarır"
    fark=x-y
    return fark
sonuc=ikiSayıTopla(10) #ikinciargman 1 kabul edildi
print(sonuc) #11
sonuc=ikinciSayıyıÇıkar() #birinci argüman 10, ikincisi 1 kabul edildi
print(sonuc) #9

```

Parametre ismi verilerek argümanlar fonksiyona geçiriliyorsa bu argümanlara **anahtar kelime argümanlar** (**keyword argument**) adı verilir;

```

def ikiSayıTopla(x,y=1): # y parametresi için 1 argümanı ön tanımlıdır
    "Bu Fonksiyon Parametreleri Toplar" #Fonksiyon Dokümantasyon Metni
    toplam=x+y
    return toplam
sonuc=ikiSayıTopla(x=10) #ikinciargman 1 kabul edildi
print(sonuc) #11

```

```
sonuc=ikiSayıTopla(x=10, y=20)
print(sonuc) #30
sonuc=ikiSayıTopla(y=10, x=20)
print(sonuc) #30
```

Fonksiyonlardaki argüman listesindeki değişkenleri değer geçirmek için anahtar kelime olarak kullanabilirsiniz. **Anahtar kelime argümanlarının** (keyword argument) kullanımı isteğe bağlıdır. Ancak, fonksiyonun argümanları yalnızca anahtar kelimeyle kabul etmesini zorlayabilirsiniz. Bunun için argümanlar listesinden önce bir yıldız işareti (*) koymalısınız. Dahili print() fonksiyonu yalnızca anahtar sözcük argümanlarına bir örnektir. Parantez içinde yazdırılacak ifadelerin listesini verebilirsiniz. Yazdırılan değerler varsayılan olarak bir boşlukla ayrılır. sep argümanını kullanarak bunun yerine başka herhangi bir ayırma karakteri belirtebilirsiniz.

```
print (1,2,3,4,5, sep='-') #1-2-3-4-5
print ("Bir","İki","Üç","Dört", sep=",") #Bir,İki,Üç,Dört
```

Bir argümanı yalnızca anahtar kelimedenden oluşan bir argüman yapmak için, kullanıcı tanımlı fonksiyonu oluştururken argümanın önüne yıldız işareti (*) koyulur.

```
from math import pow
def kuvvet(taban,*, üs):
    sonuc = pow(taban,üs)
    return sonuc

hesap = kuvvet(3, üs=4)
print(hesap) #81.0
```

Fonksiyon, bir veya daha fazla argümanın anahtar sözcüklerle değerlerini kabul edemeyecek şekilde tanımlanabilir. Bu tür argümanlara **yalnızca konumsal argümanlar** (positional-only argument) denir. Bir argümanı yalnızca konumsal yapmak için bölü (/) sembolünü kullanılır. Bu sembolden önceki tüm argümanlar yalnızca konumsal olarak ele alınacaktır.

```
from math import pow
def kuvvet(taban, üs, /):
    sonuc = pow(taban,üs) #birinci parametre taban, ikinci parametre üs kabul edilir.
    return sonuc

hesap = kuvvet(3,4)
print(hesap) #81.0
hesap =kuvvet(2,8)
print (hesap) #256.0
```

Bir başka örnek;

```
def hepsiniYaz(x,/,y,*,z): # birinci parametre her zaman ilk sırada,
                           # z de anahtar kelime zorunlu
    print(x,y,z)
    return
hepsiniYaz(100,10,z=20) # 100 10 20
hepsiniYaz(100,y=10,z=20) # 100 10 20
hepsiniYaz(100,z=20,y=10) # 100 10 20
```

Değişken sayıda argüman kabul edebilen bir fonksiyon tanımlamak isteyebiliriz. Bu argümanlara **keyfi sayıda argüman** (arbitrary argument) adı verilir. Ayrıca, değişken sayıda argüman konumsal veya anahtar kelime argümanları olabilir.

```
def hepsinitopla(*args):
    s=0
    for x in args:
        s=s+x
    return s
sonuç = hepsinitopla(3,5,7,9)
print (sonuç) #24
```

```
sonuç = hepsinitopla(150,300,450)
print (sonuç) #900
```

Keyfi sayıda argüman yanında zorunlu argüman da tanımlanabilir;

```
def katsayıİleÇarpTopla(katsayı, *args):
    s=0
    for x in args:
        s=s+x*katsayı
    return s
sonuç = katsayıİleÇarpTopla(10,3,5,7,9)
print (sonuç) #240
sonuç = katsayıİleÇarpTopla(3,150,300,450)
print (sonuç) #2700
```

Keyfi sayıda anahtar kelime argümanı da tanımlayabiliriz;

```
def keyfianahtarkelimeargüman(**kwargs):
    for k,v in kwargs.items():
        print ("{}:{}".format(k,v))
keyfianahtarkelimeargüman(adı='İlhan',soyadı='ÖZKAN',yaş=50,boy=1.8)
```

Bu programı çalıştırdığımızda aşağıdaki çıktı elde edilir;

```
adı:İlhan
soyadı:ÖZKAN
yaş:50
boy:1.8
```

Fonksiyon Açıklamaları

Python dilinde veri tipleri dinamik olarak belirlenir. Bazı fonksiyonlarda işlenecek veriler ile geri döndürülecek değerler için veri tipi belirlenmesi gerekir. Bunun için fonksiyon **açıklama (annotation)** özelliği kullanılır. Bir fonksiyon tanımında bildirilen argümanlar ve ayrıca dönüş veri tipi hakkında ek açıklayıcı meta veriler eklemenize olanak tanır.

```
def topla(x, y): #hiçbir açıklaması olmayan fonksiyon
    geçici = x+y
    return geçici
print(topla(10,20))
print(topla.__annotations__)
```

Dönüş veri tipi için de açıklama verebilirsiniz. Fonksiyon parantezinden sonra ve iki nokta üst üste sembolünden önce, eksi ve büyük karakterinden oluşan bir ok (->) koyulur. Ve dönüş tipi belirtilir.

```
def topla(x, y):
    geçici = x+y
    return geçici
print(topla(10,20)) #30
print(topla.__annotations__) #{}
def intTopla(x:int, y:int): #parametreleri int olan topla fonksiyonu
    geçici = x+y
    return geçici
print(intTopla(10,20)) #30
print(intTopla.__annotations__) #{'x': <class 'int'>, 'y': <class 'int'>}
def intToplaIntDöndür(x:int, y:int) -> int: #parametreleri ve geri döndürdüğü değer
                                         #int olan topla fonksiyonu
    geçici = x+y
    return geçici
print(intToplaIntDöndür(10,20)) #30
print(intToplaIntDöndür.__annotations__)
#{'x': <class 'int'>, 'y': <class 'int'>, 'return': <class 'int'>}
```

Değişken Kapsamı

Bir değişkenin **kapsamı** (**scope**), değişkenin kullanıcı tarafından erişilebilir olduğu yerler olarak tanımlanır.

Bir **yerel değişken** (**local variable**), belirli bir fonksiyon veya kod bloğu içinde tanımlanır. Yalnızca tanımlandığı fonksiyon veya blok tarafından erişilebilir ve sınırlı bir kapsamı vardır. Başka bir deyişle, yerel değişkenlerin kapsamı tanımlandıkları fonksiyonla sınırlıdır ve bu fonksiyon dışından bunlara erişmeye çalışmak hataya yol açacaktır.

```
def hepsinitopla(*args):
    s=0 #yerel değişken
    for x in args:
        s=s+x
    print(locals()) # {'args': (10, 20, 30, 40), 's': 100, 'x': 40}
    return s
#fonksiyon dışında s ve x değişkenlerine ulaşılamaz.
sonuc= hepsinitopla (10,20,30,40)
```

Bir **küresel değişkene** (**global variable**) programın herhangi bir bölümünden erişilebilir ve herhangi bir fonksiyon veya kod bloğunun dışında tanımlanır. Herhangi bir blok veya fonksiyona özgü değildir.

```
programcıAdı = 'İlhan ÖZKAN'
uyarlama=2.3
def birFonksiyon():
    print("Programcı Adı:", programcıAdı)
    print("Uyarlama (version):", uyarlama)
    print(locals())
def birBaşkaFonksiyon():
    temp=0
    print("Programcı Adı:", programcıAdı)
    print(temp)
    print(locals())
print(programcıAdı)
birFonksiyon()
birBaşkaFonksiyon()
print(globals())
```

Programın çıktısı aşağıdaki gibi olur;

```
İlhan ÖZKAN
Programcı Adı: İlhan ÖZKAN
Uyarlama (version): 2.3
{}
Programcı Adı: İlhan ÖZKAN
0
{'temp': 0}
{'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__':
<_frozen_importlib_external.SourceFileLoader object at 0x7ffba8057f80>, '__spec__': None,
'__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>, '__file__':
'/home/cg/root/685b2ccdb7180/main.py', '__cached__': None, 'programcıAdı': 'İlhan ÖZKAN',
'uyarlama': 2.3, 'birFonksiyon': <function birFonksiyon at 0x7ffba804a2a0>,
'birBaşkaFonksiyon': <function birBaşkaFonksiyon at 0x7ffba7e4d300>}
```

Özyinelemeli Fonksiyonlar

Özyineleme (**recursion**), bir fonksiyonun kendi çağrısını yapma tekniğidir. Bu teknik, karmaşık sorunları çözülmesi daha kolay basit sorunlara ayırmanın bir yolunu sağlar. Özyineleme, **yineleme** (**iteration**), **döngüler** (**loop**) veya tekrar yerine geçebilecek çok güçlü bir tekniktir.

Özyinelemeli (recursive) algoritmalarda, tekrarlar fonksiyonun kendi kendisini kopyalayarak çağırması ile elde edilir. Bu kopyalar işlerini bitirdikçe kaybolur. Bir problemi özyineleme ile çözmek için problem iki ana parçaya ayrılır.

1. Cevabı kesin olarak bildiğimiz **temel durum (base case)**
2. Cevabı bilinmeyen ancak cevabı yine problemin kendisi kullanılarak bulunabilecek durum.

Faktöriyel örneğini inceleyelim; Matematikte, negatif olmayan bir tam sayı olan n 'nin faktöriyeli, $n!$ ile gösterilir ve n 'den küçük veya ona eşit olan tüm pozitif tam sayıların çarpımıdır.

$$n! = n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot 3 \cdot 2 \cdot 1$$

Negatif olmayan bir tam sayı olan n 'nin faktöriyeli aynı zamanda n 'nin bir sonraki daha küçük faktöriyelle çarpımına eşittir. Yani problemin tanımı yine kendisini içerir:

$$n! = n \cdot (n - 1)!$$

Örneğin;

$$5! = 5 \cdot 4! = 5 \cdot 4 \cdot 3! = 5 \cdot 4 \cdot 3 \cdot 2! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$$

Aşağıda faktöriyel için girilen sayıdan sonra tüm sayıların çarpımıyla hesaplayan bir fonksiyon yazılmıştır.

```
def faktöriyel(n):
    if n>=1:
        çarpım=1
        for n in range(2,n+1):
            çarpım=çarpım*n
        return çarpım
    else:
        return 1

faktöriyel(5) #120
faktöriyel(7) #5040
```

Sıfır sayısı da pozitif bir sayıdır ama sonucum sıfır çıkmaması için 0 sayısının faktöriyeli 1 kabul edilir. Bu aslında faktöriyel problemi için **temel durumdur (base case)** ve **boş ürün (empty product)** kuralı olarak bilinir. Kısaca faktöriyel fonksiyonu aşağıdaki şekilde gösterilir;

$$n \in \mathbb{N} \text{ olmak üzere } f(n) = n! = \begin{cases} 1, & n = 0 \\ n \cdot f(n - 1), & n > 0 \end{cases}$$

Özyinelemeli (recursive) olarak ise bu fonksiyon aşağıdaki şekilde kodlanabilir;

```
def faktöriyel(n):
    if n==1:
        return 1 #base case
    else:
        return n*faktöriyel(n-1)
```

Girilen taban ve üs ile kuvvet hesaplama fonksiyonu özyinelemeli olarak aşağıda verilmiştir;

```
def kuvvet(taban,üs):
    if taban==0:
        return 0 #base case
    if üs==0:
        return 1 #base case
    return taban*kuvvet(taban,üs-1)

kuvvet(2,3) #8
kuvvet(3,2) #9
```

Kendi Modülümüzü Oluşturma

Bir modül, tanımları ve ifadeleri içeren içe aktarılabilir bir Python dosyasıdır. Modüller, bir `.py` dosyası oluşturarak oluşturulabilir. Yukarıdaki örneğimizde fonksiyonları `Fonksiyonlar.py` dosyasında yazalım;

```
#Fonksiyonlar.py
"""Bu Modül Okuma Yazma ve Toplama Fonksiyonu Bulunur.""" # Modül Dokümantasyon Metni
def ikiSayıTopla(x,y): # Parametrelili fonksiyon
    "Bu Fonksiyon Parametreleri Toplar" #Fonksiyon Dokümantasyon Metni
    toplam=x+y
    return toplam
def sayıYaz(sayı): # Geri değer döndürmeyen fonksiyon
    print(sayı)
    return #yazılmayabilir
def sayıOku(): # Parametresiz fonksiyon
    str=input('Bir Sayı Giriniz:')
    return float(str)
```

Bir modüldeki fonksiyonlar, modülün `içe aktarılmasıyla` (`import`) kullanılabilir. Bu durumda fonksiyonları çağıran kodumuz aşağıdaki şekilde olacak ve aynı çıktıyı verecektir;

```
import Fonksiyonlar
sayı1=Fonksiyonlar.sayıOku() # sayıOku fonksiyonu çağrılıyor
sayı2=Fonksiyonlar.sayıOku() # sayıOku fonksiyonu tekrar çağrılıyor
sonuc=Fonksiyonlar.ikiSayıTopla(sayı1,sayı2) # ikiSayıTopla fonksiyonu çağrılıyor
Fonksiyonlar.sayıYaz(sonuc) # sayıYaz fonksiyonu çağrılıyor
print(Fonksiyonlar.ikiSayıTopla.__doc__) # ikiSayıTopla fonksiyonunun tanımı nedir?
```

Oluşturduğunuz modüller, bunları içeri `aktardığınız` (`import`) dosya ile aynı klasörde olmalıdır. Ancak, önceden dahil edilmiş modüllerle birlikte Python `lib` klasörüne de koyabilirsiniz, ama mümkünse bundan kaçınılmalıdır.

Bunların dışında modüllere takma isim de verilebilir;

```
import Fonksiyonlar as Fn
sayı1=Fn.sayıOku() # sayıOku fonksiyonu çağrılıyor
sayı2=Fn.sayıOku() # sayıOku fonksiyonu tekrar çağrılıyor
sonuc=Fn.ikiSayıTopla(sayı1,sayı2) # ikiSayıTopla fonksiyonu çağrılıyor
Fn.sayıYaz(sonuc) # sayıYaz fonksiyonu çağrılıyor
print(Fn.ikiSayıTopla.__doc__) # ikiSayıTopla fonksiyonunun tanımı nedir?
```

Modüle ilişkin dokümantasyon metnini öğrenmek için aşağıdaki kod yazılabilir;

```
import Fonksiyonlar as Fn
print( Fn.__doc__ )
```

Örneğin program akışı aşağıdaki gibi olacaktır.

Bu Modül Okuma Yazma ve Toplama Fonksiyonu Bulunur.

Paket (Package) yönetimi

Python ilinde `modül` (`module`), `.py` uzantılı sınıflar, fonksiyonlar gibi nesneler içeren bir Python dosyasıdır. Python dilinde `paketler` (`package`), bir veya daha fazla modül dosyası içeren bir klasördür ve bu klasör `__init__.py` adlı özel bir dosya içerir. Bu dosya boş olabilir ancak paket listesini içerebilir.

Örnek olarak `hesap` adı bir Python paketi aşağıda verilmiştir;

<pre>D:\ ├── PythonKlasoru\ │ ├── hesap\ │ │ ├── __init__.py │ │ ├── polinom.py │ │ └── alan.py</pre>	<pre>→ → Python Betik Dosyalarının Bulunduğu Klasör → Paket Adını Belirleyen Klasör → Paket Listesini İçeren Özel Dosya → paket içindeki betik dosyası → paket içindeki bir başka betik dosyası</pre>
---	---



alanOrnek.py
polinomOrnek.py

→ Paket dışındaki betik dosyası
→ Paket dışındaki bir başka betik dosyası

Polinom modülü içeriği aşağıdaki gibidir;

```
# polinom.py
"""Bu Modül aşağıda terim katsayıları verilen polinom değerini hesaplar."""
def polinomDegeri(x,*katsayilar):
    """İlk Parametre x, ikinci parametre sabit terim katsayısı ..."""
    deger=0.0;
    indis=0
    for katsayi in katsayilar:
        deger+=katsayi*x**indis
        indis+=1
    return deger
```

Alan hesabı modülü içeriği aşağıdaki gibidir;

```
# alan.py
"""Bu Modül çeşitli geometrik şekillerin alanını hesaplar."""
from math import sqrt
def kareninAlani(x):
    """bir kenarı bilinen karenin alanı"""
    return x*x
def dikdörtgeninAlani(x,y):
    """eni ve boyu bilinen dikdörtgenin alanı"""
    return x*y
def ucgeninAlani(x,y,z):
    """Üç kenarı bilinen üçgenin alanı (heron formülü)"""
    s= (x+y+z)/2
    return sqrt(s*(s-x)*(s-y)*(s-z))
```

Özel dosyanın (__init__.py) içeriği aşağıdaki gibidir;

```
from .polinom import polinomDegeri
from .alan import kareninAlani,dikdörtgeninAlani,ucgeninAlani
```

Paket içerisindeki alan hesabını kullanacak örnek kod aşağıdaki gibidir;

```
# alanOrnek.py
from hesap.alan import ucgeninAlani
print (" 5,6,7 Üçgenin alanı:", ucgeninAlani(5,6,7))
# 5,6,7 Üçgenin alanı: 14.696938456699069
```

Paket içerisindeki polinom hesabını kullanacak örnek kod aşağıdaki gibidir;

```
# polinomOrnek.py
from hesap.polinom import polinomDegeri
print ("x=1 için Polinom Değeri :", polinomDegeri(1,5.0,1.0)) # x=1, 5x**0+1x**1
#x=1 için Polinom Değeri : 6.0
```

Eğer bu paketi sistem genelinde kullanmak için **kurmak** (**setup**) istersek hesap klasörü ile aynı klasörde olacak şekilde **setup.py** dosyası hazırlarız.

D:\	→
└─ PythonKlasoru\	→ Python Betik Dosyalarının Bulunduğu Klasör
└─ hesap\	→ Paket Adını Belirleyen Klasör
└─ __init__.py	→ Paket Listesini İçeren Özel Dosya
└─ polinom.py	→ paket içindeki betik dosyası
└─ alan.py	→ paket içindeki bir başka betik dosyası
└─ alanOrnek.py	→ Paket dışındaki betik dosyası
└─ polinomOrnek.py	→ Paket dışındaki bir başka betik dosyası
└─ setup.py	→ Paketi kurulabilir yapmak için betik dosyası

Dosyanın içeriği aşağıdaki gibi olabilir;

```
#setup.py
from setuptools import setup
setup(name='hesap',
      version='0.1',
      description='Alan ve polinom hesabını içerir',
      url='#',
      author='İlhan Özkan',
      author_email='hoydabre@gmail.com',
      license='MIT',
      packages=['hesap'],
      zip_safe=False)
```

Yukarıdaki gibi önceden hazırlanmış Python paketlerini yüklemek için **PIP** (Package Installer for Python) aracı kullanılır. Bir paketi yüklemek için komut satırında aşağıdaki komut çalıştırılır;

```
C:\Users\ILHANOZKAN>pip install <paket adı>
// Ya da
C:\Users\ILHANOZKAN>python -m pip install <paket adı>
```

Hazırladığımız hesap paketini paket yükleme aracı kullanılarak kurulumu aşağıdaki şekilde yapılır;

```
D:\PythonKlasoru> pip install .
Processing d:\PythonKlasoru
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: hesap
  Building wheel for hesap (pyproject.toml) ... done
  Created wheel for hesap: filename=hesap-0.1-py3-none-any.whl size=2016
sha256=a6827110636b8bb072e3431f1ad7664ba56081bbaec495933802a065138c0b41
  Stored in directory: C:\Users\ILHANOZKAN\AppData\Local\Temp\pip-ephem-wheel-cache-
fzr6qquh\wheels\98\62\e3\ed2d7957ca1514c88218e667de8a81a5632b3b9b01640ecca1
Successfully built hesap
Installing collected packages: hesap
Successfully installed hesap-0.1

[notice] A new release of pip is available: 25.1.1 -> 25.2
[notice] To update, run: python.exe -m pip install --upgrade pip
D:\PythonKlasoru>
```

Aynı paketi kaldırmak istersek aşağıdaki komut çalıştırılabilir;

```
D:\PythonKlasoru> pip uninstall hesap
Found existing installation: hesap 0.1
Uninstalling hesap-0.1:
  Would remove:
    c:\users\ILHANOZKAN\appdata\local\programs\python\python313\lib\site-packages\hesap-
0.1.dist-info\*
    c:\users\ILHANOZKAN\appdata\local\programs\python\python313\lib\site-packages\hesap\*
Proceed (Y/n)?
Successfully uninstalled hesap-0.1
D:\PythonKlasoru>
```

Lambda İfadeleri

Lambda ifadeleri (lambda expression) anonim fonksiyonlardır, yani bir adları yoktur. Bildiğimiz gibi, `def` anahtar kelimesi Python diline normal bir fonksiyonu tanımlamak için kullanılır. Benzer şekilde, `lambda` anahtar kelimesi de anonim bir fonksiyonu tanımlamak için kullanılır.

Lambda ifadelerinin sözdizimi aşağıdaki şekildedir;

```
lambda argümanlar : ifade
```

Aşağıda **metin** (string) olarak argüman alan üç farklı lambda fonksiyonu tanımlanmıştır;

```
str = "Python Esnek Bir Dildir!"
küçüğeCevir = lambda s: s.upper()
print(küçüğeCevir(str)) #PYTHON ESNEK BIR DILDIR!
buyugeCevir = lambda s: s.lower()
print(buyugeCevir(str)) #python esnek bir dildir!
birlestir = lambda birinci,ikinci: birinci+" "+ikinci #iki parametrelili lamda
print(birlestir("İlhan","Özkan")) #İlhan Özkan
```

Bir lamda ifadesi `if` içerebilir. Buradaki kullanım `if` talimatından farklı olduğuna dikkat ediniz.

```
durum = lambda x: "Pozitif" if x > 0 else "Negatif" if x < 0 else "Sıfır"
print(durum(1)) #Pozitif
print(durum(-1)) #Negatif
print(durum(0)) #Sıfır
tekmiçiftmi=lambda x: "Çift" if x % 2 == 0 else "Tek"
print(tekmiçiftmi(2)) #Çift
print(tekmiçiftmi(1)) #Tek
```

Lamda ifadelerini Liste (list) başlığında anlatılan liste kavrama (list comprehension) ile birleştirmek mümkündür. Böylece dönüşümleri verilere özlü bir şekilde uygulayabiliriz.

```
liste = [1, 2, 3, 4, 5, 6]
onkat = [lambda arg=x: arg * 10 for x in liste]
for i in onkat:
    print(i(),end=' ')
# 10 20 30 40 50 60
print()
```

Bir lamda fonksiyonunda demet (tuple) kullanılarak birden fazla değer geri döndürülebilir;

```
hesaplama = lambda x, y: (x + y, x * y)
sonuclar = hesaplama(5, 6)
print(sonuclar) #(11,30)
```

Python dilindeki `filter()` fonksiyonu, argüman olarak bir fonksiyon ve bir liste alır. Bu fonksiyon listedeki uygun elemanlar için `True` döndürerek filtreleme sağlar.

```
liste = [1, 2, 3, 4, 5, 6]
ciftler = filter(lambda x: x % 2 == 0, liste) # İlk parametre lamda fonksiyonudur
print(type(ciftler)) #<class 'filter'>
print(list(ciftler)) #[2, 4, 6]
```

Python dilindeki `map()` fonksiyonu, bir fonksiyon ve bir listeyi argüman olarak alır. Fonksiyon, bir lambda fonksiyonuyla çağrılır ve her bir öğe için bu fonksiyon, lamda ile değiştirilmiş öğeleri içeren yeni bir liste döndürülür.

```
liste = [1, 2, 3, 4, 5, 6]
yuzkat = map(lambda x: x *100, liste)
print(type(yuzkat)) #<class 'map'>
print(yuzkat) #[100, 200, 300, 400, 500, 600]
```

Python dilindeki `reduce()` fonksiyonu, bir fonksiyon ve bir listeyi argüman olarak alır. Fonksiyon, bir yinelemeli fonksiyonla çağrılır ve yeni bir indirgenmiş tek sonuç döndürülür. Bu, yinelemeli fonksiyon çiftleri üzerinde tekrarlayan bir işlem gerçekleştirir. `reduce()` fonksiyonu `functools` modülüne aittir.

```
from functools import reduce
liste = [1, 2, 3, 4, 5, 6]
elemanlaritopla = reduce(lambda x,y: x+y, liste) #lamda fonksiyonu ardışık iki üyeyi toplar
print(type(elemanlaritopla)) #<class 'int'> #sonuç tek bir int değeridir.
print(elemanlaritopla) #21
```

Hem `lamda` hem de `def` kelimeleri, fonksiyonları tanımlamak için kullanılabilir, ancak farklı amaçlara hizmet ederler. `def` standart yeniden kullanılabilir (reusable) fonksiyonlar oluşturmak için kullanılırken,

lamda fonksiyonları çoğunlukla geçici olarak ihtiyaç duyulan kısa ve öz işlemleri gerçekleştiren anonim fonksiyonlar için kullanılır.

NESNE YÖNELİMLİ PROGRAMLAMA

Nesne Yönelimli Programlama Paradigması

Python, sadece popüler bir **betik** (**script**) dili olarak değil, aynı zamanda nesne yönelimli programlama paradigmasını da destekler. Bu paradigmada **nesneler** (**object**) programın temel yapıtaşlarıdır. Nesneler; **durum** (**state**) ve **davranışlara** (**behavior**) sahiptir. Programlama, nesnelerin birbirlerine **ileti göndermesiyle** (**message-passing**) yapılır. Emreden paradigmanın aksine bu paradigmada programlama, nesnelerin birbirlerine ileti göndermesi ile yapılır.

Nesne yönelimli programlama paradigması aşağıdaki kavramlar üzerine kuruludur;

1. **Sınıf** (class): Nasıl ki dünyamızda evler bir **mimari plan** (**blueprint**) üzerinden inşa ediliyor, **nesneler** (**object**) de bir plan üzerinden inşa edilir. Nesnelere ilişkin verilerin tutulduğu **durumların** (**state**) ve nesnelerin göstereceği **davranışlarının** (**behavior**) tanımlandığı bu plana **sınıf** (**class**) adı verilir.
2. **Nesne** (object): Bir mimari plandan birçok ev yapılabileceği gibi, bir sınıftan birçok nesne imal edilebilir. Sınıfta uygun olarak imal edilen her **nesne** (**object**) sınıfın bir **örneğidir** (**instance**). İmal edilen her bir nesnenin kendine özgü verisi bulunur. **Durumlar** (**state**), nesnelere ilişkin verilerin tutulduğu **alanlardır** (**field**).
3. **Soyutlama** (abstraction): Bir nesnenin veya bileşenin tasarımına ilişkin detay yerine ihtiyacımızı gördüğü kadar kullanılması işlemidir. Yani nesne ve bileşenleri **soyutlama** (**abstraction**) yaparak kullanmalıyız.
4. **Sarmalama** (encapsulation): Nesneye ilişkin durum ve davranışların nesne kontrolünde olmasıdır. Nesnenin verisine ve davranışlarına dışarıdan müdahale kontrollü biçimde olmalıdır.
5. **Miras Alma** (inheritance): Bir sınıfın başka sınıflardan nitelik ve davranışları devralmasıdır.
6. **Çok Biçimlilik** (polymorphism): Çalıştırma ortamında bulunan birden çok nesneye aynı mesajın verilip karşılığında her bir nesnenin farklı davranış göstermesi işlemidir.

Sınıf ve Nesne

Kısacası sınıflar, verileri tanımlar ve bu verileri işlemek için yöntemler sağlar. Bu yöntemler sınıfın verilerini işleyecek şekilde icra edilirler. Bu nedenle, sınıflar somut uygulama ayrıntılarını sınıf verilerinden ayırarak soyutlama sağlar.

Python dilinde nesne verilerini tutan **durumlar** (**state**), **nitelikler** (**attribute**) olarak adlandırılır. Nesnenin bu durumlarını işleyecek fonksiyonlar da **yöntem** (**method**) olarak adlandırılır. Sınıflarda nitelikler ve yöntemlerden oluşur. Nitelikler değişkenler olarak, yöntemler ise fonksiyonlar olarak tanımlanır. Aşağıdaki şekilde tanımlanır;

```
class sınıf-kimliği(object):  
    """Sınıfı tanımlayan docstring"""  
  
    #Sınıf Durumları/Niteklilikleri  
    sınıf-değişkeni = ilk-değer  
  
    def __init__(self): #Nesne Yapıcı  
        örnek-değişkeni=ilk-değer #Nesne Durumları/nitelikleri  
  
    #Davranışlar/Yöntemler  
    def davranış-kimliği(self, parametre):  
        """Yöntemi Tanımlayan docstring"""  
        parametre= alan-değişkeni-kimliği+parametre  
        return parametre
```

Bir nesne, belirli bir sınıfının örneğidir. Bir mimari plandan nasıl ki birden fazla ev imal edilebilir. Bir sınıftan da istenildiği kadar **örnek** (**instance**) alınabilir. Yani **nesne** (**object**) imal edilebilir.

Python dilinde sınıf üyelerine varsayılan olarak **halka açık (public)** olarak erişilir. Yani C++/Java/C# dillerindeki **public** anahtar kelimesinin davranışını taklit eder. Yani örnek ve sınıf niteliklerine dışardan erişim ve değişiklik her zaman mümkündür.

Durumlar nesnelere ve sınıflara ait verilerdir. Tüm sınıf içerisinde ve **imal edilecek nesneler tarafından ortak kullanılacak nitelikler önce tanımlanır** ve bu nitelikler **sınıf nitelikleri (class attribute)** olarak adlandırılır. Bu niteliklere sınıf dışından erişilebilir ve değiştirilebilir. Bu nitelikler varsayılan nesne verileri için ve **tekil nesne (singleton)** oluşturulması halinde yararlıdır.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik

kişi=Kişi() #Kişi sınıfından bir örnek (instance) oluşturuldu ve
            #kişi kimlikli değişken bu nesneye referans gösterildi.
            #Kısaca Kişi tipli kişi değişkeni tanımlandı.
Kişi.kişiSayısı +=1
print (type(kişi)) #<class '__main__.Kişi'>
print (kişi.__doc__) #Kişi Sınıfı
print (Kişi.kişiSayısı) #1
başkakişi=Kişi() #Kısaca Kişi tipli başkakişi değişkeni tanımlandı.
Kişi.kişiSayısı +=1
print (Kişi.kişiSayısı) #2
```

Sınıf tanımında yer alan her **yöntem (method)**, imal edilecek ilgili nesneyi birinci parametre olarak geçirir. Bunun için **self** kelimesi kullanılır. Bir ev planını değiştirmeden farklı renklerde ev imal edilebilir. Benzer şekilde bir sınıftan da farklı veriler içeren yani farklı **durumlarda (state)** nesneler imal edilebilir. Bunu sağlayan sınıftaki **özel bir yöntem (special method)** olan **__init__()** adlı nesne **yapıcısıdır (constructor)**. Yapıcı içerisinde tanımlanan nitelikler sadece imal edilen nesneye aittir. Bu niteliklere **örnek nitelikleri (instance attribute)** adı verilir.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik

    def __init__(self):
        """Kişi Sınıfı Yapıcısı"""
        self.adı='ilhan' #adı kimlikli örnek niteliği (instance attribute)
        self.yaşı=50      #yaşı kimlikli örnek niteliği (instance attribute)
        Kişi.kişiSayısı+=1 #her nesne imalatında 1 artar

kişi=Kişi() #Kişi sınıfından bir örnek (instance) oluşturuldu ve
            #kişi kimlikli değişken bu nesneye referans gösterildi.
            #Kısaca Kişi tipli kişi değişkeni tanımlandı.
print (type(kişi)) #<class '__main__.Kişi'>
print (kişi.__doc__) #Kişi Sınıfı
print (kişi.__init__.__doc__) #Kişi Sınıfı Yapıcısı
print (Kişi.kişiSayısı) #1 #Sınıf niteliğine erişim
print (kişi.adı) #ilhan #örneğin (kişi) niteliğine erişim
print (kişi.yaşı) #50    #örneğin (kişi) değişkenine erişim
başkakişi=Kişi() #Kısaca Kişi tipli başkakişi değişkeni tanımlandı.
print (Kişi.kişiSayısı) #2
print (başkakişi.adı) #ilhan
print (başkakişi.yaşı) #50
```

Örnek niteliğinde (instance attribute) yapılan değişiklikler **yalnızca tanımlandığı nesneyi etkiler!**

Her bir örnek değişkeni bellekten silineceği zaman ise sınıfın **__del__()** isimli özel yöntemi çağrılır. Bu yöntem **yıkıcı (destructor)** adı verilir.

```
class Kişi(object):
    """Kişi Sınıfı"""
```

```

KişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik

def __init__(self):
    """Kişi Sınıfı Yapıcısı"""
    self.adı='ilhan'
    self.yaşı=50
    Kişi.KişiSayısı+=1 #her nesne imalatında 1 artar

def __del__(self):
    """Kişi Sınıfı Yıkıcısı."""
    Kişi.KişiSayısı-=1 #her nesne silindiğinde 1 eksilir

kişi=Kişi()
print (Kişi.kişiSayısı) #1
başkakişi=Kişi() #Kısaca Kişi tipli başkakişi değişkeni tanımlandı.
print (Kişi.kişiSayısı) #2
del başkakişi
print (Kişi.kişiSayısı) #1

```

Özel yöntemlerden en çok kullanılan diğer iki yöntem `__repr__` ve `__str__` yöntemleridir. `__str__` imal edilen nesnenin metin olarak karşılığını veren yöntemdir. `__str__` kullanıcılar için kullanıcı dostu çıktı oluştururken, `__repr__` ideal olarak nesneyi yeniden imal edebilmek için ayrıntılı, geliştirici odaklı metin oluşturur.

```

class Araba:
    def __init__(self, marka, model):
        self.marka = marka
        self.model = model

    def __str__(self):
        # Kullanıcı için okunabilir
        return f"{self.marka} {self.model}"

    def __repr__(self):
        # Geliştirici için detaylı
        return f"Araba('{self.marka}', '{self.model}')"

```

Bir de her sınıfta ön tanımlı olarak `__doc__`, `__name__` ve `__module__` nitelikleri bulunur.

```

class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik
    def __init__(self):
        """Kişi Sınıfı Yapıcısı Parametre Almaz!."""
        Kişi.kişiSayısı+=1
        self.adı='ilhan' #örnek niteliği
        self.yaşı=50 #örnek niteliği
    def __del__(self):
        Kişi.kişiSayısı-=1

kişi=Kişi()
başkakişi=Kişi()
print(Kişi.__doc__) #Kişi Sınıfı
print(Kişi.__name__) #Kişi
print(Kişi.__module__) #__main__

```

Sınıflardan imal edilen nesnelerin kendilerine özgü **davranışları** (**behavior**) olabilir. Aşağıda bir `Kişi` sınıfı ve bu sınıftan imal edilen `kişi` ve `başkakişi` nesnelerine `adınıSöyle()` davranışını göstermesi için mesaj gönderilmesi örneğine yer verilmiştir.

```

class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik

```

```

def __init__(self):
    """Kişi Sınıfı Yapıcısısı."""
    Kişi.kişiSayısı+=1
    self.adı='İlhan' #örnek niteliği (instance attribute)
    self.yaşı=50 #örnek niteliği (instance attribute)
def __del__(self):
    """Kişi Sınıfı Yıkıcısısı."""
    Kişi.kişiSayısı-=1
def adınıSöyle(self):
    """Kişi Sınıfından İmal Edilecek Nesnelerin Adını Konsola Yazdırır"""
    print('Adım:{}'.format(self.adı))
def yaşınıGetir(self):
    """Kişi Sınıfından İmal Edilecek Nesnelerin Yaşını Döndürür"""
    return self.yaşı

kişi=Kişi()
print('Kişi Sayısı',Kişi.kişiSayısı) #Kişi Sayısı 1
kişi.adınıSöyle() #Adım:İlhan
kişi.adı='Ali' # kişi nesnesinin örnek niteliği (instance attribute) değiştiriliyor
kişi.adınıSöyle() #Adım:Ali
başkakişi=Kişi()
başkakişi.adınıSöyle() #Adım:İlhan

```

Örnekte imal edilen tüm nesnelerin `adı` niteliği 'İlhan', `yaşı` niteliği ise 50 olacaktır. Bu durumda yapıcı parametrelili tanımlanarak her seferinde başka nitelikler yani durumlarla nesne imal edilebilir.

```

class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik
    def __init__(self, adı='İlhan', yaşı=50):
        """Kişi Sınıfı Yapıcısı Parametre Almaz!."""
        Kişi.kişiSayısı+=1
        self.adı=adı #örnek niteliği
        self.yaşı=yaşı #örnek niteliği
    def __del__(self):
        Kişi.kişiSayısı-=1

    def adınıSöyle(self):
        """Kişi Sınıfından İmal Edilecek Nesnelere Adını Konsola Yazdırır"""
        print('Adım:{}'.format(self.adı))
    def yaşınıGetir(self):
        """Kişi Sınıfından İmal Edilecek Nesnelerin Yaşını Döndürür"""
        return self.yaşı

kişi=Kişi()
print('Kişi Sayısı',Kişi.kişiSayısı) #Kişi Sayısı 1
kişi.adınıSöyle() #Adım:İlhan
kişi.adı='Ali' # kişi nesnesinin örnek niteliği (instance attribute) değiştiriliyor
kişi.adınıSöyle() #Adım:Ali
başkakişi=Kişi('Mehmet')
print('Kişi Sayısı',Kişi.kişiSayısı) #Kişi Sayısı 2
başkakişi.adınıSöyle() #Adım:Mehmet
print(başkakişi.yaşınıGetir()) #50
birbaşkakişi=Kişi('Abdullah',40)
print('Kişi Sayısı',Kişi.kişiSayısı) #Kişi Sayısı 3
birbaşkakişi.adınıSöyle() #Adım:Abdullah
print(birbaşkakişi.yaşınıGetir()) #40

```

Bir başka özel yöntem ise `__str__()` yöntemidir. Aslında Java/C# dillerinde olan `toString()` yöntemi işlevini yerine getirir. Yani imal edilen nesnenin metin olarak karşılığının ne olduğu belirleyen yöntemdir.

```

class Kişi(object):

```



```

"""Kişi Sınıfı"""
kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik
def __init__(self, adı='İlhan', yaşı=50):
    """Kişi Sınıfı Yapıcısı Parametre Almaz!."""
    Kişi.kişiSayısı+=1
    self.adı=adı #örnek niteliği
    self.yaşı=yaşı #örnek niteliği
def __del__(self):
    Kişi.kişiSayısı-=1
def adınıSöyle(self):
    """Kişi Sınıfından İmal Edilecek Nesnelere Adını Konsola Yazdırır"""
    print('Adım:{}'.format(self.adı))
def yaşıyıGetir(self):
    return self.yaşı
def __str__(self):
    return 'Adı:{},Yaşı:{}'.format(self.adı,self.yaşı)

kişi=Kişi('ilhan',50)
kişi.adınıSöyle() #Adım:İlhan
metin=kişi.__str__()
print(metin) #Tür:İnsan,Adı:ilhan,Yaşı:50
print(kişi) #Tür:İnsan,Adı:ilhan,Yaşı:50 #dolaylı olarak __str__() çağrılır
başkakişi=Kişi('Mehmet',45)
print(başkakişi) #Tür:İnsan,Adı:Mehmet,Yaşı:45

```

Bunların dışında sınıfımızda `__fonksiyon__(self)` şeklinde yöntemler de tanımlayabiliriz. Bütün bu tür yöntemlere **sihirli yöntemler** (**magic method**) denir ve Python dilindeki sınıfların önemli bir parçasını oluşturur.

Yöntemler

Yöntemler (**method**), bir sınıfın nesnesine aittir ve nesne davranışlarını gösterecek belirli işlemleri gerçekleştirmek için kullanılır. Python metotlarını üç farklı kategoriye ayırabiliriz: sınıf yöntemi, örnek yöntemi ve statik yöntem.

Sınıf yöntemi (**class method**), sınıf nitelikleri gibi sınıfın örneğine değil, sınıfa bağlı bir metottur. Sınıfın bir örneği yerine, sınıfın kendisi üzerinde çağrılabilir. Sınıf yönteminin aksine, **örnek yöntemi** (**instance method**) bir nesnenin örneklerine yani imal edilmiş nesnelere erişebilir. Ayrıca, tüm nesneler için ortak olduğu için sınıf değişkenine de erişebilir.

Sınıf yöntemi yazmanın iki yolu vardır. Birincisi dahili `classmethod()` fonksiyonunu ile örnek yöntemi sınıf yöntemine çevrilir.

```

class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik
    def __init__(self):
        """Kişi Sınıfı Yapıcısı"""
        self.adı='ilhan'
        self.yaşı=50
        Kişi.kişiSayısı+=1 #her nesne imalatında 1 artar
    def __del__(self):
        """Kişi Sınıfı Yıkıcısı."""
        Kişi.kişiSayısı-=1 #her nesne silindiğinde 1 eksilir
    def kişiSayısınıGöster(self):
        print('Kişi Sayısı:',Kişi.kişiSayısı)
    NesneSayısı = classmethod(kişiSayısınıGöster)

kişi=Kişi()
Kişi.NesneSayısı() #1
başkakişi=Kişi() #Kısaca Kişi tipli başkakişi değişkeni tanımlandı.
Kişi.NesneSayısı() #2

```

```
del başkakişi
Kişi.NesneSayısı() #1
```

İkincisinde ise sınıf yöntemi başında `@classmethod` dekoratörü kullanılır.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik
    def __init__(self):
        """Kişi Sınıfı Yapıcısı"""
        self.adı='ilhan'
        self.yaşı=50
        Kişi.kişiSayısı+=1 #her nesne imalatında 1 artar
    def __del__(self):
        """Kişi Sınıfı Yıkıcısı."""
        Kişi.kişiSayısı-=1 #her nesne silindiğinde 1 eksilir
    @classmethod
    def KişiSayısınıGöster(cls): #cls geleneksel olarak kullanılır. Başka kimlik de olabilir.
        print('Kişi Sayısı:',cls.kişiSayısı)

Kişi=Kişi()
Kişi.KişiSayısınıGöster() #1
başkakişi=Kişi()
Kişi.KişiSayısınıGöster() #2
del başkakişi
Kişi.KişiSayısınıGöster() #1
```

Statik yöntemler (**static method**), çağırılması için herhangi bir sınıfın örneğine veya nesneye ihtiyaç duymayan yöntemlerdir. Sınıf yöntemine çok benzer ancak farkı, statik yöntemin `self` nesnesine veya `cls` sınıfına referans gibi zorunlu bir argümana sahip olmamasıdır. Statik bir yöntem;

- Sıklıkla tekrar kullanılan görevleri gerçekleştirmek için yardımcı bir fonksiyon olarak kullanılabilir.
- Sınıf adını kullanarak çağırırız. Bu sayede sınıftan imal edilmiş nesne örneklerine olan bağımlılık ortadan kalkar.
- Sınıf durumundan bağımsız olarak davranışı değişmeden kaldığı için her zaman öngörülebilir.
- İlerde göreceğimiz **geçersiz kılmayı** (**overriding**) önlemek kullanılır.

Statik yöntemi yazmanın iki yolu vardır. Birincisi dahili `staticmethod()` fonksiyonunu ile örnek yöntemi statik yöntemine çevrilir.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik
    def __init__(self):
        """Kişi Sınıfı Yapıcısı"""
        self.adı='ilhan'
        self.yaşı=50
        Kişi.kişiSayısı+=1 #her nesne imalatında 1 artar
    def __del__(self):
        """Kişi Sınıfı Yıkıcısı."""
        Kişi.kişiSayısı-=1 #her nesne silindiğinde 1 eksilir
    def KişiSayısınıGöster(): #parametrede self YOK!
        print('Kişi Sayısı:',Kişi.kişiSayısı)
    NesneSayısı = staticmethod(KişiSayısınıGöster)

Kişi.NesneSayısı() #0
Kişi=Kişi()
Kişi.NesneSayısı() #1
başkakişi=Kişi()
Kişi.NesneSayısı() #2
del başkakişi
Kişi.NesneSayısı() #1
```

İkincisinde ise sınıf yöntemi başında `@staticmethod` dekoratörü kullanılır.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik
    def __init__(self):
        """Kişi Sınıfı Yapıcısı"""
        self.adı='İlhan'
        self.yaşı=50
        Kişi.kişiSayısı+=1 #her nesne imalatında 1 artar
    def __del__(self):
        """Kişi Sınıfı Yıkıcısı."""
        Kişi.kişiSayısı-=1 #her nesne silindiğinde 1 eksilir
    @staticmethod
    def KişiSayısınıGöster(): #parametrede self YOK!
        print('Kişi Sayısı:',Kişi.kişiSayısı)

Kişi.KişiSayısınıGöster() #0
kişi=Kişi()
Kişi.KişiSayısınıGöster() #1
başkakişi=Kişi()
Kişi.KişiSayısınıGöster() #2
del başkakişi
Kişi.KişiSayısınıGöster() #1
```

Erişim Belirleyiciler

Erişim belirleyiciler (**access modifier**), sınıf üyelerine yani değişkenlere ve yöntemlere sınıf dışından erişimi kısıtlamak için kullanılır. **Halka açık** (**public**), **korumalı** (**protected**) ve **mahrem** (**private**) olmak üzere üç tür erişim belirleyici vardır.

Şimdiye kadar örneklerde tanımlanan sınıflarda nitelikler ve yöntemler varsayılan olarak halka açık olarak belirlenmiştir. Yani sınıf üyelerine sınıf ve nesne dışından erişilebilir.

Bazı durumlarda imal edilecek nesnelerin bazı üyelerine dışarıdan erişim kapatılmak istenir. Bu tür üyeler **mahrem** (**private**) üyeler olarak adlandırılır. Bir üyeyi mahrem yapmak için üye kimliği önünde iki altçizgi karakteri (`__`) kullanılır. Bazı üyelere de mevcut sınıf ve bu sınıftan miras alan sınıflar (ileride anlatılacaktır) dışında erişime kapatılır. Bu tür üyeler **korumalı** (**protected**) üye olarak adlandırılır. Korumalı bir üyenin kimliği önünde tek alt çizgi (`_`) karakteri kullanılır.

```
class Kişi(object):
    """Kişi Sınıfı"""
    def __init__(self, adı, sır, maaş):
        """Kişi Sınıfı Yapıcısı"""
        self.adı=adı #halka açık (public) nitelik
        self.__sır=sır #mahrem (private) nitelik
        self._maaş=maaş #korumalı(protected) nitelik
    def kişiBilgileri(self):
        print('Ad:{},Sır:{},Maaş{}'.format(self.adı,self.__sır,self._maaş))

kişi=Kişi('İlhan','Ben bişey biliyorum',20000)
print(kişi.adı) #İlhan
#print(kişi.__sır) #Hata: 'Kişi' object has no attribute '__sır'
print(kişi._maaş) #20000
kişi.kişiBilgileri() #Ad:İlhan,Sır:Ben bişey biliyorum,Maaş20000
```

Burada imal edilen `kişi` nesnesinin `kişiBilgileri()` yöntemi erişim belirleyicisi ne olursa olsun kişinin tüm verilerine eriştiği gözden kaçırılmamalıdır.

Soyutlama (Abstraction)

Soyutlama (**abstraction**), nesne yönelimli programlamanın önemli ilkelerinden biridir. Bu, bir nesneyle ilgili yalnızca önemli verilerin dış dünyaya açıldığı ve diğer tüm detayların gizlendiği bir programlama yaklaşımını ifade eder. Bu yaklaşım, karmaşıklığı azaltmaya ve uygulama geliştirme verimliliğini artırmaya yardımcı olur.

İki tür soyutlama vardır. Biri, nesnenin gizlendiği verilerle ilgili **veri soyutlamasıdır** (**data abstraction**); bu, gizli veriye nesne içinden erişilebilen bir veri yapısı aracılığıyla ulaşılır. Diğer tür ise süreç ya da **kontrol soyutlamasıdır** (**control abstraction**). Bu ise, nesnenin gerçekleştirdiği sürecin temel uygulama detaylarının gizlenmesini ifade eder.

Aşağıda verilen Kişi sınıfını göz önüne alalım. Bu sınıftan nesne imal edildikten sonra yaşı -1, 160, 2000, ... gibi anormal değerler vermemize kimse engel olamaz.

```
class Kişi(object):
    """Kişi Sınıfı"""
    def __init__(self,adı,yaş):
        """Kişi Sınıfı Yapıcısı"""
        self.adı=adı
        self.yaş=yaş
```

Bu gibi problemlerin üstesinden **veri soyutlaması** (**data abstraction**) ile gelebiliriz. Yani **yaş** alanını **mahrem** (**private**) yapıp bu alanın değerini kontrollü bir şekilde belirleyen **set** yöntemi yazabiliriz. Benzer şekilde bu alan mahrem olduğundan **get** yöntemi yazıp nesne verisini dışarıya aktarabiliriz.

```
class Kişi(object):
    """Kişi Sınıfı"""
    def __init__(self,adı='İlhan',yaş=50):
        """Kişi Sınıfı Yapıcısı"""
        self.adı=adı
        self.__yaş=yaş
    def setYaş(self,yaş):
        if yaş<0:
            print("Hata:Yaş değeri sıfırın altında olamaz!")
        elif yaş>120:
            print("Hata:Yaş değeri en fazla 120 olabilir!")
        else:
            self.__yaş=yaş
    def getYaş(self):
        return self.__yaş
```

```
ali=Kişi('Ali',40)
ali.setYaş(200) # Hata
print(ali.getYaş()) #40
```

Veri soyutlamasına benzer şekilde imal edilen nesnenin bazı davranışlarına dışarıdan erişim kontrollü bir şekilde yapılması gerekir. Bu işleme ise **kontrol soyutlaması** (**control abstraction**) adı verilir. Aşağıda **HesapMakinesi** sınıfı ve bu sınıftan imal edilen nesnelerin **ekranıyenile** davranışı için kontrol soyutlaması yapılmıştır.

```
class HesapMakinesi(object):
    """Hesap Makinesi Sınıfı"""
    def __init__(self):
        """Hesap Makinesi Sınıfı Yapıcısı"""
        self.__sonuç=0

    def __ekranıYenile(self): #hesap makinesi nesnesinin mahrem davranışı
        print('İşlem',self.__sonuç)

    def setHesapMakinesi(self,değer):
        self.__sonuç=değer
```

```

        self.__ekraniYenile()

def makineyiSıfırla(self):
    self.__sonuç=0
    self.__ekraniYenile()

def faktöriyelHesapla(self):
    temp=1
    for x in range(1,self.__sonuç+1):
        temp=temp*x
    self.__sonuç=temp
    self.__ekraniYenile()

makine=HesapMakinesi()
makine.setHesapMakinesi(10) #10
makine.makineyiSıfırla() #0
makine.setHesapMakinesi(5) #5
makine.faktöriyelHesapla() #120
#makine.__ekraniYenile() yazılamaz çünkü mahrem bir davranıştır.

```

Sarmalama (Encapsulation)

Bir sınıf üzerinde **veri soyutlaması** (**data abstraction**) veriler üzerine kılıf çekilir, hem de **kontrol soyutlaması** (**control abstraction**) ile davranışların üzerine bir kılıf çekilir. Bir sınıf üzerinde her iki soyutlamanın yapılmasına sarmalama (**encapsulation**) adı verilir. Soyutlamaların gerekli olup olmadığına programcı karar verir.

Sarmalama işlemi örneklerde görüldüğü üzere **erişim değiştiricilerle** (**access modifier**) yapılır. Böylece imal edilen nesnelerin verileri ve davranışları güvenli bir şekilde dış dünyaya açılmış olur. Böylece, sınıf ve nesne gibi bileşenlerin içeriğini bilmeden **soyut** (**abstract**) olarak kullanmak mümkün olmaktadır. Programcı tasarladığı sınıfın soyut olarak katlanılacağını düşünerek sınıfları tasarlamalıdır.

Özetle **sarmalama** (**encapsulation**) tekniğini uygulamak, kullanımı kolay **sınıf** (**class**), **bileşen** (**component**), **kütüphane** (**library**) ve **çerçeve yapılar** (**framework**) elde etmemizi sağlar;

- n adet **veri** (**data**) ve n adet **davranış** (**behavior**) sarmalama işlemine tabi tutulursa **sınıf**,
- n adet sınıf sarmalama işlemine tabi tutulursa **bileşen**,
- n adet bileşen sarmalama işlemine tabi tutulursa **kütüphane**,
- n adet kütüphane sarmalama işlemine tabi tutulursa **çerçeve yapılar** oluşur.

Böylece bize projemizde **proje süresini ve maliyetini kısaltma üstünlüğünü** verir.

Kalıtım (Inheritance)

Öğrenci, öğretmen ve müdürün olduğu bir sistemi örnek olarak ele alalım. Bunlardan her birinden nesne imal etmek için her birine ayrı ayrı sınıf tanımlamak gerekir.

```

class Öğrenci(object):
    def __init__(self):
        self.yaş=20
        self.cinsiyet='E'
        self.ogrenciNo=0
    def yaşınıSöyle(self):
        print('Yaşım:',self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:',self.cinsiyet)
    def dersÇalış(self):
        print('Ders Çalışıyorum')

öğrenci=Öğrenci()
öğrenci.dersÇalış()
öğrenci.cinsiyetiniSöyle()

```

Öğretmen için ayrı bir sınıf;

```
class Öğretmen(object):
    def __init__(self):
        self.yaş=50
        self.cinsiyet='E'
        self.verdiğiDersKodu='BIL101'
    def yaşınıSöyle(self):
        print('Yaşım:',self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:',self.cinsiyet)
    def dersVer(self):
        print('Ders Veriyorum')
```

```
öğretmen=Öğretmen()
öğretmen.dersVer()
öğretmen.cinsiyetiniSöyle()
```

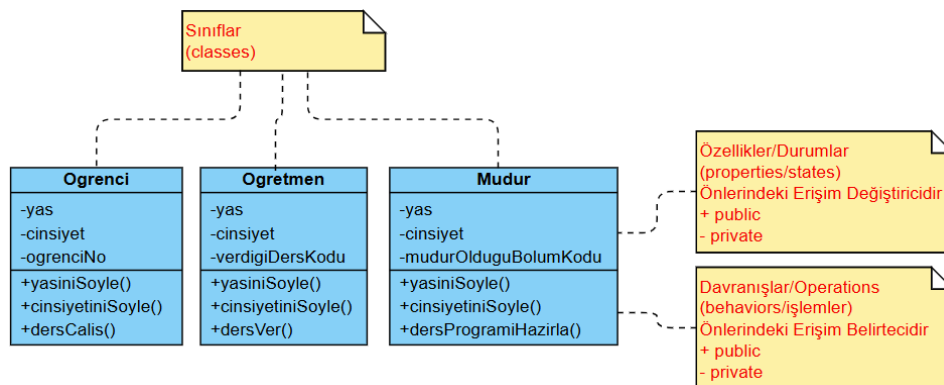
Müdür için ayrı bir sınıf;

```
class Müdür(object):
    def __init__(self):
        self.yaş=55
        self.cinsiyet='K'
        self.müdürOlduğuBölümKodu='Elektronik'
    def yaşınıSöyle(self):
        print('Yaşım:',self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:',self.cinsiyet)
    def dersProgramıHazırla(self):
        print('Ders Programı Hazırlıyorum')
```

```
müdür=Müdür()
müdür.dersProgramıHazırla()
müdür.yaşınıSöyle()
```

Bu sınıfların her birinde `yaş` ve `cinsiyet` gibi ortak **özellikler** (property) veya `yaşınıSöyle()` ve `cinsiyetSöyle()` gibi ortak **davranışlar** (behavior) bulunur. Her sınıf için bunlar tekrar tekrar tanımlanırlar. Hâlbuki bu gibi ortak özellikler ve davranışlar tek bir sınıf altında toplanabilir. Bütün bu ortak özellik ve davranışlar tek bir sınıfta toplandığı zaman daha yönetilebilir bir kod elde edilir.

Yazılım analizi yapılırken takım içerisinde iletişimi kolaylaştırmak için aşağıdaki **sınıf diyagramları** (class diagram) kullanılır. Bunlar **Unified Modelling Language-UML** dilinin bir parçasıdır.



Şekil 21. Öğrenci, Öğretmen ve Müdür UML Sınıf Diyagramları

Ortak özellik ve davranışların bir sınıfta toplanarak bu sınıftan miras alınmasına kalıtım (**inheritance**) adı verilir. Yukarıdaki örnekte ortak özellik ve davranışlar **Kişi** sınıfına toplanarak aşağıdaki şekilde kod yeniden düzenlenebilir;

Aşağıda verilen ve Kişi olarak tanımlanan **genel sınıf** (**general class**), **taban sınıf** (**base class**) veya **ebeveyn sınıf** (**parent class**) olarak adlandırılır. Daha sonra tanımlanacak Öğrenci, Öğretmen ve Müdür sınıflarının ortak özellik ve davranışlarını barındırır.

```
class Kişi(object):
    def __init__(self):
        self.yaş=20
        self.cinsiyet='E'
    def yaşınıSöyle(self):
        print('Yaşım:',self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:',self.cinsiyet)
```

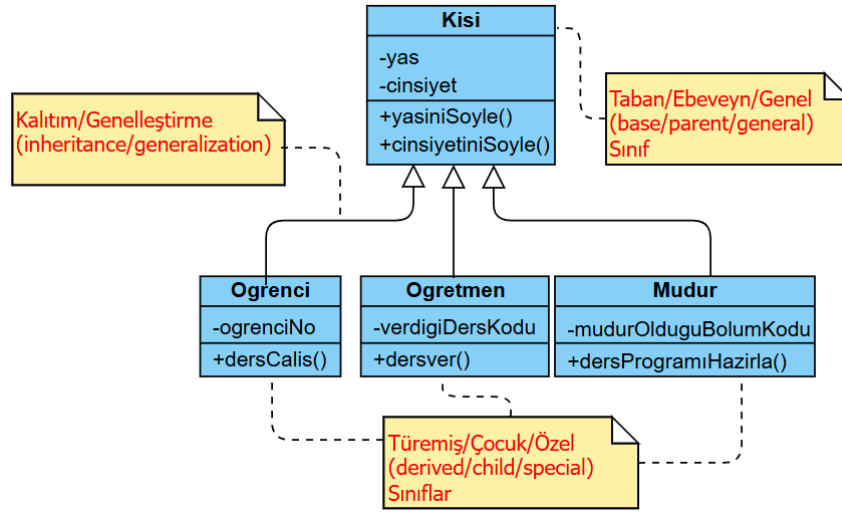
Aşağıda taban sınıftan **türemiş** (**derived**) Öğrenci, Öğretmen ve Müdür sınıfları verilmiştir. Bu sınıflar genel sınıf olan Kişi sınıfının yanında kendine özel özellik ve davranışlara da sahiptir. **Özel sınıf** (**private class**), **türemiş sınıf** (**derived class**) veya **çocuk sınıf** (**child class**) olarak adlandırılan bu sınıflar, mirasçı olup, kendilerinden imal edilmiş nesneler; hem taban sınıfın özellik ve davranışlarını, hem de kendi özellik ve davranışlarını sergilerler.

```
class Öğrenci(Kişi):
    def __init__(self):
        super().__init__() #miras alınan sınıfın yapıcısı çağrılıyor.
        self.ogrenciNo=0
    def dersÇalış(self):
        print('Ders Çalışıyorum')
class Öğretmen(Kişi):
    def __init__(self):
        super().__init__() #miras alınan sınıfın yapıcısı çağrılıyor.
        self.verdiğiDersKodu='BIL101'
    def dersVer(self):
        print('Ders Veriyorum')
class Müdür(Kişi):
    def __init__(self):
        super().__init__() #miras alınan sınıfın yapıcısı çağrılıyor.
        self.müdürOlduğuBölümKodu='Elektronik'
    def dersProgramıHazırla(self):
        print('Ders Programı Hazırlıyorum')
```

Miras alan sınıflardaki yapıcılarda taban sınıftaki nitelikler devralmak istenmiyorsa `super().__init__()` yöntemi kullanılmaz. **Kişi** sınıfından miras alan bu üç sınıftan da nesne imal edilmiş ve çeşitli davranışlar göstermesi için kendilerine **ileti gönderilmiştir** (**message-passing**).

```
öğrenci=Öğrenci()
öğrenci.dersÇalış()
öğrenci.cinsiyetiniSöyle()
öğretmen=Öğretmen()
öğretmen.dersVer()
öğretmen.cinsiyetiniSöyle()
müdür=Müdür()
müdür.dersProgramıHazırla()
müdür.yaşınıSöyle()
```

Yeni haliyle sınıf diyagramları aşağıda verilmiştir;



Şekil 22. Kişi Taban Sınıfı Eklenmiş Kalıtım UML Sınıf Diyagramı

Değişim yönetimini kolayca yapma üstünlüğünü veren kalıtımın (inheritance) iki önemli özelliği vardır; Paylaşılan bir **ortak kod (shared code)** vardır ve paylaşılan kodda yapılan **değişiklik anında alt sınıflara yansır (snap change)**.

Örneğimizden gidecek olursak, Kişi sınıfında yapılacak bir özellik veya davranış eklemesi anında bu sınıftan türemiş sınıflarda yapılmış olur. Burada önemli olan türemiş sınıfta yapıcıda **super()** yöntemi doğru kullanılmalıdır. Kötü kullanım aşağıdaki gibidir;

```

class Kopek(Hayvan):
    def __init__(self, isim):
        Hayvan.__init__(self, isim) # Kötü
  
```

İyi kullanım se aşağıdaki gibi olmalıdır;

```

class Kopek(Hayvan):
    def __init__(self, isim):
        super().__init__(isim) # İyi - Python way
  
```

Mantık olarak bir ev, birden fazla mimari plandan hazırlanmayacağından, **çoklu kalıtım (multiple inheritance)** sınıf tasarımında tercih edilmez. Sınıflar, birden fazla taban sınıftan miras alamayacak şekilde tasarlanır. Bu nedenle saf nesne yönelimli diller olan Java ve C# gibi dillerde çoklu kalıtıma izin verilmez. Python dilinde aşağıda belirtilen kalıtım türleri mevcuttur;

1. Tek Kalıtım: Bir alt sınıf, tek bir üst sınıftan kalıtım alır.
2. Çoklu Kalıtım: Bir alt sınıf, birden fazla üst sınıftan kalıtım alır.
3. Çok Düzeyli Kalıtım: Bir sınıf, başka bir sınıftan türetilen bir sınıftan türetilir.
4. Hiyerarşik Kalıtım: Birden fazla sınıf, tek bir üst sınıftan kalıtım alır.
5. Hibrit Kalıtım: Birden fazla kalıtım türünün birleşimidir.

Yöntem çözümleme sırası (method resolution order-MRO), sınıf hiyerarşisinde bir yöntemi aradığı sıradır. Çoklu kalıtım olduğunda, MRO hangi yöntemin gerçekten çağrıldığını anlamak için çok önemlidir. **Hata ayıklamada (debug)** ve kodu etkili bir şekilde okumaya yardımcı olur.

```

class A:
    def metod(self):
        print("A")

class B(A):
    def metod(self):
        print("B")

class C(A):
    def metod(self):
        print("C")
  
```



```
class D(B, C):
    pass

print(D.__mro__) # MRO kontrol etme
# (<class 'D'>, <class 'B'>, <class 'C'>, <class 'A'>, <class 'object'>)
```

Aşırı Yükleme

Nesne yönelimli programlamada bir sınıf aynı kimliğe sahip ancak farklı parametrelere işlem yapan birden fazla yönteme sahip olabilir. Bu duruma **yöntemlerin aşırı yüklenmesi** (**method overloading**) adı verilir. Yöntemleri aşırı yüklemek için parametre sayısını, parametre veri tipini veya her ikisini birden değiştirmemiz gerekir.

Java, C++ ve C# gibi diğer nesne yönelimli programlama dillerinin aksine Python, varsayılan olarak yöntem aşırı yükleme özelliğini desteklemez. Ancak bunu yapmanın yolları vardır. Aşağıdaki kodda gösterildiği gibi bir metodu birden çok kez tanımlarsanız, son tanım öncekileri geçersiz kılar. Bu nedenle, Python'da bu şekilde metod aşırı yüklemesi yapmak hataya neden olur.

```
class Hesaplayıcı:
    def toplama(self, x, y):
        return x+y
    def toplama(self, a, b, c):
        return a+b+c

nesne = Hesaplayıcı()

print (nesne.toplama(10,20,30))
print (nesne.toplama(10,20)) # Hata! Toplama fonksiyonu 3 argüman bekler.
```

Yöntem aşırı yüklemesini simüle etmek için, yöntem argümanlarına varsayılan değeri **None** olarak tanımlayarak geçici bir çözüm kullanabiliriz, böylece yöntem bir, iki veya üç argümanla kullanılabilir.

```
class Hesaplayıcı:
    def toplama(self, a = None, b = None, c = None):
        x=0
        if a !=None and b != None and c != None:
            x = a+b+c
        elif a !=None and b != None and c == None:
            x = a+b
        return x

nesne = Hesaplayıcı()
print (nesne.toplama(10,20,30)) #60
print (nesne.toplama(10,20)) #90
```

Python standart kütüphanesinde yöntem aşırı yüklemesini uygulamak için başka bir düzenleme yoktur. Ancak, bu amaçla **MultipleDispatch** adlı üçüncü taraf bir modülden bir dağıtım işlevi kullanabiliriz. Bunun için bu modül projemize dahil etmeliyiz.

```
>>> pip install multipledispatch
```

Bu modülde bir **@dispatch** dekoratörü bulunur. Aşırı yüklenecek yönteme iletilecek argüman sayısını alır.

```
from multipledispatch import dispatch
class Hesaplayıcı:
    @dispatch(int, int)
    def toplama(self, x, y):
        return x+y
    @dispatch(int, int, int)
    def add(self, a, b, c):
        return a+b+c
```

```
nesne = Hesaplayıcı()
print (nesne.add(10,20,30))
print (nesne.add(10,20))
```

Çifte sevkiyat (**double dispatch**), bir sınıfı hiçbir şekilde değiştirmeden, söz konusu sınıfa etkin biçimde yöntem eklenebilmesi işlemidir. Bu çok bilinen bir tekniktir ve **Common Lisp Object System (CLOS)** gibi programlama dilleri tarafından desteklenir. C++, C#, Java gibi birçok nesne yönelimli programlama dili **tek sevkiyat** (**single dispatch**) yöntemi ile çalışır.

Geçersiz Kılma (Overriding)

Bir yöntemi **geçersiz kılma** (**overriding**), bir alt sınıfta, üst sınıfındaki bir yöntemle aynı adı taşıyan bir yöntem tanımlamayı ifade eder. Bu durumda, Python yorumlayıcısı, çalışma zamanında hangi yöntemin çağrılacağını, başvuru olan gerçek nesneye göre belirler.

Üst sınıf yöntemlerinizi her zaman geçersiz kılabilirsiniz. Üst sınıf yöntemlerini geçersiz kılmanın nedeni, alt sınıfınızda yöntemin farklı davranış göstermesini istememizdir.

```
class Taban:
    def yöntem(self):
        print ('Taban Sınıf Nesnesinin Yöntemi Çağrıldı.')

class Türemiş(Taban):
    def yöntem(self):
        print ('Türemiş Sınıf Nesnesinin Yöntemi Çağrıldı.')

tabannesne = Taban()
tabannesne.yöntem() #Taban Sınıf Nesnesinin Yöntemi Çağrıldı.
türemişnesne = Türemiş()
türemişnesne.yöntem() #Türemiş Sınıf Nesnesinin Yöntemi Çağrıldı.
```

Çok Biçimlilik (Polymorphism)

Çok biçimlilik (**polymorphism**) terimi, bir fonksiyonun veya yöntemin farklı bağlamlarda farklı şekillerde kullanılmasını ifade eder. Kısaca **çok biçimlilik** (**polymorphism**), birden çok nesnenin olduğu bir ortamda, verilen **aynı iletiye** (**message**) karşı, nesnelerin **farklı davranış** (**behavior**) göstermeleridir (**same message-different behavior**). **Temel sınıf** (**base class**) yöntemin çok biçimliliği sağlayan yöntem olması için, aynı yöntemin farklı **alt sınıflarda** (**derived class**) farklı iş mantıkları ile **geçersiz kılınması** (**override**) gerekir.

Python dilinde, **ördek tiplmesi** (**duck typing**), **yöntem geçersiz kılma** (**method overriding**), **işleç veya yöntem aşırı yükleme** (**method overloading**) uygulamaları ile çok biçimlilik yapılabilir.

Ördek tiplmesi, bir nesnenin türünün veya sınıfının, tanımladığı yöntemlerden daha az önemli olduğu bir kavramdır. Bu kavramı kullanarak, bir nesne üzerindeki herhangi bir yöntemi, türünü kontrol etmeden, yöntem mevcut olduğu sürece çağırabilirsiniz.

```
class Ordek:
    def seslen(self):
        return "Vak, Vak!"

class BaskaKus:
    def seslen(self):
        return "Civ Civ!"

def sesCikar(ordek):
    print(ordek.seslen())

duck = Ordek()
anotherBird = BaskaKus()
```

```
sesCikar(duck)    #Vak, Vak!
sesCikar(anotherBird) #Civ, Civ!
```

Geçersiz kılma ve aşırı yükle ile çok biçimlilik aşağıdaki şekilde yapılır;

```
class Kişi(object):
    def __init__(self):
        self.yaş=20
        self.cinsiyet='E'
    def yaşınıSöyle(self):
        print('Yaşım:',self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:',self.cinsiyet)
    def raporYaz(self):
        print('Kişi Raporu: ...')

class Öğrenci(Kişi):
    def __init__(self):
        super().__init__()
        self.ogrenciNo=0
    def dersÇalış(self):
        print('Ders Çalışıyorum')
    def raporYaz(self):
        print('Öğrenci Raporu: ...')

class Öğretmen(Kişi):
    def __init__(self):
        super().__init__()
        self.verdiğiDersKodu='BIL101'
    def dersVer(self):
        print('Ders Veriyorum')
    def raporYaz(self):
        print('Öğretmen Raporu: ...')

class Müdür(Kişi):
    def __init__(self):
        super().__init__()
        self.müdürOlduğuBölümKodu='Elektronik'
    def dersProgramıHazırla(self):
        print('Ders Programı Hazırlıyorum')
    def raporYaz(self):
        print('Müdür Raporu: ...')

kişiler = [Öğrenci(), Öğretmen(),Müdür()]
for kişi in kişiler:
    kişi.raporYaz() #Her nesneye aynı mesaj veriliyor! Ama farklı davranış gösteriyorlar.
```

Yukarıdaki kod çalıştırıldığında aşağıdaki çıktı elde edilir;

```
Öğrenci Raporu: ...
Öğretmen Raporu: ...
Müdür Raporu: ...
```

Çok biçimlilik bize **çalıştırma anında** (run time) üstünlük sağlar. Bu nedenle nesneler de çalıştırma anında imal edilmiştir. Program çalıştırıldığında her **kişi** nesnesine aynı ileti verilmiş ama bu nesneler farklı davranış göstermiştir. Nesne yönelimli programlamada **dinamik bağlama** (dynamic binding) kavramı, çok biçimlilikle yakından ilişkilidir. Python dilinde dinamik bağlama, bir yöntemi veya niteliği derleme zamanı yerine çalışma zamanında çözümleme sürecidir.

Soyut Sınıf (Abstract Class)

Nesne yönelimli programlama terminolojisinde, bir sınıfın örneği/nesnesi oluşturulamıyorsa, bu sınıfa **soyut sınıf** (**abstract class**) denir. Soyut sınıfları diğer sınıfları oluşturmak için temel veya üst sınıf olarak kullanabilirsiniz.

Python dilinde soyut bir sınıf oluşturmak için, **ABC** modülünde tanımlanan **ABC** sınıfını miras alması gerekir. Bu modül, Python dilinin standart kütüphanesinde mevcuttur. Ayrıca, sınıfın en az bir **soyut yöntem** (**abstract method**) olmalıdır. Bu soyut yöntem, çağrılmayan ancak geçersiz kılınabilen yöntemdir. Soyut yöntem, `@abstractmethod` dekoratörüyle süslenir.

```
from abc import ABC, abstractmethod
class Kişi(ABC):
    def __init__(self):
        self.yaş=20
        self.cinsiyet='E'
    def yaşınıSöyle(self):
        print('Yaşım:',self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:',self.cinsiyet)
    @abstractmethod
    def raporYaz(self):
        print('Soyut Yöntem')

# Kişi=Kişi() #Hata: Soyut Sınıftan Nesne İmal Edilemez!
```

Türemiş sınıfta sanal yöntemin geçersiz kılınması aşağıdaki şekilde yapılabilir;

```
from abc import ABC, abstractmethod
class Kişi(ABC):
    def __init__(self):
        self.yaş=20
        self.cinsiyet='E'
    def yaşınıSöyle(self):
        print('Yaşım:',self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:',self.cinsiyet)
    @abstractmethod
    def raporYaz(self):
        print('Soyut Yöntem')

class Öğrenci(Kişi):
    def __init__(self):
        super().__init__()
        self.ogrenciNo=0
    def dersÇalış(self):
        print('Ders Çalışıyorum')
    def raporYaz(self):
        super().raporYaz() #istenirse soyut yöntem çağrılmayabilir.
        print('Öğrenci Raporu: ...')

öğrenci=Öğrenci()
öğrenci.raporYaz()
```

Yukarıdaki kod çalıştırıldığında aşağıdaki çıktı elde edilir;

```
Soyut Yöntem
Öğrenci Raporu: ...
```

Ara Yüzler yani Roller (Interface)

Ara yüzler (interface) nesnelerin sahip oldukları rollerdir (role). Roller bir grup davranışı (behavior) barındırır. Saf nesne yönelimli dillerde ara yüzler interface saklı kelimesiyle tanımlanırlar. Python dilinde bu anahtar kelime bulunmadığından roller, sınıf (class) olarak kodlanır.

Örnek olarak bir öğretmen; Evde baba rolünde olup babanın sahip olduğu çocuklara bakma, yemek yapma gibi davranışları gösterir. İşte öğretmen rolündedir ve öğrenmenin sahip olduğu; öğretme, sınav/değerlendirme yapma, karne hazırlama, ders notu hazırlama gibi davranışları vardır. Bu öğretmen müzisyen rolünde ise gitar çalma, akort yapma gibi davranışları da vardır. İşte bunların hepsi ayrı bir roldür ve her bir rolde o role ilişkin davranışları sergiler.

Bir sınıfa, bir veya birden fazla ara yüz uygulanabilir (interface implementation). Python dilinde interface kelimesi olmadığından ara yüzler soyut yöntemler içeren soyut sınıflar olarak tasarlanır. Bir sınıfın bir arayüzü miras almasına ara yüz gerçekleştirme (interface realization) adı da verilir.

Python dili, saf nesne yönelimli dillerin (Java, C# gibi) aksine çoklu kalıtımı (multiple inheritance) destekler, çünkü interface gibi bir anahtar kelimeye sahip olmadığından ara yüz olarak tanımlanmış birçok sınıftan miras alması gerekebilir.

Aşağıda rollerin soyut sınıflar üzerinden resmi (formal) tanımlaması yapılmıştır.

```
from abc import ABC, abstractmethod
class IBaba(ABC):
    @abstractmethod
    def cocugaBak(self):
        return
    @abstractmethod
    def HanımaYardımEt(self):
        return
class IMuzisyen(ABC):
    @abstractmethod
    def gitarCal(self):
        return
    @abstractmethod
    def piyanoCal(self):
        return
class IOgretmen(ABC):
    @abstractmethod
    def ogret(self):
        return
    @abstractmethod
    def dersNotuHazirla(self):
        return

class Kisi(IBaba, IMuzisyen, IOgretmen): # arayüz gerçekleştirme.
    def __init__(self):
        self.yaş=20
        self.cinsiyet='E'
    def yaşınıSöyle(self):
        print('Yaşım:',self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:',self.cinsiyet)
    #IBaba Interface Yöntemleri
    def cocugaBak(self):
        print('Çocuğa Bakıyorum...')
    def HanımaYardımEt(self):
        print('Hanıma Yardım Ediyorum...')
    #IMuzisyen Interface Yöntemleri
    def gitarCal(self):
        print('Gitar Çalıyorum...')
```

```

def piyanoCal(self):
    print('Piyano Çalıyorum...')
#IOgretmen Interface Yöntemleri
def ogret(self):
    print('Öğretiyorum...')
def dersNotuHazirla(self):
    print('Ders Notu Hazırlıyorum...')

kisi=Kisi()
kisi.cocugaBak() #IBaba Rolü davranışı
kisi.gitarCal() #IMuzisyen Rolü davranışı
kisi.dersNotuHazirla() #IOgretmen rolü davranışı

```

Aynı örneğin **gayri resmi** (informal) tanımlaması aşağıda verilmiştir.

```

class IBaba(object):
    def cocugaBak(self):
        pass
    def HanımaYardımEt(self):
        pass
class IMuzisyen(object):
    def gitarCal(self):
        pass
    def piyanoCal(self):
        pass
class IOgretmen(object):
    def ogret(self):
        pass
    def dersNotuHazirla(self):
        pass

class Kisi(IBaba, IMuzisyen, IOgretmen):
    def __init__(self):
        self.yaş=20
        self.cinsiyet='E'
    def yaşınıSöyle(self):
        print('Yaşım:',self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:',self.cinsiyet)
    #IBaba Interface Yöntemleri
    def cocugaBak(self):
        print('Çocuğa Bakıyorum...')
    def HanımaYardımEt(self):
        print('Hanıma Yardım Ediyorum...')
    #IMusizyen Interface Yöntemleri
    def gitarCal(self):
        print('Gitar Çalıyorum...')
    def piyanoCal(self):
        print('Piyano Çalıyorum...')
    #IOgretmen Interface Yöntemleri
    def ogret(self):
        print('Öğretiyorum...')
    def dersNotuHazirla(self):
        print('Ders Notu Hazırlıyorum...')

kisi=Kisi()
kisi.cocugaBak() #IBaba Rolü davranışı
kisi.gitarCal() #IMuzisyen Rolü davranışı
kisi.dersNotuHazirla() #IOgretmen rolü davranışı

```

İç Sınıf (Inner Class)

Python dilinde bir sınıfın içinde tanımlanan sınıfa **iç sınıf** (**inner class**) denir. İç sınıfa **iç içe sınıf** (**nested class**) adı da verilir. İç sınıftan bir nesne imal edilirse yani örneklendirildiğinde bu nesne, ana sınıf tarafından da kullanılabilir. İç sınıf nesnesi, dış sınıfın **niteliklerinden** (**attribute**) biri haline gelir. İç sınıf, **kalıtımı** (**inheritance**) kodlamadan dış sınıfın niteliklerini otomatik olarak devralır. Bir sınıfta **birden fazla iç sınıf** (**multiple inner class**) tanımlanabileceği gibi **istenildiği kadar iç içe sınıf** (**multilevel inner class**) tanımlanabilir.

```
class Sirket: #outer class
    def __init__(self):
        self.Bolum1 = self.Muhasebe()
        self.Bolum2 = self.Pazarlama()
    def SirketAdiniYaz(self):
        print("Şirket Adı: Python A.Ş")

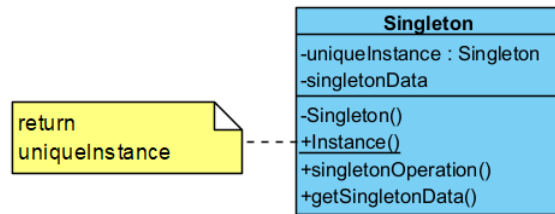
    class Muhasebe: #inner class
        def BolumunAdiniYaz(self):
            print("Bölüm Adı: Muhasebe")

    class Pazarlama: #inner class
        def BolumunAdiniYaz(self):
            print("Bölüm Adı: Pazarlama")

disNesne = Sirket()
disNesne.SirketAdiniYaz() # Şirket Adı: Python A.Ş
icNesne1 = disNesne.Bolum1
icNesne1.BolumunAdiniYaz() # Bölüm Adı: Muhasebe
icNesne2= disNesne.Bolum2
icNesne2.BolumunAdiniYaz() # Bölüm Adı: Pazarlama
```

Tekil Nesne Sınıfı (Singleton)

Tekil nesne sınıfı (**singleton class**), singleton **tasarım deseninin** (**design pattern**) uygulanmasıdır; bu da sınıfın yalnızca bir nesneye sahip olabileceği anlamına gelir. Yoğun bir işlem gerçekleştirdiğinizde bellek kullanımını optimize etmenize yardımcı olur.



Şekil 23. Tekil Deseni UML Diyagramı

Python dilinde `__new__` veya `__init__` yöntemlerinin birini kullanarak tekil nesne sınıfı oluşturabiliriz. Aşağıda `__init__` yöntemiyle sınıfın kodlaması gösterilmiştir;

```
class Singleton:
    __tekilnesne = None # mahrem (private) nesne referansı

    @staticmethod
    def nesneImalEt():
        if Singleton.__tekilnesne == None:
            Singleton()
        return Singleton.__tekilnesne

    def __init__(self):
        if Singleton.__tekilnesne != None:
            raise Exception("Object exist!")
```

```

else:
    Singleton.__tekilnesne = self

nesne1 = Singleton.nesneImalEt()
nesne2 = Singleton.nesneImalEt()
if nesne2 is nesne1:
    print("İki Nesne AYNIDIR!")

```

Alternatif olarak `__new__` yöntemi kullanılarak tekil nesne sınıfı kodlanabilir.

```

class SingletonClass:
    __tekilNesne = None #mahrem (private) tekil nesne referansı

    def __new__(cls):
        if cls.__tekilNesne is None:
            cls.__tekilNesne = super(SingletonClass, cls).__new__(cls)
        return cls.__tekilNesne

nesne1 = SingletonClass()
nesne2 = SingletonClass()
if nesne2 is nesne1:
    print("İki Nesne AYNIDIR!")

```

Yansıma (Reflection)

Nesne yönelimli programlamada **yansıma** (**reflection**), imal edilmiş bir nesnenin tipi, nitelikleri, davranışları, bir başka sınıfın alt sınıfı olup olmadığı gibi birçok konuda bilgisini çıkarma yeteneğidir. Python dilinin sağladığı standart kütüphane, bir nesnenin farklı özelliklerini yansıtan çeşitli işlevlere sahiptir. Yansımaya bazen **iç gözlem** (**introspect**) de denir. Aşağıda bu fonksiyonlar verilmiştir;

type() fonksiyonu

Bir nesnenin hangi sınıfa ait olduğunu söyler.

```

print (type(100)) #<class 'int'>
print (type(7.5)) #<class 'float'>
print (type(5+2j)) #<class 'complex'>
print (type("İlhan")) #<class 'str'>
print (type([10,20,30])) #<class 'list'>
print (type((10,20,30))) #<class 'tuple'>
print (type({'1':'bir', 2:'iki'})) #<class 'dict'>

class Sinif:
    pass
nesne=Sinif()
print(type(nesne)) #<class '__main__.Sinif'>

```

isinstance() fonksiyonu

Bir nesnenin verilen sınıfın bir örneği olup olmadığını yani verilen sınıftan imal edilip edilmediğini belirler. Aşağıdaki kod örneğinde bu fonksiyon `True` değerini geri döndürür.

```

print (isinstance(100, int))
print (isinstance(7.5, float))
print (isinstance(5+2j, complex))
print (isinstance("İlhan", str))
print (isinstance([10,20,30], list))
print (isinstance((10,20,30), tuple))
print (isinstance({'1':'bir', 2:'iki'}, dict))

class Sinif:
    pass
nesne=Sinif()

```



```
print(isinstance(nesne, Sinif))
```

issubclass() fonksiyonu

Bu fonksiyon bir sınıfın başka bir sınıfın alt sınıfı olup olmadığını kontrol eder. Sınıflar için kullanılır. Tüm Python sınıfları `object` sınıfından alt sınıflara ayrılır. Dolayısıyla, aşağıdaki `print` ifadelerinin çıktısı hepsi için `True` olacaktır.

```
class Sinif:
    pass

class AltSinif(Sinif):
    pass

print (issubclass(int, object))
print (issubclass(str, object))
print (issubclass(Sinif, object))
print (issubclass(AltSinif, object))
print (issubclass(AltSinif, Sinif))
```

callable() fonksiyonu

Bir nesne, belirli bir davranışı çağırıyorsa **çağrılabilir** (**callable**). Belirli bir davranışı gerçekleştiren bir Python fonksiyonu ise çağrılabilir bir nesnedir. Herhangi bir fonksiyon, **dahili** (**built-in**), **kullanıcı tanımlı** (**user-defined**) veya **yöntem** (**method**) çağrılabilir. `int`, `str` ve benzeri gibi dahili veri tiplerine sahip nesneler çağrılmaz. Bir sınıf örneği, `__call__()` y sahipse çağrılabilir.

```
def yontem():
    pass

print (callable("Hello")) #False
print (callable(12)) #False
print (callable(min)) #True
print (callable([1,2])) #False
print (callable(list.clear([1,2]))) #False
print (callable(yontem)) #True

class Sinif:
    def __init__(self):
        pass
    def __call__(self):
        print ("Bu Sınıftan İmal Edilen Nesneler Çağrılabilir!")

nesne = Sinif()
print ("callable?", callable(nesne)) #True
```

getattr() fonksiyonu

Bu fonksiyon, bir nesnenin adlandırılmış niteliğinin değerini almak için kullanılır.

```
class Kisi:
    def __init__(self):
        self.Adi = "İlhan"
        self.Soyadi="Özkan"
        self.Yasi=50

nesne = Kisi()
print (getattr(nesne, "Adi")) #İlhan
print (getattr(nesne, "Soyadi")) #Özkan
print (getattr(nesne, "Yasi")) #50
```

setattr() fonksiyonu

Bu fonksiyon, bir nesneye yeni bir **nitelik** (**attribute**) ekler ve ona bir değer atar. Ayrıca mevcut bir niteliğin değerini de değiştirebilir.

```
class Kisi:
    def __init__(self):
        self.Adi = "İlhan"

nesne = Kisi()
print (getattr(nesne, "Adi")) #İlhan

setattr(nesne, "Adi", "Mehmet")
setattr(nesne, "Soyadi", "Özkan")
setattr(nesne, "Yasi", 50)
print (nesne.Adi) #Mehmet
print (nesne.Soyadi) #Özkan
print (nesne.Yasi) #50
```

hasattr() fonksiyonu

Bu fonksiyon, nesnenin verilen niteliğe sahip olup olmadığını belirler. Sahipse **True** değerini, aksi durumda **False** değerini döndürür.

```
class Kisi:
    def __init__(self):
        self.Adi = "İlhan"
        self.Soyadi="Özkan"
        self.Yasi=50

nesne = Kisi()
print (hasattr(nesne, "Adi")) #True
print (hasattr(nesne, "Cinsiyeti")) #False
print (hasattr(nesne, "Kilosu")) #False
```

dir() fonksiyonu

Bu yerleşik fonksiyon argüman olmadan çağrılırsa, geçerli kapsamdaki adları döndürür. Herhangi bir nesne argüman olarak kullanıldığında, belirtilen nesnenin ve bu nesneden erişilebilen niteliklerin bir listesini döndürür.

```
print("dir(int):",dir(int))
print("dir(list):",dir(list))

class Kisi:
    def __init__(self):
        self.Adi = "İlhan"
        self.Soyadi="Özkan"
        self.Yasi=50
    def AdivSoyadiEkranaYaz(self):
        print("Adı:",self.Adi)
        print("Soyadı:", self.Soyadi)
    def YasiEkranaYaz(self):
        print("Yaşı:",self.Yas)

print("dir(Kisi):",dir(Kisi))
```

Kod çalıştırıldığında aşağıdaki çıktı elde edilir.

```
dir(int): ['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__',
 '__delattr__', '__dir__', '__divmod__', '__doc__', '__eq__', '__float__', '__floor__',
 '__floordiv__', '__format__', '__ge__', '__getattr__', '__getnewargs__', '__getstate__',
 '__gt__', '__hash__', '__index__', '__init__', '__init_subclass__', '__int__', '__invert__',
```

```
'__le__', '__lshift__', '__lt__', '__mod__', '__mul__', '__ne__', '__neg__', '__new__',
'__or__', '__pos__', '__pow__', '__radd__', '__rand__', '__rdivmod__', '__reduce__',
'__reduce_ex__', '__repr__', '__rfloordiv__', '__rlshift__', '__rmod__', '__rmul__',
'__ror__', '__round__', '__rpow__', '__rrshift__', '__rshift__', '__rsub__', '__rtruediv__',
'__rxor__', '__setattr__', '__sizeof__', '__str__', '__sub__', '__subclasshook__',
'__truediv__', '__trunc__', '__xor__', 'as_integer_ratio', 'bit_count', 'bit_length',
'conjugate', 'denominator', 'from_bytes', 'imag', 'is_integer', 'numerator', 'real',
'to_bytes']
dir(list): ['__add__', '__class__', '__class_getitem__', '__contains__', '__delattr__',
'__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattr__',
'__getitem__', '__getstate__', '__gt__', '__hash__', '__iadd__', '__imul__', '__init__',
'__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__',
'__new__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__',
'__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook__', 'append', 'clear',
'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
dir(Kisi): ['AdiveSoyadiEkранаYaz', 'YasiEkранаYaz', '__class__', '__delattr__', '__dict__',
'__dir__', '__doc__', '__eq__', '__firstlineno__', '__format__', '__ge__', '__getattr__',
'__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__',
'__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__',
'__sizeof__', '__static_attributes__', '__str__', '__subclasshook__', '__weakref__']
```

Dekoratorlar

Yazılım mühendisliğinde "Open/Closed" prensibi, bir nesnenin geliştirilmeye açık ama değiştirilmeye kapalı olması gerektiğini savunur. Python'da bir fonksiyonun veya sınıfın davranışını, onun kaynak koduna dokunmadan değiştirmemizi veya genişletmemizi sağlayan yapıya **dekoratör** (**decorator**) denir. Dekoratorlar, aslında başka bir fonksiyonu parametre olarak alan ve yeni bir fonksiyon döndüren **yüksek seviyeli fonksiyonlardır** (**higher-order functions**).

Kısaca **dekoratörler** (**decorator**), fonksiyonların veya yöntemlerin kodlarını değiştirmeden davranışlarını değiştirmenin veya genişletmenin esnek bir yoludur.

Fonksiyonların Nesne Olarak Kullanılması

Dekoratör mantığını anlamak için Python'da fonksiyonların da birer "nesne" (first-class object) olduğunu hatırlamak gerekir. Fonksiyonlar; değişkenlere atanabilir, başka fonksiyonlara parametre olarak gönderilebilir veya bir fonksiyonun içinden geri döndürülebilir.

Dekoratör Söz Dizimi ve @ Operatörü

Python'da bir fonksiyonu dekore etmek için `@dekorator_adi` ifadesi kullanılır. Bu, kodun okunabilirliğini artıran bir sentaktik şekerdir (syntactic sugar).

Temel Bir Dekoratör Tasarımı

Aşağıdaki örnekte, bir fonksiyonun ne kadar sürede çalıştığını ölçen bir mühendislik aracı tasarlayalım:

```
import time
def sure_olcer(fonksiyon):
    def sarici(*args, **kwargs):
        baslangic = time.time()
        sonuc = fonksiyon(*args, **kwargs) # Asıl fonksiyon çalıştırılıyor
        bitis = time.time()
        print(f"{fonksiyon.__name__} çalışma süresi: {bitis - baslangic} sn.")
        return sonuc
    return sarici

@sure_olcer
def veri_isle(n):
    # Ağır bir hesaplama simülasyonu
    return sum(range(n))
```

```
# Kullanım
veri_isle(100000000)
```

Parametrelili Dekoratorler ve *args, **kwargs

Bir dekoratörün her türlü fonksiyona uyum sağlayabilmesi için (farklı sayıda parametre alan fonksiyonlar gibi), sarmalayıcı (wrapper) fonksiyonun esnek bir yapıda olması gerekir. Bu noktada kitabımızın 6. bölümünde işlediğimiz *args ve **kwargs yapıları devreye girer.

@functools.wraps Kullanımı

Bir fonksiyonu dekore ettiğinizde, o fonksiyonun teknik kimliği (ismi, dokümantasyon dizini vb.) dekoratörün içindeki sarmalayıcı fonksiyonla yer değiştirir. Bu durum, hata ayıklama (debugging) süreçlerinde kafa karışıklığına yol açabilir. Bu mühendislik zafiyetini önlemek için functools.wraps kullanılır.

```
from functools import wraps
def log_kaydi(f):
    @wraps(f)
    def wrapper(*args, **kwargs):
        print(f"{f.__name__} fonksiyonu çağrıldı.")
        return f(*args, **kwargs)
    return wrapper
```

Neden Dekoratör Kullanırsınız?

- Yetkilendirme (authorization): Kullanıcının bir fonksiyonu çalıştırma yetkisi olup olmadığını kontrol etmek.
- Loglama (logging): Fonksiyon çağrılarını ve çıktılarını merkezi bir dosyada kayıt altına almak.
- Önbelleğe Alma (caching): Ağır hesaplamaların sonuçlarını belleğe alarak performansı artırmak (@lru_cache).
- Hata Yönetimi: Fonksiyonun belirli hata durumlarında otomatik olarak yeniden denenmesini (retry) sağlamak.

Dekoratorler, kodun içine enine kesen ilgiler (cross-cutting concerns) eklemek için mükemmeldir. Ana iş mantığınızı (business logic) kirlletmeden, yardımcı işlemleri bu yapılarla yönetebilirsiniz.

VERİ YAPILARI

Python dilinde kullanılan veri yapılarının her biri nesnedir. Nesneler davranışları ve özellikleri olan varlıklardır. Dolayısıyla bu dildeki veri yapılarının her birinin kendine has **yöntemleri** (**method**) bulunur. C/C++/C# ve Java gibi dillerinde bulunan int, float gibi temel veri tiplerinin aksine Python dilinde tüm **değişkenler** (**variable**) aslında var olan bir **nesneye** (**object**) tanımlanan bir referanstır.

Python dilinde dahili veri tipleri olan **bool**, **int**, **float**, **complex** ve **str** gibi veri yapılarında **değişken tanımlama** (**definition**) sırasında nesne **yapıcıları** (**constructor**) örtük olarak kullanılır. Aslında **list**, **tuple**, **set** ve **dict** veri yapılarında ise sırasıyla **[]**, **()**, **{}**, **{}** karakterleri ile örtük olarak bu veri yapılarının yapıcıları çağrılır.

Dizi (array)

Şu ana kadar en basit **veri yapıları** (**data structure**) olan **değişkenlerle** (**variable**) karşılaştık. Programlamada kullanılan bir başka veri yapısı da **dizilerdir** (**array**). Matematikte diziler; bir sıralı elemanlardan oluşan bir listedir. Sıralı elemanların sayısına **dizinin uzunluğu** (**array size**) denir. Elemanların sırası **indis** (**index**) olarak adlandırılır. Kümenin aksine aynı elemanlar dizide farklı konumlarda birkaç kez bulunabilir. Elemanlara terim adı da verilir.

Örneğin Elemanları 1'den 10'a kadar sayıların karesinden oluşan bir dizi; $(a_k)_{k=1}^{10}, a_k = k^2$ veya $(a_1, a_2, a_3, \dots, a_{10})$ veya $(1, 4, 9, \dots, 100)$ olarak gösterilir. Örneğin;

- (K, İ, T, A, P) Dizisi ile (P, A, T, İ, K) dizisi birbirinden farklıdır. Aynı elemanlardan oluşmasına rağmen elemanların sıraları farklıdır.
- (1, 3, 5, 1, 2, 1, 7) Dizisinde birden farklı indiste aynı 1 elemanı vardır. Bu dizinin geçersiz olduğu anlamına gelmez.
- Matematikte sonsuz elemanı olan sonsuz diziler tanımlanabilir. Ama programlamada hep sonlu elemanlı diziler yani **uzunluğu** (**size**) belirli diziler kullanılır.

Python dilindeki **diziler** (**array**), C, C++ ve C# ve Java gibi geleneksel programlama dillerindeki dizilere benzer. Aynı veri tipindeki üyelerden oluşan homojen bir veri yapısıdır. **Listeler** (**list**), dizilere benzer ancak elemanları farklı veri tiplerinden oluşabilir. Yani bir **liste** (**list**), homojen ya da heterojen elemanların bir koleksiyonu olabilir.

Bireysel elemanlara **indisler** (**index**) aracılığıyla erişilebilir. Python dizilerinde indisler her zaman sıfırdan başlar. Bir dizi aşağıdaki şekilde tanımlanır;

```
diziKimliği = array(tipKodu, [Başlatıcılar])
```

Python dilinde dizileri kullanmak için standart **array** modülünü **içe aktarmanız** (**import**) gerekir.

```
from array import *
tamsayıDizisi = array('i', [1, 2, 3, 1, 3])
print(tamsayıDizisi) #diziyi konsola yazma
print(tamsayıDizisi[0]) #ilk elemanı konsola yazma
print(tamsayıDizisi[1]) #ikinci elemanı konsola yazma
#Tüm elemanları yineleyici ile konsola basma:
for i in tamsayıDizisi:
    print(i)
diziuzunluğu=Len(tamsayıDizisi)
print(diziuzunluğu) #5
```

Diziyi oluşturan ilk parametre elemanların veri tipini belirler. Bu parametre;

Parametre	Eleman Veri Tipi
b	1 bayt boyutunda işaretli tam sayı
B	1 bayt boyutunda işaretli tam sayı
u	2 bayt boyutunda Unicode karakter

h	2 bayt boyutunda işaretli tam sayı
H	2 bayt boyutunda işaretli tam sayı
i	2 bayt boyutunda işaretli tam sayı
I	2 bayt boyutunda işaretli tam sayı
l	4 bayt boyutunda işaretli tam sayı
L	4 bayt boyutunda işaretli tam sayı
f	4 bayt boyutunda kayan noktalı sayı
d	8 bayt boyutunda kayan noktalı sayı

Tablo 16. Dizi Eleman Veri Tiplerini Belirleyen Parametreler

Dizilerin sonuna bir ekleme `append` ile veya indis belirterek istenilen yere `insert` ile yeni eleman eklenebilir.

```
from array import *
tamsayıDizisi = array('i', [1,2,3,1,3])
tamsayıDizisi.append(100) # dizi sonuna 100 sayısını ekleme
print(tamsayıDizisi) # array('i', [1, 2, 3, 1, 3, 100])
tamsayıDizisi.insert(0,200) # ilk eleman olarak 200 ekleme
print(tamsayıDizisi) # array('i', [200, 1, 2, 3, 1, 3, 100])
```

Dizilerin sonundaki eleman `pop` ile silinir veya indis belirterek istenilen konumdaki eleman `remove` ile yeni eleman silinebilir.

```
from array import *
tamsayıDizisi = array('i', [1,2,3,1,3])
tamsayıDizisi.pop() # dizi sonundaki eleman silinir
print(tamsayıDizisi) # array('i', [1, 2, 3, 1])
tamsayıDizisi.remove(2) # 2 değerine sahip eleman silinir
print(tamsayıDizisi) # array('i', [1, 3, 1])
```

Bir dizi aynı tipten elemanları olan bir başka dizi ile `extend` yöntemi ile birleştirilebilir. Ayrıca bir listeden de dizi sonuna `fromList` yöntemi ile eleman eklenebilir.

```
from array import *
dizi = array('i', [1,2,3,4,5])
eklenecekDizi = array('i', [7,8,9,10])
dizi.extend(eklenecekDizi)
print(dizi) #array('i', [1, 2, 3, 4, 5, 7, 8, 9, 10])
liste=[11,12,13]
dizi.fromList(liste)
print(dizi) #array('i', [1, 2, 3, 4, 5, 7, 8, 9, 10, 11, 12, 13])
```

Argüman olarak verilen değer ile eşleşen ilk indisi `index` döndürür.

```
from array import *
dizi = array('i', [1,2,3,4,1])
print(dizi.index(4)) #3
print(dizi.index(1)) #0
```

Dizideki eleman konumları tersine çevrilmek istenirse `reverse` yöntemi kullanılır.

```
from array import *
dizi = array('i', [1,2,3,4,1])
dizi.reverse()
print(dizi) #array('i', [1, 4, 3, 2, 1])
```

Bellekteki dizi tamponunun (`buffer`) başlangıç adresini ve dizideki öge sayısını `buffer_info` yöntemi verir.

```
from array import *
dizi = array('i', [1,2,3,4,1])
print(dizi.buffer_info()) #(140514698296784, 5)
```

Bir öğenin bir dizide kaç kez görüldüğünü `count` yöntemi döndürecektir.

```
from array import *
dizi = array('i', [1,2,3,4,1])
print(dizi.count(1)) #2
```

Bir `dizgi(string)` `encode` yöntemiyle her bir karakteri sayıya kodlanır. Ardından bayt dizisine `frombytes` yöntemi ile eklenebilir. Tersine bir bayt dizisinden `decode` ile karakterlere dönüştürülecek sayılara dönüştürülebilir.

```
from array import *
baytdizizi = array('b')
baytdizizi.frombytes('İlhan'.encode())
print(baytdizizi) #array('b', [-60, -80, 108, 104, 97, 110])
metin=baytdizizi.tobytes()
print(metin) #b'\xc4\xbc\x01han'
print(metin.decode()) #İlhan
```

Dizilerde bir kısım hazır/dahili (bult-in) fonksiyonlar kullanılır;

```
from array import *
dizi = array('i', [1,2,3,4,1])
print('Eleman Sayısı:', len(dizi)) #5
print('En Küçük Eleman:', min(dizi)) #1
print('En Büyük Eleman:', max(dizi)) #4
```

Bu duruma örnek `len` fonksiyonunu içeren olarak 10 adet satış miktarlarının, ortalamaya olan uzaklıkları yani `sapma (deviation)` ve karesi alınmış sapmalar yani `varyans (variance)` ile standart sapmayı yazdıran program örnek verilebilir.

```
from math import sqrt
satışlar=[100.0,850.0,500.0,600.0,750.0,650.0,450.0,800.0,900.0,110.0]
toplam=0.0
for satış in satışlar:
    toplam+=satış
ortalama=toplam/len(satışlar)
print('Ortalama', ortalama)
varyanstoplam=0.0
for satış in satışlar:
    sapma=satış-ortalama
    print('Sapma',sapma)
    varyanstoplam=sapma*sapma
standartsapma=sqrt(varyanstoplam/len(satışlar))
print('Standart Sapma',standartsapma)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Ortalama 571.0
Sapma -471.0
Sapma 279.0
Sapma -71.0
Sapma 29.0
Sapma 179.0
Sapma 79.0
Sapma -121.0
Sapma 229.0
Sapma 329.0
Sapma -461.0
Standart Sapma 145.7810001337623
```

Bir dizide indis negatif kullanılabilir. Bu durumda son elemandan geriye doğru elemana erişilir

```
from array import *
dizi = array('i', [1,2,3,4,1])
```

```
print(dizi[-2]) #4 #sondan ikinci eleman
print(dizi[-4]) #2 #sondan dördüncü eleman
```

Dizilerde **dilimleme (slicing)**, belirli bir diziden alt dizi oluşturmanın bir yoludur. Bu işlemde, dizinin bir kısmını veya parçasını çıkarırız. Python'da dilimleme işlemini gerçekleştirmek için dilim işlecini `[:]` kullanırız.

```
from array import *
dizi = array('i', [1,2,3,4,1])
altdizi = dizi[0:2] #ilk elemandan itibaren 2. indise kadar elemanı alarak alt dizi oluştur
print(altdizi) #array('i', [1, 2])
altdizi2 =dizi[1:3] #ikinci elemandan itibaren 3. indise kadar elemanı alarak alt dizi oluştur
print(altdizi2) #array('i', [2, 3])
```

Dilimlemede negatif indisler de kullanılabilir.

```
from array import *
dizi = array('i', [1,2,3,4,1])
altdizi = dizi[-3:-1] # sondan 3. elemandan sondan 1. elamana kadar alt dizi oluştur
print(altdizi) # array('i', [3, 4])
altdizi2 =dizi[-4:-2] # sondan 4. elemandan sondan 2. elamana kadar alt dizi oluştur
print(altdizi2) # array('i', [2, 3])
```

Liste (list)

Listeler (list) dizilerden farklı olarak homojen veri içermezler. Yani aşağıdaki gibi elemanları her tipten nesne olabilir ve aynı yöntemlere sahiptir.

```
liste=[1,'a',3.14, 1+2j, [4,5,6],(7,8,9)]
print(liste) #[1, 'a', 3.14, (1+2j), [4, 5, 6], (7, 8, 9)]
```

C/C++, C# ve Java dillerinin aksine iki boyutlu diziler yani **matrisler (matrix)**, liste içeren listeler şeklinde tanımlanabilir.

```
matris=[[1,2,3],[4,5,6],[7,8,9]]
print(matris[0]) #[1, 2, 3] # birinci satır
print(matris[1]) #[4, 5, 6] # ikinci satır
print(matris[2]) #[7, 8, 9] # üçüncü satır
print(matris[0][1]) #2 # birinci satır ikinci sütun elemanı
print(matris[1][1]) #5 # ikinci satır ikinci sütun elemanı
print(matris[2][1]) #8 # üçüncü satır ikinci sütun elemanı
matris[2][2]=100
print(matris) #[[1, 2, 3], [4, 5, 6], [7, 8, 100]]
```

Benzer şekilde üç boyutlu ya da çok boyutlu diziler tanımlanabilir.

```
üçboyutlu=[[11,12,13],[14,15,16],[17,18,19]],\
           [[21,22,23],[24,25,26],[27,28,29]],\
           [[31,32,33],[34,35,36],[37,38,39]]]
print(üçboyutlu[0]) # [[11, 12, 13], [14, 15, 16], [17, 18, 19]] # ilk matris
print(üçboyutlu[1]) # [[21, 22, 23], [24, 25, 26], [27, 28, 29]] # ikinci matris
print(üçboyutlu[2][0]) # [31, 32, 33] # üçüncü matrisin ilk satırı
print(üçboyutlu[1][0][1]) # 22 # ikinci matrisin ilk satırının ikinci sütun elemanı
```

Dizilerde olan ve dizi ve liste nesnelerinin kendine has yöntemleri vardır;

Yöntem	Açıklama
append()	Listenin sonuna bir eleman ekler
clear()	Tüm elemanları siler
copy()	Listenin bir kopyasını döndürür
count()	Belirtilen değerde listenin sahip olduğu eleman sayısını döndürür
extend()	Listeye bir başka liste ekleyerek genişletir
index()	Verilen bir değerın listede bulunan ilk indisini döner

insert()	Belirtilen indis konumuma yeni bir eleman ekler
pop()	Belirtilen indis konumundan bir elemanı siler
remove()	Belirtilen değerdeki ilk elemanı siler
reverse()	Listeyi ters çevirir
sort()	Değerlere göre listeyi sıralar

Tablo 17. Dizi ve Liste Nesnesi Yöntemleri

Liste yöntemleri, bir listenin son giren elemanı ilk çıkar (**Last In First Out-LIFO**) mantığı yani **yığın (stack)** mantığında kullanmayı da sağlar;

```
yığın=[1,2,3]
yığın.append(4) # sonuna 4 eklendi
yığın.append(5) # sonuna 5 eklendi
çıkan=yığın.pop()
print(çıkan) #5
```

Bir listedeki yinelenen değerleri kaldırmak, listeyi bir kümeye (yani, farklı nesnelerin sırasız bir koleksiyonu) dönüştürerek yapılabilir.

```
meyveler = ["armut", "elma", "erik", "elma", "armut"]
tekilmeyveler=list(set(meyveler))
print(tekilmeyveler) #['erik', 'armut', 'elma']
```

Liste ve dizilerde elemanlar üzerinde **yinelemeler (iteration)** aşağıdaki şekilde yapılabilir;

```
liste=[10,20,30,40,50]
for (indis,eleman) in enumerate(liste):
    print('{}.indisteki eleman={}'.format(indis,eleman))

for i in range(0,len(liste)):
    print(liste[i])
```

Bu kod çalıştığında vereceği çıktı aşağıda verilmiştir;

```
0.indisteki eleman=10
1.indisteki eleman=20
2.indisteki eleman=30
3.indisteki eleman=40
4.indisteki eleman=50
10
20
30
40
50
```

Listede bir elemanın varlığı **in** **işleci (operator)** ile test edildiği daha önce anlatılmıştı. Bir **yineleme (iterable)** içerisinde tüm değerlerin **True** olarak değerlendirip değerlendirmedini belirlemek için dahili **all()** fonksiyonunu kullanabilirsiniz.

```
liste=[1,2,3,0] #en az bir elemanı 0
all(liste) #False
liste=['a','b',0] #en az bir elemanı 0
all(liste) #False
liste=['a','b','c'] #hiç 0 değerli elemanı yok
all(liste) #True
```

Benzer şekilde, dahili **any()** fonksiyonu bir yinelemede bir veya daha fazla değer **True** olarak değerlendirip değerlendirmedini belirler.

```
liste=['a','b','c']
any(liste) #True
liste=[0,0,0] #Tüm değerler 0
any(liste) #False
```

Tersinen bir liste döndüren ters yani `reversed()` dahili fonksiyonunu kullanarak tersine bir yineleyici elde ederiz.

```
sayılar=[1,2,3,4]
for eleman in reversed(sayılar):
    print(eleman, end='-') #4-3-2-1-
terstensayılar=sayılar[::-1] #yineleyici değil de tersine bir dizi için::-1 kullanılır
print(terstensayılar) # [4, 3, 2, 1]
```

Bir **liste kavrama** (**list comprehension**), bir **ifade** (**expression**) ile liste üzerinde **yineleme** (**iterable**) ile her bir elemana uygulayarak yeni bir liste oluşturur. Sözde kodu aşağıda verilmiştir;

```
[ <expression> for <element> in <iterable> ]
#yada
[ <expression> for <element> in <iterable> if <condition> ]
```

Aşağıda bir listedeki sayıların karelerinden yeni bir liste oluşturan örnek verilmiştir;

```
liste=[1,2,3,4,5]
kareler=[x*x for x in liste]
print(kareler) #[1, 4, 9, 16, 25]

kelimeler=['ilhan','Özkan', 'trabzon']
büyükkelimeler=[s.upper() for s in kelimeler]
print(büyükkelimeler) #['ILHAN', 'ÖZKAN', 'TRABZON']

onakadarçiftsayılar=[x for x in range(10) if x % 2 == 0]
print(onakadarçiftsayılar) # [0, 2, 4, 6, 8]

yıldızlı=[x if x in 'aeiou' else '*' for x in 'Şifre']
print(yıldızlı) # ['*', 'i', '*', '*', 'e']
```

İki liste **+** işleci ile birleştirilebilir.

```
liste1=[1,2,3]
liste2=['a','b','c']
birleşikliste=liste1+liste2
print(birleşikliste) #[1, 2, 3, 'a', 'b', 'c']
```

Dahili **sıkıştırma** (**zip**) fonksiyonu, iki listenin karşılıklı elemanlarından oluşan bir **demet** (**tuple**) üretir. Eleman sayısı az olan kadar demet üretilir.

```
liste1=[1,2,3] #3 elemanlı
liste2=['a','b','c'] #üç elemanlı
liste3=[10.5, 20.6] #iki elemanlı
for x in zip(liste1,liste2):
    print(x)
for x in zip(liste1,liste3):
    print(x)
```

Programın çıktısı aşağıdaki gibi olacaktır;

```
(1, 'a')
(2, 'b')
(3, 'c')
(1, 10.5)
(2, 20.6)
```

Bir listenin belirli sayıda elemanı olması isteniyorsa aşağıdaki gibi tanımlanabilir.

```
liste1 = [None] * 10 #veri tipi belirlenmemiş 10 elemanlı liste
print(liste1) #[None, None, None, None, None, None, None, None, None, None]
liste2= ['a']*5 #veri tipi str belirlenmiş 5 elemanlı liste
print(liste2) #['a', 'a', 'a', 'a', 'a']
```

Dizgi (str)

Python dilinde metinleri işlemek için **dizgi** (**string**) veri yapısı **str** ile tanımlanır. Dizgiler, Unicode karakterlerinin sıralı bir **dizisidir** (**array**). C++ dilindeki **string** veri yapısı gibidir. Yani tüm dizgi nesnelere ilişkin yöntemler, orijinal dizgiyi bozmaz ve yeni bir **str** nesnesi döndürür.

Yöntem	Açıklama
capitalize()	İlk harfi büyüterek döndürür
casefold()	Tüm harfleri küçüğe çevirerek döndürür
center()	Metni belirtilen karakter sayısına göre ortasına yerleştirir
count()	Belirtilen metnin dizgi içerisinde kaç defa geçtiğini döndürür
encode()	Metnin kodlanmış (encoded) halini döndürür.
endswith()	Belirtilen metin ile bitiyorsa dizgiyi döndürür.
expandtabs()	Belirtilen sekme miktarına göre dizgi içerisindeki sekmeleri (\t) genişleterek döndürür.
find()	Belirtilen metni dizgi içerisinde arar.
format()	Dizgi metnini biçimlendirme metni olarak kullanır. Biçimlendirilmiş metni döndürür.
format_map()	Dizgiyi bir sözlükten alınan değerler ile biçimlendirir
index()	Dizgide belirtilen bir değeri arar ve bulunduğu konumu döndürür
isalnum()	Dizgideki tüm karakterler alfasayısal (alphanumeric) ise True değerini döndürür
isalpha()	Dizgideki tüm karakterler alfabetik ise True değerini döndürür
isascii()	Dizgideki tüm karakterler ASCII karakterleriyse True değerini döndürür
isdecimal()	Dizgideki tüm karakterler ondalık sayıysa True değerini döndürür
isdigit()	Dizgideki tüm karakterler rakamsa True değerini döndürür
identifier()	Dizgi bir değişken kimliği (identifier) ise True değerini döndürür
islower()	Dizgideki tüm karakterler küçük harf ise True değerini döndürür
isnumeric()	Dizgideki tüm karakterler sayısal ise True değerini döndürür
isprintable()	Dizgideki tüm karakterler yazıcıda yazdırılabilirse True değerini döndürür
isspace()	Dizgideki tüm karakterler beyaz boşluksa (whitespace) True değerini döndürür
istitle()	Dizgi bir başlığın kurallarına (Her Sözcüğün İlk Harfi Büyük) uyuyorsa True değerini döndürür
isupper()	Dizgideki tüm karakterler büyük harf ise True değerini döndürür
join()	Bir yineleyicinin (iterable) öğelerini bir dizgiye dönüştürür
ljust()	Dizginin sola yaslanmış halini döndürür
lower()	Bir dizgiyi küçük harfe dönüştürür
lstrip()	Dizginin soldan kırılmış (boşluğu alınmış) halini döndürür
maketrans()	Çevirilerde kullanılacak bir çeviri tablosu döndürür
partition()	Dizenin üç parçaya bölündüğü bir demet (tuple) döndürür
replace()	Belirtilen bir değer bir başka değerle değiştirildiği bir dizgi döndürür
rfind()	Dizgide belirtilen bir değer aranır ve bulunduğu son konumu döndürür
rindex()	Dizgide belirtilen bir değer aranır ve bulunduğu son konumu döndürür
rjust()	Dizginin sağa yaslanmış bir sürümünü döndürür
rpartition()	Dizginin üç parçaya bölündüğü bir demet (tuple) döndürür
rsplit()	Dizgiyi sondan başa doğru belirtilen ayırıcı ile böler ve bir liste döndürür
rstrip()	Dizginin sağdan kırılmış (boşlukları alınmış) şeklini döndürür
split()	Dizgiyi belirtilen ayırıcı ile böler ve bir liste döndürür
splitlines()	Dizgiyi satır sonlarında böler ve bir liste döndürür

Yöntem	Açıklama
startswith()	Dizgi belirtilen değerle başlıyorsa True değerini döndürür
strip()	Dizinin kırılmış (boşlukları alınmış) şeklini döndürür
swapcase()	Harfleri değiştirir, küçük harf büyük harfe, büyük harf küçük harfe dönüşür ve dönüşmüş halini döndürür.
title()	Her kelimenin ilk karakterini büyük harfe dönüşmüş halini döndürür
translate()	Dizginin çevrilmiş halini döndürür
upper()	Bir dizgiyi büyük harfe dönüştürür
zfill()	Dizginin başını belirtilen sayıda 0 değerle doldurur

Tablo 18. Dizgi Nesnesi Yöntemleri

Aşağıdaki örnekte dizgi nesnesine ilişkin yöntemlerin birkaçı yer almaktadır.

```
metin=' ilhan özkan '
print(metin.lower()) # ilhan özkan
print(metin.upper()) # ILHAN ÖZKAN
print(metin.title()) # İlhan Özkan
print(metin.strip()) #ilhan özkan
print(metin.find('kan')) #9
print(len(metin)) #13 #Dahili fonksiyonlar da dizgilerle kullanılabilir.
print(metin.islower()) #True
print(metin.isprintable()) #True
```

Dizgiler üzerinde toplama (+) işleci sağ ve solundaki metni birleştirir. Çarpma (*) işleci ise **işlenenlerden (operand)** biri tamsayı olmak kaydıyla metni belirtilen tamsayı kadar çoğaltır;

```
metin1='İlhan'
metin2='özkan'
print(metin1,metin2) #İlhan özkan
print(metin1+metin2) #İlhanözkan
print(metin1*2) #İlhanİlhan
print(metin1*4) #İlhanİlhanİlhanİlhan
print(2*metin1) #İlhanİlhan
```

Birleştirilecek dizgi **değişmezleri (literal)** yan yana yazılarak da dizgiler oluşturulabilir;

```
metin='İlhan' " " "özkan" ',' ""Merhaba!""
print(metin) #İlhan özkan,Merhaba!
```

Dizgilere tek veya çift tırnak arasında yazılan değişmezler ile başlatılır. Metnin içerisinde geçecek tek ve çift tırnak karakterleri için değişmez içinde C dilindeki gibi ters bölü karakteri ile belirtilir.

```
metin='İlhan\'ın soyadı "Özkan" dır'
print(metin) #İlhan'ın soyadı "Özkan" dır
metin="İlhan\'ın soyadı \"Özkan\" dır"
print(metin) #İlhan'ın soyadı "Özkan" dır
```

Ters bölü ile belirtilen bu karakterlere **kaçış dizisi (escape sequence)** karakterleri adı verilir.

Karakter	Açıklama
\'	Tek Tırnak
\"	Çift Tırnak
\\	Ters Bölü
\n	Yeni satır (new line): İmleç (cursor), ↵ tuşu davranışı sergiler.)
\b	İmleç, bir karakter geri gider. Yani geri (backspace) tuşu ⇐ davranışı sergiler.
\r	Satır başı (carriage return): İmleç, ⏪ tuşu davranışı sergiler.)
\t	Sekme (Konsolda 8 ve katları sütuna ilerlenir): İmleç, → tuşu davranışı sergiler.
\f	Yeni sayfa (form feed): Yazıcıda yeni sayfaya geç anlamında kullanılır.
\ooo	Sekizlik (octal) Sayı Sisteminde Karakter Kodu
\xhh	Onlatalık (hexadecimal) Sayı Sisteminde Karakter Kodu

Tablo 19. Kaçış Dizisi Karakterleri

Aşağıda birkaç örnek verilmiştir;

```
metin = "\110\145\154\154\157"
print(metin)
metin = "\x48\x65\x6c\x6c\x66"
print(metin)
metin="İlhan\nÖzkan" # \n ile konsolda satır başı yapılır
print(metin)
metin="İlhan\tÖzkan" #\t ile konsolda 8 ve katları olan sütuna ilerlenir.
print(metin)
metin="İlhan\bÖzkan" #\b ile konsolda son konumdan bir karakter geri ilerlenir.
print(metin)
metin="Merhaba\rİlhan" #\r ile konsolda mevcut satırın başına ilerlenir.
print(metin)
```

Örneğin çalıştırıldığında çıktısı aşağıdaki gibi olacaktır

```
Hello
Hello
İlhan
Özkan
İlhan   Özkan
İlhaÖzkan
İlhanba
```

Demet (tuple)

Demet (tuple), değiştirilemez bir değerler listesidir ve aşağıdaki şekilde oluşturulabilirler.

```
demet1=() #elemanı olmayan bir demet (tuple)
demet2='a','b','c'
demet3='a',
demet4=('a')
demet5=('a',)
demet6=tuple(range(5)) #yapıcı kullanarak aralık nesnesinden demet yapma
print(demet6) #(0, 1, 2, 3, 4)
demet7=tuple("Demet") #yapıcı kullanarak str nesnesinden demet yapma
print(demet7) #('D', 'e', 'm', 'e', 't')
```

Demeti oluşturan nesneler **değiştirilemez** (immutable). Ancak yeni elemanlar ile genişletilebilir veya iki demet birleştirilebilir.

```
demet=(1,2,3,4)
#demet[2]=100 #TypeError: 'tuple' object does not support item assignment
baskaDemet=('a','b','c')
yeniDemet=demet+baskaDemet
print(yeniDemet) #(1, 2, 3, 4, 'a', 'b', 'c')
yeniDemet+=(1+4j,"İlhan")
print(yeniDemet) #(1, 2, 3, 4, 'a', 'b', 'c', (1+4j), 'İlhan')
```

Demetlere ilişkin yöntemler aşağıda verilmiştir.

Yöntem	Açıklama
count()	Belirtilen değerin demet içinde kaç kez bulunduğunu döndürür.
index()	Belirtilen değeri demet içinde arar ve bulunduğu yeri döndürür.

Bir demetin yalnızca bazı öğelerine ihtiyaç duyuluyorsa, altçizgi sembolü (_) geçici bir değişken adı olarak kullanılabilir ve **yer tutucu** (placeholder) olarak görev yapar.

```
demet = 1, 2, 3, 4
_, x, y, _ = demet # ikinci eleman x , üçüncü eleman y
if x == 2:
```

```
print(x+y) #5
```

Benzer şekilde bir demetin bitişim birkaç öğesine ihtiyaç duyuluyorsa, yıldız sembolü (*) ile geçici bir değişken adı kullanılabilir ve buna **paket açma** (**unpacking**) adı verilir.

```
demet = (1, 2, 3, 4)
a,*b,c = demet # ikinci ve üçüncü elemanı içeren bir liste b değişkenini oluşturur
print(b) # [2,3]
```

Küme (set)

Küme (set), herhangi bir veri tipinde ve **tekil (unique)** elemanlardan oluşan sırasız bir koleksiyondur. Bir nesne, ömrü boyunca asla değişmeyen bir **karma (hash)** değerine sahipse (ki bu nesne bir **hash()** yöntemine ihtiyaç duyar) ve diğer nesnelerle **karşılaştırılabilir** (bir **eq()** yöntemine ihtiyaç duyar) **karmalanabilir** (**hashable**). Eşitliği karşılaştıran karmalanabilir nesnelerin, aynı karma değerine sahip olması gerekir. Karma, bir nesneye eşsiz bir sayı olarak karşılık gelen değerlerdir.

```
a = {1, 2, 'a'}
print(a)
# Çıktı: {1, 2, 'a'}
b = {1, 1, 2, 2} # bir değer bir kümede yalnızca 1 kez yer alabilir
print(b)
# Çıktı: {1, 2}
c = set() # boş küme oluşturma
print(c)
# Çıktı: set()
d = set([1, 2, 3, 3, 4]) # listeden küme oluşturma
print(d)
# Çıktı: {1, 2, 3, 4}
```

Kümeler sırasız olduğundan indeks kullanılarak erişilemez. Ancak for döngüsü ile elemanlar üzerinde gezinilebilir.

```
renkler = {'kırmızı', 'yeşil', 'mavi'}
for renk in renkler:
    print(renk)
# Çıktı: mavi
# Çıktı: yeşil
# Çıktı: kırmızı
# in işleci ile eleman kontrolü
'kırmızı' in renkler
# Çıktı: True
'sarı' in renkler
# Çıktı: False
```

Küme elemanlarına aşağıdaki şekilde ekleme ve çıkarma yapılabilir;

```
meyveler = {'elma', 'armut'}
print(meyveler)
# Çıktı: {'elma', 'armut'}

# add() - tek eleman ekleme
meyveler.add('muz')
print(meyveler)
# Çıktı: {'elma', 'armut', 'muz'}

# update() - birden fazla eleman ekleme
meyveler.update(['portakal', 'çilek'])
print(meyveler)
# Çıktı: {'elma', 'armut', 'muz', 'portakal', 'çilek'}

# remove() - eleman çıkarma (yoksa hata verir)
meyveler.remove('armut')
```

```
print(meyveler)
# Çıktı: {'elma', 'muz', 'portakal', 'çilek'}

# discard() - eleman çıkarma (yoksa hata vermez)
meyveler.discard('kayısı') # hata vermez
meyveler.discard('muz')
print(meyveler)
# Çıktı: {'elma', 'portakal', 'çilek'}

# pop() - rastgele bir eleman çıkarır
eleman = meyveler.pop()
print(f"Çıkarılan: {eleman}, Kalan: {meyveler}")
# Çıktı: Çıkarılan: elma, Kalan: {'portakal', 'çilek'}

# clear() - tüm elemanları siler
meyveler.clear()
print(meyveler)
# Çıktı: set()
```

Kümeler matematikten bildiğimiz küme işlemlerini destekler. Aşağıda örnekleri verilmiştir;

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# Birleşim (union) - |
print(A | B)
# Çıktı: {1, 2, 3, 4, 5, 6, 7, 8}
print(A.union(B))
# Çıktı: {1, 2, 3, 4, 5, 6, 7, 8}

# Kesişim (intersection) - &
print(A & B)
# Çıktı: {4, 5}
print(A.intersection(B))
# Çıktı: {4, 5}

# Fark (difference) - -
print(A - B) # A'da olup B'de olmayanlar
# Çıktı: {1, 2, 3}
print(A.difference(B))
# Çıktı: {1, 2, 3}
print(B - A) # B'de olup A'da olmayanlar
# Çıktı: {6, 7, 8}

# Simetrik fark (symmetric_difference) - ^
print(A ^ B) # Ya A'da ya B'de ama ikisinde birden olmayanlar
# Çıktı: {1, 2, 3, 6, 7, 8}
print(A.symmetric_difference(B))
# Çıktı: {1, 2, 3, 6, 7, 8}
```

Alt küme işlemleri da aşağıdaki şekilde yapılır;

```
A = {1, 2, 3}
B = {1, 2, 3, 4, 5}
C = {1, 2}

# issubset() - alt küme mi?
print(A.issubset(B)) # A, B'nin alt kümesi mi?
# Çıktı: True
print(C.issubset(A)) # C, A'nın alt kümesi mi?
# Çıktı: True

# issuperset() - üst küme mi?
```

```
print(B.issuperset(A)) # B, A'nın üst kümesi mi?
# Çıktı: True
print(A.issuperset(C)) # A, C'nin üst kümesi mi?
# Çıktı: True

# isdisjoint() - kesişimleri boş mu?
D = {6, 7, 8}
print(A.isdisjoint(D)) # A ve D'nin ortak elemanı var mı?
# Çıktı: True
print(A.isdisjoint(B))
# Çıktı: False
```

Küme eleman sayısı ve küme kopyalama;

```
sayilar = {1, 2, 3, 4, 5}

# len() - eleman sayısı
print(len(sayilar))
# Çıktı: 5

# copy() - kümeyi kopyala
yeni_sayilar = sayilar.copy()
yeni_sayilar.add(6)
print(sayilar)
# Çıktı: {1, 2, 3, 4, 5}
print(yeni_sayilar)
# Çıktı: {1, 2, 3, 4, 5, 6}
```

Değiştirilemez kümeler (**frozenset**) de tanımlanabilir. Değiştirilemez kümeler, küme gibidir ancak elemanları **değiştirilemez** (**immutable**). Bu sayede sözlüklerde anahtar veya başka bir kümenin elemanı olarak kullanılabilir.

```
donuk_kume = frozenset([1, 2, 3, 4])
print(donuk_kume)
# Çıktı: frozenset({1, 2, 3, 4})

# Değiştirilemez
# donuk_kume.add(5) # AttributeError hatası verir

# Küme işlemleri çalışır
diger_kume = frozenset([3, 4, 5, 6])
print(donuk_kume | diger_kume)
# Çıktı: frozenset({1, 2, 3, 4, 5, 6})

# Sözlükte anahtar olarak kullanılabilir
sozluk = {donuk_kume: "değer"}
print(sozluk)
# Çıktı: {frozenset({1, 2, 3, 4}): 'değer'}
```

Kümeye tekrarlanan elemanları kaldırma aşağıdaki şekilde yapılabilir;

```
liste = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4]
tekil_liste = list(set(liste))
print(tekil_liste)
# Çıktı: [1, 2, 3, 4]
```

İki farklı kümenin ortak elemanlarını bulma aşağıdaki şekilde yapılabilir;

```
ogrenci1_dersler = {'matematik', 'fizik', 'kimya', 'biyoloji'}
ogrenci2_dersler = {'fizik', 'kimya', 'tarih', 'edebiyat'}
ortak_dersler = ogrenci1_dersler & ogrenci2_dersler
print(ortak_dersler)
# Çıktı: {'fizik', 'kimya'}
```


İki farklı kümenin farklı elemanlarını bulma aşağıdaki şekilde yapılabilir;

```
tum_dersler = ogrenci1_dersler | ogrenci2_dersler
print(tum_dersler)
# Çıktı: {'matematik', 'fizik', 'kimya', 'biyoloji', 'tarih', 'edebiyat'}
sadece_ogrenci1 = ogrenci1_dersler - ogrenci2_dersler
print(sadece_ogrenci1)
# Çıktı: {'matematik', 'biyoloji'}
```

Sözlük (dict)

Sözlük (**dict**), **anahtar-değer** (**key-value**) **çiftlerini** (**pair**) tekil olarak tutan bir koleksiyondur. Anahtarlar **karmalanabilir** (**hashable**).

```
a = {1: 'Elma', 2: 'Armut', 3: 'Muz'}
print(a)
# Çıktı: {1: 'Elma', 2: 'Armut', 3: 'Muz'}

b = {'a': [1, 2, 3], 'b': (1, 1), 'c': 'Özkan'}
print(b)
# Çıktı: {'a': [1, 2, 3], 'b': (1, 1), 'c': 'Özkan'}

# Boş sözlük oluşturma
c = {}
print(c)
# Çıktı: {}
d = dict()
print(d)
# Çıktı: {}
```

Sözlük elemanlarına erişim aşağıdaki şekilde yapılabilir;

```
ogrenci = {'isim': 'Ali', 'yas': 20, 'bolum': 'Bilgisayar'}

# Köşeli parantez ile erişim
print(ogrenci['isim'])
# Çıktı: Ali
print(ogrenci['yas'])
# Çıktı: 20

# get() metodu ile erişim
print(ogrenci.get('bolum'))
# Çıktı: Bilgisayar
print(ogrenci.get('not')) # Olmayan anahtar, None döner
# Çıktı: None
print(ogrenci.get('not', 0)) # Varsayılan değer belirtme
# Çıktı: 0

# Köşeli parantez ile olmayan anahtar KeyError verir
# print(ogrenci['not']) # KeyError: 'not'
```

Sözlüğe eleman ekleme ve güncelleme aşağıdaki şekilde yapılabilir;

```
kitap = {'baslik': 'Python Programlama', 'yazar': 'İlhan Özkan'}
print(kitap)
# Çıktı: {'baslik': 'Python Programlama', 'yazar': 'İlhan Özkan'}

# Yeni anahtar-değer ekleme
kitap['yil'] = 2025
kitap['sayfa'] = 500
print(kitap)
# Çıktı: {'baslik': 'Python Programlama', 'yazar': 'İlhan Özkan', 'yil': 2025, 'sayfa': 500}
```

```
# Mevcut değeri güncelleme
kitap['sayfa'] = 550
print(kitap)
# Çıktı: {'baslik': 'Python Programlama', 'yazar': 'İlhan Özkan', 'yıl': 2025, 'sayfa': 550}

# update() ile birden fazla ekleme/güncelleme
kitap.update({'dil': 'Türkçe', 'fiyat': 150})
print(kitap)
# Çıktı: {'baslik': 'Python Programlama', 'yazar': 'İlhan Özkan', 'yıl': 2025, 'sayfa': 550,
'dil': 'Türkçe', 'fiyat': 150}
```

Sözlükten eleman silme aşağıdaki şekilde yapılabilir;

```
urun = {'isim': 'Laptop', 'fiyat': 15000, 'stok': 5, 'kategori': 'Elektronik'}

# del ile silme
del urun['stok']
print(urun)
# Çıktı: {'isim': 'Laptop', 'fiyat': 15000, 'kategori': 'Elektronik'}

# pop() - anahtarı siler ve değeri döner
kategori = urun.pop('kategori')
print(f"Silinen: {kategori}, Kalan: {urun}")
# Çıktı: Silinen: Elektronik, Kalan: {'isim': 'Laptop', 'fiyat': 15000}

# popitem() - son eklenen anahtar-değer çiftini siler (Python 3.7+)
son = urun.popitem()
print(f"Silinen çift: {son}, Kalan: {urun}")
# Çıktı: Silinen çift: ('fiyat', 15000), Kalan: {'isim': 'Laptop'}

# clear() - tüm elemanları siler
urun.clear()
print(urun)
# Çıktı: {}
```

Sözlükte anahtar, değer ve anahtar-değer çiftleri üzerinde işlemler aşağıdaki şekilde yapılabilir;

```
notlar = {'Ali': 85, 'Ayşe': 92, 'Mehmet': 78, 'Fatma': 95}

# keys() - tüm anahtarları döner
print(notlar.keys())
# Çıktı: dict_keys(['Ali', 'Ayşe', 'Mehmet', 'Fatma'])

# values() - tüm değerleri döner
print(notlar.values())
# Çıktı: dict_values([85, 92, 78, 95])

# items() - tüm anahtar-değer çiftlerini döner
print(notlar.items())
# Çıktı: dict_items([('Ali', 85), ('Ayşe', 92), ('Mehmet', 78), ('Fatma', 95)])

# Döngü ile gezinme
for isim in notlar.keys():
    print(isim)
# Çıktı: Ali
# Çıktı: Ayşe
# Çıktı: Mehmet
# Çıktı: Fatma

for not_degeri in notlar.values():
    print(not_degeri)
# Çıktı: 85
# Çıktı: 92
```

```
# Çıktı: 78
# Çıktı: 95

for isim, not_degeri in notlar.items():
    print(f"{isim}: {not_degeri}")
# Çıktı: Ali: 85
# Çıktı: Ayşe: 92
# Çıktı: Mehmet: 78
# Çıktı: Fatma: 95

araba = {'marka': 'Toyota', 'model': 'Corolla', 'yil': 2023}

# in operatörü - anahtar var mı?
print('marka' in araba)
# Çıktı: True
print('renk' in araba)
# Çıktı: False

# not in operatörü
print('fiyat' not in araba)
# Çıktı: True

# len() - eleman sayısı
print(len(araba))
# Çıktı: 3
```

İç içe sözlükler aşağıdaki gibi işlenir;

```
okullar = {
    'lise1': {
        'isim': 'Atatürk Lisesi',
        'ogrenci_sayisi': 500,
        'sehir': 'Ankara'
    },
    'lise2': {
        'isim': 'Fen Lisesi',
        'ogrenci_sayisi': 300,
        'sehir': 'İstanbul'
    }
}

# İç içe sözlüğe erişim
print(okullar['lise1']['isim'])
# Çıktı: Atatürk Lisesi
print(okullar['lise2']['ogrenci_sayisi'])
# Çıktı: 300

# Tüm okullarda gezinme
for okul_id, okul_bilgi in okullar.items():
    print(f"\n{okul_id}:")
    for anahtar, deger in okul_bilgi.items():
        print(f"  {anahtar}: {deger}")

# lise1:
#   isim: Atatürk Lisesi
#   ogrenci_sayisi: 500
#   sehir: Ankara

# lise2:
#   isim: Fen Lisesi
#   ogrenci_sayisi: 300
#   sehir: İstanbul
```

Sözlüğe varsayılan değer `setdefault()` yöntemi ile eklenir;

```
sayac = {}

# setdefault() - anahtar yoksa ekler ve değeri döner
print(sayac.setdefault('a', 0))
# Çıktı: 0
print(sayac)
# Çıktı: {'a': 0}

# Anahtar varsa mevcut değeri döner
sayac['a'] = 5
print(sayac.setdefault('a', 0))
# Çıktı: 5
print(sayac)
# Çıktı: {'a': 5}

# Kelime sayma örneği
metin = "merhaba dünya merhaba python python python"
kelime_sayaci = {}
for kelime in metin.split():
    kelime_sayaci[kelime] = kelime_sayaci.get(kelime, 0) + 1
print(kelime_sayaci)
# Çıktı: {'merhaba': 2, 'dünya': 1, 'python': 3}
```

Sözlüğü kopyalama aşağıdaki şekilde yapılabilir;

```
orijinal = {'a': 1, 'b': 2, 'c': 3}

# Sığ kopyalama (shallow copy)
kopya1 = orijinal.copy()
kopya1['a'] = 100
print(orijinal)
# Çıktı: {'a': 1, 'b': 2, 'c': 3}
print(kopya1)
# Çıktı: {'a': 100, 'b': 2, 'c': 3}

# dict() ile kopyalama
kopya2 = dict(orijinal)
print(kopya2)
# Çıktı: {'a': 1, 'b': 2, 'c': 3}
```

Listelerde olduğu gibi sözlüklerde de **kavrama** (**comprehension**) vardır. Kavrama ile, bir **ifadenin** (**expression**) sözlük üzerinde **yineleme** (**iterable**) ile her bir elemana uygulayarak yeni bir sözlük oluşturulur.

```
# Basit sözlük comprehension
kareler = {x: x**2 for x in range(1, 6)}
print(kareler)
# Çıktı: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Koşullu sözlük comprehension
cift_kareler = {x: x**2 for x in range(1, 11) if x % 2 == 0}
print(cift_kareler)
# Çıktı: {2: 4, 4: 16, 6: 36, 8: 64, 10: 100}

# İki listeden sözlük oluşturma
isimler = ['Ali', 'Ayşe', 'Mehmet']
yaslar = [25, 30, 28]
isim_yas = {isim: yas for isim, yas in zip(isimler, yaslar)}
print(isim_yas)
# Çıktı: {'Ali': 25, 'Ayşe': 30, 'Mehmet': 28}
```

```
# Sözlüğü ters çevirme
orijinal = {'a': 1, 'b': 2, 'c': 3}
ters = {deger: anahtar for anahtar, deger in orijinal.items()}
print(ters)
# Çıktı: {1: 'a', 2: 'b', 3: 'c'}
```

Aşağıda bir öğrenci sistemine ilişkin örnek verilmiştir;

```
ogrenci = {
    'numara': '12345',
    'ad': 'Ali',
    'soyad': 'Veli',
    'notlar': {
        'matematik': 85,
        'fizik': 78,
        'kimya': 92
    }
}

ortalama = sum(ogrenci['notlar'].values()) / len(ogrenci['notlar'])
print(f"{ogrenci['ad']} {ogrenci['soyad']}'nin ortalaması: {ortalama:.2f}")
# Çıktı: Ali Veli'nin ortalaması: 85.00
```

Sözlükle ilgili aşağıda bir başka telefon defleri örneği verilmiştir;

```
rehber = {
    'Ali': '555-1234',
    'Ayşe': '555-5678',
    'Mehmet': '555-9012'
}

# Arama
isim = 'Ayşe'
if isim in rehber:
    print(f"{isim}'nin telefonu: {rehber[isim]}")
else:
    print(f"{isim} rehberde yok")
# Çıktı: Ayşe'nin telefonu: 555-5678
```

İSTİSNA YÖNETİMİ

Programlama dünyasında hatalar kaçınılmazdır. Ancak bir yazılımcı için önemli olan hatanın oluşması değil, bu hatanın yönetilebilir olmasıdır. Programın **çalışma anında (runtime)** karşılaştığı beklenmedik durumlara **İstisna (exception)** denir. İstisna yönetimi, programın bu hatalar karşısında tamamen çökmesini engellemek ve kullanıcıya anlamlı geri bildirimler vererek sistemin güvenli bir duruma (**safe state**) geçmesini sağlamak için kullanılır.

Kitabımızın başında belirttiğimiz "nelerin yapılacağını değil, işin nasıl yapılacağını" bilme ilkesi burada devreye girer. **İstisna yönetimi (exception handling)**, sadece hataları gizlemek için değil, sistemin sınırlarını belirlemek için kullanılır.

try-except Bloğu

Python'da istisnalar **try-except** mekanizması ile kontrol edilir. **try** bloğu, hata potansiyeli taşıyan, riskli kodların yazıldığı bölümdür. **except** bloğu ise eğer **try** bloğu içinde bir hata oluşursa, programın akışı anında bu bloğa sıçrar. Söz dizimi aşağıda belirtilmiştir;

```
try:
    # Riskli kodlar bu blokta yazılır
    sayi = int(input("Bir sayı giriniz: "))
    sonuc = 100 / sayi
except ValueError:
    # Veri tipi uyumsuzluğunda bu blok çalışır
    print("Hata: Lütfen geçerli bir tam sayı giriniz!")
except ZeroDivisionError:
    # Sıfıra bölünme hatasında bu blok çalışır
    print("Hata: Bir sayıyı sıfıra bölemezsiniz!")
```

Bazı durumlarda **else** ve **finally** bloğu da kullanılır.

- **else** Bloğu: Eğer **try** bloğunda hiçbir hata oluşmazsa çalışır. İşlemin başarıyla tamamlandığını teyit etmek için kullanılır.
- **finally** Bloğu: Hata oluşsa da oluşmasa da mutlaka çalışır. Özellikle dosya kapatma, veri tabanı bağlantısı kesme veya donanım kaynaklarını serbest bırakma gibi "temizlik" işlemleri için hayati önem taşır.

Python dilinde diğer nesne yönelimli dillerde olduğu gibi tüm istisnalar aslında birer nesnedir (object) ve **BaseException** sınıfından türetilmiştir. Hataları yakalarken "özelden genele" doğru bir sıra izlenmelidir.

```
try:
    dosya = open("veriler.txt", "r")
    # ... işlemler ...
except FileNotFoundError:
    print("Dosya bulunamadı!")
except Exception as e:
    # Tüm hataları yakalayan genel sınıf
    print(f"Beklenmedik bir hata oluştu: {e}")
finally:
    print("Sistem kaynakları kontrol ediliyor...")
```

Neden try-except Kullanmalıyız?

- **Süreklilik:** Programın küçük bir hata yüzünden tamamen durmasını engeller.
- **Güvenlik:** Kullanıcıya sistemin iç yapısını **ele veren (traceback)** karmaşık hata mesajları yerine, filtrelenmiş ve güvenli mesajlar gösterilmesini sağlar.
- **Hata Ayıklama (debug):** Hatanın nerede ve neden oluştuğuna dair **iz kaydı (log)** tutulmasını kolaylaştırır.

Python'da her hata aslında bize sistemin neresinde bir mantık veya kaynak sorunu olduğunu söyler. Aşağıdaki tablo, geliştirme sürecinde en sık karşılaşılabilecek istisnaları ve bunların bilgisayar mimarisindeki karşılıklarını özetlemektedir;

<i>İstisna (Exception)</i>	<i>Ne Anlama Gelir?</i>	<i>Mühendisçe Yaklaşım</i>
SyntaxError	Yazım hatası. Python'un dil kurallarına aykırı kod yazılmıştır.	Kod daha derleme/yorumlanma aşamasına geçememiştir.
IndentationError	Girintileme hatası. Blok yapısı bozulmuştur.	Python'un kapsam (scope) belirleme kuralı ihlal edilmiştir.
NameError	Tanımlanmamış bir değişken veya fonksiyon adı kullanılmıştır.	Bellekte (RAM) karşılığı olmayan bir isme referans verilmiştir.
TypeError	Bir veri tipi, desteklemediği bir işleme tabi tutulmuştur.	Nesne arayüzü (interface) ile yapılan işlem uyumsuzdur.
ValueError	Veri tipi doğru olsa da içeriği (değeri) işlem için uygun değildir.	Fonksiyonun kabul edilebilir parametre sınırları dışına çıkmıştır.
ZeroDivisionError	Bir sayı sıfıra bölünmeye çalışılmıştır.	Matematiksel bir tanımsızlık ve işlemci seviyesinde bir hata durumudur.
IndexError	Bir listenin veya dizinin sınırları dışında bir indise erişilmeye çalışılmıştır.	Bellekte tahsis edilen alanın (buffer) dışına taşma girişimidir.
KeyError	Sözlükte (dict) bulunmayan bir anahtar (key) çağırılmıştır.	Hash tablosunda aranan referans bulunamamıştır.
FileNotFoundError	Erişilmek istenen dosya belirtilen yolda mevcut değildir.	Dosya sistemi (File System) üzerinde bir girdi/çıkı (I/O) hatasıdır.
AttributeError	Bir nesneye, sahip olmadığı bir özellik veya metot ile erişilmeye çalışılmıştır.	Sınıf hiyerarsisi veya nesne yapısı yanlış kurgulanmıştır.

Tablo 20.İstisna Türleri

Python bir hata verdiğinde, terminalde **geriye doğru izlenen (traceback)** bir çıktı üretir. Son fonksiyon çağrıları için bu bir **yığın izidir (stack trace)**. En alt satır hatanın türünü ve nedenini söyler. Üstteki satırlar ise hatanın kodun hangi katmanlarından geçerek (fonksiyon çağrı yığını) buraya ulaştığını gösterir. Hata ayıklarken her zaman en alttan başlayıp yukarı doğru "yığın boyunca" takip yapmalısınız.

İstisnai Duruma Düşme

Şu ana kadar istisnaların Python yorumlayıcısı tarafından otomatik olarak oluşturulup sanal bir istisna havuzuna atılmasını (sıfıra bölme, hatalı tip vb.) ve bizim bunları **except** ile **yakalamamızı** işledik. Ancak profesyonel bir yazılım mimarisinde, bazen kod teknik olarak doğru olsa da mantıksal olarak hatalı bir durum oluşabilir. Bu durumda mühendis, hatayı kendisi tetiklemeli yani hataya ait istisna nesnesini kendisi imal edip istisna havuzuna **atmalıdır (raise)**. İşte bu işleme **istisnai duruma düşme (raising an exception)** adı verilir.

Kendi istisna nesnelerimizi aşağıdaki durumlarda kullanabiliriz;

1. İş Kurallarını Denetlemek: Örneğin, bir bankacılık uygulamasında bakiyenin altına düşen bir çekim işlemi teknik bir hata değildir ama mantıksal bir hatadır.
2. Hata Zinciri Oluşturmak: Alt katmanlarda oluşan bir hatayı yakalayıp, daha üst katmana daha anlamlı bir mesajla iletmek için.
3. Zorunlu Alan Kontrolü: Fonksiyonun çalışması için kritik olan verilerin eksikliği durumunda süreci durdurmak için.

Kullanıcı tanımlı bir istisna nesnesi **raise** talimatı ile verilir. Sözdizimi aşağıda verilmiştir;

```
raise <İstisnaTürü>("Hata Mesajı")
```

Buradaki istisna türü Tablo-20’de verilen istisna sınıflarından biri olabilir. Örnek olarak; bir mühendislik projesinde, bir sıcaklık sensöründen gelen verinin fiziksel sınırlar (-50°C ile +150°C) dışında olması durumunda sistemin durdurulması gerektiğini varsayalım. Buna ilişkin kod aşağıda verilmiştir;

```
def sicaklik_kontrol_et(deger):
    if deger < -50 or deger > 150:
        # Teknik bir hata yok ama fiziksel bir imkansızlık/hata var
        raise ValueError(f"Geçersiz sıcaklık verisi: {deger}. Sensör arızalı olabilir!")
    print(f"Sıcaklık normal: {deger}")

try:
    sicaklik_kontrol_et(200)
except ValueError as e:
    print(f"Sistem Durduruldu: {e}")
```

Bazen bir hatayı yakalayıp, üzerine kendi açıklamamızı ekleyerek yeni bir istisna nesnesi imal ederiz. Buna **hata zincirleme** (**exception chaining**) denir.

```
try:
    # Veritabanına bağlanma denemesi
    baglanti_kur()
except ConnectionError as orijinal_hata:
    # Hatayı yakalayıp daha üst seviye bir mesajla fırlatıyoruz
    raise RuntimeError("Veritabanı erişimi başarısız, sistem başlatılamıyor.") from
    orijinal_hata
```

Kendi İstisna Sınıflarımızı Oluşturma

Büyük projelerde Python’un standart hata isimleri (ValueError, TypeError vb.) yetersiz kalabilir. Bu durumda, kitabımızın **kalıtım** (**Inheritance**) prensibini kullanarak kendi hata/istisna sınıflarımızı (**custom exception**) oluşturabiliriz.

```
class SensorArizasi(Exception):
    """Sensör donanım hataları için özel istisna sınıfı"""
    pass

# Kullanımı:
raise SensorArizasi("Donanımdan sinyal alınamıyor!")

class YasHatasiError(ValueError):
    """Geçersiz yaş değeri hatası"""
    def __init__(self, yas, mesaj="Yaş 0-150 arası olmalı"):
        self.yas = yas
        self.mesaj = f"{mesaj}. Girilen: {yas}"
        super().__init__(self.mesaj)
```


PYTHON STANDART KÜTÜPHANESİ

Gerçek dünya kullanım senaryolarında bu kütüphane olmazsa olmazdır. Standart kütüphanedeki (**standart library**) çok kullanılan modüller, `datetime`, `os`, `sys`, `json` ve `math` modülleridir.

Zaman ve Tarih Yönetimi

Mühendislik uygulamalarında, verinin "ne" olduğu kadar "ne zaman" üretildiği de kritiktir. Bir sensör verisinin **zaman damgası** (**timestamp**), bir işlemin süresi veya bir sistemin log kayıtları için zaman yönetimi hataya yer bırakmamalıdır. Python, bu karmaşıklığı yönetmek için `datetime` modülünü sunar.

Zamanın Temsili ve Temel Sınıflar

`datetime` modülü, zamanı farklı katmanlarda ele alan dört ana sınıftan oluşur:

1. `date`: Sadece yıl, ay ve gün bilgilerini tutar (Takvim verisi).
2. `time`: Sadece saat, dakika, saniye ve mikro saniye bilgilerini tutar (Saat verisi).
3. `datetime`: Hem tarih hem de saat bilgilerini bir arada tutar.
4. `timedelta`: İki zaman noktası arasındaki farkı (süreyi) temsil eder.

Sistem Zamanına Erişim

Bir mühendis için en temel işlem, sistemin mevcut zamanını okumaktır.

```
from datetime import datetime
simdi = datetime.now()
print(f"Sistem Zamanı: {simdi}")
print(f"Yıl: {simdi.year}, Ay: {simdi.month}, Gün: {simdi.day}")
```

Zaman Aritmetiği

Zaman üzerinde toplama ve çıkarma işlemleri yapmak için `timedelta` sınıfı kullanılır. Bu sınıf, özellikle periyodik işlemler (örneğin; "bundan 30 gün sonra ne olacak?") için kullanılır.

```
from datetime import datetime, timedelta
bugun = datetime.now()
bir_hafta_sonra = bugun + timedelta(days=7)
print(f"Bir hafta sonraki tarih: {bir_hafta_sonra.date()}")
```

Zaman aritmetiğinde doğrudan tam sayı toplamak yerine `timedelta` kullanmak, artık yıllar (**leap years**) ve ayların gün sayıları gibi karmaşık takvim hesaplamalarını Python'un otomatik yapmasını sağlar.

Biçimlendirme

Zaman verisi genellikle dış dünyadan metin (string) olarak gelir veya dış dünyaya metin olarak sunulur. İki fonksiyon biçimlendirme için kullanılır;

- `strftime` (**string format time**): Zaman objesini metne dönüştürür.
- `strptime` (**string parse time**): Metin halindeki zamanı objeye dönüştürür.

```
# Obje -> Metin
print(bugun.strftime("%d/%m/%Y %H:%M:%S")) # 16/02/2026 14:30:00

# Metin -> Obje
tarih_metni = "2026-05-19"
tarih_objesi = datetime.strptime(tarih_metni, "%Y-%m-%d")
```

Zaman Dilimleri

Mühendislikte en çok yapılan hatalardan biri, **zaman dilimini** (**timezone**) göz ardı etmektir.

- Saf (**naive**): Zaman dilimi bilgisi içermeyen, sadece yerel saati gösteren objelerdir.
- Farkında (**aware**): UTC (Evrensel Zaman) referans alınarak zaman dilimi bilgisi eklenmiş objelerdir.

Küresel sistemlerde her zaman UTC kullanılması bir mühendislik standardıdır. Örnek olarak bir algoritmanın çalışma hızını ölçmek için **datetime** farklarından yararlanabiliriz:

```
baslangic = datetime.now()

# ... Algoritma veya İşlem ...
toplam = sum(range(1000000))

bitis = datetime.now()
gecen_sure = bitis - baslangic
print(f"İşlem süresi: {gecen_sure.total_seconds()} saniye.")
```

Metin İşleme ve Düzenli İfadeler

Yazılım dünyasında **ham metin verisi** (**raw text**) genellikle düzensizdir. Bir mühendis için binlerce satırlık bir hata **iz kaydından** (**log**) sadece IP adreslerini ayıklamak veya bir veri setindeki tarih formatlarını standart hale getirmek manuel olarak imkansızdır. **Düzenli ifadeler** (**regex**), metinler içerisinde belirli desenleri tanımlamak ve bu desenlere uyan verileri bulmak, değiştirmek veya doğrulamak için kullanılan özelleşmiş bir dildir.

Python'da bu işlemler için standart kütüphanede yer alan **re** modülü kullanılır.

Temel Karakterler ve Belirleyiciler

Regex, kendine has bir sembol dili kullanır. **Belirleyiciler** (**meta character**) aşağıda verilmiştir;

Desen (pattern)	Açıklama
.	Herhangi bir karakteri temsil eder.
^	Satır başı karakterini temsil eder.
\$	Satır sonu karakterini temsil eder.
*	Bir karakterin sıfır veya daha fazla tekrarını temsil eder.
+	Bir karakterin bir veya daha fazla tekrarını temsil eder.
?	Bir karakterin sıfır veya bir kez tekrarını temsil eder.
{n}	Bir karakterin tam n kez tekrarını temsil eder.
[abc]	a, b veya c karakterlerinden birini temsil eder.
\d	Herhangi bir rakamı (0-9) temsil eder.
\w	Herhangi bir alfanümerik karakteri (harf, rakam, alt çizgi) temsil eder.
\s	Boşluk karakteri temsil eder.

re Modülü Temel Fonksiyonları

Python'da en sık kullanılan Regex yöntemleri şunlardır:

- **re.search()**: Metin içinde deseni arar ve ilk bulduğu konumu (**match object**) döndürür.
- **re.findall()**: Desene uyan tüm kısımları bir liste olarak döndürür.
- **re.sub()**: Desene uyan kısımları başka bir metinle değiştirir.
- **re.compile()**: Bir deseni önceden derleyerek (pre-compile) performans artışı sağlar.

Gruplandırma ve Veri Ayıklama

Parantez **()** kullanarak desenin belirli kısımlarını gruplandırabilir ve sadece o gruplara erişebiliriz.

E-posta doğrulama ve ayıklama örneği verilmiştir;

```
import re

metin = "İletişim için ilhan.ozkan@kurum.com veya test_kullanici@gmail.com kullanın."
# E-posta deseni: [karakterler]@[alan_adi].[uzanti]
desen = r"([\w.-]+)@([\w.-]+)"
eslesmeler = re.findall(desen, metin)

for kullanıcı, alan in eslesmeler:
    print(f"Kullanıcı: {kullanıcı}, Alan Adı: {alan}")
```

Açgözlü Eşleşme

Regex varsayılan olarak açgözlüdür (**greedy**), yani bulabildiği en uzun eşleşmeyi alır. Mühendislik uygulamalarında (örneğin HTML etiketlerini ayıklarken) bazen en kısa eşleşmeye ihtiyaç duyarız. Bunun için `?` operatörü kullanılır.

Regex ifadeleri güçlüdür ancak karmaşıklıkla okunabilirliği ve bakım maliyeti (**maintenance cost**) artar. Bir mühendis olarak, çok karmaşık bir regex yazmak yerine, işlemi mantıksal adımlara bölmek veya açıklayıcı yorumlar eklemek (**verbose modu**) kodun kalitesini artıracaktır.

Derlenmiş Desenler ve Performans

Eğer aynı desen binlerce satırlık bir veri üzerinde döngü içinde kullanılacaksa, deseni bir kez derlemek işlemci zamanından tasarruf sağlar.

```
# Deseni bir kez derle
ip_deseni = re.compile(r"\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}")

# Döngü içinde hızlıca kullan
with open("access.log", "r") as f:
    for satir in f:
        if ip_deseni.search(satir):
            # ... işlem ...
```

Dosya Yönetimi ve Bağlam Yöneticileri

Bir yazılımın dış dünya ile veri alışverişi yapmasının en temel yolu dosya sistemidir. Mühendislik uygulamalarında log kayıtları, konfigürasyon dosyaları veya sensör verilerinin kalıcı hale getirilmesi için dosya işlemleri hayati önem taşır. Ancak dosyalar, işletim sistemi tarafından sınırlı sayıda sunulan **kaynakladır** (**resource**). Açılan her dosyanın işi bittiğinde kapatılması, dosya ile işlemlerin en kısa sürede bitirilmesi, sistem kaynaklarının tükenmemesi için bir zorunluluktur.

Dosya Açma ve Erişim Modları

Python'da bir dosyaya erişmek için `open()` fonksiyonu kullanılır. Bu fonksiyon, işletim sistemi seviyesindeki **dosya tanıtıcısı** (**file descriptor**) nesnesine bir referans oluşturur.

```
# Temel kullanım (Önerilmeyen yöntem)
dosya = open("veriler.txt", "w")
dosya.write("Mühendislik Verisi")
dosya.close() # Kapatma unutulursa kaynak sızıntısı oluşur!
```

Erişim Modları:

1. **'r'**: Okuma (read) modu. Dosya yoksa istisna oluşur.
2. **'w'**: Yazma (write) modu. Dosya varsa içeriği siler, yoksa oluşturur.
3. **'a'**: Ekleme (append) modu. Dosyanın sonuna veri ekler.
4. **'b'**: Verilerin bilgisayar belleğindeki gibi tutulduğu ikili (binary) mod. Resim, ses veya derlenmiş dosya işlemleri için kullanılır.
5. **'r+'**: Hem okuma ve hem de yazma Modu.
6. **'rb'**: İkili dosyayı okuma Modu.

7. **'wb'**: İkili dosyayı yazma Modu.

Bağlam Yöneticileri

Mühendislikte "hata toleransı" esastır. Eğer dosya açıkken bir hata oluşursa, klasik yöntemle `close()` komutu hiçbir zaman çalışmayabilir. Python bu riski ortadan kaldırmak için bağlam yöneticisi (**context manager**) yapısını sunar. Bu yapı sayesinde dosya `with` bloğu kullanıldığında, işlem bittikten sonra veya bir hata oluşsa dahi dosya otomatik olarak kapatılır.

```
with open("deney_sonuc_lari.csv", "r") as dosya:
    icerik = dosya.read()
    # İşlem bittiğinde sistem dosyayı otomatik kapatır.
```

`with` kullanımı, C++ dilindeki Kaynak Edinimi Başlatma İşlemidir (**Resource Acquisition Is Initialization-RAII**) prensibine benzer. Kaynak kullanımı, kodun kapsamı (scope) ile sınırlıdır.

Dosya Okuma Teknikleri

Büyük veri setleriyle (**big data**) çalışırken tüm dosyayı bir kerede belleğe (RAM) yüklemek sistemi kilitleyebilir. Mühendisçe yaklaşım, veriyi satırlar halinde işlemektir. Genelde gözle okunabilir metin dosyalarını işlerken kullanılır.

```
with open("buyuk_veri_seti.txt", "r") as dosya:
    for satir in dosya:
        # Her adımda sadece bir satır bellekte tutulur
        print(satir.strip())
```

Yapılandırılmış Veri Saklama

Standart metin dosyaları yerine, verileri **yapılandırılmış** (**structured**) bir formatta saklamak veri bütünlüğü için önemlidir. Verileri gözle okunabilir şekilde barındıran ve en çok kullanılan formatlar **XML** (**Extensible Markup Language**) ve **JSON** (**Java Script Object Notation**) formatıdır. Python dilinin `json` modülü ile nesneleri doğrudan dosyaya kaydedebiliriz.

```
import json
veriler = {"sensor_id": 101, "sicaklik": 24.5}
with open("config.json", "w") as f:
    json.dump(veriler, f) # Veriyi JSON formatında dosyaya yazar
```

Dosya ve Dizin Kontrolleri

Bir dosyayı açmadan önce varlığını kontrol etmek, programın çökmesini engeller. Bu durumu `os` nesnesi ile kontrol edebiliriz.

```
import os

if os.path.exists("rapor.txt"):
    print("Dosya zaten mevcut.")
else:
    print("Yeni dosya oluşturuluyor...")
```

Sistem Parametreleri ve Etkileşimi

Python'un `sys` modülü, **yorumlayıcının** (**interpreter**) **çalışma anındaki** (**runtime**) ortamına ve sistem kaynaklarına doğrudan erişim sağlar. `os` modülü işletim sistemiyle ilgilenirken, `sys` modülü daha çok Python'un kendisi ve bilgisayar donanımı arasındaki düşük seviyeli parametrelerle ilgilenir. Bir mühendis için bu modül, bellek yönetimi ve sistem entegrasyonu açısından vazgeçilmezdir.

`sys` modülü, Python'un **abstraction** (**soyutlama**) seviyesinden bir adım aşağı inerek donanıma yaklaşmamızı sağlar. Özellikle bellek optimizasyonu ve terminal tabanlı araçlar geliştirirken bu modülün sunduğu parametreler yazılımın performansını doğrudan etkiler.

Komut Satırı Argümanları

Yazılan bir **betiğin** (**script**), terminal üzerinden dışarıdan parametreler alarak çalışması gerekebilir. `sys.argv`, komut satırından girilen argümanları bir liste olarak saklar.

1. `sys.argv[0]`: Çalıştırılan dosyanın adıdır.
2. `sys.argv[1:]`: Dışarıdan girilen parametrelerdir.

```
import sys
# terminal: python hesapla.py 10 20
print(f"Çalışan dosya: {sys.argv[0]}")
if len(sys.argv) > 2:
    sayi1 = int(sys.argv[1])
    sayi2 = int(sys.argv[2])
    print(f"Toplam: {sayi1 + sayi2}")
```

Bellek Yönetimi

Bir mühendis için yazdığı kodun ne kadar bellek (RAM) tükettiği hayati önem taşır. `sys.getsizeof()` fonksiyonu, bir Python nesnesinin bellekte kapladığı alanı **bayt** (**byte**) cinsinden döndürür.

```
import sys

liste = [i for i in range(1000)]
sozluk = {i: i for i in range(1000)}

print(f"Listenin bellek boyutu: {sys.getsizeof(liste)} byte")
print(f"Sözlüğün bellek boyutu: {sys.getsizeof(sozluk)} byte")
```

Standart Giriş, Çıkış ve Hata Akışları

Bilgisayar mimarisinde I/O (Girdi/Çıktı) işlemleri üç ana kanal üzerinden yürür: `stdin` (standart giriş), `stdout` (standart çıkış) ve `stderr` (standart hata).

- `sys.stdout`: `print()` fonksiyonunun arka planda kullandığı kanaldır.
- `sys.stderr`: Hata mesajlarını sisteme iletmek için kullanılır. Mühendislik uygulamalarında hataları normal çıktılardan ayırmak için bu kanalın kullanılması standart bir pratiktir.

Dosya İşlemlerinde Özel Hata Türleri

En çok olan hata türleri aşağıda verilmiştir;

```
try:
    with open('dosya.txt', 'r') as f:
        icerik = f.read()
except FileNotFoundError:
    print("Dosya bulunamadı!")
except PermissionError:
    print("Dosya erişim izni yok!")
except IOError as e:
    print(f"Dosya hatası: {e}")
```

Dosyaların Ön Tanımlı Olarak Aranacağı Dizinler

Python bir modülü **içe aktarırken** (**import**), belirli dizinleri kontrol eder. `sys.path`, bu dizinlerin listesini tutar. Eğer kendi yazdığınız kütüphaneleri Python'un bulmasını istiyorsanız, bu listeye müdahale edebilirsiniz.

```
import sys
# Python'un kütüphane arama yollarına yeni bir dizin eklemek
sys.path.append("C:/KendiKutuphanelerim")
```

Programdan Çıkış

Bir hata durumunda veya görev tamamlandığında programın akışını güvenli bir şekilde sonlandırmak ve işletim sistemine bir **çıkış kodu** (**exit code**) döndürmek için kullanılır.

- `sys.exit(0)`: Başarılı çıkış.
- `sys.exit(1)`: Hatalı çıkış (Sistem mühendisliğinde hata takip mekanizmaları için kritiktir).

Yorumlayıcı ve Platform Bilgileri

Yazılımın hangi platformda (Windows, Linux, macOS) ve hangi Python sürümünde çalıştığını denetlemek, kodun **taşınabilirliği** (**portability**) için önemlidir.

```
print(f"İşletim Sistemi Platformu: {sys.platform}")
print(f"Python Sürümü: {sys.version}")
```

Matematiksel Hesaplamalar

Bilgisayar mimarisinde matematiksel işlemler işlemcinin **aritmetik işlem birimi** (**arithmetic logic unit-ALU**) birimi tarafından yürütülür. Python'un `math` modülü, C diliyle yazılmış olan standart matematik kütüphanesine doğrudan erişim sağlayarak, karmaşık hesaplamaların hem hızlı hem de standartlara uygun (IEEE 754) yapılmasını sağlar¹⁶.

Mühendislik hesaplamalarında **kayan noktalı sayıların** (**floating point**) işlemci seviyesindeki davranışı bazen istenmeyen yuvarlama hatalarına neden olabilir.

- `math` Modülü: C dilinin standart matematik kütüphanesi üzerine inşa edilmiştir. Trigonometri, logaritma ve sabitler (π , e) gibi donanım destekli hızlı hesaplama araçlarını sunar.
- `decimal` Modülü: Finansal veya yüksek hassasiyetli mühendislik hesaplamaları için float veri tipinin zafiyetlerini giderir. $0.1 + 0.2$ 'nin tam olarak 0.3 etmesini sağlayan deterministik bir hesaplama yapısı sunar.

Matematiksel Sabitler

Mühendislik formüllerinde kullanılan evrensel sabitler, modül içerisinde yüksek hassasiyetle tanımlanmıştır.

- `math.pi`: π sayısı (3.141592...).
- `math.e`: Euler sayısı (2.718281...).
- `math.tau`: π değerini temsil eden sabit.

Sayı Yuvarlama ve Sınır Değerler

Sayısal analizlerde, kayan noktalı bir sayının (float) tam sayıya nasıl dönüştürüleceği işlemin sonucunu etkiler.

- `math.ceil(x)`: Sayıyı yukarıya, en yakın tam sayıya yuvarlar.
- `math.floor(x)`: Sayıyı aşağıya, en yakın tam sayıya yuvarlar.
- `math.trunc(x)`: Sayının ondalık kısmını atarak sadece tam sayı kısmını bırakır.

Üs, Logaritmik ve Güç Fonksiyonları

Donanım tasarımı veya veri analitiği gibi alanlarda kullanılan temel fonksiyonlardır.

- `math.pow(x, y)`: x sayısının y inci kuvvetini alır.
- `math.sqrt(x)`: Sayının karekökünü hesaplar.
- `math.log(x, base)`: Belirtilen tabanda logaritma hesaplar.

¹⁶ <https://docs.python.org/3/library/math.html>

Trigonometrik İşlemler

Trigonometrik fonksiyonları **Radyan** cinsinden hesaplar. Mühendislikte sıkça kullanılan derece-radyan dönüşümleri için `math` modülü yardımcı fonksiyonlar sunar.

- `math.sin()`, `math.cos()`, `math.tan()`: Temel trigonometrik fonksiyonlar.
- `math.radians(x)`: Dereceyi radyana çevirir.
- `math.degrees(x)`: Radyanı dereceye çevirir.

```
import math

derece = 90
radyan = math.radians(derece)

print(f"90 derecenin sinüsü: {math.sin(radyan)}") # Çıktı: 1.0
```

Özel Durumlar

Bilgisayar bilimlerinde bazı matematiksel sonuçlar standart sayılarla ifade edilemez.

- `math.inf`: **Sonsuz** (**infinity**) değerini temsil eder.
- `math.nan`: **Sayı olmayan** (**not a number-NaN**) değerini temsil eder. Hatalı veri girişlerini kontrol etmek için kritiktir.

`math.isclose()` fonksiyonu, iki kayan noktalı sayının birbirine "yeterince yakın" olup olmadığını kontrol etmek için kullanılır. `if a == b` kontrolü float sayılarda yuvarlama hataları nedeniyle risklidir; bu yüzden toleranslı bir karşılaştırma her zaman daha güvenlidir.

Pisagor teoremini kullanarak bir dik üçgenin hipotenüsünü hesaplayan program örneği aşağıda verilmiştir;

```
import math

def hipotenus_hesapla(a, b):
    # math.hypot doğrudan karelerin toplamının kökünü alır
    return math.hypot(a, b)

kenar1 = 3
kenar2 = 4
print(f"Hipotenüs: {hipotenus_hesapla(kenar1, kenar2)}") # Çıktı: 5.0
```

Bilimsel Hesaplama

NumPy (**Numerical Python**), Python'da bilimsel hesaplama için temel kütüphanedir. Çok boyutlu **diziler** (**array**) ve bu diziler üzerinde yapılan matematiksel işlemler için optimize edilmiş fonksiyonlar sağlar.

NumPy, veri bilimi, makine öğrenmesi ve bilimsel hesaplama projelerinin temelini oluşturur. Python'un standart listelerine göre çok daha hızlı ve bellek verimlidir. Bu nedenle çok önemlidir.

`pip install numpy` ile kurulumu yapılır. Kullanımı ise `numpy` dosya.py şeklindedir.

```
import time

# Python listesi ile işlem
python_liste = list(range(1000000))
baslangic = time.time()
python_sonuc = [x * 2 for x in python_liste]
python_sure = time.time() - baslangic

# NumPy array ile işlem
numpy_array = np.arange(1000000)
baslangic = time.time()
```

```
numpy_sonuc = numpy_array * 2
numpy_sure = time.time() - baslangic

print(f"Python listesi: {python_sure:.4f} saniye")
# Çıktı: Python listesi: 0.0523 saniye
print(f"NumPy array: {numpy_sure:.6f} saniye")
# Çıktı: NumPy array: 0.001234 saniye
print(f"Hız artışı: {python_sure/numpy_sure:.1f}x")
# Çıktı: Hız artışı: 42.4x

# NumPy çok daha hızlıdır!
```

NumPy Dizileri

NumPy'nin temel veri yapısı ndarray (**n-dimensional array**) yani çok boyutlu dizidir.

```
# Python listesinden dizi oluşturma
liste = [1, 2, 3, 4, 5]
arr = np.array(liste)
print(arr)
# Çıktı: [1 2 3 4 5]
print(type(arr))
# Çıktı: <class 'numpy.ndarray'>

# İki boyutlu dizi (matris)
matris = np.array([[1, 2, 3], [4, 5, 6]])
print(matris)
# Çıktı: [[1 2 3]
# Çıktı:  [4 5 6]]

# Üç boyutlu dizi
ucboyutlu = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print(ucboyutlu.shape)
# Çıktı: (2, 2, 2)
```

Özel Dizi Oluşturma Fonksiyonları

```
# Sıfırlarla dolu array
sifirlar = np.zeros((3, 4))
print(sifirlar)
# Çıktı: [[0. 0. 0. 0.]
# Çıktı:  [0. 0. 0. 0.]
# Çıktı:  [0. 0. 0. 0.]]

# Birlerle dolu array
birler = np.ones((2, 3))
print(birler)
# Çıktı: [[1. 1. 1.]
# Çıktı:  [1. 1. 1.]]

# Belirli bir değerle dolu array
dolu = np.full((2, 2), 7)
print(dolu)
# Çıktı: [[7 7]
# Çıktı:  [7 7]]

# Birim matris (Identity matrix)
birim = np.eye(3)
print(birim)
# Çıktı: [[1. 0. 0.]
# Çıktı:  [0. 1. 0.]
# Çıktı:  [0. 0. 1.]]
```



```
# Belirli aralıkta sayılar
aralik = np.arange(0, 10, 2)
print(aralik)
# Çıktı: [0 2 4 6 8]

# Eşit aralıklı sayılar
esit_aralik = np.linspace(0, 1, 5)
print(esit_aralik)
# Çıktı: [0.  0.25 0.5  0.75 1.  ]

# Rastgele sayılar
rastgele = np.random.rand(3, 3) # 0-1 arası
print(rastgele)
# Çıktı: [[0.5488 0.7152 0.6028]
# Çıktı: [0.5449 0.4237 0.6459]
# Çıktı: [0.4376 0.8918 0.9637]]

# Normal dağılım
normal = np.random.randn(3, 3) # Ortalama 0, standart sapma 1
print(normal)
# Çıktı: [[-0.2342  0.3452 -0.9423]
# Çıktı: [ 1.1234 -0.4563  0.2345]
# Çıktı: [ 0.5678 -0.1234  0.8901]]
```

Dizi Özellikleri

```
# Boyut (dimensions)
print(f"Boyut (ndim): {arr.ndim}")
# Çıktı: Boyut (ndim): 2

# Şekil (shape) - her boyuttaki eleman sayısı
print(f"Şekil (shape): {arr.shape}")
# Çıktı: Şekil (shape): (3, 4)

# Toplam eleman sayısı
print(f"Boyut (size): {arr.size}")
# Çıktı: Boyut (size): 12

# Veri tipi
print(f"Veri tipi (dtype): {arr.dtype}")
# Çıktı: Veri tipi (dtype): int64

# Bellekte kapladığı alan (bytes)
print(f"Her elemanın boyutu: {arr.itemsize} bytes")
# Çıktı: Her elemanın boyutu: 8 bytes
print(f"Toplam bellek: {arr.nbytes} bytes")
# Çıktı: Toplam bellek: 96 bytes
```

Dizi İndisleri ve Dilimleme

```
# Tek eleman erişimi
print(arr[0])
# Çıktı: 10
print(arr[-1])
# Çıktı: 50

# Dilimleme (slicing)
print(arr[1:4])
# Çıktı: [20 30 40]
print(arr[:,2])
# Çıktı: [10 30 50]
```

```
# İki boyutlu array
matris = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# Tek eleman erişimi
print(matris[0, 0])
# Çıktı: 1
print(matris[1, 2])
# Çıktı: 6

# Satır erişimi
print(matris[1])
# Çıktı: [4 5 6]

# Sütun erişimi
print(matris[:, 1])
# Çıktı: [2 5 8]

# Alt matris
print(matris[0:2, 1:3])
# Çıktı: [[2 3]
# Çıktı: [5 6]]

# Boolean indeksleme
arr = np.array([1, 2, 3, 4, 5])
maske = arr > 3
print(arr[maske])
# Çıktı: [4 5]

# Koşullu indeksleme
print(arr[arr % 2 == 0])
# Çıktı: [2 4]
Array Şekillendirme
print(arr)
# Çıktı: [ 0  1  2  3  4  5  6  7  8  9 10 11]

# Yeniden şekillendirme (reshape)
matris = arr.reshape(3, 4)
print(matris)
# Çıktı: [[ 0  1  2  3]
# Çıktı: [ 4  5  6  7]
# Çıktı: [ 8  9 10 11]]

# Otomatik boyut hesaplama (-1 kullanarak)
matris2 = arr.reshape(4, -1) # 4 satır, otomatik sütun
print(matris2)
# Çıktı: [[ 0  1  2]
# Çıktı: [ 3  4  5]
# Çıktı: [ 6  7  8]
# Çıktı: [ 9 10 11]]

# Düzleştirme (flatten)
duz = matris.flatten()
print(duz)
# Çıktı: [ 0  1  2  3  4  5  6  7  8  9 10 11]

# Transpozisyon (satır ve sütunları yer değiştirme)
transpoz = matris.T
print(transpoz)
# Çıktı: [[ 0  4  8]
# Çıktı: [ 1  5  9]
# Çıktı: [ 2  6 10]
```

```
# Çıktı: [ 3  7 11]
```

Matematiksel İşlemler

Temel Aritmetik

```
b = np.array([10, 20, 30, 40])
```

```
# Toplama
```

```
print(a + b)
```

```
# Çıktı: [11 22 33 44]
```

```
# Çıkarma
```

```
print(b - a)
```

```
# Çıktı: [ 9 18 27 36]
```

```
# Çarpma (element-wise)
```

```
print(a * b)
```

```
# Çıktı: [ 10  40  90 160]
```

```
# Bölme
```

```
print(b / a)
```

```
# Çıktı: [10. 10. 10. 10.]
```

```
# Üs alma
```

```
print(a ** 2)
```

```
# Çıktı: [ 1  4  9 16]
```

```
# Skaler işlemler
```

```
print(a + 10)
```

```
# Çıktı: [11 12 13 14]
```

```
print(a * 2)
```

```
# Çıktı: [2 4 6 8]
```

İstatistiksel Fonksiyonlar

```
# Toplam
```

```
print(f"Toplam: {np.sum(arr)}")
```

```
# Çıktı: Toplam: 45
```

```
print(f"Satır toplamaları: {np.sum(arr, axis=1)}")
```

```
# Çıktı: Satır toplamaları: [ 6 15 24]
```

```
print(f"Sütun toplamaları: {np.sum(arr, axis=0)}")
```

```
# Çıktı: Sütun toplamaları: [12 15 18]
```

```
# Ortalama
```

```
print(f"Ortalama: {np.mean(arr)}")
```

```
# Çıktı: Ortalama: 5.0
```

```
# Standart sapma
```

```
print(f"Standart sapma: {np.std(arr):.2f}")
```

```
# Çıktı: Standart sapma: 2.58
```

```
# Minimum ve maksimum
```

```
print(f"Minimum: {np.min(arr)}")
```

```
# Çıktı: Minimum: 1
```

```
print(f"Maksimum: {np.max(arr)}")
```

```
# Çıktı: Maksimum: 9
```

```
# Minimum ve maksimumun indeksi
```

```
print(f"Min indeks: {np.argmin(arr)}")
```

```
# Çıktı: Min indeks: 0
```

```
print(f"Max indeks: {np.argmax(arr)}")
```

```
# Çıktı: Max indeks: 8
```

```
# Medyan
print(f"Medyan: {np.median(arr)}")
# Çıktı: Medyan: 5.0

# Yüzdelik dilimler (percentiles)
print(f"25. yüzdelik: {np.percentile(arr, 25)}")
# Çıktı: 25. yüzdelik: 3.0
print(f"75. yüzdelik: {np.percentile(arr, 75)}")
# Çıktı: 75. yüzdelik: 7.0
```

Matematiksel Fonksiyonlar

```
# Trigonometrik fonksiyonlar
print(f"Sinüs: {np.sin(arr)}")
# Çıktı: Sinüs: [0.    0.5   0.71 0.87 1.   ]
print(f"Kosinüs: {np.cos(arr)}")
# Çıktı: Kosinüs: [1.    0.87 0.71 0.5   0.   ]

# Üstel ve logaritma
arr2 = np.array([1, 2, 3, 4, 5])
print(f"e^x: {np.exp(arr2)}")
# Çıktı: e^x: [ 2.72  7.39 20.09 54.60 148.41]
print(f"log(x): {np.log(arr2)}")
# Çıktı: log(x): [0.    0.69 1.10 1.39 1.61]
print(f"log10(x): {np.log10(arr2)}")
# Çıktı: log10(x): [0.    0.30 0.48 0.60 0.70]

# Karekök
print(f"√x: {np.sqrt(arr2)}")
# Çıktı: √x: [1.    1.41 1.73 2.    2.24]

# Mutlak değer
arr3 = np.array([-1, -2, 3, -4, 5])
print(f"|x|: {np.abs(arr3)}")
# Çıktı: |x|: [1 2 3 4 5]

# Yuvarlama
arr4 = np.array([1.234, 2.567, 3.891])
print(f"Round: {np.round(arr4, 2)}")
# Çıktı: Round: [1.23 2.57 3.89]
print(f"Floor: {np.floor(arr4)}")
# Çıktı: Floor: [1. 2. 3.]
print(f"Ceil: {np.ceil(arr4)}")
# Çıktı: Ceil: [2. 3. 4.]

Lineer Cebir İşlemleri
# Matris çarpımı
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Element-wise çarpım
print(f"Element-wise çarpım:\n{A * B}")
# Çıktı: Element-wise çarpım:
# Çıktı: [[ 5 12]
# Çıktı:  [21 32]]

# Matris çarpımı (dot product)
print(f"Matris çarpımı:\n{np.dot(A, B)}")
# Çıktı: Matris çarpımı:
# Çıktı: [[19 22]
# Çıktı:  [43 50]]

# @ operatörü ile matris çarpımı
print(f"A @ B:\n{A @ B}")
```

```
# Çıktı: A @ B:
# Çıktı: [[19 22]
# Çıktı:  [43 50]]

# Determinant
det = np.linalg.det(A)
print(f"Determinant: {det}")
# Çıktı: Determinant: -2.0

# Ters matris (inverse)
A_inv = np.linalg.inv(A)
print(f"Ters matris:\n{A_inv}")
# Çıktı: Ters matris:
# Çıktı: [[-2.   1. ]
# Çıktı:  [ 1.5 -0.5]]

# Özdeğerler ve özvektörler
ozdegerler, ozvektorler = np.linalg.eig(A)
print(f"Özdeğerler: {ozdegerler}")
# Çıktı: Özdeğerler: [-0.37  5.37]
print(f"Özvektörler:\n{ozvektorler}")
# Çıktı: Özvektörler:
# Çıktı: [[-0.82 -0.42]
# Çıktı:  [ 0.57 -0.91]]
```

Yayın

NumPy modülünde, farklı şekillerdeki dizileri otomatik olarak uyumlu hale getirmek için **yayın** (**broadcasting**) yapılır.

```
# Skaler ile arrayb
arr = np.array([1, 2, 3])
print(arr + 10)
# Çıktı: [11 12 13]

# Farklı şekillerde array'ler
a = np.array([[1, 2, 3], [4, 5, 6]]) # (2, 3)
b = np.array([10, 20, 30])          # (3,)

print(a + b)
# Çıktı: [[11 22 33]
# Çıktı:  [14 25 36]]

# Sütun vektörü ile matris
a = np.array([[1, 2, 3], [4, 5, 6]]) # (2, 3)
b = np.array([[10], [20]])           # (2, 1)

print(a + b)
# Çıktı: [[11 12 13]
# Çıktı:  [24 25 26]]
Array Birleştirme ve Ayırma
# Array birleştirme
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Dikey birleştirme (vstack)
dikey = np.vstack((a, b))
print(dikey)
# Çıktı: [[1 2 3]
# Çıktı:  [4 5 6]]

# Yatay birleştirme (hstack)
```

```

yatay = np.hstack((a, b))
print(yatay)
# Çıktı: [1 2 3 4 5 6]

# Concatenate (axis belirtilerek)
birlesmis = np.concatenate((a, b))
print(birlesmis)
# Çıktı: [1 2 3 4 5 6]

# Array ayırma
arr = np.array([1, 2, 3, 4, 5, 6])

# Eşit parçalara ayırma
parcalar = np.split(arr, 3)
print(parcalar)
# Çıktı: [array([1, 2]), array([3, 4]), array([5, 6])]

# Belirli indekslerde ayırma
parcalar2 = np.split(arr, [2, 4])
print(parcalar2)
# Çıktı: [array([1, 2]), array([3, 4]), array([5, 6])]

Kopyalama ve View
# View (görünüm) - aynı veriyi gösterir
arr = np.array([1, 2, 3, 4, 5])
view = arr.view()
view[0] = 999
print(arr)
# Çıktı: [999  2  3  4  5] # Orijinal array değişti!

# Copy (kopya) - bağımsız yeni array
arr = np.array([1, 2, 3, 4, 5])
kopya = arr.copy()
kopya[0] = 999
print(arr)
# Çıktı: [1 2 3 4 5] # Orijinal array değişmedi
print(kopya)
# Çıktı: [999  2  3  4  5]

```

Pratik Örnekler

Veri Normalizasyonu Örneği

```

# Min-Max normalizasyon (0-1 arası)
veri = np.array([10, 20, 30, 40, 50])

min_val = np.min(veri)
max_val = np.max(veri)
normalize_edilmis = (veri - min_val) / (max_val - min_val)

print(normalize_edilmis)
# Çıktı: [0.  0.25 0.5  0.75 1.  ]

# Z-score normalizasyon (standartlaştırma)
ortalama = np.mean(veri)
std = np.std(veri)
standartlastirilmis = (veri - ortalama) / std

print(standartlastirilmis)
# Çıktı: [-1.41 -0.71  0.   0.71  1.41]

```

Mesafe Hesaplama Örneği

```

# İki nokta arasındaki Öklid mesafesi
nokta1 = np.array([1, 2, 3])

```

```
nokta2 = np.array([4, 5, 6])

# mesafe = np.sqrt(np.sum((nokta1 - nokta2) ** 2))
print(f"Öklid mesafesi: {mesafe:.2f}")
# Çıktı: Öklid mesafesi: 5.20

# Alternatif: linalg.norm kullanarak
mesafe2 = np.linalg.norm(nokta1 - nokta2)
print(f"Mesafe (linalg.norm): {mesafe2:.2f}")
# Çıktı: Mesafe (linalg.norm): 5.20
```

Görüntü İşleme

```
# Gri tonlamalı görüntü (8-bit, 0-255 arası)
goruntu = np.random.randint(0, 256, (5, 5), dtype=np.uint8)
print("Orijinal görüntü:")
print(goruntu)

# Parlaklık artırma
parlak_goruntu = np.clip(goruntu + 50, 0, 255).astype(np.uint8)
print("\nParlak görüntü:")
print(parlak_goruntu)

# Kontrast artırma
kontrast_goruntu = np.clip(goruntu * 1.5, 0, 255).astype(np.uint8)
print("\nYüksek kontrastlı görüntü:")
print(kontrast_goruntu)

# Negatif görüntü
negatif_goruntu = 255 - goruntu
print("\nNegatif görüntü:")
print(negatif_goruntu)
```

Polinomial Regresyon

```
# Veri noktaları
x = np.array([1, 2, 3, 4, 5])
y = np.array([2.1, 3.9, 6.2, 8.1, 9.8])

# 2. derece polinom fit etme
katsayilar = np.polyfit(x, y, 2)
print(f"Katsayılar (a, b, c): {katsayilar}")
# Çıktı: Katsayılar (a, b, c): [-0.04  2.09  0.04]

# Tahmin yapma
x_yeni = np.linspace(1, 5, 50)
y_tahmin = np.polyval(katsayilar, x_yeni)

# Hata hesaplama
y_fit = np.polyval(katsayilar, x)
mse = np.mean((y - y_fit) ** 2)
print(f"Ortalama Kare Hata (MSE): {mse:.4f}")
# Çıktı: Ortalama Kare Hata (MSE): 0.0123
```

Performans İpuçları

```
# 1. Vektörleştirme (Vectorization) kullanın
# Kötü: Döngü ile
arr = np.arange(1000000)
sonuc = np.zeros(1000000)
for i in range(len(arr)):
    sonuc[i] = arr[i] * 2
```

```
# İyi: Vektörleştirilmiş
arr = np.arange(1000000)
sonuc = arr * 2

# 2. In-place işlemler kullanın (bellek tasarrufu)
arr = np.arange(1000000)
arr *= 2 # arr = arr * 2 yerine

# 3. Uygun veri tipi seçin
float64 yerine float32 kullanmak bellek kullanımını yarıya indirir
arr_buyuk = np.zeros(1000000, dtype=np.float64) # 8 MB
arr_kucuk = np.zeros(1000000, dtype=np.float32) # 4 MB

# 4. Broadcasting kullanın
# Kötü
a = np.arange(1000).reshape(10, 100)
b = np.arange(100)
sonuc = np.zeros_like(a)
for i in range(10):
    sonuc[i] = a[i] + b

# İyi
sonuc = a + b # Broadcasting otomatik
```

En İyi NumPy Uygulamaları

```
# 1. Doğru veri tipini kullan
entero = np.array([1, 2, 3], dtype=np.int32) # Tam sayılar için
ondalik = np.array([1.0, 2.0], dtype=np.float32) # Ondalıklı için
karmasik = np.array([1+2j], dtype=np.complex64) # Karmaşık sayılar için

# 2. Shape kontrolü yapın
def guvenli_toplama(a, b):
    """Shape uyumluluğunu kontrol eden toplama"""
    if a.shape != b.shape:
        raise ValueError(f"Shape uyumsuzluğu: {a.shape} vs {b.shape}")
    return a + b

# 3. Bellek yönetimi
# Büyük array'leri del ile silin
buyuk_array = np.zeros((10000, 10000))
# İşlemler...
del buyuk_array # Belleği serbest bırak

# 4. Kod okunabilirliği için isim verin
sutun_ortalamalari = np.mean(matris, axis=0) # axis=0 yerine açıklayıcı
satir_ortalamalari = np.mean(matris, axis=1) # axis=1 yerine açıklayıcı
```

Bilimsel Hesaplama Özeti

Veri bilimi ve analizi, Makine öğrenmesi, Görüntü işleme, Bilimsel hesaplama, Finans ve ekonomi ile Mühendislik simülasyonları alanlarında çok kullanılan NumPy'nin temel özellikleri:

- Hız: Python listelerinden 10-100x daha hızlı
- Çok Boyutlu Diziler: `ndarray` veri yapısı
- Matematiksel İşlemler: Vektörleştirilmiş işlemler
- Yayın: Farklı boyutlardaki dizileri otomatik uyumlu hale getirme
- Lineer Cebir: Matris işlemleri, özdeğerler, determinant
- İstatistik: Ortalama, standart sapma, medyan, vb.
- Rastgele Sayılar: Çeşitli dağılımlardan rastgele sayı üretimi

Olasılık ve İstatistiksel Analiz Araçları

Mühendislikte sistem davranışlarını simüle etmek veya toplanan verileri analiz etmek için matematiksel modellerden yararlanırız. Python, bu ihtiyaçlar için `random` ve `statistics` modüllerini sunar.

Rastlantısallık ve Simülasyon

Bilgisayar bilimlerinde gerçek rastlantısallık zordur; bu yüzden **sözde rastlantısal** (**pseudo-random**) sayılar kullanılır. `random` modülü, bu sayıları üretmek için deterministik algoritmalar kullanır.

- `random.random()`: 0,0 ile 1,0 arasında rastgele bir float sayı üretir.
- `random.randint(a, b)`: Belirtilen aralıkta (a ve b dahil) bir tam sayı üretir.
- `random.choice(liste)`: Bir liste içinden rastgele bir eleman seçer.
- `random.seed(x)`: Rastlantısal sayı üreticini sabitler. **Simülasyonların tekrarlanabilirliği** (**reproducibility**) için mühendislikte hayati önem taşır.

Temel Veri Analizi

Deneyisel verilerin merkezi eğilimlerini ve yayılımını ölçmek için kullanılır.

- `statistics.mean()`: Verilerin aritmetik ortalamasını hesaplar.
- `statistics.median()`: Verilerin orta değerini bulur (aykırı değerlerin etkisini azaltmak için kullanılır).
- `statistics.stdev()`: Standart sapmayı hesaplar. Verinin ne kadar güvenilir veya değişken olduğunu ölçen temel mühendislik metriğidir.

Yüksek Performanslı Döngü Yönetimi

Büyük veri setleri üzerinde iterasyon yaparken **bellek verimliliği** (**memory efficiency**) en öncelikli konudur. `itertools` modülü, karmaşık döngüleri ve kombinasyonel işlemleri bellek dostu **yineleyiciler** (**iterator**) ile yönetmemizi sağlar.

Sonsuz ve Koşullu Yineleyiciler

- `itertools.count(start, step)`: Belirli bir sayıdan başlayarak sonsuza kadar artan bir sayı dizisi üretir. Sayaç mekanizmaları için idealdir.
- `itertools.cycle(iterable)`: Bir veri grubunu sonsuz bir döngüye sokar.

Kombinasyonel Araçlar

Karmaşık olasılık hesaplamalarında ve sistem konfigürasyonlarını test ederken kullanılır:

- `itertools.product()`: Kartezyen çarpım oluşturur.
- `itertools.permutations()`: Elemanların tüm sıralı dizilişlerini hesaplar.
- `itertools.combinations()`: Elemanların sırasız gruplarını (kombinasyonlarını) oluşturur.

`itertools` fonksiyonları veriyi belleğe bir anda yüklemeyi, **ihtiyaç duyuldukça üretir** (**lazy evaluation**). Bu sayede milyonlarca kombinasyon içeren işlemler bile sistemi kilitlemeden yürütülebilir.

İleri Seviye Veri Yapıları

Standart liste ve sözlüklerin performans olarak yetersiz kaldığı algoritmalarda, özelleşmiş koleksiyonlar (**collection**) kullanılır.

`namedtuple`: Veriyi hem indis hem de anahtar kelime ile erişim sağlayan, bellek dostu **değiştirilemez** (**immutable**) bir yapıdır.

`deque` (**double-ended queue**): Algoritma bölümlerinde işlediğimiz "Kuyruk" yapısının, her iki uçtan da O(1) karmaşıklığıyla veri ekleme-çıkarma yapabilen optimize edilmiş halidir.

JSON Veri Değişim Formatı

Günümüzde yazılımlar tek başlarına değil, bir ağ (network) üzerinden diğer sistemlerle konuşarak çalışırlar. Farklı dillerle (Python, C++, C#) yazılmış sistemlerin birbirini anlayabilmesi için ortak bir veri diline ihtiyaç vardır. **JSON** (JavaScript Object Notation), bu ihtiyacı karşılayan, dilden bağımsız, metin tabanlı ve insan tarafından okunabilir en popüler veri değişim formatıdır.

JSON Yapısı ve Python Nesneleriyle İlişkisi

JSON yapısı, bir mühendis için yabancı değildir; çünkü Python'daki **sözlük** (dict) ve **liste** (list) yapılarının birebir metin karşılığıdır.

- Nesneler: Süslü parantez { } içinde tutulan **anahtar-değer** (key-value) çiftleridir. (Python'daki dict)
- Diziler: Köşeli parantez [] içinde tutulan sıralı veri gruplarıdır. (Python'daki list)

Serileştirme

Python nesnelerini (bellekteki canlı verileri) bir dosyaya yazılacak veya ağdan gönderilecek bir metin formatına dönüştürme işlemine **serileştirme** (serialization) denir. Bunun tersi işleme ise **deserileştirme** (deserialization) adı verilir.

- `json.dumps()`: Python nesnesini bir JSON karakter dizisine (string) dönüştürür.
- `json.loads()`: JSON formatındaki bir metni Python nesnesine dönüştürür.

Dosya Tabanlı İşlemler

Eğer veri bir string değişkeni yerine doğrudan bir dosyaya yazılacaksa, "s" takısı olmayan fonksiyonlar kullanılır.

JSON ile Veri Bütünlüğü

JSON kullanmak, verinin tip güvenliğini sağlar. Örneğin, Python'daki bir **None** değeri JSON'da **null** olarak, **True** değeri ise **true** olarak saklanır. Bu sayede veri, başka bir dile (örneğin C#'a) aktarıldığında o dilin standartlarına göre doğru şekilde yorumlanır.

JSON verisi metin tabanlı olduğu için **utf-8** karakter kodlamasıyla saklanır. Büyük veri setlerinde JSON dosyaları çok fazla yer kaplayabilir; bu durumlarda veriyi sıkıştırmak veya **pickle** gibi **ikili** (binary) formatları kullanmak performans açısından değerlendirilmelidir.

Örnek Uygulama

Aşağıda sistem bilgisi ve veri saklama konulu örnek verilmiştir

```
import os
import sys
import json

def sistem_ozeti_hazirla():
    # Sistemden bilgi toplama
    bilgi = {
        "isletim_sistemi": os.name,
        "python_surumu": sys.version.split()[0],
        "mevcut_dizin": os.getcwd()
    }

    # Veriyi JSON formatına dönüştürme (Serileştirme)
    json_cikti = json.dumps(bilgi, indent=4)
    print("Sistem Analiz Raporu:")
    print(json_cikti)

sistem_ozeti_hazirla()
```

Bu programın çıktısı aşağıdaki gibi olacaktır;

```
Sistem Analiz Raporu:
{
    "isletim_sistemi": "nt",
    "python_surumu": "3.13.4",
    "mevcut_dizin": "D:\\Python\\PSamples"
}
```

TEST VE HATA AYIKLAMA

Yazılım geliştirmede test yazmak, kodun doğru çalıştığından emin olmak için kritik öneme sahiptir. Python'da test yazmak için `unittest` (standart kütüphane) ve `pytest` (üçüncü taraf, daha popüler) gibi araçlar kullanılır.

Test, kodunuzun beklediğiniz gibi çalıştığını doğrulayan küçük programlardır. Test yazmanın faydaları aşağıda sıralanmıştır:

- Hataları erken tespit eder
- Kod değişikliklerinde güven sağlar (refactoring)
- Kodun nasıl kullanılacağını gösterir (dokümantasyon)
- Daha iyi tasarım yapmaya zorlar
- Hata ayıklama süresini azaltır

Yazılım Doğrulamada Birim Testi

Yazılım mühendisliğinde bir fonksiyonun veya sınıfın beklenen işi yapıp yapmadığını otomatik olarak kontrol eden küçük test kodlarına **birim testi** (**unit test**) denir. Bir mühendis için test yazmak, tasarımın doğruluğunu ispatlamaktır. Kodun bir noktasında yapılan değişikliğin, sistemin başka bir yerini bozup bozmadığını (**regression**) anlamanın tek yolu budur.

Python, bu süreci yönetmek için standart kütüphanesinde `unittest` modülünü sunar.

unittest Modülü Temel Yapısı

Bir test dosyası oluştururken `unittest.TestCase` sınıfından kalıtım alan sınıflar yazılır. Test yöntemlerinin ismi mutlaka `test_` ifadesi ile başlamalıdır.

Örnek Senaryo olarak matematiksel bir fonksiyonun test edilmesini ele alalım.

```
# hesaplama.py
def toplama(a, b):
    return a + b
```

Şimdi bu fonksiyonu test eden sınıfımızı yazalım:

```
import unittest
from hesaplama import toplama

class TestMatematik(unittest.TestCase):
    def test_pozitif_toplama(self):
        # assertEquals: sonucun beklenen değere eşit olup olmadığını kontrol eder
        self.assertEqual(toplama(10, 5), 15)

    def test_negatif_toplama(self):
        self.assertEqual(toplama(-1, -1), -2)

if __name__ == '__main__':
    unittest.main()
```

İddia Yöntemleri

`unittest` içinde durumları kontrol etmek için çeşitli iddia yöntemleri (**assertion**) bulunur:

- `self.assertEqual(a, b)`: `a == b` olduğunu kontrol eder.
- `self.assertTrue(x)`: `x` değerinin `True` olduğunu kontrol eder.
- `self.assertRaises(İstisna)`: Kodun belirli bir istisnai duruma düşüp düşmediğini denetler.

Test Hazırlığı ve Temizliği

Her testten önce bir veri tabanı bağlantısı kurmak veya bir sensör nesnesi oluşturmak gerekebilir.

- `setUp()`: Her test metodundan önce otomatik çalışır.
- `tearDown()`: Her test metodundan sonra otomatik çalışır (temizlik işlemleri için).

```
class TestSensor(unittest.TestCase):
    def setUp(self):
        # Test öncesi sensör nesnesini hazırla
        self.sensor = Sensor(id=101)

    def test_okuma_araligi(self):
        deger = self.sensor.oku()
        self.assertTrue(0 <= deger <= 100)
```

pytest Modülü

Bu bölümde pytest'e odaklanacağız çünkü daha basit ve güçlüdür. Kurulum yapmak için `pip install pytest` komutu çalıştırılır.

pytest ile test dosyaları `test_` ile başlar veya `test` ile biter. Test fonksiyonları da `test` ile başlamalıdır. Hesaplamalar yapan bir `hesap.py` dosyasını modül olarak hazırlayalım;

```
# hesap.py
def topla(a: int, b: int) -> int:
    """İki sayıyı toplar."""
    return a + b

def carp(a: int, b: int) -> int:
    """İki sayıyı çarpar."""
    return a * b

def bol(a: float, b: float) -> float:
    """İki sayıyı böler."""
    if b == 0:
        raise ValueError("Sıfıra bölme hatası!")
    return a / b
```

Hesaplama dosyasının testini yapacak bir dosyayı aşağıdaki gibi hazırlayabiliriz;

```
# test_hesap.py
from hesaplama import topla, carp, bol
import pytest

def test_topla():
    """toplam fonksiyonunu test eder."""
    assert topla(2, 3) == 5
    assert topla(-1, 1) == 0
    assert topla(0, 0) == 0

def test_carp():
    """carp fonksiyonunu test eder."""
    assert carp(2, 3) == 6
    assert carp(-2, 3) == -6
    assert carp(0, 5) == 0

def test_bol():
    """bol fonksiyonunu test eder."""
    assert bol(6, 2) == 3.0
    assert bol(5, 2) == 2.5

def test_bol_sifira_bolme():
    """Sıfıra bölme hatasını test eder."""
```

```
with pytest.raises(ValueError):
    bol(10, 0)
```

Testleri çalıştırmak için `pytest test_hesaplama.py` komutu çalıştırılır. Çalıştırma sonucu aşağıdaki çıktı elde edilir;

```
D:\Python\PSamples> pytest test_hesap.py
===== test session starts =====
platform win32 -- Python 3.13.4, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\Python\PSamples
collected 4 items

test_hesap.py .... [100%]

===== 4 passed in 0.04s =====
D:\Python\PSamples>
```

assert Fonksiyonu

Bu fonksiyon yani `assert` (iddia etmek/doğrulamak) fonksiyonu, yazılımın hata toleransı ve savunmacı programlama (defensive programming) prensipleri çerçevesinde kullanılır.

Yazılım geliştirme sürecinde, belirli bir noktada bir koşulun mutlaka doğru (True) olması gerektiğini varsayabiliriz. `assert` fonksiyonu, bu varsayımların doğruluğunu çalışma anında (runtime) kontrol eden bir güvenlik mekanizmasıdır. Eğer belirtilen koşul yanlış (False) ise, program akışı anında kesilir ve bir `AssertionError` istisna nesnesi yaratılır. Söz dizimi aşağıda verilmiştir;

```
assert <koşul>, <hata_mesajı>
```

Bir mühendis için `assert`, kodun sadece düzgün çalışmasını değil, aynı zamanda beklenmedik durumlara karşı kendini imha etmesini (fail-fast) sağlar. Hatalı bir veriyle işleme devam edip sistemin geri dönülemez bir zarara uğraması yerine, hatanın kaynağında durdurulması tercih edilir. Aşağıdaki durumlarda kullanılabilir;

- Girdi Doğrulama: Bir fonksiyona gelen parametrelerin, mantıksal sınırlar içerisinde olup olmadığını denetler.
- Değişmezlerin (invariant) Kontrolü: Döngülerin başında veya sonunda bir değer asla değişmemesi gereken durumları denetler.
- Hata Ayıklama (debug): Kodun iç mantığını test etmek için kullanılır.

Bir personelin maaş artış oranını hesaplayan bir fonksiyonda, oran negatif olamaz. Bu, yazılımcının bir varsayımdır. Aşağıda buna ilişkin kod örneği verilmiştir;

```
def maas_artir(mevcut_maas, artis_orani):
    # Artis oranının negatif olamayacağını garanti altına alıyoruz
    assert artis_orani >= 0, "Artış oranı negatif olamaz!"

    yeni_maas = mevcut_maas * (1 + artis_orani)
    return yeni_maas

# Doğru kullanım
print(maas_artir(10000, 0.20))

# Hatalı kullanım (Program burada AssertionError verecektir)
print(maas_artir(10000, -0.10))
```

Yazılımcının yaptığı mantıksal hataları veya asla gerçekleşmemesi gereken durumları (programcı hataları) tespit etmek için `assert` fonksiyonu kullanılır. İstisna yönetimi için kullanılan `try-except` bloğu ise kullanıcı hataları (hatalı dosya yolu, yanlış şifre vb.) veya dış etkenler gibi beklenen hataları yönetmek için kullanılır.

Python yorumlayıcısı -O (**optimize**) bayrağı ile çalıştırıldığında tüm `assert` ifadeleri görmezden gelinir. Bu nedenle, **iş kritik** (**business-critical**) kontroller için `assert` yerine if blokları ve `raise` ifadeleri tercih edilmelidir. `assert` daha çok geliştirme ve test aşamasındaki iç denetim mekanizmasıdır.

`pytest` modülü Python'un dahili `assert` talimatını kullanır:

```
# Eşitlik kontrolü
assert 2 + 2 == 4

# Eşitsizlik kontrolü
assert 3 != 4

# Büyüklük kontrolü
assert 5 > 3
assert 3 < 5

# True/False kontrolü
assert True
assert not False

# Liste içeriği
assert [1, 2, 3] == [1, 2, 3]
assert 1 in [1, 2, 3]

# String içeriği
assert "python" in "python programlama"
assert "merhaba".startswith("mer")
```

Birden Fazla Girdiyle Test

Aynı testin farklı girdilerle çalıştırılması için **parametrelendirme** (**parametrize**) kullanılır:

```
# test_hesaplama.py
@pytest.mark.parametrize("a, b, beklenen", [
    (1, 2, 3),
    (5, 5, 10),
    (-1, 1, 0),
    (0, 0, 0),
    (100, 200, 300)
])
def test_topla_parametrize(a, b, beklenen):
    from hesaplama import topla
    assert topla(a, b) == beklenen
```

Bu testin çıktısı aşağıda verilmiştir;

```
$ pytest test_hesaplama.py::test_topla_parametrize -v
===== test session starts =====
test_hesaplama.py::test_topla_parametrize[1-2-3] PASSED      [ 20%]
test_hesaplama.py::test_topla_parametrize[5-5-10] PASSED     [ 40%]
test_hesaplama.py::test_topla_parametrize[-1-1-0] PASSED     [ 60%]
test_hesaplama.py::test_topla_parametrize[0-0-0] PASSED      [ 80%]
test_hesaplama.py::test_topla_parametrize[100-200-300] PASSED [====100%]

===== 5 passed in 0.05s =====
```

Testler için ortak bir veri ya da kaynak havuzu oluşturulabilir. Bu verilere **demirbaş** (**fixture**) adı verilir;

```
@pytest.fixture
def ornek_liste():
    """Test için örnek liste hazırlar."""
    return [1, 2, 3, 4, 5]

@pytest.fixture
```

```
def ornek_sozluk():
    """Test için örnek sözlük hazırlar."""
    return {"isim": "Ali", "yas": 25, "sehir": "Ankara"}

def test_liste_uzunlugu(ornek_liste):
    assert len(ornek_liste) == 5

def test_liste_toplam(ornek_liste):
    assert sum(ornek_liste) == 15

def test_sozluk_anahtarlari(ornek_sozluk):
    assert "isim" in ornek_sozluk
    assert "yas" in ornek_sozluk

def test_sozluk_degerleri(ornek_sozluk):
    assert ornek_sozluk["isim"] == "Ali"
    assert ornek_sozluk["yas"] == 25
```

Demirbaşların (**fixture**) ne zaman oluşturulup yok edileceğini belirlenebilir. Buna **demirbaş kapsamı** (**fixture scope**) adı verilir.

```
# Her test için yeni oluştur (varsayılan)
@pytest.fixture(scope="function")
def her_test_icin():
    print("\nHer test için oluşturuldu")
    return "veri"

# Tüm modül için bir kez oluştur
@pytest.fixture(scope="module")
def modul_icin_bir_kez():
    print("\nModül için bir kez oluşturuldu")
    veri = {"baglanti": "veritabani_baglantisi"}
    yield veri # yield kullanımı
    print("\nModül sonu - temizlik yapıldı")

# Tüm test oturumu için bir kez
@pytest.fixture(scope="session")
def oturum_icin_bir_kez():
    print("\nOturum başlangıcında oluşturuldu")
    return "global_veri"
```

Sahte Nesneler

Dış bağımlılıkları test etmek için **sahte** (**mock**) nesneler kullanılır;

```
# hava_durumu.py
import requests

def sehir_sicaklik(sehir: str) -> float:
    """Bir şehrin sıcaklığını API'den alır."""
    url = f"https://api.weather.com/{sehir}"
    yanit = requests.get(url)
    veri = yanit.json()
    return veri["sicaklik"]
```

```
# test_hava_durumu.py
from hava_durumu import sehir_sicaklik
from unittest.mock import Mock, patch

def test_sehir_sicaklik():
    """API çağrısını mock'luyoruz."""
```



```

with patch('hava_durumu.requests.get') as mock_get:
    # Mock yanıt oluştur
    mock_yanıt = Mock()
    mock_yanıt.json.return_value = {"sicaklik": 25.5}
    mock_get.return_value = mock_yanıt

    # Testi çalıştır
    sicaklik = sehir_sicaklik("Ankara")

    # Kontroller
    assert sicaklik == 25.5
    mock_get.assert_called_once_with("https://api.weather.com/Ankara")

```

pytest-mock Modülü

pytest-mock modülü daha kolay **sahte** (**mock**) nesne kullanımı sağlar. `pip install pytest-mock` komutu ile kurulum yapılır.

```

"""pytest-mock ile daha basit kullanım."""
# Mock oluştur
mock_get = mocker.patch('hava_durumu.requests.get')
mock_get.return_value.json.return_value = {"sicaklik": 30.0}

# Test
sicaklik = sehir_sicaklik("İstanbul")

# Kontrol
assert sicaklik == 30.0

```

Veri Tabanı Testi

Aşağıda **demirbaş** (**fixture**) yöntemi ile bir veri tabanı testi örneği verilmiştir;

```

import sqlite3

@pytest.fixture
def veritabani():
    """Test veritabanı oluşturur."""
    baglanti = sqlite3.connect(":memory:") # Bellekte veritabanı
    cursor = baglanti.cursor()

    # Tablo oluştur
    cursor.execute("""
        CREATE TABLE kullanicilar (
            id INTEGER PRIMARY KEY,
            isim TEXT NOT NULL,
            yas INTEGER
        )
    """)

    # Örnek veri ekle
    cursor.execute("INSERT INTO kullanicilar (isim, yas) VALUES ('Ali', 25)")
    cursor.execute("INSERT INTO kullanicilar (isim, yas) VALUES ('Ayşe', 30)")
    baglanti.commit()

    yield baglanti # Test için veritabanını ver

    baglanti.close() # Test sonrası temizle

def test_kullanici_sayisi(veritabani):
    cursor = veritabani.cursor()
    cursor.execute("SELECT COUNT(*) FROM kullanicilar")
    sayi = cursor.fetchone()[0]

```

```

    assert sayi == 2

def test_kullanici_ekle(veritabani):
    cursor = veritabani.cursor()
    cursor.execute("INSERT INTO kullanicilar (isim, yas) VALUES ('Mehmet', 28)")
    veritabani.commit()

    cursor.execute("SELECT COUNT(*) FROM kullanicilar")
    sayi = cursor.fetchone()[0]
    assert sayi == 3

```

Test Kapsama Oranı

Kodunuzun ne kadarının test edildiğini görmek için **test kapsama oranı** (coverage) kullanılır. `pip install pytest-cov` konutu ile kurulum yapılır.

```

$ pytest --cov=hesaplama test_hesap.py
===== test session starts =====
collected 4 items

test_hesap.py .... [100%]

----- coverage: platform win32, python 3.10.0 -----
Name           Stmts   Miss  Cover
-----
hesap.py         8       0   100%
-----
TOTAL            8       0   100%

===== 4 passed in 0.10s =====

```

HTML raporu oluşturmak için `pytest --cov=hesaplama --cov-report=html test_hesap.py` komutu kullanılabilir.

Test Kapsama Oranı (Coverage) Hedefleri

- % 80 ile 100 arası mükemmel,
- %60 ile 80 arası iyi,
- %40 ile 60 arası kabul edilebilir,
- %0 ile 40 arası yetersiz sayılır.

Testleri Gruplandırma

Testleri işaretleyerek gruplandırabiliriz.

```

@pytest.mark.hizli
def test_hizli_islem():
    assert 1 + 1 == 2

@pytest.mark.yavas
def test_yavas_islem():
    import time
    time.sleep(2)
    assert True

@pytest.mark.veritabani
def test_veritabani_baglantisi():
    # Veritabanı testi
    pass

```

```
@pytest.mark.network
def test_api_cagrisi():
    # API testi
    pass
```

Bu testleri çalıştırmak için aşağıdaki komutlar çalıştırılabilir;

```
$ pytest -m hizli # Sadece hızlı testler
$ pytest -m "not yavas" # Yavaş testler hariç
$ pytest -m "veritabani or network" # Veritabanı veya network testleri
```

Paylaşımlı Demirbaşlar

Aşağıdaki dosyadaki **demirbaşlar** (**fixture**) tüm test dosyalarında kullanılabilir.

```
# conftest.py
import pytest

@pytest.fixture
def uygulama_ayarlari():
    """Tüm testlerde kullanılabilir."""
    return {
        "veritabani": "test.db",
        "debug": True,
        "port": 5000
    }

@pytest.fixture(autouse=True)
def test_baslangic_bitis():
    """Her testten önce ve sonra otomatik çalışır."""
    print("\n--- Test başlıyor ---")
    yield
    print("\n--- Test bitti ---")
```

Zaman Uyumsuz Fonksiyon Testleri

Zaman uyumsuz (**asynchron**) fonksiyonları test etmek için `pytest-asyncio` kullanılır. `pip install pytest-asyncio` komutu ile kurulum yapılır.

```
# async_islemler.py
import asyncio

async def asenkron_topla(a: int, b: int) -> int:
    await asyncio.sleep(0.1) # Simüle edilmiş asenkron işlem
    return a + b

async def api_cagir(url: str) -> dict:
    await asyncio.sleep(0.5)
    return {"durum": "basarili", "veri": "test"}
```

Bu işlemlerin bulunduğu dosyayı test etmek için aşağıdaki kod yazılabilir;

```
# test_async_islemler.py
import pytest
from async_islemler import asenkron_topla, api_cagir

@pytest.mark.asyncio
async def test_asenkron_topla():
    sonuc = await asenkron_topla(2, 3)
    assert sonuc == 5

@pytest.mark.asyncio
async def test_api_cagir():
    sonuc = await api_cagir("https://api.example.com")
    assert sonuc["durum"] == "basarili"
```

Test Odaklı Yazılım Geliştirme

Test odaklı yazılım geliştirme (**Test Driven Development-TDD**), önce test yazıp sonra kodu yazma metodolojisi. Bu metodolojide ilk önce başarısız bir test yazılır (**red**), sonrasında testi geçecek minimum kod yazılır (**green**), en sonunda kod iyileştirilir (**refactor**).

```
# 1. Önce test yaz (başarısız olacak)
def test_kullanici_olustur():
    kullanıcı = Kullanici("Ali", "ali@example.com")
    assert kullanıcı.isim == "Ali"
    assert kullanıcı.email == "ali@example.com"

# 2. Kodu yaz (testi geçirecek)
class Kullanici:
    def __init__(self, isim: str, email: str):
        self.isim = isim
        self.email = email

# 3. Kodu iyileştir
class Kullanici:
    def __init__(self, isim: str, email: str):
        if not isim:
            raise ValueError("İsim boş olamaz")
        if "@" not in email:
            raise ValueError("Geçersiz email")
        self.isim = isim
        self.email = email
```

En İyi Uygulamalar

- Her fonksiyon için en az bir test yazılmalı
- Test isimleri açıklayıcı olmalı (`test_topla_iki_pozitif_sayi`)
- Testler birbirinden bağımsız olmalı ve hızlı çalışmalı
- Sahte nesne (**mock**) kullanarak dış bağımlılıkları izole edilmeli
- Test kapsama oranı (**coverage**) %100'e yakın olmalı (kritik kod için)
- Demirbaşlar (**fixture**) tekrar kullanılabilir yapılmalı
- Testleri her sürüm sırasında düzenli çalıştırılmalı (CI/CD)
- Test kodu da temiz tutulmalı

Test İçeren Örnek Proje

Test içeren örnek bir proje aşağıdaki gibi olmalıdır;

```
proje/
├── src/
│   ├── __init__.py
│   ├── hesaplama.py
│   └── kullanıcı.py
├── tests/
│   ├── __init__.py
│   ├── conftest.py
│   ├── test_hesaplama.py
│   └── test_kullanici.py
├── pytest.ini
└── requirements.txt
```

Bu projede yer alan `pytest.ini` dosyası aşağıdaki gibi olmalıdır;

```
[pytest]
testpaths = tests
python_files = test_*.py
python_classes = Test*
python_functions = test_*
```

```
markers =  
    hizli: Hızlı testler  
    yavas: Yavaş testler  
    veritabani: Veritabanı testleri  
    network: Network gerektiren testler
```

ZAMAN UYUMSUZ PROGRAMLAMA

Zaman uyumsuz (**asynchron**) programlama, bir programın birden fazla işlemi aynı anda (eşzamanlı olarak) yapabilmesini sağlar. Python'da asenkron programlama için `asyncio` kütüphanesi kullanılır. Bu özellik Python 3.4'te tanıtılmış, Python 3.5'te `async/await` talimatları da eklenmiştir.

Zaman Uyumlu ve Uyumsuz Programlama

Zaman Uyumlu (**sincron**) programlamada, her işlem bir önceki işlemin bitmesini bekler;

```
import time

def gorev1():
    print("Görev 1 başladı")
    time.sleep(2) # 2 saniye bekle
    print("Görev 1 bitti")

def gorev2():
    print("Görev 2 başladı")
    time.sleep(1) # 1 saniye bekle
    print("Görev 2 bitti")

baslangic = time.time()
gorev1()
# Çıktı: Görev 1 başladı
# Çıktı: Görev 1 bitti
gorev2()
# Çıktı: Görev 2 başladı
# Çıktı: Görev 2 bitti
print(f"Toplam süre: {time.time() - baslangic:.2f} saniye")
# Çıktı: Toplam süre: 3.00 saniye
```

Zaman Uyumsuz (**asynchron**) programlamada ise görevler beklerken diğer görevler çalışabilir;

```
import asyncio

async def gorev1():
    print("Görev 1 başladı")
    await asyncio.sleep(2) # 2 saniye bekle (asenkron)
    print("Görev 1 bitti")

async def gorev2():
    print("Görev 2 başladı")
    await asyncio.sleep(1) # 1 saniye bekle (asenkron)
    print("Görev 2 bitti")

async def main():
    baslangic = time.time()
    await asyncio.gather(gorev1(), gorev2())
    print(f"Toplam süre: {time.time() - baslangic:.2f} saniye")

asyncio.run(main())
# Çıktı: Görev 1 başladı
# Çıktı: Görev 2 başladı
# Çıktı: Görev 2 bitti
# Çıktı: Görev 1 bitti
# Çıktı: Toplam süre: 2.00 saniye
```

Görüldüğü gibi asenkron programlamada toplam süre 2 saniye oldu çünkü görevler paralel çalıştı.

async ve await Anahtar Kelimeleri

Bu kelimeler zaman uyumsuz programlama için olmazsa olmazlardır;

async: Bir fonksiyonu zaman uyumsuz (**coroutine**) yapar.

await: Asenkron bir işlemin tamamlanmasını bekler, beklerken diğer görevlerin çalışmasına izin verir.

```
async def selam_ver(isim: str, bekleme: int):
    print(f"{isim} için bekleniyor...")
    await asyncio.sleep(bekleme)
    print(f"Merhaba {isim}!")
    return f"{isim} selamlandı"

async def main():
    sonuc = await selam_ver("Ali", 2)
    print(sonuc)

asyncio.run(main())
# Çıktı: Ali için bekleniyor...
# Çıktı: Merhaba Ali!
# Çıktı: Ali selamlandı
```

Birden Fazla Görevi Birlikte Çalıştırma

Bu kütüphanedeki **gather()** fonksiyonu ile birden fazla asenkron görevi aynı anda başlatabilir ve hepsinin bitmesini bekleyebiliriz:

```
async def indir(dosya: str, sure: int):
    print(f"{dosya} indiriliyor...")
    await asyncio.sleep(sure)
    print(f"{dosya} indirildi!")
    return f"{dosya} hazır"

async def main():
    sonuclar = await asyncio.gather(
        indir("resim1.jpg", 2),
        indir("resim2.jpg", 3),
        indir("resim3.jpg", 1)
    )
    print("Tüm indirmeler tamamlandı:")
    for sonuc in sonuclar:
        print(f" - {sonuc}")

asyncio.run(main())
# Çıktı: resim1.jpg indiriliyor...
# Çıktı: resim2.jpg indiriliyor...
# Çıktı: resim3.jpg indiriliyor...
# Çıktı: resim3.jpg indirildi!
# Çıktı: resim1.jpg indirildi!
# Çıktı: resim2.jpg indirildi!
# Çıktı: Tüm indirmeler tamamlandı:
# Çıktı: - resim1.jpg hazır
# Çıktı: - resim2.jpg hazır
# Çıktı: - resim3.jpg hazır
```

Görev Oluşturma

Zaman uyumsuz bir görev, **create_task()** ile bir coroutine'i **Task** nesnesine dönüştürüp arka planda çalıştırabiliriz:

```
async def sayac(isim: str, sayi: int):
```

```
for i in range(sayi):
    print(f"{isim}: {i+1}")
    await asyncio.sleep(1)
return f"{isim} tamamlandı"

async def main():
    # Görevleri oluştur (hemen başlatılır)
    gorev1 = asyncio.create_task(sayac("A", 3))
    gorev2 = asyncio.create_task(sayac("B", 4))

    # Her iki görevin tamamlanmasını bekle
    sonuc1 = await gorev1
    sonuc2 = await gorev2

    print(sonuc1)
    print(sonuc2)

asyncio.run(main())
# Çıktı: A: 1
# Çıktı: B: 1
# Çıktı: A: 2
# Çıktı: B: 2
# Çıktı: A: 3
# Çıktı: B: 3
# Çıktı: B: 4
# Çıktı: A tamamlandı
# Çıktı: B tamamlandı
```

Asenkron Dosya İşlemleri

Normal dosya işlemleri **zaman uyumludur** (**sincron**) ve programı bloke eder. **aiofiles** kütüphanesi ile **zaman uyumsuz** (**asynchron**) dosya işlemleri yapabiliriz:

```
import aiofiles

async def dosya_yaz(dosya_adi: str, icerik: str):
    async with aiofiles.open(dosya_adi, 'w', encoding='utf-8') as f:
        await f.write(icerik)
    print(f"{dosya_adi} yazıldı")

async def dosya_oku(dosya_adi: str) -> str:
    async with aiofiles.open(dosya_adi, 'r', encoding='utf-8') as f:
        icerik = await f.read()
    print(f"{dosya_adi} okundu")
    return icerik

async def main():
    # Paralel yazma
    await asyncio.gather(
        dosya_yaz("dosya1.txt", "İçerik 1"),
        dosya_yaz("dosya2.txt", "İçerik 2"),
        dosya_yaz("dosya3.txt", "İçerik 3")
    )

    # Paralel okuma
    sonuclar = await asyncio.gather(
        dosya_oku("dosya1.txt"),
        dosya_oku("dosya2.txt"),
        dosya_oku("dosya3.txt")
    )

    for i, icerik in enumerate(sonuclar, 1):
```



```
print(f"Dosya {i}: {icerik}")
```

```
# asyncio.run(main())
```

Asenkron HTTP İstekleri

Web istekleri genellikle yavaştır. `aiohttp` ile birden fazla isteği aynı anda yapabiliriz:

```
import aiohttp

async def sayfa_getir(url: str):
    async with aiohttp.ClientSession() as oturum:
        async with oturum.get(url) as yanit:
            icerik = await yanit.text()
            print(f"{url}: {len(icerik)} karakter")
            return len(icerik)

async def main():
    urllar = [
        "https://python.org",
        "https://github.com",
        "https://stackoverflow.com"
    ]

    sonuclar = await asyncio.gather(*[sayfa_getir(url) for url in urllar])

    print(f"Toplam: {sum(sonuclar)} karakter")

# asyncio.run(main())
```

Asenkron Bağlam Yöneticileri

Asenkron **bağlam yöneticileri** (**context manager**) `async with` kullanılarak tanımlanır:

```
class VeriTabani:
    async def __aenter__(self):
        print("Veritabanına bağlanılıyor...")
        await asyncio.sleep(1)
        print("Bağlantı kuruldu")
        return self

    async def __aexit__(self, exc_type, exc_val, exc_tb):
        print("Bağlantı kapatılıyor...")
        await asyncio.sleep(0.5)
        print("Bağlantı kapatıldı")

    async def sorgu_calistir(self, sorgu: str):
        print(f"Sorgu çalıştırılıyor: {sorgu}")
        await asyncio.sleep(0.5)
        return f"Sonuç: {sorgu}"

async def main():
    async with VeriTabani() as vt:
        sonuc = await vt.sorgu_calistir("SELECT * FROM users")
        print(sonuc)

asyncio.run(main())
# Çıktı: Veritabanına bağlanılıyor...
# Çıktı: Bağlantı kuruldu
# Çıktı: Sorgu çalıştırılıyor: SELECT * FROM users
# Çıktı: Sonuç: SELECT * FROM users
# Çıktı: Bağlantı kapatılıyor...
```

```
# Çıktı: Bağlantı kapatıldı
```

Asenkron Yineleyiciler

Asenkron **yineleyiciler** (**iterator**) için `async for` ile kullanılır:

```
class AsenkronSayac:
    def __init__(self, baslangic: int, bitis: int):
        self.mevcut = baslangic
        self.bitis = bitis

    def __aiter__(self):
        return self

    async def __anext__(self):
        if self.mevcut >= self.bitis:
            raise StopAsyncIteration

        await asyncio.sleep(0.5) # Her sayıda 0.5 saniye bekle
        self.mevcut += 1
        return self.mevcut - 1

async def main():
    async for sayi in AsenkronSayac(1, 6):
        print(f"Sayı: {sayi}")

asyncio.run(main())
# Çıktı: Sayı: 1
# Çıktı: Sayı: 2
# Çıktı: Sayı: 3
# Çıktı: Sayı: 4
# Çıktı: Sayı: 5
```

Zaman Aşımı

Bir görevin belirli bir sürede tamamlanmasını zorlamak için `wait_for()` kullanılır:

```
async def uzun_islem():
    print("Uzun işlem başladı")
    await asyncio.sleep(5)
    print("Uzun işlem bitti")
    return "Tamamlandı"

async def main():
    try:
        sonuc = await asyncio.wait_for(uzun_islem(), timeout=2.0)
        print(sonuc)
    except asyncio.TimeoutError:
        print("İşlem zaman aşımına uğradı!")

asyncio.run(main())
# Çıktı: Uzun işlem başladı
# Çıktı: İşlem zaman aşımına uğradı!
```

Asenkron Kuyruk

Üretici-tüketici desen (**producer-consumer pattern**) için asenkron kuyruk kullanılır:

```
async def uretici(kuyruk: asyncio.Queue, urun_sayisi: int):
    for i in range(urun_sayisi):
        urun = f"Ürün-{i+1}"
        await kuyruk.put(urun)
```

```
    print(f"Üretildi: {urun}")
    await asyncio.sleep(0.5)

# Tüketicilere bittiği sinyali
await kuyruk.put(None)

async def tuketici(kuyruk: asyncio.Queue, isim: str):
    while True:
        urun = await kuyruk.get()
        if urun is None:
            # Bittiği sinyali alındı, diğer tüketiciler için geri koy
            await kuyruk.put(None)
            break

        print(f"{isim} işledi: {urun}")
        await asyncio.sleep(1)
        kuyruk.task_done()

async def main():
    kuyruk = asyncio.Queue()

    uretici_gorev = asyncio.create_task(uretici(kuyruk, 5))
    tuketici_gorevleri = [
        asyncio.create_task(tuketici(kuyruk, "Tüketici-1")),
        asyncio.create_task(tuketici(kuyruk, "Tüketici-2"))
    ]

    await uretici_gorev
    await asyncio.gather(*tuketici_gorevleri)

# asyncio.run(main())
```

Hata Yönetimi

Asenkron fonksiyonlarda da normal `try-except` kullanılır:

```
async def hata_olustur():
    await asyncio.sleep(1)
    raise ValueError("Bir hata oluştu!")

async def guvenli_calistir():
    try:
        await hata_olustur()
    except ValueError as e:
        print(f"Hata yakalandı: {e}")
    finally:
        print("Temizlik işlemleri yapıldı")

asyncio.run(guvenli_calistir())
# Çıktı: Hata yakalandı: Bir hata oluştu!
# Çıktı: Temizlik işlemleri yapıldı
```

İlk Tamamlanan

Birden fazla görevden ilk tamamlananı işlemek için `as_completed()` fonksiyonu kullanılır:

```
async def gorev(isim: str, sure: int):
    await asyncio.sleep(sure)
    return f"{isim} tamamlandı ({sure}s)"

async def main():
    gorevler = [
```

```
        gorev("Hızlı", 1),
        gorev("Orta", 2),
        gorev("Yavaş", 3)
    ]

    for tamamlanan in asyncio.as_completed(gorevler):
        sonuc = await tamamlanan
        print(sonuc)

asyncio.run(main())
# Çıktı: Hızlı tamamlandı (1s)
# Çıktı: Orta tamamlandı (2s)
# Çıktı: Yavaş tamamlandı (3s)
```

Özet ve En İyi Uygulamalar

- Girdi/çıkıtlı yoğun işlemler için `asyncio` idealdir
- İşlemci yoğun kullanılacağı işlemler (**CPU-bound**) için `multiprocessing` kullanılır
- `await` kullanmadan `async` fonksiyon çağrılmaz
- Bloke eden fonksiyonlar asenkron blokta kullanılmaz
- `time.sleep()` yerine `await asyncio.sleep()` kullanılır
- Uzun süren işlemler için `asyncio.wait_for()` ile zaman aşımı (**timeout**) eklenir
- Birden fazla görev için `gather()` veya `create_task()` kullanılır
- Zaman uyumsuz işlemler için asenkron kütüphaneler kullanılır (`aiohttp`, `aiofiles` vb.)
- Olay döngüsü (**event loop**), `asyncio.run()` ile başlatılır
- Hata yönetimi için `try-except` kullanılır
- Bağlam Yönetici (**context manager**) için `async with` kullanılır
- Yineleyiciler (**iterator**) için `async for` kullanın

TÜR İPUÇLARI

Python dinamik tipli bir dildir. Yani bir değişkenin veri tipi **çalışma zamanında** (runtime) belirlenir. Ancak büyük projelerde bu durum hatalara sebep olabilir. İşte bu noktada **tür ipuçları** (type hints) devreye girer. Tür ipuçları Python 3.5 ile tanıtılmış ve sonraki sürümlerde geliştirilmiştir.

Tür İpuçlarının Temelleri

Tür ipuçları, değişkenlere, fonksiyon parametrelerine ve dönüş değerlerine beklenen veri tipini belirtmemizi sağlar. Bu sadece bir ipucudur; Python bu tipleri çalışma zamanında kontrol etmez. Ancak mypy gibi araçlar bu ipuçlarını kullanarak kod analizini yapar.

```
def toplam(a: int, b: int) -> int:
    return a + b
sonuc: int = toplam(5, 3)
print(sonuc)
# Çıktı: 8
```

Yukarıdaki örnekte a ve b parametrelerinin int tipinde olması gerektiği, fonksiyonun int döndüreceği ve sonuc değişkeninin int olduğu belirtilmiştir.

Temel Veri Tipleri

```
isim: str = "İlhan"
yas: int = 35
boy: float = 1.75
aktif: bool = True
sayılar: list = [1, 2, 3]
koordinat: tuple = (10, 20)
ayarlar: dict = {"dil": "tr"}
```

Gelişmiş Tür İpuçları

Daha karmaşık tür ipuçları için typing modülü kullanılır:

```
from typing import List, Dict, Optional, Union, Tuple

# Liste, elemanları int
sayılar: List[int] = [1, 2, 3, 4, 5]

# Sözlük, anahtarları str, değerleri int
puanlar: Dict[str, int] = {"Ali": 85, "Ayşe": 92}

# None olabilir
def bul_ogrenci(id: int) -> Optional[str]:
    if id == 1:
        return "Ahmet"
    return None

# Birden fazla tip kabul eder (Python 3.10+ için | kullanılabilir)
def isle(veri: Union[int, str]) -> str:
    return str(veri)

# Python 3.10+ ile daha basit sözdizimi
def isle_yeni(veri: int | str) -> str:
    return str(veri)
```

Python 3.10+ Modern Özellikler

Eşleşme Desenleri

Python 3.10 ile gelen en önemli özelliklerden biri **eşleşme desenleridir** (pattern matching). `match-case` ifadesi ile karmaşık koşullu ifadeleri daha okunabilir hale getirebiliriz:

```
def http_durumu(kod: int) -> str:
    match kod:
        case 200:
            return "Başarılı"
        case 404:
            return "Bulunamadı"
        case 500:
            return "Sunucu Hatası"
        case _:
            return "Bilinmeyen Durum"

print(http_durumu(200))
# Çıktı: Başarılı
```

Daha karmaşık eşleşmeler de yapılabilir:

```
def komut_isle(komut: tuple) -> str:
    match komut:
        case ["git", yön]:
            return f"{yön} yönüne gidiliyor"
        case ["al", nesne, miktar]:
            return f"{miktar} adet {nesne} alınıyor"
        case ["dur"]:
            return "Duruldu"
        case _:
            return "Bilinmeyen komut"

print(komut_isle(["git", "kuzey"]))
# Çıktı: kuzey yönüne gidiliyor
print(komut_isle(["al", "elma", 5]))
# Çıktı: 5 adet elma alınıyor
```

Union İşleci

Python 3.10'dan önce birden fazla tipi belirtmek için Union kullanıyorduk. Artık daha basit `|` işlecini kullanabiliriz:

```
# Eski yöntem (hala geçerli)
from typing import Union
def islem(sayi: Union[int, float]) -> Union[int, float]:
    return sayi * 2

# Yeni yöntem (Python 3.10+)
def islem_yeni(sayi: int | float) -> int | float:
    return sayi * 2

# None için de kullanılabilir
def bul(anahtar: str) -> str | None:
    sozluk = {"a": "değer"}
    return sozluk.get(anahtar)
```

Statik Tip Kontrolü

Python kodundaki tür ipuçlarını kontrol eden bir araç `mypy`'dir. Kodunuzu çalıştırmadan önce tip hatalarını bulmanızı sağlar. Kurulumu `pip install mypy` komutu ile yapılır.

Örnek bir hatalı kod:

```
# ornek.py
# def toplam(a: int, b: int) -> int:
#     return a + b

# sonuc: int = toplam(5, "10") # Hata: str kullanıldı
```

mypy ile kontrol:

```
$ mypy ornek.py
ornek.py:4: error: Argument 2 to "toplam" has incompatible type "str"; expected "int"
```

Generic Tipler ve TypeVar

Generic tipler, fonksiyonların farklı tiplerle çalışmasını sağlar:

```
from typing import TypeVar, List

T = TypeVar('T') # Generic tip değişkeni

def ilk_eleman(liste: List[T]) -> T | None:
    return liste[0] if liste else None

sayilar = [1, 2, 3]
print(ilk_eleman(sayilar)) # mypy int olarak anlar
# Çıktı: 1

kelimeler = ["merhaba", "dünya"]
print(ilk_eleman(kelimeler)) # mypy str olarak anlar
# Çıktı: merhaba
```

Literal ve Final

Literal, bir değişkenin sadece belirli değerleri alabileceğini belirtir:

```
from typing import Literal

def yon_degistir(yon: Literal["kuzey", "güney", "doğu", "batı"]) -> str:
    return f"{yon} yönüne dönülüyor"

print(yon_degistir("kuzey")) # Geçerli
# Çıktı: kuzey yönüne dönülüyor
# print(yon_degistir("yukarı")) # mypy hata verir
```

Final, bir değişkenin değiştirilmemesi gerektiğini belirtir:

```
from typing import Final

MAX_BAGLANTI: Final[int] = 100
# MAX_BAGLANTI = 200 # mypy hata verir
```

Protocol ve Structural Subtyping

Protocol, bir sınıfın hangi yöntemlere sahip olması gerektiğini belirtir. Kalıtım gerektirmez:

```
from typing import Protocol

class Cizilebilir(Protocol):
    def ciz(self) -> None:
        ...

class Daire:
    def ciz(self) -> None:
```

```

        print("Daire çiziliyor")

class Kare:
    def ciz(self) -> None:
        print("Kare çiziliyor")

def sekil_ciz(sekil: Cizilebilir) -> None:
    sekil.ciz()

sekil_ciz(Daire()) # Geçerli - ciz() metodu var
# Çıktı: Daire çiziliyor
sekil_ciz(Kare()) # Geçerli - ciz() metodu var
# Çıktı: Kare çiziliyor

```

TypedDict

TypedDict, **sözlüklerin** (dict) yapısını tanımlamak için kullanılır:

```

from typing import TypedDict

class Ogrenci(TypedDict):
    isim: str
    yas: int
    sinif: str

ogrenci: Ogrenci = {
    "isim": "Ahmet",
    "yas": 15,
    "sinif": "9-A"
}

def ogrenci_bilgisi(o: Ogrenci) -> str:
    return f"{o['isim']}, {o['yas']} yaşında, {o['sinif']} sınıfında"

print(ogrenci_bilgisi(ogrenci))
# Çıktı: Ahmet, 15 yaşında, 9-A sınıfında

```

Callable

Fonksiyonları parametre olarak alan fonksiyonlar için **callable** kullanılır:

```

from typing import Callable

def uygula(func: Callable[[int, int], int], a: int, b: int) -> int:
    return func(a, b)

def topla(x: int, y: int) -> int:
    return x + y

def carp(x: int, y: int) -> int:
    return x * y

print(uygula(topla, 5, 3))
# Çıktı: 8
print(uygula(carp, 5, 3))
# Çıktı: 15

```

Özet ve En İyi Uygulamalar

- Tür ipuçları opsiyoneldir ancak büyük projelerde çok yararlıdır
- **mypy** ile kodunuzu düzenli olarak kontrol edin
- Python 3.10+ kullanıyorsanız **|** operatörünü tercih edin

- Generic tipler kod tekrarını önler
- Protocol, duck typing'i tip güvenli hale getirir
- Tüm fonksiyonlarda dönüş tipini belirtin
- Public API'lerde tür ipuçları kullanmak özellikle önemlidir
- IDE'ler (PyCharm, VS Code) tür ipuçlarını kullanarak daha iyi otomatik tamamlama sağlar
- Karmaşık veri yapılarında `TypedDict` kullanın
- Değiştirilmemesi gereken değerler için `Final` kullanın

WEB GELİŞTİRME

FastAPI, modern, hızlı (yüksek performanslı) ve Python 3.6+ ile web API'leri oluşturmak için tasarlanmış bir web **çerçevesidir** (**framework**). Otomatik API dokümantasyonu, **tür ipuçları** (**type hints**) desteği ve yüksek performansı ile popülerdir.

FastAPI'nin Özellikleri;

- Hızlı: NodeJS ve Go kadar hızlı performans
- Kolay: Öğrenmesi ve kullanması kolay
- Otomatik Dokümantasyon: Swagger UI ve ReDoc otomatik oluşturulur
- Type Hints: Python type hints ile güçlü tip kontrolü
- Asenkron Destek: async/await tam desteği
- Veri Doğrulama (validation): Pydantic ile otomatik veri doğrulama
- Dependency Injection: Bağımlılık enjeksiyonu sistemi

Kurulum için aşağıdaki iki komut çalıştırılır;

```
$ pip install fastapi
$ pip install "uvicorn[standard]" # ASGI sunucusu
```

İlk FastAPI Uygulaması

İlk uygulamamız aşağıdaki gibi olsun;

```
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def ana_sayfa():
    return {"mesaj": "Merhaba FastAPI!"}

@app.get("/selam/{isim}")
def selam_ver(isim: str):
    return {"mesaj": f"Merhaba {isim}!"}
```

Uygulamayı çalıştırma:

```
$ uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['D:\\Python\\Samples\\fastapi']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [13032] using WatchFiles
INFO: Started server process [3376]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Tarayıcıda aşağıdaki URL'ler açılır;

- <http://127.0.0.1:8000> - Ana sayfa
- <http://127.0.0.1:8000/selam/Alı> - Selamlaşma
- <http://127.0.0.1:8000/docs> - Otomatik API dokümantasyonu (Swagger UI)
- <http://127.0.0.1:8000/redoc> - Alternatif dokümantasyon (ReDoc)

Path Parametreleri

URL'den değer almak için path parametreleri kullanılır;

```
def kullanıcı_getir(kullanıcı_id: int):
    return {"kullanıcı_id": kullanıcı_id, "isim": "Ali"}
```

```
@app.get("/urunler/{kategori}/{urun_id}")
def urun_detay(kategori: str, urun_id: int):
    return {
        "kategori": kategori,
        "urun_id": urun_id,
        "isim": f"{kategori} Ürünü {urun_id}"
    }
```

Query Parametreleri

URL'deki query string'den değerler aşağıdaki şekilde alınabilir;

```
@app.get("/ara")
def ara(q: str, limit: int = 10, offset: int = 0):
    """
    q: arama terimi (zorunlu)
    limit: sonuç sayısı (varsayılan 10)
    offset: başlangıç noktası (varsayılan 0)
    """
    return {
        "arama": q,
        "limit": limit,
        "offset": offset,
        "sonuclar": [f"Sonuç {i}" for i in range(offset, offset + limit)]
    }

@app.get("/filtrele")
def filtrele(kategori: Optional[str] = None, min_fiyat: Optional[float] = None):
    """Opsiyonel parametreler"""
    filtre = {}
    if kategori:
        filtre["kategori"] = kategori
    if min_fiyat:
        filtre["min_fiyat"] = min_fiyat
    return {"filtre": filtre}
```

Buradaki uygulama aşağıdaki örneklerde verilen query string'den değerler alabilir;

- <http://127.0.0.1:8000/ara?q=python>
- <http://127.0.0.1:8000/ara?q=python&limit=5&offset=10>
- http://127.0.0.1:8000/filtrele?kategori=kitap&min_fiyat=50

Request Body

POST isteklerinde veri göndermek için Pydantic modelleri kullanılır:

```
from typing import Optional

class Kullanici(BaseModel):
    isim: str = Field(..., min_length=2, max_length=50)
    email: EmailStr
    yas: int = Field(..., gt=0, lt=150)
    aktif: bool = True
    aciklama: Optional[str] = None

@app.post("/kullanici")
def kullanici_olustur(kullanici: Kullanici):
    return {
        "mesaj": "Kullanıcı oluşturuldu",
        "kullanici": kullanici
    }

class UrunGuncelle(BaseModel):
```

```
    isim: Optional[str] = None
    fiyat: Optional[float] = Field(None, gt=0)
    stok: Optional[int] = Field(None, ge=0)

@app.put("/urun/{urun_id}")
def urun_guncelle(urun_id: int, urun: UrunGuncelle):
    return {
        "urun_id": urun_id,
        "guncellenen": urun.dict(exclude_unset=True) # Sadece gönderilen alanlar
    }
```

HTTP Yöntemleri

Beş adet http yöntemine ilişkin örnekler aşağıda verilmiştir;

```
# GET - Veri okuma
@app.get("/urunler")
def urunleri_listele():
    return [{"id": 1, "isim": "Ürün 1"}, {"id": 2, "isim": "Ürün 2"}]

# POST - Yeni veri oluşturma
@app.post("/urunler", status_code=status.HTTP_201_CREATED)
def urun_olustur(urun: dict):
    return {"mesaj": "Ürün oluşturuldu", "urun": urun}

# PUT - Veri güncelleme (tüm alanlar)
@app.put("/urunler/{urun_id}")
def urun_guncelle_put(urun_id: int, urun: dict):
    return {"mesaj": f"Ürün {urun_id} güncellendi", "urun": urun}

# PATCH - Kısmi güncelleme
@app.patch("/urunler/{urun_id}")
def urun_guncelle_patch(urun_id: int, urun: dict):
    return {"mesaj": f"Ürün {urun_id} kısmen güncellendi", "urun": urun}

# DELETE - Veri silme
@app.delete("/urunler/{urun_id}", status_code=status.HTTP_204_NO_CONTENT)
def urun_sil(urun_id: int):
    return None # 204 için içerik dönmeyin
```

Response Models

Dönen veriyi şekillendirmek için `response_model` kullanılır:

```
#     email: EmailStr
#     sifre: str

class KullaniciCikis(BaseModel):
    id: int
    isim: str
    email: EmailStr
    # sifre alanı yok!

@app.post("/giris", response_model=KullaniciCikis)
def giris_yap(kullanici: KullaniciGiris):
    # Veritabanından kullanıcı bul ve doğrula
    return {
        "id": 1,
        "isim": "Ali Veli",
        "email": kullanici.email,
        "sifre": "hash123" # Bu alan response'a dahil edilmez
    }
```

Hata Yönetimi

```
# Basit hata
@app.get("/kullanici/{kullanici_id}")
def kullanici_getir(kullanici_id: int):
    if kullanici_id > 100:
        raise HTTPException(status_code=404, detail="Kullanıcı bulunamadı")
    return {"kullanici_id": kullanici_id}

# Detaylı hata
@app.get("/urun/{urun_id}")
def urun_getir(urun_id: int):
    if urun_id < 1:
        raise HTTPException(
            status_code=400,
            detail="Ürün ID 1'den küçük olamaz",
            headers={"X-Error": "Geçersiz ID"}
        )
    return {"urun_id": urun_id}
```

Dependency Injection

Ortak kodları paylaşmak ve bağımlılıkları enjekte etmek için:

```
# Basit dependency
def ortak_parametreler(q: Optional[str] = None, skip: int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}

@app.get("/items/")
def itemleri_oku(common: dict = Depends(ortak_parametreler)):
    return common

# Token kontrolü
def token_kontrol(x_token: str = Header(...)):
    if x_token != "gizli-token":
        raise HTTPException(status_code=400, detail="Geçersiz token")
    return x_token

@app.get("/korunmus")
def korunmus_endpoint(token: str = Depends(token_kontrol)):
    return {"mesaj": "Erişim izni verildi"}

# Veritabanı bağlantısı
def veritabanı_baglantisi():
    db = {"baglanti": "aktif"}
    try:
        yield db
    finally:
        print("Veritabanı bağlantısı kapatıldı")

@app.get("/veriler")
def veri_oku(db: dict = Depends(veritabanı_baglantisi)):
    return {"veritabanı": db["baglanti"], "veriler": [1, 2, 3]}
```

Asenkron Endpoint'ler

```
import aiohttp

@app.get("/sync")
def senkron_endpoint():
    # Normal fonksiyon
```

```

    return {"tip": "senkron"}

@app.get("/async")
async def asenkron_endpoint():
    # Asenkron fonksiyon
    await asyncio.sleep(1)
    return {"tip": "asenkron"}

@app.get("/harici-api")
async def harici_api_cagir():
    async with aiohttp.ClientSession() as session:
        async with session.get("https://api.github.com") as response:
            veri = await response.json()
            return {"durum": response.status, "veri": veri}

```

Dosya Yükleme

Dosya yükleme (file upload) örneği aşağıda verilmiştir;

```

from typing import List

@app.post("/dosya-yukle")
async def dosya_yukle(dosya: UploadFile = File(...)):
    icerik = await dosya.read()
    return {
        "dosya_adi": dosya.filename,
        "boyut": len(icerik),
        "icerik_tipi": dosya.content_type
    }

@app.post("/coklu-dosya")
async def coklu_dosya_yukle(dosyalar: List[UploadFile] = File(...)):
    return [
        {
            "dosya_adi": dosya.filename,
            "boyut": len(await dosya.read())
        }
        for dosya in dosyalar
    ]

```

Çapraz Kaynak Paylaşımı

Çapraz kaynak paylaşımı (Cross-Origin Resource Sharing-CORS) tekniğinde **önyüz** (frontend) uygulamalarından API'ye erişim kontrol altına alınır;

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000", "https://myapp.com"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Geliştirme için tüm kaynaklara izin ver (production'da kullanma!)
allow_origins=["*"]

```

Arka Planda Görev Çalıştırma

İstek tamamlandıktan sonra **arka planda görev çalıştırma** (background task):

```

def email_gonder(email: str, mesaj: str):
    # Email gönderme simülasyonu

```

```
import time
time.sleep(5) # Uzun süren işlem
print(f"{email} adresine email gönderildi: {mesaj}")

@app.post("/kayit")
async def kayit_ol(email: EmailStr, arka_plan: BackgroundTasks):
    arka_plan.add_task(email_gonder, email, "Hoş geldiniz!")
    return {"mesaj": "Kayıt başarılı, email gönderiliyor..."}
```

Veritabanı Entegrasyonu (SQLAlchemy)

```
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, Session

# Veritabanı kurulumu
SQLALCHEMY_DATABASE_URL = "sqlite:///./test.db"
engine = create_engine(SQLALCHEMY_DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

# Model tanımı
class KullaniciDB(Base):
    __tablename__ = "kullanici"
    id = Column(Integer, primary_key=True, index=True)
    isim = Column(String)
    email = Column(String, unique=True, index=True)

Base.metadata.create_all(bind=engine)

# Dependency
def veritabani_oturumu():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# CRUD (Create-Read-Update-Delete) işlemleri
@app.post("/kullanici/")
def kullanici_olustur(kullanici: Kullanici, db: Session = Depends(veritabani_oturumu)):
    db_kullanici = KullaniciDB(isim=kullanici.isim, email=kullanici.email)
    db.add(db_kullanici)
    db.commit()
    db.refresh(db_kullanici)
    return db_kullanici

@app.get("/kullanici/")
def kullanicilari_listele(skip: int = 0, limit: int = 10, db: Session = Depends(veritabani_oturumu)):
    kullanicilar = db.query(KullaniciDB).offset(skip).limit(limit).all()
    return kullanicilar

# Authentication - JWT Token
from jose import JWTError, jwt
from passlib.context import CryptContext

SECRET_KEY = "gizli-anahtar-buraya"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
```

```
def sifre_hash(sifre: str) -> str:
    return pwd_context.hash(sifre)

def sifre_dogrula(sifre: str, hash_sifre: str) -> bool:
    return pwd_context.verify(sifre, hash_sifre)

def token_olustur(veri: dict):
    to_encode = veri.copy()
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

@app.post("/token")
def giris(email: str, sifre: str):
    # Kullanıcı doğrulama (basitleştirilmiş)
    if email == "test@test.com" and sifre == "sifre123":
        token = token_olustur({"sub": email})
        return {"access_token": token, "token_type": "bearer"}
    raise HTTPException(status_code=401, detail="Geçersiz kimlik bilgileri")
```

Modüler Yapı

Büyük projelerde kodları modüllere ayırmak için `APIRouter` yapısı kullanılır:

```
# routers/kullaniciilar.py
from fastapi import APIRouter

router = APIRouter(
    prefix="/kullaniciilar",
    tags=["kullaniciilar"]
)

@router.get("/")
def kullaniciilari_listele():
    return [{"id": 1, "isim": "Ali"}]

@router.get("/{kullanici_id}")
def kullanici_detay(kullanici_id: int):
    return {"id": kullanici_id, "isim": "Ali"}
```

```
# main.py
from fastapi import FastAPI
from routers import kullaniciilar

app = FastAPI()

app.include_router(kullaniciilar.router)
```

Gerçek Zamanlı İletişim

Gerçek zamanlı iletişim için `WebSocket` yapısı kullanılır:

```
@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    try:
        while True:
            veri = await websocket.receive_text()
            await websocket.send_text(f"Mesaj alındı: {veri}")
    except:
```



```
print("Bağlantı kapatıldı")
```

API'leri Test Etme

İlk yöntem **tarayıcı** (**browser**) üzerinden test etmektir.

```
http://localhost:8000/docs
```

İkinci yöntem CURL dili ile test etmektir;

```
curl -X GET "http://localhost:8000/items/5"
```

Üçüncü yöntem ise Python dili ile **request** ile test etmektir;

```
import requests
response = requests.get("http://localhost:8000/items/5")
print(response.json())
```

En İyi Uygulamalar

- Type hints her yerde kullanın
- Response models ile veri şekillendirme yapın
- Dependency injection ile kod tekrarını önleyin
- Async endpoint'leri I/O işlemleri için kullanın
- Router'larla kodları modüllere ayırın
- Environment variables ile yapılandırma yapın
- Logging ekleyin
- Rate limiting uygulayın
- HTTPS kullanın (production'da)
- Input validation'ı Pydantic'e bırakın
- Database connection pooling kullanın
- Background tasks ile uzun işlemleri yönetin

Web Proje Yapısı

```
my-fastapi-app/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── models/
│   │   ├── __init__.py
│   │   └── kullanıcı.py
│   ├── routers/
│   │   ├── __init__.py
│   │   ├── kullanıcılar.py
│   │   └── ürünler.py
│   ├── schemas/
│   │   ├── __init__.py
│   │   └── kullanıcı.py
│   ├── database.py
│   └── dependencies.py
├── tests/
│   ├── __init__.py
│   └── test_main.py
├── requirements.txt
└── .env
```

EN İYİ UYGULAMALAR VE TEMİZ KOD

Bu bölümde Python'da profesyonel kod yazma prensiplerini, kod kalitesini artırma yöntemlerini ve yaygın kullanılan [en iyi uygulamaları](#) (best practice) inceleyeceğiz.

Python Stil Rehberi

PEP 8, Python kodu için resmi stil rehberidir. Kodun okunabilirliğini artırır.

Girinti

```
# DOĞRU
def hesapla(x, y):
    sonuc = x + y
    return sonuc

# YANLIŞ - Tab kullanma, 4 boşluk kullan!
def hesapla(x, y):
    sonuc = x + y
    return sonuc
```

Satır Uzunluğu

```
# DOĞRU - Maksimum 79 karakter
def uzun_fonksiyon_ismi(parametre1, parametre2,
                        parametre3, parametre4):
    return parametre1 + parametre2 + parametre3 + parametre4

# Uzun string'ler için
mesaj = (
    "Bu çok uzun bir mesajdır. "
    "Birden fazla satıra bölünmüştür. "
    "Okunabilirliği artırır."
)
```

İsmlendirme Kuralları

```
# Değişkenler ve fonksiyonlar: snake_case
kullanici_adi = "Ali"
toplam_fiyat = 100.50

def hesapla_toplam(fiyat, vergi):
    return fiyat * (1 + vergi)

# Sınıflar: PascalCase
class KullaniciYoneticisi:
    pass

class OgrenciKayit:
    pass

# Sabitler: UPPER_SNAKE_CASE
MAX_BAGLANTI = 100
API_ANAHTARI = "gizli-anahtar"
PI_DEGERI = 3.14159

# Private (özel) Üyeler: önünde _
class Hesap:
    def __init__(self):
        self._bakiye = 0 # Private
        self.__hesap_no = "12345" # Strongly private
```

Modülleri İç Aktarma Düzeni

İlk önce standart kütüphaneler, sonra üçüncü parti kütüphaneler, en son kendi modüllerimizi **iç** aktarmalıyız (**import**);

```
# 1. Standart kütüphane
import os
import sys
from datetime import datetime

# 2. Üçüncü parti kütüphaneler
import numpy as np
import pandas as pd
from fastapi import FastAPI

# 3. Kendi modülleriniz
from myapp.models import Kullanici
from myapp.utils import hesapla_toplam
```

Boşluklar

```
# DOĞRU
x = 5
y = x + 10
liste = [1, 2, 3]
sozluk = {"a": 1, "b": 2}

def fonksiyon(a, b, c):
    return a + b + c

# YANLIŞ
x=5
y=x+10
liste=[1,2,3]
sozluk={"a":1,"b":2}

def fonksiyon(a,b,c):
    return a+b+c
```

Dokümantasyon Metinleri

Fonksiyon, sınıf ve modüllerin dokümantasyonu için `docstring` kullanılmalıdır:

```
"""
Verilen sayıların toplamını hesaplar.

Args:
    sayilar (list[int]): Toplanacak sayılar listesi

Returns:
    int: Sayıların toplamı

Raises:
    ValueError: Liste boşsa

Examples:
    >>> toplam_hesapla([1, 2, 3])
    6
    >>> toplam_hesapla([10, 20])
    30
"""
if not sayilar:
```

```

        raise ValueError("Liste boş olamaz")
        return sum(sayilar)

class Kullanici:
    """
    Kullanıcı bilgilerini yöneten sınıf.

    Attributes:
        isim (str): Kullanıcının adı
        email (str): Kullanıcının email adresi
        yas (int): Kullanıcının yaşı
    """

    def __init__(self, isim: str, email: str, yas: int):
        self.isim = isim
        self.email = email
        self.yas = yas

```

Tür İpuçlarını Kullanın

Kod okunabilirliği ve IDE desteği için **tür ipuçlarını** (**type hint**) kullanın:

```

def filtrele(
    sayilar: List[int],
    kosul: Callable[[int], bool]
) -> List[int]:
    """Verilen koşulu sağlayan sayıları filtreler."""
    return [s for s in sayilar if kosul(s)]

def kullanici_bul(
    kullanici_id: int
) -> Optional[Dict[str, Union[str, int]]]:
    """Kullanıcı bulunursa dict, bulunamazsa None döner."""
    if kullanici_id > 0:
        return {"id": kullanici_id, "isim": "Ali"}
    return None

```

SOLID İlkeleri

S-Single Responsibility (Tek Sorumluluk)

Her sınıf veya fonksiyon tek bir işten sorumlu olmalı:

```

# YANLIŞ - Birden fazla sorumluluk
class Kullanici:
    def __init__(self, isim, email):
        self.isim = isim
        self.email = email

    def kaydet(self):
        # Veritabanına kaydet
        pass

    def email_gonder(self):
        # Email gönder
        pass

    def rapor_olustur(self):
        # Rapor oluştur
        pass

# DOĞRU - Her sınıf tek bir işten sorumlu

```

```
class Kullanici:
    def __init__(self, isim, email):
        self.isim = isim
        self.email = email

class KullaniciRepository:
    def kaydet(self, kullanici: Kullanici):
        # Veritabanına kaydet
        pass

class EmailServisi:
    def gonder(self, alici: str, mesaj: str):
        # Email gönder
        pass

class RaporServisi:
    def olustur(self, kullanici: Kullanici):
        # Rapor oluşturun
        pass
```

O-Open/Closed (Açık/Kapalı)

Genişletmeye açık, değişikliğe kapalı olmalı:

```
# YANLIŞ - Yeni şekil eklemek için sınıfı değiştirmek gerekir
class AlanHesaplama:
    def hesapla(self, sekil):
        if sekil.tip == "daire":
            return 3.14 * sekil.yaricap ** 2
        elif sekil.tip == "kare":
            return sekil.kenar ** 2

# DOĞRU - Abstract base class kullanımı
from abc import ABC, abstractmethod

class Sekil(ABC):
    @abstractmethod
    def alan_hesapla(self) -> float:
        pass

class Daire(Sekil):
    def __init__(self, yaricap: float):
        self.yaricap = yaricap

    def alan_hesapla(self) -> float:
        return 3.14 * self.yaricap ** 2

class Kare(Sekil):
    def __init__(self, kenar: float):
        self.kenar = kenar

    def alan_hesapla(self) -> float:
        return self.kenar ** 2

# Yeni şekil eklemek kolay:
class Ucgen(Sekil):
    def __init__(self, taban: float, yukseklik: float):
        self.taban = taban
        self.yukseklik = yukseklik

    def alan_hesapla(self) -> float:
        return (self.taban * self.yukseklik) / 2
```

L-Liskov Substitution (Liskov İkame)

Alt sınıf, üst sınıfın yerine kullanılabilirmeli:

```
# DOĞRU
# class Kus:
#     def ucabilir_mi(self) -> bool:
#         return True

# class Serce(Kus):
#     pass # Uçabilir

# class Penguen(Kus):
#     def ucabilir_mi(self) -> bool:
#         return False # Uçamaz ama davranış tutarlı

# Kullanım
# def kus_ucur(kus: Kus):
#     if kus.ucabilir_mi():
#         print(f"{kus.__class__.__name__} uçuyor")
#     else:
#         print(f"{kus.__class__.__name__} uçamıyor")
```

I-Interface Segregation (Arayüz Ayrımı)

Sınıflar kullanmadıkları yöntemleri kodlamaya zorlanmamalı:

```
# YANLIŞ
class Isci(ABC):
    @abstractmethod
    def calis(self): pass

    @abstractmethod
    def yemek_ye(self): pass

    @abstractmethod
    def uyku_mo_ver(self): pass

class Robot(Isci): # Robot yemek yemez ve uyumaz!
    def calis(self): pass
    def yemek_ye(self): pass # Gereksiz
    def uyku_mo_ver(self): pass # Gereksiz

# DOĞRU - Küçük interface'ler
class Calisabilir(ABC):
    @abstractmethod
    def calis(self): pass

class Beslenebilir(ABC):
    @abstractmethod
    def yemek_ye(self): pass

class Dinlenebilir(ABC):
    @abstractmethod
    def uyku_mo_ver(self): pass

class Insan(Calisabilir, Beslenebilir, Dinlenebilir):
    def calis(self): pass
    def yemek_ye(self): pass
    def uyku_mo_ver(self): pass

class Robot(Calisabilir):
```

```
def calis(self): pass
```

D-Dependency Inversion

Bağımlılık tersine çevirme olarak bilinen bu prensip üst seviye modüller alt seviye modüllere bağımlı olmamasını ister:

```
# YANLIŞ - Doğrudan bağımlılık
class MySQLVeritabani:
    def kaydet(self, veri):
        print("MySQL'e kaydedildi")

class KullaniciServisi:
    def __init__(self):
        self.db = MySQLVeritabani() # Sıkı bağlantı

    def kullanıcı_ekle(self, kullanıcı):
        self.db.kaydet(kullanıcı)

# DOĞRU - Abstraction kullanımı
class Veritabani(ABC):
    @abstractmethod
    def kaydet(self, veri): pass

class MySQLVeritabani(Veritabani):
    def kaydet(self, veri):
        print("MySQL'e kaydedildi")

class PostgreSQLVeritabani(Veritabani):
    def kaydet(self, veri):
        print("PostgreSQL'e kaydedildi")

class KullaniciServisi:
    def __init__(self, veritabani: Veritabani):
        self.db = veritabani # Dependency injection

    def kullanıcı_ekle(self, kullanıcı):
        self.db.kaydet(kullanıcı)

# Kullanım
mysql_db = MySQLVeritabani()
servis = KullaniciServisi(mysql_db)

postgres_db = PostgreSQLVeritabani()
servis2 = KullaniciServisi(postgres_db)
```

Tekrarlardan Kaçınma İlkesi

Bu ilki şu şekilde ifade edilir: "Her bilgi parçasının bir sistem içinde tek, açık ve güvenilir bir temsili olmalıdır". Yani aynı görevi gören kod tekrarından kaçının (**Don't Repeat Yourself-DRY**):

```
# YANLIŞ - Kod tekrarı
def kullanıcı_ekle(isim, email):
    if not isim:
        print("Hata: İsim boş olamaz")
        return False
    if not email:
        print("Hata: Email boş olamaz")
        return False
    if "@" not in email:
        print("Hata: Geçersiz email")
        return False
```

```
# Kullanıcı ekle
return True

def urun_ekle(isim, fiyat):
    if not isim:
        print("Hata: İsim boş olamaz")
        return False
    if fiyat < 0:
        print("Hata: Fiyat negatif olamaz")
        return False
    # Ürün ekle
    return True

# DOĞRU - Validation fonksiyonu
def dogrula(deger, hata_mesaji):
    if not deger:
        print(f"Hata: {hata_mesaji}")
        return False
    return True

def kullanıcı_ekle(isim, email):
    if not dogrula(isim, "İsim boş olamaz"):
        return False
    if not dogrula(email, "Email boş olamaz"):
        return False
    if "@" not in email:
        print("Hata: Geçersiz email")
        return False
    # Kullanıcı ekle
    return True
```

Aptala Anlatır Gibi Basit İlkesi

Bu tasarım ve geliştirme ilkesi, sistemlerin, ürünlerin ve süreçlerin gereksiz karmaşıklıktan kaçınılarak basit tutulması gerektiğini savunur. Basit tutun, gereksiz karmaşıklıktan kaçının (**Keep It Simple, Stupid-KISS**):

```
# YANLIŞ - Gereksiz karmaşık
def cift_mi(sayi):
    if sayi % 2 == 0:
        return True
    else:
        return False

# DOĞRU - Basit
def cift_mi(sayi):
    return sayi % 2 == 0
```

Code Smell'ler ve Çözümleri

Long Method (Uzun Fonksiyon)

```
# YANLIŞ
def siparis_isle(siparis):
    # 100+ satır kod
    # Stok kontrolü
    # Fiyat hesaplama
    # Kargo hesaplama
    # Email gönderme
    # Veritabanı kayıt
    pass
```



```
# DOĞRU
def siparis_isle(siparis):
    stok_kontrol(siparis)
    fiyat = fiyat_hesapla(siparis)
    kargo = kargo_hesapla(siparis)
    email_gonder(siparis)
    veritabanina_kaydet(siparis)
```

Magic Numbers (Sihirli Sayılar)

```
# YANLIŞ
def indirim_hesapla(fiyat):
    if fiyat > 1000:
        return fiyat * 0.15
    elif fiyat > 500:
        return fiyat * 0.10
    return fiyat * 0.05

# DOĞRU
YUKSEK_INDIRIM_SINIRI = 1000
ORTA_INDIRIM_SINIRI = 500
YUKSEK_INDIRIM_ORANI = 0.15
ORTA_INDIRIM_ORANI = 0.10
DUSUK_INDIRIM_ORANI = 0.05

def indirim_hesapla(fiyat):
    if fiyat > YUKSEK_INDIRIM_SINIRI:
        return fiyat * YUKSEK_INDIRIM_ORANI
    elif fiyat > ORTA_INDIRIM_SINIRI:
        return fiyat * ORTA_INDIRIM_ORANI
    return fiyat * DUSUK_INDIRIM_ORANI
```

Kod Kalitesi Araçları

Kod biçimlendirme aracı (Black):

```
pip install black
black dosya.py
```

Kod Analiz aracı (Pylint):

```
pip install pylint
pylint dosya.py
```

Tip kontrolü aracı (mypy):

```
pip install mypy
mypy dosya.py
```

Hızlı [SQL biçimlendirici](#) (linter) aracı (ruff):

```
pip install ruff
ruff check dosya.py
```

İz Kayıtları Oluşturma

[İz kaydı](#) (log) oluşturmak için `print()` yerine `logging` kullanın:

```
# Logging ayarları
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)
```

```

logger = logging.getLogger(__name__)

def kullanıcı_ekle(isim, email):
    logger.info(f"Yeni kullanıcı ekleniyor: {isim}")
    try:
        # Kullanıcı ekle
        logger.debug(f"Email doğrulanıyor: {email}")
        # ...
        logger.info("Kullanıcı başarıyla eklendi")
        return True
    except Exception as e:
        logger.error(f"Kullanıcı eklenirken hata: {str(e)}")
        return False

```

Konfigürasyon Yönetimi

```

# config.py
import os
from dotenv import load_dotenv

load_dotenv()

class Config:
    DEBUG = os.getenv("DEBUG", "False") == "True"
    DATABASE_URL = os.getenv("DATABASE_URL")
    SECRET_KEY = os.getenv("SECRET_KEY")
    MAX_BAGLANTI = int(os.getenv("MAX_BAGLANTI", "100"))

```

Konfigürasyona yönelik tüm bilgiler `env` dosyasında olmalıdır;

```

# .env dosyası
DEBUG=True
DATABASE_URL=postgresql://user:pass@localhost/db
SECRET_KEY=gizli-anahtar
DB_PASSWORD=Password123
MAX_BAGLANTI=50

```

Aşağıda güvenlikle ilgili doğru ve yanlış kullanım gösterilmiştir;

```

# YANLIŞ Kullanım:
password = "12345" # Kodda hardcoded
sql = f"SELECT * FROM users WHERE id = {user_id}" # SQL injection

# GÜVENLİ Kullanım
import os
password = os.getenv('DB_PASSWORD')

# Parametrelili query
cursor.execute("SELECT * FROM users WHERE id = ?", (user_id,))

```

Bağlam Yöneticisi

Herhangi bir programlama dilinde, dosya işlemleri veya veri tabanı bağlantıları gibi kaynakların kullanımı çok yaygındır, ancak sınırlıdır. Bu nedenle, asıl sorun, bu kaynakların kullanımdan sonra serbest bırakılmasını sağlamaktır. Serbest bırakılmazlarsa, kaynak sızıntısına yol açar ve sistemin yavaşlamasına veya çökmesine neden olabilir.

Python'ın **bağlam yöneticileri** (**context manager**), kaynakları otomatik olarak kurmak ve temizlemek için düzgün bir yol sağlar ve hatalar oluşsa bile düzgün bir şekilde yönetilmelerini sağlar.

Bağlam Yöneticilerine Neden İhtiyacımız Var?

- **Otomatik temizleme:** Dosyalar veya bağlantılar gibi kaynakları manuel çaba gerektirmeden serbest bırakır.
- **Kaynak sızıntısını önleme:** Hatalar oluşsa bile düzgün serbest bırakmayı sağlar.
- **Daha temiz kod:** Ayrıntılı `try-finally` bloklarını ile değiştirir.
- **İstisna güvenli:** İstisnalarda bile her zaman temizleme işlemi gerçekleştirir.
- **Özel kontrol:** `__enter__` ve `__exit__` kullanarak herhangi bir kaynağı yönetmenizi sağlar.

```
# YANLIŞ
dosya = open("veri.txt", "r")
icerik = dosya.read()
dosya.close() # Hata olursa kapatılmayabilir!

# DOĞRU
with open("veri.txt", "r") as dosya:
    icerik = dosya.read()
    # Otomatik kapatılır

# Kendi context manager'iniz
from contextlib import contextmanager

@contextmanager
def veritabani_baglantisi():
    baglanti = veritabani_baglan()
    try:
        yield baglanti
    finally:
        baglanti.kapat()

with veritabani_baglantisi() as db:
    db.sorgu_calistir("SELECT * FROM users")
```

Liste veya Sözlük Kavrama

Kavramalar (**comprehension**) gereksiz uzun yazılmamalıdır.

```
# YANLIŞ - Gereksiz uzun
kareler = []
for i in range(10):
    kareler.append(i ** 2)

# DOĞRU - Kısa ve öz
kareler = [i ** 2 for i in range(10)]

# Dict comprehension
sozluk = {i: i ** 2 for i in range(10)}

# Koşullu
cift_kareler = [i ** 2 for i in range(10) if i % 2 == 0]
```

En İyi Uygulamalar

- PEP 8'e uyun - Kod stili tutarlı olsun
- Type hints kullanın - Kod okunabilirliği artar
- Docstring yazın - Her fonksiyon dokümanite edilmeli
- DRY prensibine uyun - Kod tekrardan kaçın
- SOLID prensiplerini uygulayın - Sürdürülebilir kod yazın
- Test yazın - Her fonksiyon test edilmeli
- Logging kullanın - `print()` yerine logging
- Environment variables kullanın - Hassas bilgileri kodda saklamayın

- Virtual environment kullanın - Proje bağımlılıklarını izole edin
- Git kullanın - Versiyon kontrolü yapın
- Code review yapın - Kodunuzu başkalarına inceletin
- Linter ve formatter kullanın - Kod kalitesini artırın
- CI/CD pipeline kurun - Otomatik test ve deployment
- Exception handling yapın - Hataları yakalayın
- Security best practices uygulayın - Güvenlik açıklarından kaçının

ŞEKİL LİSTESİ

Şekil 1.	Motorola 6802, Intel 8086 İşlemciler ile 2732 EPROM Belleği	9
Şekil 2.	Frekansı 1 olan Kare Dalga İşareti	10
Şekil 3.	Basit bir İşlemcinin Yapısı	10
Şekil 4.	Python Yorumlayıcısının Çalışması	24
Şekil 5.	IDLE Kabuk Programı	24
Şekil 6.	Akış Diyagramları Genel Şekilleri	48
Şekil 7.	Örnek Akış Diyagramı	48
Şekil 8.	If Talimatı İcra Akışı	49
Şekil 9.	If-Else Talimatı Sözde Kodu ve İcra Akışı	50
Şekil 10.	Programın If Talimatlarıyla Yazılması Halinde İcra Akışı	51
Şekil 11.	Programın If-Else Talimatlarıyla Yazılması Halinde İcra Akışı	52
Şekil 12.	Switch Talimatı Sözde Kodu ve İcra Akışı	52
Şekil 13.	Bir Döngünün İcra Akışı	54
Şekil 14.	For Talimatı İcra Akışı	54
Şekil 15.	For-Else Talimatı İcra Akışı	56
Şekil 16.	While Talimatı İcra Akışı	57
Şekil 17.	While Döngüsünde Break Talimatı İcra Akışı	58
Şekil 18.	For ve For-Else Döngülerinde Break Talimatı İcra Akışı	59
Şekil 19.	While Döngüsünde Continue Talimatı İcra Akışı	59
Şekil 20.	For ve For-Else Döngülerinde Continue Talimatı İcra Akışı	60
Şekil 21.	Öğrenci, Öğretmen ve Müdür UML Sınıf Diyagramları	85
Şekil 22.	Kişİ Taban Sınıfı Eklenmiş Kalıtım UML Sınıf Diyagramı	87
Şekil 23.	Tekil Deseni UML Diyagramı	94

TABLO LİSTESİ

Tablo 1.	Bazı İşlemcilerin Adres ve Veri Yolu Genişlikleri.....	9
Tablo 2.	6802 Emir Seti Örneği ve Sembolik İsim Listesi.....	12
Tablo 3.	Tamsayı Değişkenler ve Sayı Sınırları	17
Tablo 4.	Kayan Noktalı Sayılar ve Sınırları.....	17
Tablo 5.	İki ile çarpma fonksiyonu.....	18
Tablo 6.	Aritmetik İşleçler.....	38
Tablo 7.	Karşılaştırma İşleçleri.....	39
Tablo 8.	Atama İşleçleri	40
Tablo 9.	Mantıksal İşleçler	40
Tablo 10.	Bir Düzeyi İşleçler.....	41
Tablo 11.	Üyelik İşleçleri	41
Tablo 12.	Üyelik İşleçleri	42
Tablo 13.	İşleç Öncelikleri	42
Tablo 14.	Biçim Belirleyiciler.....	45
Tablo 15.	If Talimatı İçeren Bir C Programının İcrası.....	49
Tablo 16.	Dizi Eleman Veri Tiplerini Belirleyen Parametreler	101
Tablo 17.	Dizi ve Liste Nesnesi Yöntemleri	104
Tablo 18.	Dizgi Nesnesi Yöntemleri	107
Tablo 19.	Kaçış Dizisi Karakterleri.....	108
Tablo 20.	İstisna Türleri	118

Dizin

abstract class, 91
abstract method, 91
abstraction, 76, 83
access modifier, 82
access modifier, 84
accumulator, 11
actual parameter.*Bakın* argument
address bus, 8, 9
ALU, 125
annotation, 68
arbitrary argument, 67
argument, 18
array, 19, 32, 100, 106
assembly code, 12
assertion, 139
assignment operator, 39
asynchron, 146, 149
attribute, 76, 97
background task, 165
base, 17
base case, 70
base class, 86, 89
behavior, 19, 20, 76, 78, 85
best practice, 169
big data, 123
binary, 16, 17
binary digit, 9
bit.*Bakın* binary digit
bitwise operator, 40
broadcasting, 132
built-in function, 62
bus, 8
byte, 9
byte code, 24
call, 18
callable, 96
Central Processing Unit, 8
child class, 86
CISC.*Bakın* Complex Instruction Set Computing
class, 76, 84, 92
class attribute, 77
class diagram, 85
class method, 80
collection, 136
command prompt, 23, 28
comparision operator, 39
compiler, 12, 22
Complex Instruction Set Computing, 11
component, 84
comprehension, 115, 178
computer program, 11
condition, 49, 50, 57
conditional choice, 47, 49
conditional code, 49, 50
console, 13
constructor, 77
context manager, 123, 152, 177
control abstraction, 83, 84
control expression, 53
control operation, 47
control statement, 54
control structure, 19
coroutine, 150
CORS.*Bakın* Cross-Origin Resource Sharing
coverage, 145
CPU.*Bakın* Central Processing Unit
Cross-Origin Resource Sharing, 165
data, 8
data abstraction, 83, 84
data bus, 9
data structure, 19, 100
data type, 17, 18, 26
debug, 87, 117, 141
decision making, 49, 50
decode, 11
decorator, 98
default argument, 66
derived class, 86, 89
deserialization, 137
design pattern, 94
destructor, 77
dict, 34, 112, 137, 159
divide and conquer, 62
docstring, 65
Don't Repeat Yourself, 174
double dispatch, 89
DRY.*Bakın* Don't Repeat Yourself
duck typing, 89
dynamic binding, 90
dynamic type system, 28
else code, 50
empty product, 70
encapsulation, 84
encoded, 106
enumeration, 34
escape sequence, 107
exception, 117
exception chaining, 119
exception handling, 117
execute, 11
exit code, 125
explicit casting, 37

exponent, 38
 expression, 18, 36, 38, 52, 105, 115
 fetch, 11
 field, 76
 file descriptor, 122
 file upload, 165
 fixture, 142, 144, 146
 fixture scope, 143
 floating point, 125
 floating point number, 17
 floor division, 38
 flowchart, 48
 formal argument, 66
 format specifier, 45
 fraction, 17
 framework, 84
 frontend, 165
 frozenset, 111
 function, 17
 general class, 86
 global variable, 69
 hasable, 34
 hash, 34, 109
 hashable, 34, 109, 112
 hexadecimal, 45
 high level programming language, 14
 Inheritance, 119
 identification, 47
 identifier definition, 36
 identity operator, 42
 immutable, 33, 108, 111, 136
 imperative programming, 19
 implicit casting, 37
 import, 64, 71, 100, 124, 170
 index, 33, 100
 infinity, 126
 information, 8
 inheritance, 86, 87
 inner class, 94
 input output operation, 36
 instance, 76
 instance attribute, 77
 instance method, 80
 instruction, 11, 15
 instruction code, 11, 14, 15, 25
 instruction set, 11
 interface, 92
 interface implementation. *Bakın* interface realization
 interface realization, 92
 interpreter, 23
 introspect, 95
 invariant, 141
 iterable, 32, 34, 55, 105, 115
 iteration, 69, 104
 iterator, 136, 153
JSON, 123, 137
 jump, 47
 Keep It Simple, Stupid, 175
 key-value, 112
 key-value pair, 34
 keyword, 25
 keyword argument, 66
KISS. *Bakın* Keep It Simple, Stupid
 lambda expression, 73
 Last In First Out, 104
 library, 84
 linter, 176
 list, 33, 103, 137
 list comprehension, 74, 105
 literal, 16, 26, 27, 107
 local variable, 69
 log, 117, 176
 logical operator, 40
 logical sequence, 15, 25, 47
 loop, 54, 69
 loop code, 57
 low level programming, 14
 machine language, 11
 magic method, 80
 main function, 19
 mantissa, 17
 matrix, 103
 membership operator, 41
 memory, 8
 message-passing, 19, 20, 76, 86
 method, 76, 77, 80
 method overloading, 88, 89
 method resolution order, 87
 mnemonic, 12, 14, 15
 mock, 143, 144
 modulus, 38
MRO. *Bakın* method resolution order
 multiple inheritance, 87, 92
 mutable, 33
 naming convention, 27
NaN, 126
 native code, 22
 nested class, 94
 nested loop, 57
 Not a Number, 31
 object, 19, 26, 76, 100
 object oriented programming, 20
 objects, 20
 opcode. *Bakın* instruction code
 operand, 38, 107

operating system, 12
 operator, 36, 38
 operator precedence, 42
 outhur loop, 57
 overriding, 81, 89
 Package Installer for Python, 73
 parametrize, 142
 parent class, 86
 pattern, 52
 pattern matching, 157
 physical sequence, 47
 placeholder, 108
 pointer, 19
 polymorphism, 89
 positional argument, 66
 positional-only argument, 67
 private, 82
 private class, 86
 programming, 11
 property, 85
 protected, 82
 public, 77, 82
 punch card, 14
RAII, 123
 raising an exception, 118
 range, 55
 recursion, 69
 recursive, 70
 Reduced Instruction Set Computing, 11
 reflection, 20, 95
 register, 11
 relational loop, 47
 relational `operator.Bakin` comparision
 operator
 reserved word `Bakin` keyword
 resource, 122
 return, 18
 RISC `Bakin` Reduced Instruction Set Computing
 role `Bakin` interface
 run time, 90
 runtime, 117
 scope, 69
 script, 23
 sentinel value, 58
 separator, 37
 sequential operation, 36
 serialization, 137
 set, 34, 109
 single dispatch, 89
 singleton class, 94
 singletone, 77
 slice, 33
 slicing, 103
 spaghetti code, 16
 special method, 77
 stack, 104
 stack trace, 118
 standart library, 120
 state, 19, 20, 76
 statement, 15, 16, 18, 25, 36
 static method, 81
 static type system, 28
 string, 31, 36, 37, 73, 102, 106
 strongly typed, 37
 structural programming, 18
 synchron, 149
TDD, 147
 terminal, 13
 timestamp, 120
 timezone, 121
 top down, 62
 traceback, 117
 translator, 22
 tuple, 32, 54, 74, 105, 108
 type hints, 156, 161, 171
 UML `Bakin` Unified Modelling Language
 Unified Modelling Language, 85
 unique, 34, 35, 109
 unit test, 139
 user defined function, 62
 variable, 16, 19, 26, 100
XML, 123
 zip, 105